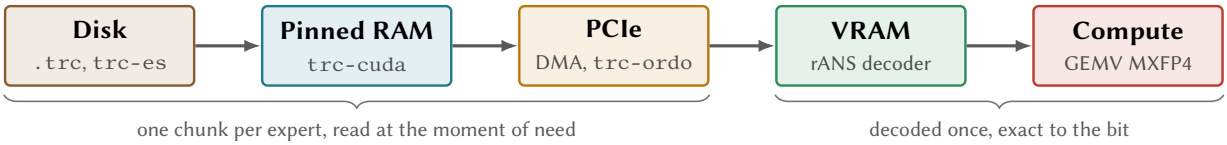
troncons

Volume II: the system

Architecture, `.trc` format, kernels, cache and engine; the exact reference and the judge; milestone J1 and its independent trial

Research thesis — the program, built and measured



63.4 GB
model served (gpt-oss-120b), four times the RAM

384 MB
peak memory of the process; llama.cpp: 10.9 GB

×2.22
1.965 tokens/s, integrity guard included, against the best llama.cpp baseline

0 / 5 120
steps whose logits differ between coded and raw file

Stone

Toronto, 16 October 2026

Follows Volume I, *the measured foundations*. Every figure is regenerated from the measurement data committed in the `troncons` repository; every number cites the file it comes from.

Abstract

Volume I of *troncons* measured an ordinary laptop (16 GB of RAM, an 8 GB RTX 4060, an encrypted NVMe disk), proved the roof of the problem and decided, by calculation, to build an exact architecture first: serve through the graphics card a model much larger than all the memory of the machine, its experts compressed losslessly on disk and decoded at the moment a computation needs them, within a memory window that does not depend on the size of the model. This second volume is the volume of the system: the program, built in Rust with CUDA kernels by a team of five lots with frozen interfaces, and its first milestone, judged on gpt-oss-120b (63.4 GB, four times the RAM, eight times the card's memory) against twelve criteria declared before the first line of code.

The volume gives the architecture and structure of the program; the `.trc` format to the bit, one chunk per expert and streams stored by refinement level; the byte path, from the disk to the card's memory, through unbuffered reads and pinned memory; the kernels, among them the rANS decoder with per-lane tables, with the proof of their exactness and determinism; a double-precision reference and the theory of the judge, founded on the Kullback–Leibler divergence; the expert cache, the scheduler and the engine, with their lemmas.

The results fit in four numbers, each recomputed by a second path from the committed passes. *Memory*: a 384 MB peak for the 63.4 GB model, the same to within 3.7 MB as for a model five times smaller, where llama.cpp holds 10.9 GB (11.4 GB in its fastest configuration); memory no longer follows the size of the model. *Exactness*: logits identical to the bit between the coded and the raw file over 5 120 steps, from one build and one order of computation to another. *Speed*: 1.965 tokens per second with the integrity guard, 2.22 times the best configuration of llama.cpp, and a prefill seven times faster; two earlier ratios, ×3.85 then ×2.79, are struck out because the baseline was badly tuned (sweeping it raised it from 0.57 to 0.88 token/s). *Correctness*: closer to the exact mathematics of the model than llama.cpp (KL 0.0031 against 0.0607 on the 120b), and, with an f32 KV cache, equal to them up to the format: the whole gap came from the f16 rounding of the cache.

A judge who wrote neither the specification nor the code redid the numbers: his decoder, written from the specification alone, reproduces the chunks to the bit; his own double-precision forward pass confirms the reference up to the rounding of the format. He found two serious defects (a pass that hangs without a message, an orphan child process that blocks its driver) and five medium ones, among them a corruption of raw bytes invisible down to the text. The round of corrections corrected or measured them in one morning, each with a control that fails on the old binary; a production guard now turns any corruption of a chunk read into a named fatal error, for 1.6 % of speed at equal disk. Ten criteria out of twelve hold; the other two await a decision of the arbiter.

What remains is the research contribution, the base + complement split of milestone J2; an unencrypted disk, which will tell what BitLocker costs on a real model; Stone's biomimetic study, the weight genome; and the gate that will give this engine back to relais. The thesis exists in two complete versions, English and French; every figure is regenerated from the committed measurement data.

Résumé

Le Tome I de *troncons* avait mesuré un portable ordinaire (16 Go de RAM, une RTX 4060 de 8 Go, un disque NVMe chiffré), démontré le toit du problème et décidé, par le calcul, de construire d'abord une architecture exacte : servir par la carte graphique un modèle bien plus gros que toute la mémoire de la machine, ses experts compressés sans perte sur le disque et décodés à l'instant où un calcul en a besoin, dans une fenêtre de mémoire qui ne dépend pas de la taille du modèle. Ce second tome est celui du système : le programme, construit en Rust avec des noyaux CUDA par une équipe de cinq lots aux interfaces gelées, et son premier jalon, jugé sur gpt-oss-120b (63,4 Go, quatre fois la RAM, huit fois la mémoire de la carte) contre douze critères déclarés avant la première ligne de code.

Le tome donne l'architecture et la structure du programme ; le format `.trc` au bit près, un tronçon par expert et des flux rangés par niveau de raffinement ; le chemin des octets, du disque à la mémoire de la carte, par la lecture sans tampon et la mémoire épinglée ; les noyaux, dont le décodeur rANS à table par voie, avec la preuve de leur exactitude et de leur déterminisme ; une référence en double précision et la théorie du juge, fondée sur la divergence de Kullback et Leibler ; le cache d'experts, l'ordonnanceur et le moteur, avec leurs lemmes.

Les résultats tiennent en quatre nombres, chacun recalculé par un second chemin depuis les passes commitées. *Mémoire* : 384 Mo de pointe pour le modèle de 63,4 Go, la même à 3,7 Mo près que pour un modèle cinq fois plus petit, quand llama.cpp en tient 10,9 Go (11,4 Go dans sa configuration la plus rapide) ; la mémoire ne suit plus la taille du modèle. *Exactitude* : des logits identiques au bit entre le fichier codé et le fichier brut sur 5 120 pas, d'une construction et d'un ordre de calcul à l'autre. *Vitesse* : 1,965 jeton par seconde avec la garde d'intégrité, 2,22 fois la meilleure configuration de llama.cpp, et un préremplissage sept fois plus rapide ; deux rapports antérieurs, ×3,85 puis ×2,79, sont barrés parce que la base était mal réglée (la balayer l'a portée de 0,57 à 0,88 jeton/s). *Justesse* : plus près des mathématiques exactes du modèle que llama.cpp (KL 0,0031 contre 0,0607 sur le 120b), et, avec un cache KV en f32, égal à elles au format près : tout l'écart venait de l'arrondi f16 du cache.

Un juge qui n'avait écrit ni la spécification ni le code a refait les chiffres : son décodeur, écrit depuis la seule spécification, redonne les tronçons au bit près ; sa propre passe avant en double précision confirme la référence à l'arrondi du format près. Il a trouvé deux défauts graves (une passe qui se fige sans message, un processus enfant orphelin qui bloque son pilote) et cinq moyens, dont une corruption d'octets bruts invisible jusque dans le texte. La ronde de corrections les a corrigés ou mesurés en une matinée, chacun avec un témoin qui tombe sur l'ancien binaire ; une garde de production rend désormais toute corruption d'un tronçon lu en erreur fatale nommée, pour 1,6 % de vitesse à disque égal. Dix critères tiennent sur douze ; les deux autres attendent une décision de l'arbitre.

Restent la contribution de recherche, la découpe base + complément du jalon J2 ; un disque non chiffré, qui dira ce que BitLocker coûte sur un vrai modèle ; l'étude biomimétique de Stone, le génome des poids ; et la porte qui rendra ce moteur à relais. Le mémoire existe en deux versions complètes, anglaise et française ; chaque figure se régénère depuis les données de mesure commitées.

Contents

Abstract iii

Résumé v

List of Figures xiii

List of Tables xvii

Listings xix

Notation and conventions xxi

1 Introduction: from calculation to program 1
- 1.1 The mission, and what Stone asked of this volume . . . 1
- 1.2 Where we start: Volume I . . . 2
- 1.3 The question of this volume . . . 2
- 1.4 What this volume contributes . . . 2
- 1.5 The method . . . 3
- 1.6 Plan of the volume . . . 3

I The program 5

2 Architecture and structure of the program 7
- 2.1 The requirements, and the decision that follows . . . 7
  - 2.1.1 Why Rust and CUDA, by the measurements . . . 8
  - 2.1.2 The dependencies, and why so few . . . 8
- 2.2 The workspace: fourteen members, an acyclic graph . . . 9
- 2.3 The overall picture . . . 10
- 2.4 The seams: what is frozen between the lots . . . 11
  - 2.4.1 The determinism rule, or why bit equality makes sense . . . 11
- 2.5 The interfaces, as they are . . . 11
- 2.6 The exactness chain . . . 15
- 2.7 The bounded window, a property of the architecture . . . 16
- 2.8 What the independent trial changed in the architecture . . . 16

3 The `.trc` format to the bit 23
- 3.1 The principles . . . 23
- 3.2 The overall layout . . . 24
- 3.3 The fixed header . . . 24
- 3.4 META, RESIDENTS, TABLES, DIRECTORY . . . 26
  - 3.4.1 META: the GGUF pairs, copied . . . 26

3.4.2 RESIDENTS: the always-read tensors . . . 27
3.4.3 TABLES: the rANS frequencies . . . 27
3.4.4 DIRECTORY: the table of contents . . . 27
3.5 A chunk part . . . 28
3.6 The rANS stream . . . 29
3.6.1 Serialisation . . . 29
3.6.2 Normative decoding . . . 29
3.6.3 Why it is exact . . . 30
3.6.4 Thirty-two lanes at once . . . 31
3.7 Contexts: the table chosen by the decoder . . . 32
3.8 The stream kinds . . . 33
3.9 The exact reconstruction of an expert . . . 33
3.9.1 The normative procedure . . . 33
3.9.2 The original expert, and why nothing needs reordering . . . 34
3.9.3 The expert slot on the card . . . 35
3.10 The fingerprints . . . 35
3.11 The bounded-memory coder . . . 36
3.11.1 Why two passes . . . 36
3.11.2 Same bytes, bounded memory . . . 36
3.12 C1 on 768 and 4,608 experts . . . 38
3.12.1 The first path: five checks per expert . . . 38
3.12.2 The second path: no line in common . . . 38
3.13 The trial's independent decoder . . . 40
3.13.1 What writing the decoder taught about the text . . . 42
3.14 Versions, and what is reserved for J2 . . . 42

4 The path of the bytes 45
4.1 What the path must do . . . 45
4.2 `trc-cuda`: the driver API, with no library linked . . . 46
4.3 `trc-es`: reading without buffering, through a completion port . . . 47
4.3.1 The structures, bit for bit . . . 47
4.3.2 Mutual exclusion, proved . . . 48
4.4 Pinning: trial E0 (a) . . . 51
4.5 The link: trial E0 (b) and the single copy engine . . . 52
4.6 Reading the router once per layer: trial E0 (c) . . . 53
4.6.1 The mailbox, bit for bit . . . 53
4.6.2 The threshold, and the first model . . . 54
4.6.3 What the measurement changed in the model . . . 55
4.7 The disk alone: step E1 . . . 56
4.7.1 The threshold, and what a clean pass is . . . 56
4.7.2 One session under load, one session quiet . . . 57
4.7.3 Little's law, a second path for the throughput . . . 57
4.8 End to end: step E2 . . . 58
4.8.1 The threshold, set before the measurement . . . 58
4.8.2 Exact transport, and what each check sees . . . 60
4.8.3 Throughput: the verdict fails, and the witness says why . . . 61
4.9 The discovery: two regimes depending on length alone . . . 62

4.10 The path in the engine . . . 64
4.11 What this chapter struck out . . . 66

5 The kernels on the card **71**
5.1 What the card must do . . . 71
5.2 The per-lane-table decoder (K1) . . . 72
5.2.1 The stream, the tile, the table . . . 72
5.2.2 Two kernels, one slot . . . 73
5.2.3 Exactness, proved . . . 74
5.2.4 The word window . . . 75
5.2.5 The per-lane table, and determinism . . . 76
5.2.6 Witnesses and breaks . . . 77
5.2.7 Why a single expert is slow: the latency model . . . 77
5.2.8 Throughputs, and a verdict that fails by 0.9 and 1.1 % . . . 78
5.3 The MXFP4 matrix–vector product (K2) . . . 80
5.3.1 The algorithm . . . 80
5.3.2 Exact to the bit on integers . . . 81
5.3.3 On real weights: the error bound . . . 82
5.3.4 The roof . . . 82
5.4 The forward pass: twelve kernels (K3) . . . 82
5.4.1 The kernels and their structures . . . 82
5.4.2 The witness protocol . . . 84
5.4.3 A single rounding: the f64 kernels . . . 85
5.4.4 The exact kernels . . . 86
5.4.5 Attention, the smallest margin . . . 87
5.4.6 Selections, and determinism as a whole . . . 88
5.4.7 Results, kernel by kernel . . . 90
5.4.8 The end-to-end chain . . . 90
5.5 The round: the KV cache in f32 . . . 93
5.6 What this chapter struck out . . . 94

II The truth and the memory **97**

6 The exact reference and the judge **99**
6.1 Why judge against a mathematical truth . . . 100
6.2 The forward pass in double precision . . . 100
6.2.1 Exact weights . . . 100
6.2.2 Two paths, one function . . . 101
6.2.3 Tiling the product, and why it changes no bit . . . 101
6.2.4 The YaRN ramp: OpenAI's definition and llama.cpp's form . . . 102
6.3 The Kullback-Leibler divergence, and what it measures . . . 105
6.3.1 Definition and first properties . . . 105
6.3.2 The KL of a small perturbation of the logits . . . 106
6.3.3 Top-1 agreement, and its intervals . . . 106
6.3.4 Judging two engines on the same positions . . . 106
6.4 The instrument and its witnesses . . . 107
6.4.1 The logits file and its floor . . . 107

6.4.2 The defect the identity witness found: a coded judged file is not renormalised 107
6.4.3 The witnesses: what the instrument sees, and what it does not 108
6.5 The discovery: a mixture of experts amplifies the smallest errors 110
6.5.1 The numbers 110
6.5.2 The mechanism: the routing switch 111
6.5.3 The imposed-routing experiment 113
6.5.4 The switches, counted 113
6.5.5 What this does to the criteria 114
6.6 The f16 KV cache carries the switches, f32 reaches the truth 115
6.6.1 The first token of C12 115
6.6.2 The share of switches carried by the cache rounding 115
6.6.3 Why f32 must reach the truth 115
6.6.4 Measured 116
6.7 What remains 118

7 The cache, the scheduler and the engine 121
7.1 The expert-cache problem 122
7.2 The offline optimum: Belady 122
7.3 LRU: stack, competitive bound, worst case 123
7.4 troncons' cache, and why it equals its simulators 124
7.4.1 The interface 124
7.4.2 Two eviction rules, one result 125
7.4.3 The inclusive RAM tier 125
7.4.4 Witness O1 126
7.5 The cliff of the shared LRU 127
7.6 The locality of gpt-oss 128
7.6.1 Reuse: popularity and recency 128
7.6.2 The policies 129
7.7 The on-demand scheduler 131
7.7.1 The path of a token 131
7.7.2 The order is safe 131
7.7.3 The engine computes what the kernels validated, to the bit 132
7.7.4 The serial floor, and what the engine does with it 134
7.8 Streamed prefill 134
7.9 Prefetching, tried and abandoned 136
7.10 The integrity guard, and what it costs 136
7.11 The engine against the baseline: criterion C6 137
7.12 What remains 138

III The judgement 141

8 Results of milestone J1 143
8.1 What is measured, and how 143
8.2 The criteria at a glance 144
8.3 Exactness: coded equals raw, to the bit, over 5 120 steps 146
8.4 Memory: 384 MB for 63 GB 147

8.5 Speed . . . 149
8.5.1 The serial floor (C5) . . . 149
8.5.2 The baseline, and the history of its numbers . . . 150
8.5.3 C6 redone: guard included . . . 151
8.5.4 Stream prefill . . . 152
8.5.5 The cost of the integrity guard . . . 152
8.5.6 What J1+ kept, killed, left open . . . 153
8.6 Fidelity: how far from the exact mathematics of the model . . . 153
8.6.1 The instrument . . . 153
8.6.2 C4 on both models . . . 154
8.6.3 The f16 KV cache carried the whole gap . . . 154
8.7 What the results say together . . . 156

9 The independent trial and the round of corrections 159
9.1 The protocol . . . 159
9.2 Step 1: is the specification true? . . . 160
9.3 Do the controls fail? . . . 161
9.4 A third double-precision forward pass . . . 162
9.5 The speed and memory session . . . 163
9.6 Is the baseline honest? . . . 164
9.7 Is the text right? . . . 165
9.8 The findings . . . 166
9.9 The round of corrections . . . 166
9.9.1 G1: the hung pass . . . 166
9.9.2 G2: the orphan that holds the pipes . . . 169
9.9.3 M1: the production guard . . . 169
9.9.4 M2 to M5, and the minor findings . . . 171
9.9.5 What remains for the arbiter . . . 171
9.10 What the trial could not check . . . 172

10 Discussion, limits and future work 173
10.1 What Volume II establishes . . . 173
10.2 Volume I's predictions, against the measurements . . . 173
10.3 The struck-out numbers . . . 174
10.4 The ideas killed by numbers . . . 174
10.5 The limits . . . 174
10.6 Future work . . . 175
10.6.1 First of all: a single series, and the arbiter's decisions . . . 175
10.6.2 The D: disk without BitLocker . . . 176
10.6.3 J2: the base + complement split . . . 177
10.6.4 The compatible gate for relais . . . 177
10.6.5 The weight genome: Stone's biomimetic study . . . 178

Conclusion 181

A Measurement protocols of the engine 183
A.1 What is inherited from Volume I . . . 183
A.2 The end criteria of each step . . . 183

A.3 A threshold precedes its pass: the time of the commit is proof . . . 185
A.4 ABBA series and palindromes . . . 185
A.5 The lock of the card and the shared machine . . . 186
A.6 The witness of the witnesses . . . 187
A.7 The independent trial . . . 187
A.8 The round of corrections, block by block . . . 187
A.9 Reproducing the builds . . . 188

B Tools of the engine **191**
B.1 The chain of Volume II . . . 191
B.2 The `troncons` program . . . 192
B.3 The committed tools . . . 193
B.4 The figures and their data . . . 198
B.5 The logs . . . 200

Bibliography **203**

List of Figures

2.1 The kernels' build chain . . . 9
2.2 The graph of internal dependencies . . . 19
2.3 Lines of code, tests and tools per member . . . 19
2.4 The overall picture . . . 20
2.5 The five lots and their seams . . . 20
2.6 The exactness chain . . . 21

3.1 The .trc file, its sections . . . 24
3.2 The fixed header, byte by byte . . . 25
3.3 The META section . . . 26
3.4 An entry of the residents table . . . 27
3.5 A record of the TABLES section . . . 28
3.6 A directory entry . . . 28
3.7 A part header and a stream descriptor . . . 28
3.8 A mean chunk and a CODES_CTX stream . . . 29
3.9 The header of a rANS stream . . . 29
3.10 The 32-bit LUT entry . . . 30
3.11 The rANS tile and the per-lane table . . . 32
3.12 The two-pass coder and its window . . . 36
3.13 Peak memory of the coder and the check . . . 37
3.14 Bits per weight per tensor and layer . . . 40
3.15 Chunk size per layer . . . 41
3.16 The two paths of C1 . . . 42

4.1 The stack of the byte path . . . 46
4.2 The automaton of a `trc-es` request . . . 49
4.3 E0 (b): DMA from pinned memory . . . 52
4.4 The single copy engine . . . 53
4.5 The mailbox slot of a layer . . . 54
4.6 Model D2, to scale . . . 56
4.7 E0 (c): cost of a synchronization, empty and loaded . . . 57
4.8 E1: the reader on the disk alone, two sessions . . . 59
4.9 Little's law against the wall clock . . . 60
4.10 The E2 window and the life of a slot . . . 61
4.11 E2: experts served per pass . . . 63
4.12 E2: the stages . . . 64
4.13 E2: timeline of the first 40 chunks . . . 65
4.14 The discovery: throughput against read length . . . 67
4.15 The engine: two 120b layers, every miss timestamped . . . 68
4.16 The engine: 12 MiB reads (L12) . . . 69

5.1 The rANS tile and the per-lane table . . . 72
5.2 The bits: LUT entry, MXFP4 block, code, E8M0 scale . . . 73
5.3 The SoA slot of a gpt-oss expert, to scale . . . 73
5.4 The three-register word window . . . 75
5.5 K1: breakdown of the time of an expert . . . 79
5.6 K1: decoder throughput per pass . . . 79
5.7 K2: GEMV error on real weights . . . 83
5.8 K2: throughput of the MXFP4 GEMV . . . 83
5.9 The forward pass of a layer, kernel by kernel . . . 84
5.10 The structures passed to the kernels . . . 84
5.11 K3: each kernel's margin to its threshold . . . 91
5.12 K3: KL per position, free against imposed routing . . . 92
5.13 K3: routing decisions and their flips . . . 93
5.14 K3: the "window" break and its step at 128 . . . 94
5.15 The KV cache: f16 against f32, 120b . . . 95

6.1 The YaRN ramp, OpenAI against llama.cpp . . . 104
6.2 One entry of the .kld file, bit for bit . . . 107
6.3 The paths of the truth . . . 110
6.4 The KL per position is a tail . . . 111
6.5 The routing switch . . . 112
6.6 The tree of R1's experiments . . . 113
6.7 KL per position against the router margin . . . 114
6.8 Routing agreement per layer, engine against truth . . . 116
6.9 KL against the truth, f16 against f32 KV cache . . . 117

7.1 The automaton of an expert slot . . . 122
7.2 The two tiers of the cache . . . 126
7.3 The cliff of the shared LRU . . . 128
7.4 The measured cliff . . . 129
7.5 The proposition checked on 17 traces . . . 130
7.6 Hit rate versus budget, five policies . . . 131
7.7 The three streams, the host and the disk . . . 132
7.8 The measured timeline of a layer . . . 135
7.9 troncons against swept llama.cpp . . . 138

8.1 Peak memory against model size . . . 147
8.2 The serial floor, stage by stage . . . 149
8.3 Generation: troncons against llama.cpp . . . 151
8.4 Routing agreement with the truth, per layer . . . 155
8.5 KL per position against the f64 truth . . . 157

9.1 The paths of the truth . . . 162
9.2 The trial's session, pass by pass . . . 164
9.3 The exit on error, before and after the round . . . 168
9.4 The round robin of arrival buffers . . . 168
9.5 The production guard . . . 170

10.1 The milestones and the work to come 175

List of Tables

2.1 CUDA versus Vulkan: what decided . . . 8
2.2 The dependencies (seam D) . . . 9
2.3 The members of the workspace . . . 10
2.4 The seams and their history . . . 12
2.5 What the correction round changed . . . 17

3.1 The sections of both files . . . 26
3.2 The stream kinds . . . 33
3.3 C1: the verdict . . . 39
3.4 The six breaks of the second path . . . 43
3.5 The breaks of the trial's decoder . . . 43
3.6 Precisions of the text awaiting the arbiter . . . 44
3.7 What the v1.1 split costs . . . 44

4.1 A `trc-es` read in memory . . . 48
4.2 E0 (a): pinning 0.25 to 4 GB . . . 51
4.3 E2: the detection table of the break . . . 62
4.4 The hypotheses, killed or kept by the numbers . . . 66

5.1 Kernel resources on Ada . . . 72
5.2 K1: witnesses and breaks . . . 78
5.3 K1 and K2: throughputs against thresholds . . . 80
5.4 The buffers of the forward pass . . . 85
5.5 K3: the twelve kernels against the f64 reference . . . 90
5.6 K3: the end-to-end chain . . . 91
5.7 The KV cache in f16 against f32 . . . 95

6.1 The two blocks the reference reads . . . 101
6.2 R0 and R1 on gpt-oss-20b . . . 109
6.3 gpt-oss amplifies small errors . . . 110
6.4 f16 against f32 KV cache, against the f64 truth . . . 116

7.1 The interface of the cache . . . 125
7.2 Witness O1 . . . 126
7.3 Reuse from one token to the next . . . 129
7.4 The serial floor of the 120b . . . 134

8.1 The measurement sessions of milestone J1 . . . 144
8.2 Milestone J1, criterion by criterion . . . 145
8.3 C3 and C11: coded against raw . . . 146
8.4 The terms of the window that grow with the model . . . 148
8.5 The swept llama.cpp baseline . . . 151

8.6 C4 against the f64 truth . . . 154
8.7 f16 against f32 KV cache . . . 156

9.1 Step 1: a third decoder from § 4 alone . . . 160
9.2 The breaks of the judge's decoder . . . 161
9.3 The breaks replayed by the judge . . . 161
9.4 C8 by two instruments . . . 165
9.5 The first token of C12 . . . 165
9.6 The trial's findings, and what the round made of them . . . 167

10.1 Volume I's predictions and Volume II's measurements . . . 174
10.2 The struck-out numbers of Volume II . . . 175
10.3 The ideas killed by numbers . . . 176
10.4 The mechanisms of the living and their measurable hypotheses . . . 178

A.1 The end criteria of the steps . . . 184
A.2 The threshold files and their date . . . 185
A.4 The tools that break the witnesses . . . 187
A.5 The blocks of the round of corrections . . . 188

B.1 The options of the troncons program . . . 192
B.2 The tools of the engine and of its measurements . . . 193
B.4 The figures and the files they read . . . 198
B.6 The committed logs of the engine . . . 200

Listings

2.1 The interface of `trc-format` . . . 11
2.2 The interface of `trc-codec` . . . 12
2.3 The interfaces of `trc-cuda` and `trc-es` . . . 13
2.4 The interface of `trc-cache` . . . 13
2.5 The engine's configuration . . . 14

3.1 The core of the rANS decoder . . . 30
3.2 Reconstructing a v1.0 matrix . . . 34
3.3 The trial's vectorised rANS . . . 41

4.1 Loading the driver table . . . 47
4.2 Submit, launch, harvest . . . 48
4.3 The simulated router of trial E0 (c) . . . 55
4.4 The raw cumulative disk counter . . . 58

5.1 A warp step . . . 74
5.2 The loop of `corps_codes` . . . 76
5.3 The exact E8M0 scale and the product of a block . . . 81
5.4 The YaRN frequency in f64 . . . 85
5.5 The combiner . . . 86
5.6 Decode attention . . . 87
5.7 The router . . . 88

6.1 The 2 × 2 kernel of the reference . . . 103
6.2 The YaRN ramp in the reference . . . 105
6.3 KL of a decoded entry . . . 108

7.1 The next use in Belady . . . 124
7.2 The decision of a layer . . . 133

9.1 The choice of an arrival buffer in stream prefill, after the round (`moteur/crates/trc-ordo/src/lib.rs`, excerpt; the code's comments are in French). . . . 168
9.2 The guard turns a wrong chunk into a named fatal error (`moteur/crates/trc-ordo/src/garde.rs`, excerpt; the message is in French). . . . 170

Notation and conventions

The conventions are Volume I's. Units are decimal: 1 GB = 10^9 bytes, 1 GB/s = 10^9 bytes per second; binary multiples are written KiB, MiB, GiB (a chunk read of 12 MiB is 12×2^{20} bytes). Every number carries its provenance: [meas.] measured here on Stone's machine with a committed tool and log, [calc.] computed from named numbers, [lit.] literature (publisher's number, not reproduced). A sentence that a later measurement contradicted is struck out where it stood (~~like this~~), with what replaced it. A file of the repository cited as the source of a number is set as `mesures/moteur/O/csv/`; its path is relative to the root of the `troncons` repository.

Names of the work

Name	Meaning
troncons	the program (a Rust workspace with CUDA kernels, under `moteur/`), and the project
chunk	the bytes of one expert (its three matrices and their biases) in a `.trc` file, read in one request
slot	the place of one decoded expert in video memory (13.25 MB); the cache decides which expert a slot holds
lots E, F, K, O, R	the five work packages of the team, each sole owner of its files: E the byte path (`trc-cuda`, `trc-es`), F the format and coder, K the kernels, O the cache, scheduler and engine, R the reference and the judge
seams F, C, N, A, J, D	the interfaces frozen between lots before construction (Table 2.4)
E0, E1, E2, F1–F3, K1–K3, R0–R2, O1–O4	the steps of each lot, each with its witness, its break and its declared threshold (`decisions/ARCHITECTURE.md` § 6.5; Chapter A)
J1, J1+, J2	the milestones: J1 exact service of gpt-oss-120b in a bounded window; J1+ speed against the baseline; J2 the base + complement split in the engine (future)
C1–C12	the twelve criteria of J1 and J1+, declared on 15 October 2026 before the first line of code (Chapter 8)
G1, G2; M1–M5; m1–m6	the findings of the independent trial: serious, medium, minor (Chapter 9)
mode A0	no expert cache in RAM: every miss of the video-memory cache is read from disk
coded / raw file	the same `.trc` with its streams coded by rANS, or stored as they are (flag bit 2); the raw file is the reference path of C3

Name	Meaning
the reference	the f64 forward pass of gpt-oss on the processor (`trc-reference`), the "truth" of criterion C4
the baseline	llama.cpp b11515 (CUDA 12.4), best measured configuration `fit -t 16` on the 120b

Symbols

Each chapter defines its symbols where it uses them. Those below are shared by several chapters; a letter with two meanings in the volume is listed with the scope of each.

Symbol	Meaning
The model (gpt-oss)	
L	number of layers (36 for the 120b, 24 for the 20b), except in the rANS sections of Chapters 3 to 5 and 9, where $L = 2^{16}$ is the lower bound of the state (below)
E	routed experts per layer (128 and 32)
k	experts chosen per token and layer (4); in Chapter 3, k also counts the sub-steps of a rANS macro-step
ℓ, t	index of a layer, index of a token (a step)
d	layers of look-ahead of the arrival buffers ($d = 3$, Proposition 2.3); elsewhere a dimension, defined in place
z, p, q	logits; the next-token distribution of the truth (p) and of an engine (q)
The format and rANS	
M	precision of the rANS probabilities, $M = 2^{12} = 4\,096$
x, x_l	32-bit rANS state, of lane $l \in \{0, \dots, 31\}$
L (rANS)	lower bound of the normalised state, 2^{16}; every lane ends a tile on L
f_s, c_s	quantised frequency and cumulative of symbol s ($\sum_s f_s = M$)
P, N, T, W	in a stream header: bytes produced, macro-steps per tile (256), tiles, 16-bit words; N has another meaning in Chapter 7 (below)
e	E8M0 scale of an MXFP4 block: value 2^{e-127} in the OCP specification [1]; the code and Chapters 5 and 6 write a weight $K[c]\,2^{e-128}$, where $K = \{0, \pm1, \pm2, \pm3, \pm4, \pm6, \pm8, \pm12\}$ holds the E2M1 magnitudes doubled (the same value); median of its row and offset as in Volume I
Arithmetic and the judge	
u, u_{16}, u_{32}	unit roundoff; of f16 (2^{-11}) and of f32 (2^{-24})
$\mathrm{KL}(p \,\|\, q)$	Kullback–Leibler divergence $\sum_i p_i \ln(p_i/q_i)$, in nats, of an engine q from the truth p, per position [2]
G_{moteurs}, G_{cfg}	measured KL between two correct engines, between two configurations of one engine (step R0)
Δ, δ	a gap (of logits, of margins); defined in place
Cache, scheduler, disk	
N (cache)	capacity of a cache, in slots (326 in J1 on the 120b)

Symbol	Meaning
h, h_v	hit rate of the expert cache; the one predicted by Volume I for the video-memory cache
X, N, R	in Little's law $X = N/R$ (Chapters 4 and 8): throughput, mean requests in flight, residence time [3]
W_i, O_i	throughput of stage i, bytes a token makes it carry (Volume I); the roof $t \geq \max_i O_i/W_i$
B	bytes of chunks read per step ($\approx$ 0.71 GB on the 120b, Chapter 2); in Chapter 3, blocks per row of a matrix

References to Volume I

This volume cites Volume I in two ways. "Volume I, chapter n" is a chapter of the first thesis. "Study 0n" (or "0n § m") is one of the seven committed studies from which Volume I was written, under `fondations/`; its sections are those of the Markdown file. The correspondence:

Study	File	Volume I, chapter
01	`fondations/01-machine.md`	3 The machine and its physics
02	`fondations/02-quantification.md`	5 Extreme quantization
03	`fondations/03-entropie.md`	6 Lossless compression
04	`fondations/04-chargement.md`	7 On-demand loading
05	`fondations/05-sparsite.md`	8 Sparsity and speculative decoding
06	`fondations/06-modeles.md`	9 The target models
07	`fondations/07-ce-qui-manque.md`	4 Formulation, 10 Architecture
decision	`decisions/ARCHITECTURE.md`	10 ; in this volume, Chapters 2 and 3

The decision rule used throughout, "06 § 7.5", is study 06's: a difference is real only if it exceeds twice the larger noise of the two programs and 3 % (Chapter A).

Chapter 1

Introduction: from calculation to program

Where we start, what this chapter establishes, what remains

Where we start: Volume I, a machine measured stage by stage, a proved roof, four measured levers, a token equation, three computed architectures and a decision: build the exact architecture first, in a format that prepares the contribution. No program was running. **This volume establishes** the program, troncons, built by a team of five lots; its format, its byte path, its kernels, its reference, its cache and its engine, each with its proofs and benches; the results of its first milestone on gpt-oss-120b; the trial an independent judge put it through, and the round of corrections that followed. **What remains**: the research contribution (the base + complement split, milestone J2), the unencrypted disk, the study of the weight genome and the gate for relais (Chapter 10).

1.1 The mission, and what Stone asked of this volume

The mission was written by Stone on 15 October 2026, and Volume I quotes it in full. It asks for "une vrai solution pour prendre un modele ultra lourd quelcoque d'une ai le faire tourner proprement sans ram ni rien" (*a real solution to take any ultra-heavy AI model and run it cleanly without RAM or anything*), then: "je ne veux pas que des recherches déja existante je veux que tu fasse une vrai recherche de spécialiste puis tu implémente la solution fonctionnelle avec une team d'ingénieur" (*I do not want only research that already exists, I want you to do real specialist research and then implement the working solution with a team of engineers*). Volume I was the research; this volume is the implementation. Stone asked that it be written with the same demands (in French, typing errors included):

> *pareil tout ce que l'équipe fera calcul et tout ça on fera un mémoire comme l'autre architectuew areucture genre vraiment comme un projet de recherche c'est dailleur le cas mais vraiment calcul theorie et tout*

(*the same: everything the team does, calculations and all, we will make a thesis like the other one, architecture, structure, really like a research project, which it is, but really calculation, theory and all*), and he had set the bar the same day: "des shemas des explications des commentaires he veux que ce soit légendaire ce projet" (*diagrams, explanations, comments, I want this project to be legendary*). This volume therefore follows the form of the first: every chapter says where it starts, what it establishes and what remains; every structure is given to the bit, every calculation posed step by step, every measurement drawn from its committed data; every chapter closes on what could be wrong.

1.2 Where we start: Volume I

The machine. An ordinary laptop: i7-13620H processor, 16 GB of RAM on a single module, an 8 GB RTX 4060 Laptop card, a 1 TB NVMe disk encrypted by BitLocker. Volume I measured each of its stages with tools written for the purpose, each with a control forced to fail: the disk delivers 2.82 GB/s, 47 % of its rating, because the volume is encrypted and goes through Intel RST VMD; the host-to-card link 12.2 GB/s; the card's memory 251.3 GB/s.

The roof. From these throughputs, Volume I proved the bound of a chain of stages: the time of a token is at least $\max_i O_i/W_i$, where W_i is the throughput of stage i and O_i the bytes a token must push through it. If the card decompresses, compression never becomes useless; if the processor decompresses, it does beyond a factor $r^\star = 4.33$.

The levers. Extreme quantization (a 2-bit trellis closing 90 % of the gap between the optimal scalar quantizer and the Shannon bound); lossless coding (the MXFP4 scales coded by the median of their row, a rate of 1.102 6 on the experts of gpt-oss-120b, an rANS decoder proved exact at 82 GB/s under CUDA); on-demand loading, where the cache hit rate decides everything; speculation, which pays only under an expert reuse that independent routing does not give.

The equation, and the decision. A single equation, the time of a token of a mixture of experts served by the card, contains the four levers; its proved elasticities say that on this machine the disk holds the maximum, and that faster kernels would return less than 2 %. Three architectures were computed on it: A, exact; B, which serves a base without its complement where a byte is expensive, a new mechanism; C, which reads less often. After the debate of two advocates, the arbiter decided to build A first, in Rust with CUDA kernels, in a format that prepares B (`decisions/ARCHITECTURE.md`). This is where this volume begins.

1.3 The question of this volume

> *Does the program do what the calculation promised: serve a model several times larger than all the memory of the machine, exactly, within a memory window that does not depend on its size, faster than the best baseline measured; and does a judge who did not write it confirm it?*

The question is judged on twelve criteria, declared on 15 October 2026 before the first line of code, each with a threshold derived from a named quantity (`decisions/ARCHITECTURE.md`, § 7). The answer is in Chapter 8; the trial, in Chapter 9.

1.4 What this volume contributes

1. **A program that works**: troncons runs gpt-oss-120b (63.4 GB, four times the RAM and eight times the card's memory) on Stone's machine, with 384 MB of peak memory, the same to within 3.7 MB as for a model five times smaller.
2. **A self-contained format, to the bit**, `.trc`, one chunk per expert, streams stored by refinement level to welcome B without costing A anything, and a third decoder, written by the judge from the specification alone, that reproduces it to the bit.

3. **Exactness, proved and measured**: logits identical to the bit between the coded and the raw file over 5 120 steps, from one build and one order of computation to another.
4. **Speed**: 1.965 tokens per second guard included, 2.22 times the best llama.cpp baseline measured, and a stream prefill seven times faster; two earlier ratios, ×3.85 then ×2.79, struck out where they stood because the baseline was badly tuned.
5. **Correctness, against the exact mathematics of the model**, through a double-precision reference confirmed by a third independent computation up to rounding; and the measurement that the f16 rounding of the KV cache carried the engine's whole gap to the truth.
6. **A production integrity guard** that turns any corruption of a chunk read into a named fatal error, proved, for 1.6 % of speed at equal disk.
7. **An independent trial and a round of corrections**: two serious and five medium defects found by a judge, corrected or measured in one morning, each with a control that fails on the old binary.

1.5 The method

The rules are Volume I's: every measurement has its control, and every new control must be able to fail; a threshold derives from a named quantity and is declared before the measurement, by a dated commit; a cost is judged in ABBA series, with a program's distance to itself as the noise; every number we depend on is redone by a second path; a sentence contradicted by a measurement is struck out, and what changed is said. This volume adds three. *Frozen seams*: the interfaces between lots are fixed in writing before construction, and change only by a decision of the arbiter. *The independent trial*: an agent who wrote neither the specification nor the code redoes the numbers with his own tools. *The second path for the thesis's numbers*: the numbers of the results chapters are recomputed from the passes themselves by a committed tool (`memoire/outils/tome2_resultats.py`), which stops with an error at the first discrepancy with a published number.

Every number carries its provenance: [meas.] measured here, by a committed tool and log; [calc.] computed from named numbers; [lit.] literature, not reproduced.

1.6 Plan of the volume

The volume follows the path of a byte, from the program to the result. Chapter 2 gives the architecture and structure of the program: the requirements, the language and the kernels, the workspace, the lots and their seams. Chapter 3 gives the `.trc` format to the bit, section by section, and the exact reconstruction of an expert. Chapter 4 follows the byte path, from the disk to the card's memory, through unbuffered reads and pinned memory. Chapter 5 describes the kernels on the card, the rANS decoder with per-lane tables and the forward pass of gpt-oss, with the proof of their exactness and determinism. Chapter 6 sets out the exact double-precision reference and the theory of the judge. Chapter 7 describes the expert cache, the scheduler and the engine, with the theory of the cache. The last three chapters judge: Chapter 8 gives the results of milestone J1, Chapter 9 the independent trial and the round of corrections, Chapter 10 the discussion, the limits and the future work, before the conclusion.

Part I

The program

Chapter 2

Architecture and structure of the program

Where we start, what this chapter establishes, what remains

At the start, Volume I ended on a decision and a specification: build **troncons**, an engine of our own, in Rust with CUDA kernels, whose first milestone (J1) serves gpt-oss-120b (63.4 GB) *exactly*, experts losslessly coded, read from disk at the size of one expert, inside a memory window that does not depend on the size of the model. The specification (`decisions/ARCHITECTURE.md`, commit `f0b0fea`) fixed the modules, their interfaces and five lots of engineers with disjoint files; not one line of the program existed. **This chapter establishes** what was built: fourteen members of a Rust workspace (eleven libraries, three executables, 15,655 lines of code, 2,194 of tests, 2,151 of tools), twenty CUDA kernels loaded by the driver alone, six seams frozen between the lots and what became of them; it proves that the bit-for-bit equality of the logits between the coded and the raw file (criterion C3) is not luck but the consequence of four seam contracts (theorem 2.2), and it says what the independent trial changed in the architecture (an integrity guard, an exit on error, an optional f32 KV cache). **What remains**: the arbiter's decisions that the correction round brought to him (seam N for the f32 KV cache; precisions of the format; criteria C7 and C8), the term of the window that grows with the model (0.9 MB unexplained), and the root cause of the hang G1. Sources: `decisions/ARCHITECTURE.md` § 0 to 6; `moteur/Cargo.toml` and the members' `Cargo.toml`; `moteur/O-pour-le-memoire.md` (parts II and III); `moteur/E-pour-le-memoire.md`; `moteur/K-pour-le-memoire.md`; `mesures/epreuve-J1.md`.

In brief

A program that runs a model five times larger than the machine's memory is cut along *who knows what*. The format knows where every byte is; the byte path knows how to carry it from disk to card without damage; the kernels know how to compute with it, always in the same order; the cache knows *where* an expert is, never *what it contains*; the judge knows how to measure without believing anyone. Each piece of knowledge is a lot, each boundary a seam written in advance. This cut is what makes exactness provable: one does not prove a program of twenty thousand lines, one proves that its boundaries carry equality across.

2.1 The requirements, and the decision that follows

Stone's words (SOCLE, 2026-10-15) ask for "a real solution to take any ultra-heavy AI model and run it cleanly" and "not only research that already exists" (our translation of his French). The arbiter drew three requirements from them that must hold together (ARCHITECTURE.md § 1.1):

E1, a solution that works: a program that serves a model much larger than memory, cleanly: correct output, bounded memory, speed above the known baseline;
E2, a contribution: a mechanism or an assembly that does not yet exist, proven by measurement;
E3, a first working program soon, without closing the door on E2.

The order chosen (delegated answer of 2026-10-15, reversible) is **exact A first** (J1), the quality of contribution B measured in parallel without an engine, then B inside the engine at milestone J2. Volume I gives the figures; this chapter describes what A became.

2.1.1 Why Rust and CUDA, by the measurements

The choice of language is not a taste. The project's already measured building blocks (rANS coder, IOCP disk bench, GGUF reader) were in Rust; a program juggling aligned buffers, asynchronous I/O queues and device pointers gains from a compiler that refuses wrong lifetimes. The choice of CUDA over Vulkan is settled by measurements (Table 2.1): the overlap of disk, link and compute that the maximum of the token equation assumes had only been measured on this machine with CUDA streams.

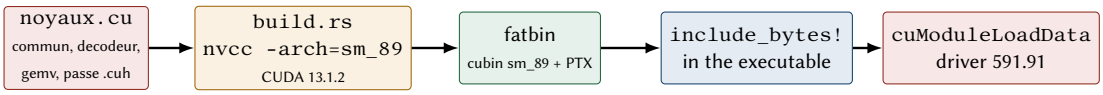
Question	Vulkan (wgpu 30)	CUDA 13.1
rANS v4 decoder, same stream, ABBA	70.2 and 71.6 GB/s	**80.8 and 82.1 GB/s** (+14.9 %) [meas.]
4-bit GEMV in VRAM	249 GB/s	231 to 237 GB/s (shared card): a tie [meas.]
Copy–compute overlap	not measured	hidden share 0.98 to 1.01 [meas.]
Pinned memory	driver buffer, 12.2 GB/s	`cuMemHostAlloc`, +21 % over pageable [meas.]
Reading the disk straight into the memory the DMA reads	impossible without host-memory import (absent from wgpu)	aligned pinned memory, handed as is to `ReadFile`
Copy and compute on two queues	a single queue per device	streams and copy engine

Table 2.1: CUDA versus Vulkan, the measurements that decided the kernels' language (`ARCHITECTURE.md` § 2.3, after studies 01 to 04 of Volume I). The price, stated with the decision: the program requires an NVIDIA card; the measured WGSL kernels remain the second path, and a Vulkan port opens after J2.

The kernels are compiled *at build time* by `nvcc` and embedded in the executable; at run time the driver alone loads them (Figure 2.1). The driver API manual page (v13.4, updated 2026-10-12, consulted 2026-10-16) states that `cuModuleLoadData` accepts a cubin or fatbin image output by `nvcc`, or NULL-terminated PTX [4]: this is what allows shipping *no* CUDA runtime DLL. The image holds the `sm_89` cubin (Ada, the machine's RTX 4060) and the `compute_89` PTX, which the driver can recompile for a newer card. No cuBLAS (no dense matrix multiplication in J1), no NVRTC.

2.1.2 The dependencies, and why so few

Seam D (Section 2.4) freezes the dependencies and their exact versions. Table 2.2 lists them with their licence; all are in the machine's cargo cache, and the build is offline (`--offline`). A Cargo workspace shares one `Cargo.lock` and one output directory, and its members inherit versions from



At build time only: `nvcc` is called directly when `cl.exe` is on the PATH (the former `cmd /C` line broke the recipe, minor finding m2 of the trial). At run time: no CUDA runtime DLL, the driver alone loads the image. Left of the executable box: the build machine; right: any machine with an NVIDIA driver supporting CUDA 13.1.

Figure 2.1: The kernels' build chain. The `build.rs` of `trc-noyaux` (options `-O3 -std=c++17`, *without* `--use_fast_math`, so that subnormal E8M0 scales stay exact and the binary computes what the bench measured) produces a fatbin; `include_bytes!` puts it into the executable; `trc_cuda::Module::charger` hands it to the driver. Source: `moteur/crates/trc-noyaux/build.rs`.

the `[workspace.dependencies]` table through `workspace = true` [5]: a version is written in one place only.

Crate	Version	Licence	Use
`rayon`	1.12.0	MIT or Apache-2.0	parallelism of the coder, the check, the reference
`sha2`	0.10.9	MIT or Apache-2.0	SHA-256 and fingerprint64
`windows-sys`	0.61.2	MIT or Apache-2.0	IOCP, `VirtualAlloc`, PDH, process memory
`half`	2.7.1	MIT or Apache-2.0	f16 in the reference
`libloading`	0.8.9	ISC	loading `llama.dll` (tokenisation)
`serde`, `serde_json`	1.0.229, 1.0.151	MIT or Apache-2.0	logs
`anyhow`, `thiserror`	1.0.104, 2.0.21	MIT or Apache-2.0	errors

Table 2.2: The dependencies (seam D), versions read in `moteur/Cargo.toml`. No other without a written decision; no xxHash (absent from the cache): the SHA-256 prefix, accelerated by the processor's SHA instructions, is enough for a witness. `llama.dll` b11515 (MIT) serves only for tokenisation, through its public API, and as the comparison baseline.

2.2 The workspace: fourteen members, an acyclic graph

The engine lives under `moteur/`, a Cargo workspace whose members are found by two patterns, `crates/*` and `bins/*`. A build rule came out of it, written by lot O at the head of `moteur/Cargo.toml` after measuring it with cargo 1.98.1: Cargo refuses the *whole* workspace as long as one folder matching a pattern lacks a `Cargo.toml`; each lot therefore writes its crate's `Cargo.toml` before any other file. Lot F learnt it the hard way: as long as `crates/trc-noyaux/` had no manifest, it had to build through a scratch workspace outside the repository (`moteur/REPRISE-F.md`).

Table 2.3 says what each member does and who owns it; Figure 2.3 gives its size. The count comes from a committed tool (`memoire/outils/tome2_format.py`) that counts the lines of the `.rs`, `.cu` and `.cuh` files and sorts them into code, tests (`tests/` folder, `tests.rs` file) and tools (`banc/`, `examples/`, `src/bin/`): 15,655 lines of code, 2,194 of tests, 2,151 of tools, 20,000 in all [calc.].

Member	Lot	Role	Code
`trc-format`	F	structures of the `.trc` file, checked reading, writing, fingerprints	1,303
`trc-codec`	F	processor rANS (coder, context decoder), reconstruction of an expert: the oracle	907
`trc-codeur`	F	executable: GGUF → `.trc`, `.vocab.gguf` companion, check C1	1,141
`trc-cuda`	E	the CUDA driver API, wrapped; the only crate that calls `nvcuda.dll`	897
`trc-es`	E	unbuffered reads through a completion port (IOCP)	314
`banc-chemin`	E	executable: trials E0, end-to-end bench E2 (disk → decoded VRAM)	2,341
`trc-noyaux`	K	twenty CUDA kernels, their wrappers, their bench	2,219
`trc-reference`	R	gpt-oss forward pass in f64 on the processor, read from the GGUF	1,592
`trc-jetons`	R	tokenisation through `llama.dll`	145
`trc-juge`	R	logits, KL, system counters, ABBA series	874
`trc-cache`	O	expert cache, policies, `rejouer` simulator (same code)	752
`trc-modele`	O	gpt-oss: hyperparameters, device buffers, layers	737
`trc-ordo`	O	the engine: layer scheduler, prefill, guard	1,750
`troncons`	O	executable: `generer`, `evaluer`, `mesurer`, `disque`	683

Table 2.3: The fourteen members, their lot (ARCHITECTURE.md § 6.1: each lot is the sole owner of its files) and their lines of code [calc.]. Tests and tools are in Figure 2.3.

2.3 The overall picture

Figure 2.4 puts the members in their place in time and on the machine. **Offline**, once per model, `trc-codeur` rewrites the original GGUF into a self-contained `.trc` file (the next chapter describes it to the bit) and checks, expert by expert, that decoding gives back the original (C1). **Online**, at every token, `troncons` calls the engine of `trc-ordo`, which asks the cache where the four experts chosen by each layer's router are, reads the missing ones through `trc-es`, has them copied and decoded by `trc-cuda` and `trc-noyaux`, then launches the layer through `trc-modele`.

Three device streams and one host thread. The engine orders the card through three CUDA streams: *compute* (high priority), *copy*, *decode*. They meet only through events. The host waits once per layer for the "router ℓ" event recorded right after the `routeur_topk` kernel, reads the four indices from a mailbox in mapped memory (protocol of lot E's trial E0 c: sequence number written last, after a system fence), queries the cache, submits the misses' reads and enqueues copies and decodes. The cost of this synchronisation is measured: 18.5 µs per layer under load, 0.67 ms per token of the 120b, 23 times below the declared threshold of 15.7 ms [meas.] (`moteur/E-pour-le-memoire.md`, D2 bis).

Lemma 2.1 (The order of the three streams is safe)**.** *Under the assumptions (H1) a stream executes its operations in order and without overlap; (H2) an event wait on a stream holds back everything enqueued after it until the end of what the event captured; (H3) the host decides layer ℓ only after seeing the "router ℓ" event finished; (H4) an expert requested by layer ℓ stays pinned until* `liberer`(ℓ)*, called after the end of the layer is enqueued, and an eviction victim is never pinned nor in flight; no expert slot is read by a GEMV while a decode writes it, and no arrival buffer is rewritten before the copy or decode reading it has finished.*

Proof. Let s be a slot decoded for layer ℓ' and evicted by a request of layer ℓ, later in the layer stream. By (H4), the eviction follows `liberer`(ℓ'), hence the enqueuing of the end of layer ℓ' on the compute

stream; the new decode into s is enqueued after the decision of layer ℓ, hence, by (H3), after the host has seen "router ℓ" finished, itself enqueued after the end of ℓ' on the same stream: by (H1), all GEMVs that read s are finished when the decode starts. Conversely, the GEMVs of layer ℓ that read s are enqueued after waiting for the "decoded" event of s (H2). For arrival buffers, each reuse follows a layer's decision, hence the end of the previous layers, which waited for their decodes, which waited for their copies; explicit waits (copy after the previous decode of the same buffer, read after the copy of the same pinned buffer) make it true even without (H3). Lot O's full proof: `moteur/O-pour-le-memoire.md` § 6.1. ■

The correction round found that this argument did not cover the *streamed prefill* (J1+), where several reads of a packet are in flight at once: the buffer of the s-th read was s mod 12, and a slow read could still be writing its buffer when the twelfth one after it took it back. The corrected rule takes the next free buffer; it is proven (lot O's lemma 4 bis), and the case did not occur in the pass replayed under the trial's load (0 conflicts) [meas.]. It is one of the two possible causes of hang G1; the root cause is not established (Section 2.8).

2.4 The seams: what is frozen between the lots

Five lots built troncons in parallel, each the owner of its files (Figure 2.5). What links them is written in advance, in the specification, and changes only by a dated decision of the arbiter: these are the *seams* (Table 2.4). A seam may be *extended* without a decision (a function added, a field added) if nothing existing changes; it never changes in a code commit.

2.4.1 The determinism rule, or why bit equality makes sense

Seam N contains a rule that is not an interface but a discipline: *no kernel uses floating-point atomic addition; every reduction follows a fixed tree that depends only on the sizes.* Floating-point addition is not associative: $(a + b) + c$ and $a + (b + c)$ generally differ by one rounding. A reduction through atomics adds in the order in which threads arrive, which changes from one run to the next; two identical passes would yield different logits, and criterion C3 ("zero different bits") would lose its meaning. With a fixed tree, each kernel is a *function* of the bytes it reads. Lot K tested it down to the detail: the `combiner` kernel uses `__fmul_rn` and `__fadd_rn`, never contracted into an fma; an fma put back in its place breaks equality with the same sum on the host from element 649 onwards (`moteur/K-pour-le-memoire.md`, § 7.3.6) [meas.]. One rounding fewer changes bits.

2.5 The interfaces, as they are

The following listings are excerpts of the committed code, shortened (comments summarised, bodies omitted; comments are kept in French, the language of the code); they show what a lot *promises* to the others.

The format (`trc-format`, lot F). A `Fichier` opens by checking everything that can be checked without reading the chunks: header, section fingerprints, table sums, directory ranks, file size. The rest of the program never reads a byte of the `.trc` without going through its structures.

Listing 2.1: The interface of `trc-format` (excerpt of `moteur/crates/trc-format/src/lib.rs`).

```
pub const MAGIE: [u8; 8] = *b"TRONCONS";
pub const ALIGNEMENT: u64 = 4096;
```

	Between	What is frozen	What it became (as of 2026-10-16)
F	F → E, K, O, R	the `.trc` format to the bit	no byte changed; nine precisions by lot F and three imprecisions found by the trial (m4), all compatible with the written files, awaiting the arbiter (next chapter)
C	E → K, O	the API of `trc-cuda`	extended without changing anything (context with wait mode, attributes, `Flux::fini`, `Evenement::nouveau_bloquant`, `MemoireCarte::remplir`, `HoteEpingle::lire_volatil_u32`, `copier_carte_carte`…)
N	K → O	kernels' binary interface, buffer layout, determinism rule	held; the f32 KV cache (finding M5) is added *outside the seam*, as an option (`--kv-f32`, two new kernels), without touching the f16 kernels; the decision is the arbiter's
A	O ↔ R	API of `trc-cache`, trace format	unchanged; the simulator is the cache's own code
J	O, E → R	format of measurement logs	extended (process I/O counters, error log `<journal>.erreur.json`)
D	all	dependencies and versions	unchanged (Table 2.2)

Table 2.4: The six seams and what became of them. Sources: `ARCHITECTURE.md` § 6.2; `moteur/REPRISE-F.md`; `moteur/E-pour-le-memoire.md` § 2.3; `moteur/O-pour-le-memoire.md` § 14; `mesures/epreuve-J1.md`, finding m4.

```
pub const VERSION_MAJEURE: u16 = 1;   // un lecteur 1.x lit toute 1.y, y <= 1
pub const VERSION_MINEURE: u16 = 1;
impl Fichier {
    pub fn ouvrir(chemin: &Path) -> Result<Fichier, ErreurFormat>; // tout se vérifie
    pub fn entete(&self) -> &Entete;
    pub fn residents(&self) -> &[TenseurResident];
    pub fn table(&self, couche: u16, matrice: u8, sorte: Sorte, contexte: u8) -> Option<&Table>;
    pub fn entree(&self, couche: u16, expert: u16) -> &EntreeRepertoire;
}
pub fn lire_partie(octets: &[u8]) -> Result<Partie<'_>, ErreurFormat>;
pub fn empreinte64(octets: &[u8]) -> u64;   // 8 premiers octets de SHA-256, petit-boutiste
```

The codec (`trc-codec`, lot F). It contains the processor rANS coder and decoder, and above all *the reference reconstruction of an expert*: the oracle that the card's decoder must match byte for byte. A symbol's context is a function the decoder can compute before decoding it (`None`: skipped symbol).

Listing 2.2: The interface of `trc-codec` (excerpt of `moteur/crates/trc-codec/src/lib.rs` and `rans.rs`).

```
pub fn coder(source: &[u8], geometrie: u8, contexte: &dyn Fn(usize) -> Option<u8>,
```

```
            tables: &[TableRans]) -> Vec<u8>;            // flux serialise
pub fn decoder(flux: &[u8], contexte: &mut dyn FnMut(usize, &[u8]) -> Option<u8>,
               tables: &[TableRans]) -> Result<Vec<u8>, ErreurCodec>; // voies hors de L : erreur
pub fn reconstruire(fichier: &Fichier, couche: u16, expert: u16, parties: &[&[u8]],
                    exact: bool) -> Result<ExpertDecode, ErreurCodec>;
impl ExpertDecode { pub fn octets_origine(&self) -> Vec<u8>; pub fn empreinte(&self) -> u64; }
```

The byte path (`trc-cuda`, `trc-es`, lot E). `trc-cuda` loads `nvcuda.dll` from System32 and wraps each entry point by its exact exported name (a `table!` macro names at load time any missing symbol); no other crate calls the driver. `trc-es` reads unbuffered (`FILE_FLAG_NO_BUFFERING | FILE_FLAG_OVERLAPPED`): offset, length and destination address must be multiples of 4,096, which fixes the format's alignment [6].

Listing 2.3: The interfaces of `trc-cuda` and `trc-es` (seam C; excerpts).

```
impl HoteEpingle { pub fn allouer(c: &Contexte, octets: usize, projetee: bool) -> Result<Self>; }
pub fn copier_vers_carte(dst: VueCarte, src: &HoteEpingle, decalage: usize, octets: usize,
                         f: &Flux) -> Result<()>;
impl Evenement { pub fn enregistrer(&self, f: &Flux) -> Result<()>; pub fn fini(&self) -> Result<bool>; }
impl Flux { pub fn attendre(&self, e: &Evenement) -> Result<()>; }
pub struct Requete { pub jeton: u64, pub decalage: u64, pub longueur: u32, pub destination: *mut u8 }
impl Lecteur {
    pub fn ouvrir(chemin: &Path, profondeur: usize) -> Result<Self>;
    pub fn soumettre(&self, r: Requete) -> Result<()>;   // multiples de 4 096
    pub fn recolter(&self, attente_ms: u32, sortie: &mut Vec<Achevement>) -> Result<()>;
}
```

The cache (`trc-cache`, lot O, seam A). The cache returns a *decision* per requested expert: hit (the slot where it is), miss served by warm RAM, or miss served by disk. An expert requested by the current layer is pinned until `liberer`. The `rejouer` simulator is the same code, called on a trace: the witness "engine count = simulator count, to the unit" then becomes possible, and it held over 5,120 steps of the 20b (454,210 hits, 37,310 misses, 36,948 evictions on both sides; break: 427,984 / 63,536 / 63,174) [meas.] (`moteur/O-pour-le-memoire.md` § 6.4).

Listing 2.4: The interface of `trc-cache` (seam A, ARCHITECTURE.md § 5.7).

```
pub enum Decision {
    Succes { emplacement: u32 },
    DefautRam { emplacement: u32, tiede: u32 },        // troncon code deja en RAM epinglee
    DefautDisque { emplacement: u32, tiede: Option<u32> } }
impl<P: Politique> Cache<P> {
    pub fn demander(&mut self, couche: u16, experts: &[u16], t: u64) -> Vec<Decision>;
    pub fn pret(&mut self, emplacement: u32);           // decodage termine
    pub fn liberer(&mut self, couche: u16);             // fin des GEMV : desepingle
```

```
}
pub fn rejouer<P: Politique>(trace: &Trace, emplacements: u32, tiedes: u32, vram:
     P, ram: P) -> Compteurs;
```

The engine (`trc-ordo`, lot O). Its configuration shows the architecture better than a long text: every field after `brut` is an extension, added without changing anything, and almost each one bears the name of a measurement or of a finding (Listing 2.5).

Listing 2.5: The engine's configuration (`moteur/crates/trc-ordo/src/lib.rs`, comments summarised).

```
pub struct Config {
    pub fichier: PathBuf,
    pub ram_tiede_go: f64,           // 0 : mode A0, aucun cache en RAM
    pub enchassement_en_ram: bool,   // sinon une ligne de token_embd lue par
    jeton
    pub profondeur_es: usize,        // lectures confiees au systeme a la fois (4)
    pub couches_d_avance: usize,     // d : fenetre de d * k tampons d'arrivee (3)
    pub prechargement: bool,         // J1+ : mesure, tue (-11 %)
    pub contexte: u32,               // 4 096 positions en J1
    pub temoin: bool,                // chaque expert decode redescend et se
    compare
    pub brut: bool,                  // fichier brut exige (drapeau bit 2)
    // ---- extensions
    pub emplacements: Option<u32>,   // sinon : autant que le plafond de VRAM le
    permet
    pub vram_plafond_mio: u64,       // 8 188 - 909 - 100 (critere C10)
    pub longueur_lecture: Option<u32>, // 12 Mio : regime rapide du disque (lot E
    )
    pub chronogramme: bool,
    pub battement: bool,             // constat G1 : dater un blocage
    pub garde: bool,                 // constat M1 : vrai par defaut
    pub casser: Option<(String, u64)>, // temoin de la garde, rien en production
    pub kv_f32: bool,                // constat M5 : hors couture N, faux par
    defaut
}
impl Moteur {
    pub fn nouveau(c: &Config) -> Result<Moteur>;
    pub fn pas(&mut self, jeton: u32, position: u32, logits: Option<&mut [f32]>)
    -> Result<u32>;
    pub fn preremplir(&mut self, jetons: &[u32], logits: Option<&mut dyn FnMut(
    usize, &[f32]) -> Result<()>>) -> Result<u32>;
}
```

The kernels (`trc-noyaux`, lot K, seam N). Twenty `extern "C"` kernels: the two-stage decoder (`decoder_echelles` then `decoder_codes`, and their batched versions), `copier_brut`, three GEMVs (MXFP4, Q8_0, f32, plus a batched MXFP4 GEMV), `rmsnorm`, `rope_yarn`, `ecrire_kv` and `attention` (and their f32 variants from the round), `routeur_topk`, `swiglu_oai`, `combiner`, `ligne_q8_0`, `argmax` [calc.] (read in `moteur/crates/trc-noyaux/noyaux/*.cuh`). The layout they pass to one another, the expert slot, is frozen in the next chapter (Section 3.9.3).

2.6 The exactness chain

Criterion C3 of the milestone required that the logits of the coded file and those of the raw file, in the same engine, differ by *no bit* over 5,120 steps. It is a strange requirement at first sight: do two floating-point computations along different paths (an entropy decoder on one side, a copy on the other) really give the same bits? The answer is yes, and it rests on no luck: it is proven from the seams (Figure 2.6).

Theorem 2.2 (The exactness chain)**.** *Assume:*

(C1) for every expert x, the processor reconstruction of x's chunk equals the original expert of the GGUF, byte for byte;
(K1) the card's decoder writes into the slot exactly the bytes (codes, scales, biases) of the processor reconstruction, and `copier_brut` *writes the raw streams there in the same layout;*
(C2) the bytes of a chunk that arrive in VRAM are those of the file;
(N) each kernel is a deterministic function of the bytes it reads and of its parameters;
(L) no slot is read while it is being written (lemma 2.1);
(A) the cache only chooses where *an expert is, never* which bytes *it holds.*

Then, for any token sequence, the logits returned by the engine on the coded file and on the raw file are equal to the bit, whatever the number of slots, the cache policy and the completion order of the reads.

Proof. The slots. Let x be an expert and $E_c(x)$, $E_b(x)$ the bytes of its slot after decoding (coded pass) and after copying (raw pass). By (C2), the decoder and the copy read the file's bytes. By (K1) and (C1), $E_c(x)$ is the SoA layout of the codes, scales and biases of the original expert; the raw streams of the raw file are, by construction of the coder (`flux_bruts`), the same scales, codes and biases, which `copier_brut` arranges in the same layout: $E_b(x) = E_c(x)$. *The other inputs.* Resident tensors are copied as they are in both files; the KV cache depends only on earlier computations; kernel parameters (positions, sizes) do not depend on the file. *Induction.* Order the computations by (token t, layer ℓ). Assume the states (residual, KV cache) equal to the bit in both passes before (t, ℓ). The attention and router kernels read equal bytes: by (N), they return the same bits, hence the same four indices and the same weights. By (A), the chosen slots may differ from one pass to the other, but each holds $E(x)$ for the requested expert x, and, by (L), these bytes are those of the finished decode at the time of reading. The experts' GEMVs therefore read equal bytes and return the same bits; `combiner` adds them in the router's order $i = 0, 1, 2, 3$, which is the same, not in slot order. The state after (t, ℓ) is equal to the bit. The output (norm, GEMV of the output embedding, argmax) reads only the final residual. ■

The theorem says why C3 is *expected*, not tolerated: a single bit of difference is a defect, to be hunted. It also says what it does not say: it does not judge the correctness of the model (that is criterion C4, against an f64 reference), only that entropy coding changes nothing in the computation.

What was measured. On gpt-oss-20b and gpt-oss-120b, 10 windows of 512 tokens (5,120 steps): 0 different logit fingerprints between coded and raw, `.kld` files equal byte for byte, perplexity 461.978277060145 (20b) and 422.20558522791305 (120b) on both sides [meas.]. The passes did not have the same number of slots (362 against 361 on the 20b, 325 against 326 on the 120b), hence not the same sequence of misses: the "whatever the cache" part is observed, not only deduced (`moteur/O-pour-le-memoire.md` § 6.3). The independent trial redid it from a clean clone, with the streamed prefill instead of the step loop: its `.kld` files equal lot O's byte for byte (SHA-256 `5b212fe9`…) [meas.] (`mesures/epreuve-J1.md` § 0).

2.7 The bounded window, a property of the architecture

"without RAM or anything" ("sans ram ni rien"), Stone asked. The reading adopted by the project (SOCLE): *the memory in use no longer depends on the size of the model*, only on a bounded working window. This property is not a lucky measurement: it can be read in the structure.

Proposition 2.3 (Host memory, term by term)**.** *In mode A0 (no warm cache), the engine's host memory is*

$$M = M_{\text{fixe}} + d \cdot k \cdot \ell_{\text{al}} + M_{\text{rép}},$$

where M_{fixe} does not depend on the model (CUDA context, kernel image, libraries), $d \cdot k$ is the number of pinned arrival buffers ($d = 3$ layers ahead, $k = 4$ active experts) of length ℓ_{al} (the largest chunk rounded to 4,096, or a fixed read length), and $M_{\text{rép}}$ gathers what grows with the number of experts $L \cdot E$ (directory, cache lengths, tables, trace).

Proof. Expert weights stay in host memory only for the duration of their copy to the card, in the $d \cdot k$ arrival buffers (lemma 2.1: a buffer is reused only after the copy that read it); decoded slots are in VRAM. Resident tensors are mounted in VRAM at opening, except the input embedding (`token_embd`, 0.615 GB), one row of which is read from disk per token into an 8 KiB buffer. The open file keeps only its structures in memory: header, tables, directory, residents table. Only the directory and the per-expert objects grow with $L \cdot E$. ■

What was measured, and what the proposition miscounted. Peak working set: **384.2 MB** for gpt-oss-120b (63.4 GB) and 380.5 MB for gpt-oss-20b (12.1 GB), against 10.9 and 9.42 GB for llama.cpp on the same models [meas.] (`moteur/O-pour-le-memoire.md` § 5 and § 6.6); llama.cpp's fastest configuration on the 120b, `fit -t 16`, swept by the round, holds 11.4 (Section 8.4). A model 5.2 times larger costs 1 % more memory. The declared ceiling of C7 (1.9 GB) holds, with a margin of five (386.8 MB at most, guard included). But the 120b − 20b *gap*, declared at 0.5 MB plus twice the noise, is measured at 3.70 MB (lot O) and 2.94 MB (trial): **it fails**. The proposition had forgotten terms: the window itself grows with the largest chunk ($12 \times (12{,}251{,}136 - 12{,}144{,}640) = 1{,}277{,}952$ B), the timestamped events per possible miss (657 to 660 B each), the directory held twice. The sum of the named terms is ≈ 2.05 MB; about 0.9 MB remains that these terms do not explain (granularity of small allocations): **the named-term model is not closed** (`moteur/O-pour-le-memoire.md` § 13.6). The physics remains: memory does not follow the size of the model; what was badly set was the tolerance.

2.8 What the independent trial changed in the architecture

The trial (`mesures/epreuve-J1.md`, 2026-10-16) was held by a judge who had written neither the specification nor a line of the code. It confirmed the specification and the exactness criteria; it also found two serious findings and one medium finding that touched the architecture, not the computation. The correction round dealt with them (Table 2.5).

The guard, a consequence of the format. The guard needed no extra byte: the directory already held the fingerprint64 of the useful bytes of every part (field 32, next chapter). Its cost is set down before it is measured: a step of the 120b reads $B \approx 0.71$ GB of chunks; fingerprinting runs at 1.07 to 1.10 GB/s per thread [meas.] (`examples/debit_empreinte.rs`), i.e. ≈ 0.65 s of core time per step, spread over four threads (≈ 0.17 s of wall time, below the ≈ 0.5 s of a step); only the hashing of the last

	Finding	Correction	Witness and cost
G1	a pass can freeze forever and silently: on an error, destroying the engine spun inside `nvcuda64.dll`	the error is written before any destruction (`<journal>.erreur.json`), then the process ends without destroying the context; a heartbeat per layer of the prefill; arrival buffers of the stream corrected	the old binary waits 90 s on the pipes, the new one returns in 1.5 to 4.9 s; root cause not reproduced
G2	after an error, an orphaned `nvidia-smi` holds the parent's pipes	the child is killed by `Drop` and by a "kill on close" job; null input; non-inheritable standard handles	three cases, 0 orphans (the old one: one per case)
M1	production does not guard the raw streams of a chunk (MEDIANS, BIAS: 0.36 % of the bytes) nor the embedding row: a corruption passes without error	a **guard**: the fingerprint64 of the useful bytes of every chunk read is compared with the directory by four threads, before returning a token; every row read is compared with a table built at opening	named error on the corrupted copy (layer 0, expert 30); −1.6 % tokens/s at equal disk throughput, 5.2 to 5.4 ms of wait per token, +0.95 s at opening, +1.9 MB
M5	the f16 KV cache decides the first token of the C12 text	optional f32 KV cache (`--kv-f32`), outside seam N	troncons equal to the f64 truth up to the format on the 120b (KL at the format's floor, 100 % top-1 agreement); 326 → 314 slots

Table 2.5: The trial's findings that touched the architecture, and their correction (`moteur/O-pour-le-memoire.md` part III; logs under `mesures/moteur/O/corrections/`).

read remains on the critical path, at most 12.25 MB/1.07 GB/s ≈ 11.4 ms [calc.]. Measured: 5.2 to 5.4 ms of wait per token over the round's four guarded passes, and −1.6 % tokens/s at equal disk throughput [meas.] (`mesures/moteur/O/corrections/csv/vitesse-2026-10-16-passes.csv`). Lot O's note wrote ~~5.2 to 5.3 ms~~ and ~~−1.7 %~~: pass 07 (5.42 ms) was not counted there, and −1.7 % came from rounded throughputs (Section 8.5.5).

Speed against the baseline, redone. Milestone J1+ (C6) compared troncons with the best measured configuration of llama.cpp. The figure changed twice, and each time it is struck through, not removed. Lot O had published ~~×3.85~~ against `fit -t 6` (0.568 tok/s), an under-tuned baseline; the trial found `-t 10` better and wrote ~~×2.79 to 2.89~~; the round swept the baseline (`mesures/moteur/O/corrections/csv/base-2026-10-16-passes.csv`, eleven passes, declared palindrome): the best is `fit -t 16`, **0.884 tok/s** (0.900 and 0.867), which beats `-t 6` by 54 % [meas.]. Against it, troncons *guard included* does **1.965 tok/s** on average over four passes (1.703; 1.900; 2.126; 2.131; noise 10.97 %, `csv/vitesse-2026-10-16-passes.csv`): × **2.22**, beyond the rule's threshold (2 × 10.97 % + 3 % = 24.9 %) [meas.]. C6 holds. The baseline may still be better (more than 16 threads, `--mlock`): not swept.

What could be wrong

- **The chain theorem assumes (N) for every kernel.** It is checked kernel by kernel (each call made twice, outputs equal to the bit, lot K) and by C3 end to end, not proven for the compiler: an `nvcc` that contracted an operation differently from one build to the next would change bits. The three fatbin images built with and without `vcvars` are equal byte for byte (finding m2), which bounds this risk for one version of `nvcc`, not for another.
- **The root cause of G1 is not established**: the arrival-buffer defect is proven possible, not observed; the old binary, replayed under the same load, also finished (338 s).
- **The window model leaves 0.9 MB unexplained**, and the 120b − 20b gap of C7 fails as long as the arbiter has not decided (fixed-size window proposed: 0.99 MB gap measured).
- **The line count** sorts a file by its place (`tests/`, `banc/`…), not by its content: a test module lodged in `src/` counts as code.
- **The guard's cost** rests on one pair of passes at equal disk throughput; the declared series says −12.3 % under a threshold of 26.8 % (noise 11.9 %), not real (Section 8.5.5).
- **The choice of CUDA** rests on the decoder (+14.9 %) and on two functions absent from wgpu, not on a measurement of the full path under Vulkan; a Vulkan without wgpu (with host-memory import and a transfer queue) might have done as well.

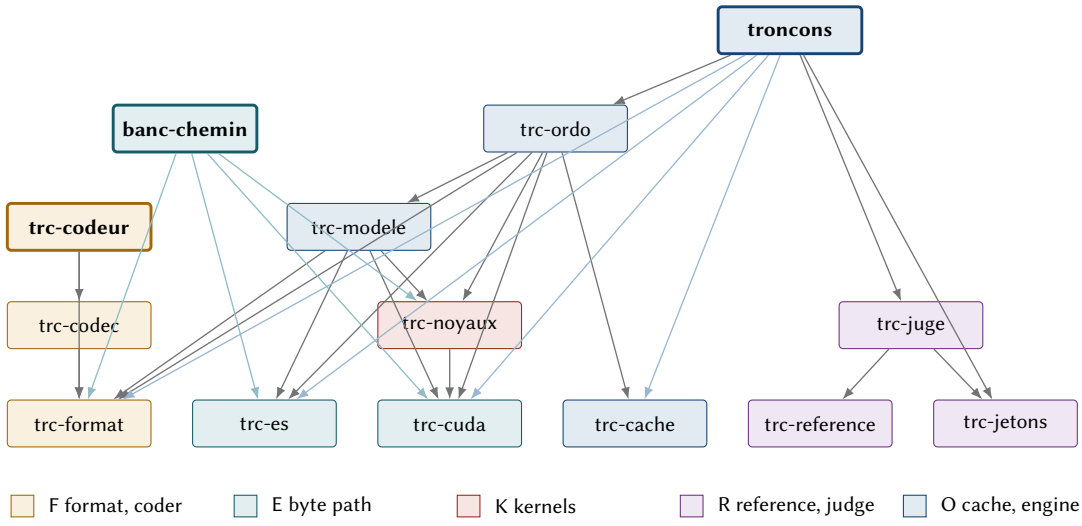


Figure 2.2: The graph of the workspace's internal dependencies, read in the `Cargo.toml` files (`memoire/figures/donnees/tome2/caisses.csv`). Six leaves without internal dependency, five levels, no cycle. Two structural facts can be read in it. **The engine does not depend on the processor codec**: `trc-ordo` and `trc-modele` do not import `trc-codec`; the card's decoder (`trc-noyaux`) is a second decoder, written from the specification, and its equality with `trc-codec` is a witness, not a given. **The reference does not depend on the format**: `trc-reference` reads the original GGUF with its own reader, which makes it an independent path for judging the engine.

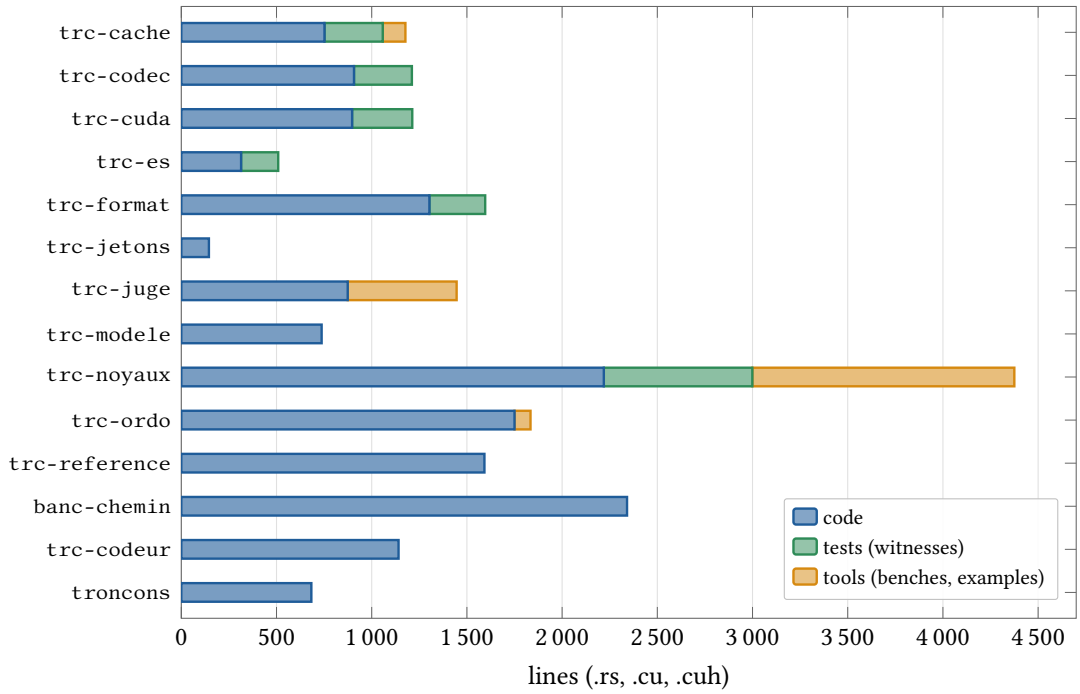


Figure 2.3: Lines of code, tests and tools per workspace member [calc.]. `trc-noyaux` is the largest: its kernels, their wrappers, 780 lines of on-card tests and a 1,376-line bench. The crates without tests in their folder (`trc-modele`, `trc-ordo`, `trc-reference`) are judged by external witnesses (chain K3, criteria C3 and C4, witness O2), committed under `mesures/moteur/`. Data: `memoire/figures/donnees/tome2/caisses.csv`.

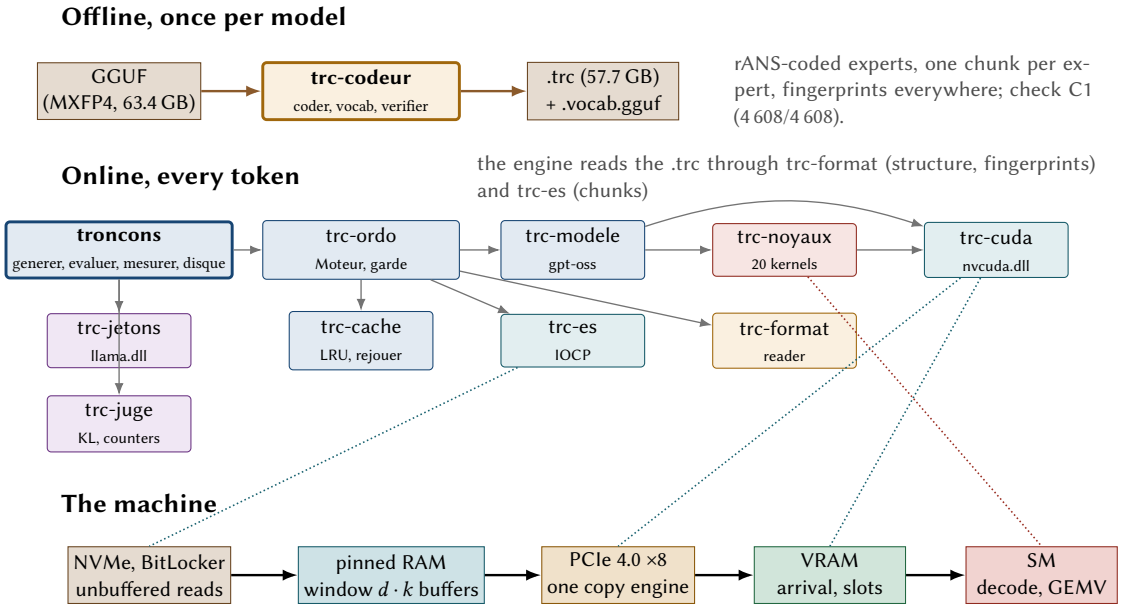
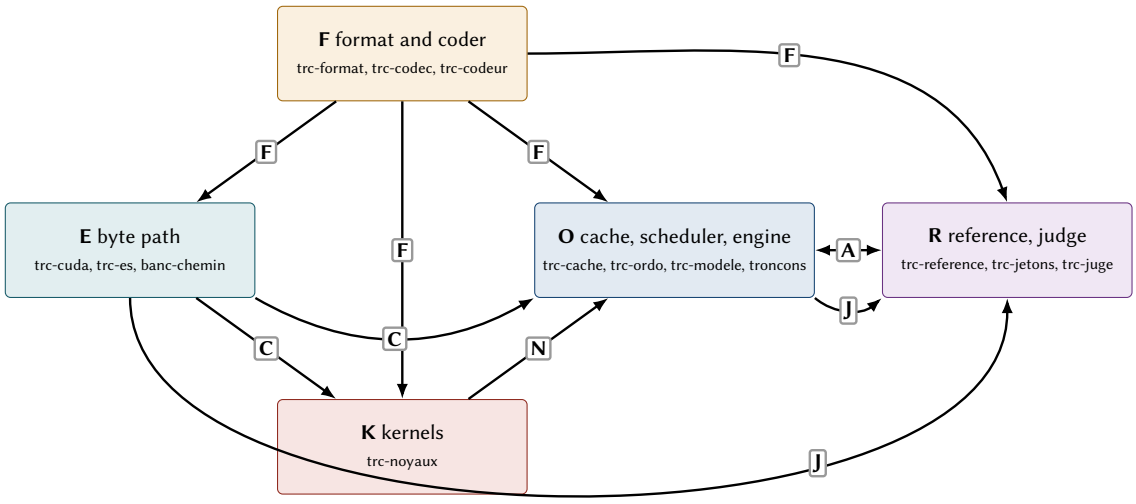


Figure 2.4: The overall picture: the coder offline, the engine online, and the machine underneath (dotted: which crate talks to which organ). A chunk read from disk lands in pinned RAM, which the disk writes and the card's copy engine reads: the same physical page, without any copy by the processor. The card has a single copy engine under WDDM (attribute `ASYNC_ENGINE_COUNT` = 1, recorded by lot E [meas.]): forward and backward copies form a single queue.



F: the .trc format to the bit (§ 4 of the specification). C: the API of trc-cuda. N: the kernels' binary interface, buffer layouts and the determinism rule. A: the cache API and the routing-trace format. J: the format of measurement logs. D, not drawn, binds everyone: the dependencies and their exact versions. An arrow goes from the lot that defines the seam to the lot that uses it.

Figure 2.5: The five lots and their frozen seams (version 1, 2026-10-15; `ARCHITECTURE.md` § 6.2). Fallback foreseen by the specification: with four engineers, F and R merge; with three, {F, R}, {E, O}, {K}, lot K staying alone because the forward pass is the longest pole.

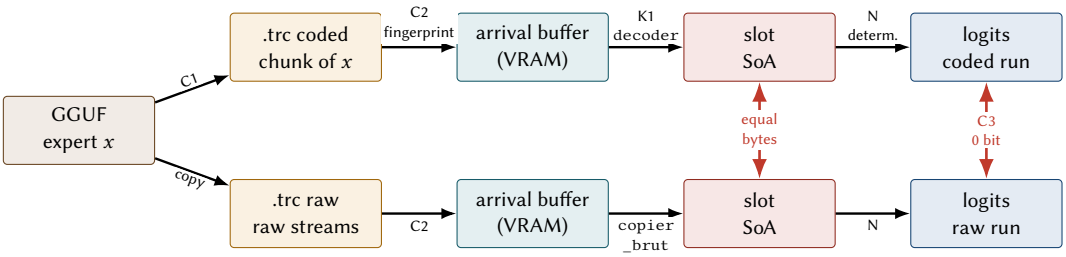


Figure 2.6: The exactness chain. Top, the coded pass; bottom, the raw pass (the same model written without entropy coding, flag bit 2). Each arrow carries the equality that guards it and the lot that proved it. The two paths meet at the expert slot, which holds the same bytes; from there, the determinism rule makes the equality of the logits inevitable.

Chapter 3

The `.trc` format to the bit

Where we start, what this chapter establishes, what remains

At the start, the arbiter's specification (`decisions/ARCHITECTURE.md` § 4) described to the bit a file that did not exist: one self-contained file per model, aligned on 4,096 bytes, one chunk per expert, rANS streams with the table chosen per lane. The arbiter's measurement F1 had fixed its coding (same rate as the three class streams of study 03, without reassembly); nobody had written the coder, nor a decoder of this format. **This chapter establishes** the format as the real files carry it: every field of every structure, drawn and filled with the values read in `gpt-oss-120b.trc` (57,730,662,400 bytes) and `gpt-oss-20b.trc`; the lemmas that make interleaved rANS an exact code, and why its two free checks do not guard the table; the exact reconstruction of an expert, proven; the bounded-memory coder and its two propositions (same bytes, memory independent of the model); criterion C1 held on 768 and 4,608 experts by two paths that share no line; and the gesture that tells whether a specification is true: a judge who wrote none of it decoded it, from the text alone, to the bit. **What remains**: version 1.1 with two levels (J2), reserved to the bit but not written; thirteen precisions of the text (nine from lot F, three from the trial's judge, one from the judge of Volume II), compatible with the written files, which the arbiter must carry into the specification. Sources: `decisions/ARCHITECTURE.md` § 3 and 4; `moteur/F-pour-le-memoire.md`, `moteur/REPRISE-F.md`; `mesures/moteur/F/derivations.md`, `verdicts.md`, `00-seuils-declares.md`, `journaux/`, `csv/`; `mesures/epreuve-J1.md` § 2; `mesures/moteur/epreuve-J1/decodeur_s4.py`.

In brief

A `.trc` file is a book whose every page is an expert. The table of contents (the directory) says where each page starts, how much it weighs and which fingerprint it must have; each page is read in a single gesture, because it lies in one piece on disk; and each page is written in a shorthand (rANS) that thirty-two readers decipher at the same time, each knowing in advance, from the line it stands on, which alphabet to use. Nothing is guessed: every structure carries its size, its number of entries and its fingerprint, and a reader refuses what it does not know.

3.1 The principles

Six principles, each with its reason (`ARCHITECTURE.md` § 4.1):

1. **One self-contained file per model**, written once from the original GGUF by `trc-codeur`. It contains everything the engine reads; a companion `<name>.vocab.gguf` carries the vocabulary for `llama.dll`. Reason (delegated answer of 2026-10-15, reversible): a general instrument has a single artefact per model; the cost is copying the always-read tensors (2.3 GB on the 120b).
2. **Little-endian** everywhere; strings in UTF-8; sizes in bytes.

3. **Alignment on 4,096 bytes** for every section and everything read by an unbuffered read: Windows then requires offset, length and address to be multiples of the sector size [6], and 4,096 covers the machine's NVMe.
4. **One chunk = one expert** (its three matrices and their biases), in one piece: a miss costs one request, one completion, one fingerprint, one slot. The throughput gain of one 12 MiB read over three of 4.2 MiB (+9 %) was below the decision threshold (20 %): the choice is made for simplicity, not speed.
5. **Refinement levels**: a chunk is the sequence of its parts of level 0, 1, …; each part is aligned on 4,096, and any prefix that stops at the end of a part decodes on its own. Reader, cache and scheduler only know "read up to the end of level *j*".
6. **Nothing is guessed**: a reader refuses an unknown major version, stream kind or flag.

3.2 The overall layout

A .trc file is a fixed header of 4,096 bytes followed by five sections, each aligned on 4,096 (Figure 3.1). The offsets and lengths of Table 3.1 are those carried by the headers of the two real files, re-read by the second path of C1 (Section 3.12).

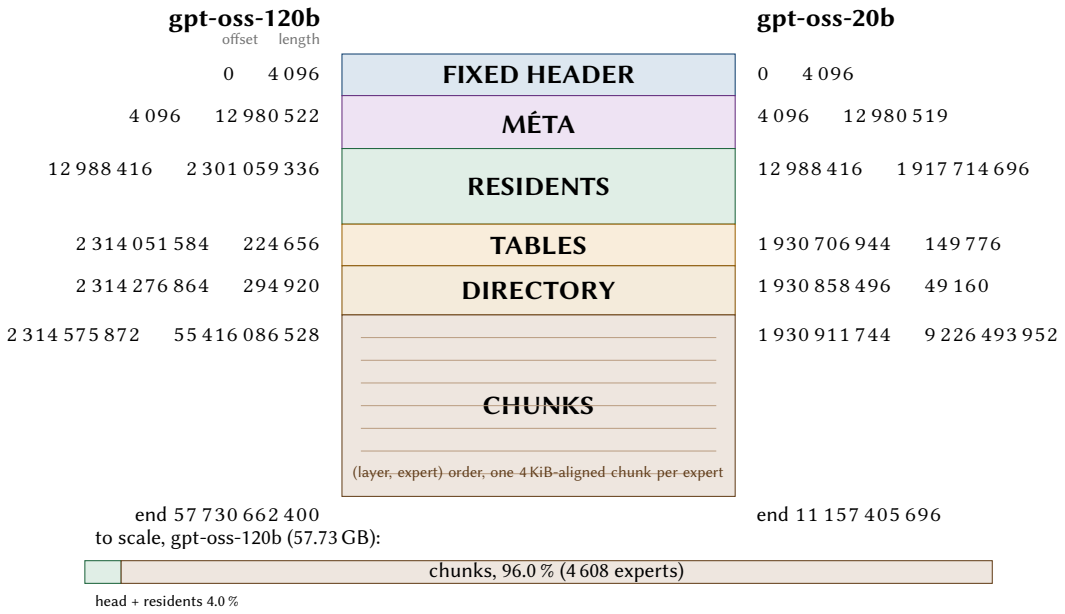


Figure 3.1: The .trc file and its sections, at the real offsets of both files (in bytes; box heights not proportional). The bottom bar is to scale: on gpt-oss-120b, 96.0 % of the file are chunks. Data: mesures/moteur/F/journaux/second-chemin-{20b,120b}.json, field entete.sections.

3.3 The fixed header

The header fits in 4,096 bytes (Figure 3.2). It says what the file is (magic, version, flags), where it comes from (SHA-256 and size of the source GGUF), the model's geometry (layers, experts, active experts, dimensions of the three matrices, MXFP4 format), and where the sections are. Its 3,824 reserved bytes must be zero, which the reader checks; its last eight bytes are the fingerprint64 (Section 3.10) of the first 4,088. A truncated file is refused on opening: its real size differs from field 24.

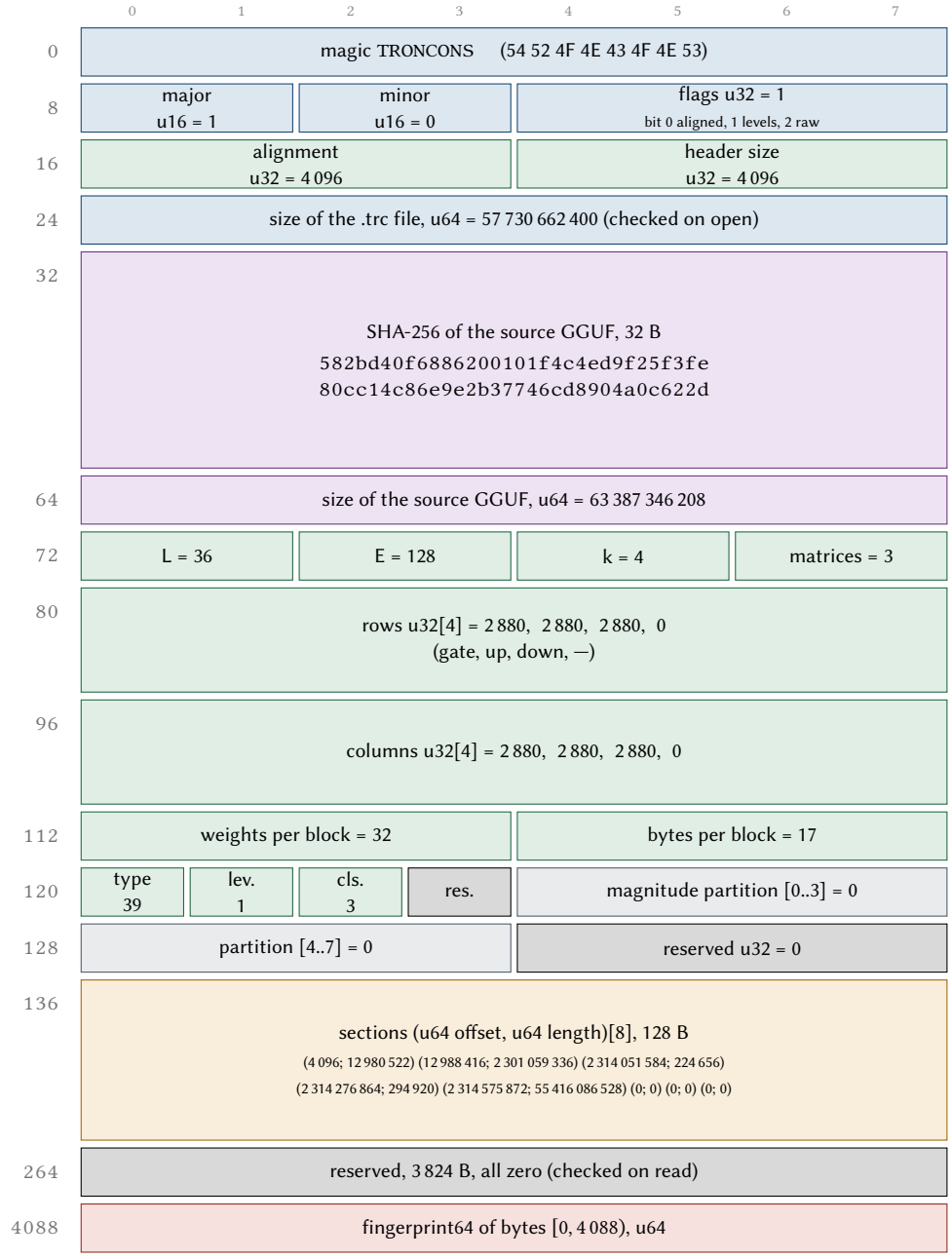


Figure 3.2: The fixed header, eight bytes per row (offsets on the left), except two compressed rows: the sections (128 bytes, offsets 136 to 263, on three rows) and the reserved area (3,824 bytes, from 264 to 4,087, on a single one), with the values read in `gpt-oss-120b.trc`. For the 20b: size 11,157,405,696, SHA-256 `27cd6c43...5901`, source size 12,109,566,624, L = 24, E = 32, the same dimensions. Type 39 is MXFP4 in ggml's numbering; the magnitude partition only serves version 1.1. Colours: identity (blue), provenance (violet), geometry (green), sections (orange), reserved (grey), fingerprint (red).

	gpt-oss-120b		gpt-oss-20b	
Section	offset	length	offset	length
0 META	4,096	12,980,522	4,096	12,980,519
1 RESIDENTS	12,988,416	2,301,059,336	12,988,416	1,917,714,696
2 TABLES	2,314,051,584	224,656	1,930,706,944	149,776
3 DIRECTORY	2,314,276,864	294,920	1,930,858,496	49,160
4 CHUNKS	2,314,575,872	55,416,086,528	1,930,911,744	9,226,493,952
end of file		57,730,662,400		11,157,405,696

Table 3.1: The sections of both files, in bytes (`memoire/figures/donnees/tome2/sections.csv`). Lengths are *useful*: they count the section's final fingerprint, not the padding leading to the next section (lot F's precision, the trial's finding m4). Example: META of the 120b ends at 4,096 + 12,980,522 = 12,984,618, and RESIDENTS starts at the next multiple of 4,096, 3,171 × 4,096 = 12,988,416. The last section ends exactly at the end of the file.

The flags. Bit 0: chunks aligned on 4,096 (always 1). Bit 1: at least two levels (v1.1). Bit 2: *raw* file, whose streams are not coded, used as the reference path for criterion C3. An unknown flag makes the file refused, even re-fingerprinted: it is one of the breaks of `trc-format`'s witness (`moteur/REPRISE-F.md`, F1).

3.4 META, RESIDENTS, TABLES, DIRECTORY

3.4.1 META: the GGUF pairs, copied

The META section starts with a `u64` (the number of pairs), then copies byte for byte the GGUF key-value pairs, in their order and in GGUF v3 format (64-bit key length, key bytes, 32-bit type, value) [7]. The coder adds three pairs: `troncons.codeur.version` ("trc-codeur 0.1.0"), `troncons.codeur.date` ("2026-10-15") and `troncons.source.nom`. Both files have 36 + 3 = 39 pairs [meas.] (Figure 3.3). The section ends with the fingerprint64 of everything before it. Its 12.98 MB are almost all vocabulary (201,088 tokens); the three bytes of difference between 120b and 20b come from the strings naming the model. The engine reads the hyperparameters there: 64 query heads, 8 key and value heads, head dimension 64, rotation base 150,000, YaRN with factor 32 over an original context of 4,096 (fast β 32, slow β 1), sliding window of 128.

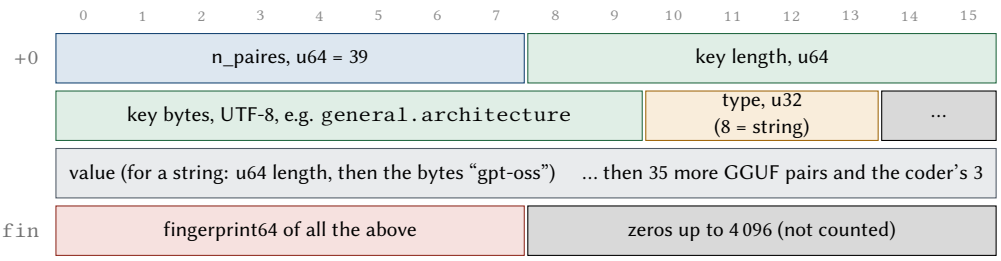


Figure 3.3: The META section, in symbolic rows (keys and values have variable lengths): the count, the pairs in GGUF v3 format copied byte for byte, the final fingerprint64, then zeros that the section's length does not count.

3.4.2 RESIDENTS: the always-read tensors

All GGUF tensors except the experts (`blk.*.ffn_{gate,up,down}_exps.{weight,bias}`) are copied as they are: $687 - 36 \times 6 = 471$ on the 120b, $459 - 24 \times 6 = 315$ on the 20b [calc.]. The section starts with a table (a `u32` count, a reserved `u32`, then one 128-byte entry per tensor, Figure 3.4), zeros up to the multiple of 4,096 from the start of the section, then the data zone, where each tensor is aligned on 256 in its ggml layout. The table's fingerprint64 is stored in the last eight bytes of the section: on the 120b, at 2,301,059,328 bytes from its start, for a length of 2,301,059,336 [meas.] (lot F's precision 2: the specification says "after the data zone"; the end of the last tensor being aligned on 256, the three possible readings coincide here).

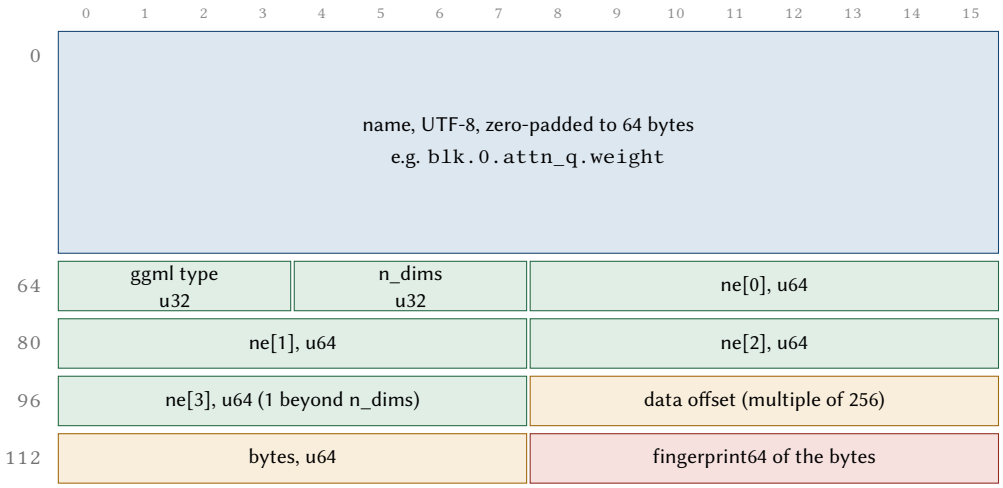


Figure 3.4: An entry of the residents table (128 bytes, sixteen per row). `ne` follows ggml's convention: `ne[0]` is the fastest dimension. The offset is counted from the start of the data zone.

The input embedding `token_embd` (Q8_0, 0.615 GB) is *never* mounted in VRAM: the engine reads one row of it per token. A row of 2,880 weights in Q8_0 is 90 blocks of 34 bytes, i.e. 3,060 bytes, read within an aligned 8 KiB read [calc.].

3.4.3 TABLES: the rANS frequencies

A record (Figure 3.5) carries the layer, the matrix, the kind of stream it serves, the context, and a payload: 16 or 256 frequencies on 16 bits, or 4 centres in f32 (v1.1). A frequency is an integer from 0 to 4,096; a table's sum is exactly 4,096; 0 marks an absent symbol. Tables are shared by all experts of a layer, one per (layer, matrix, kind, context). In v1.0, each matrix of a layer has four tables: DELTA (context 0) and CODES_CTX (contexts 0, 1, 2), i.e. $36 \times 3 \times 4 = 432$ records of $8 + 512 = 520$ bytes on the 120b; the section is $8 + 432 \times 520 + 8 = 224{,}656$ bytes, count and fingerprint included, exactly the recorded length [calc.]. The 20b has 288 (149,776 bytes). The engine mounts them in VRAM as LUTs of 4,096 32-bit words (16 KiB each): $432 \times 16{,}384 = 7{,}077{,}888$ bytes (7.08 MB, 6.75 MiB) for the 120b [calc.]. The specification (`decisions/ARCHITECTURE.md` § 4.6) writes "$36 \times 3 \times 4 \times 16$ KiB = ~~6.9 MB~~": the product is right, its value is right neither in MB nor in MiB; this is precision T2 of Table 3.6.

3.4.4 DIRECTORY: the table of contents

The directory has one 64-byte entry per (layer, expert), in that order, then the section's fingerprint64: $4{,}608 \times 64 + 8 = 294{,}920$ bytes on the 120b, $768 \times 64 + 8 = 49{,}160$ on the 20b [calc.]. Each entry (Figure 3.6) says where the chunk starts, the length and useful bytes of each of its two levels, and

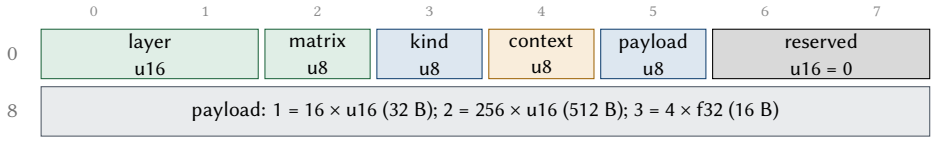


Figure 3.5: A record of the TABLES section: eight header bytes, then the payload.

three fingerprints: that of the *original* expert, and that of the useful bytes of each level. The level-1 part immediately follows the level-0 part: exact mode reads both in a single request. It is the only structure of the file that grows with the number of experts, and it stays below 0.3 MB on the 120b.

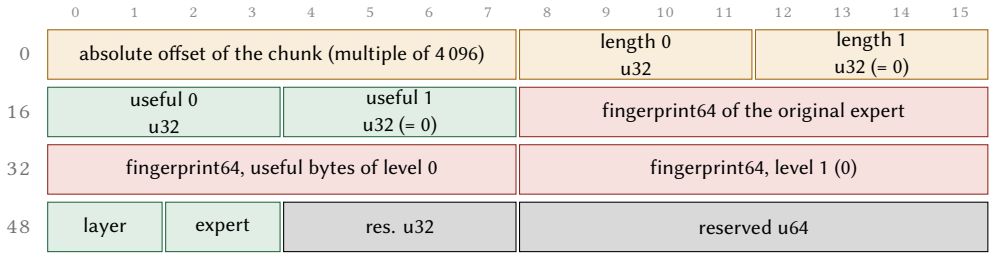


Figure 3.6: A directory entry (64 bytes). In v1.0, the level-1 fields are zero.

3.5 A chunk part

A part starts with a 64-byte header and a table of 16-byte stream descriptors (Figure 3.7), then the streams, each aligned on 16, then zeros up to the multiple of 4,096. In v1.0, a level-0 part carries twelve streams, four per matrix: MEDIANES_j, BIAIS_j, DELTA_j, CODES_CTX_j for $j = 0$ (gate), 1 (up), 2 (down). Figure 3.8 gives their mean sizes, measured over the 13,824 matrices of the 120b.

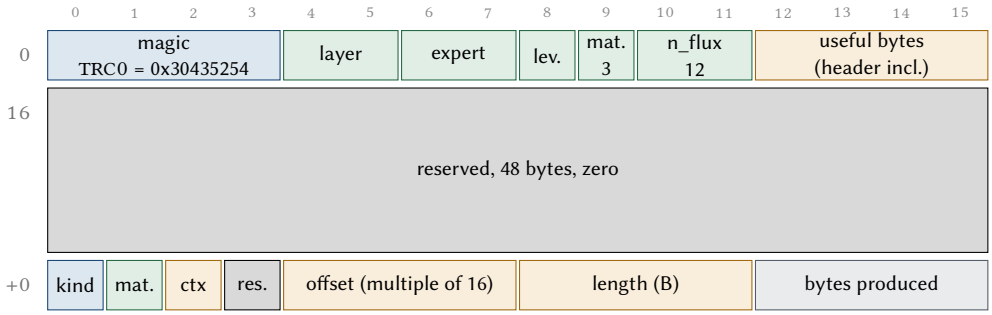
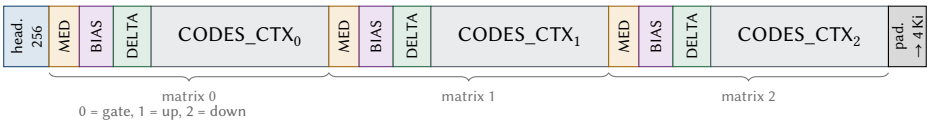


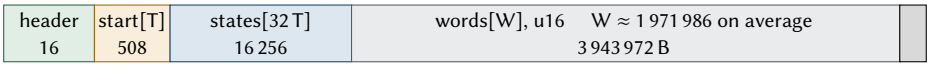
Figure 3.7: Top, a part header (64 bytes: magic TRC0 at level 0, TRC1 at level 1). Bottom, a stream descriptor (16 bytes). The fixed context is 0 for DELTA and 0xFF (none) for raw streams and variable-context streams (lot F's precision 4); the bytes produced by a rANS stream equal its P field, padding included (precision 5).

Padding, a plausibility check. If the useful length of a part were uniformly distributed modulo 4,096, the mean padding would be $\frac{1}{4{,}096}\sum_{i=0}^{4{,}095} i = 2{,}047.5$ bytes. Measured: 2,036 bytes on the 120b, 2,029 on the 20b (mean chunk minus mean useful bytes) [calc.]. Padding therefore costs 0.017 % of the file, and principle 3 is paid at that price.



Mean sizes over the 13 824 matrices of gpt-oss-120b: MEDIANS 2 880 B (raw), BIAS 11 520 B (raw), DELTA 32 772 B, CODES_CTX 3 960 750 B; part header and stream table 64 + 12 × 16 = 256 B; zero padding to the next multiple of 4 096: 2 036 B on average. Mean chunk: 12 026 060 B, against 13 253 760 B of original MXFP4 expert. Box widths are not to scale (CODES_CTX is 98.8 % of the chunk).

One CODES_CTX stream, serialised



For a 2 880 × 2 880 matrix: P = 16 · 259 200 = 4 147 200 bytes produced, 32 400 macro-steps, T = ⌈32 400/256⌉ = 127 tiles; fixed part 16 + 4T + 128T = 16 780 B, i.e. 0.42 % of the stream. The last box is the padding to a multiple of 16 (at most 15 B).

Figure 3.8: A mean chunk of gpt-oss-120b, stream by stream, then the serialised CODES_CTX stream. Data: `memoire/figures/donnees/tome2/flux-moyens.csv` and `calculs-format.csv`, drawn from `mesures/moteur/F/journaux/verifier-120b.json`.

3.6 The rANS stream

3.6.1 Serialisation

A rANS stream (Figure 3.9) carries in 16 bytes what is needed to decode it: P, the number of bytes it produces (a multiple of 128); N, the number of macro-steps per tile (256); the geometry (1: one byte per symbol, 4 sub-steps per output word; 2: one nibble, 8 sub-steps); T, the number of tiles, $T = \lceil (P/128)/N \rceil$; W, the number of 16-bit words of the shared stream. Then come `début[T]` (the index of each tile's first word), `états[32 T]` (the initial state of each lane of each tile), `mots[W]`, then zeros up to the multiple of 16.

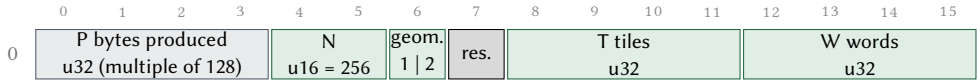


Figure 3.9: The header of a rANS stream (16 bytes).

The calculation, for one gpt-oss matrix. A matrix of $R = 2{,}880$ rows and $C = 2{,}880$ columns has $B = C/32 = 90$ blocks per row, $R \cdot B = 259{,}200$ blocks. Its codes are $16 \times 259{,}200 = 4{,}147{,}200$ bytes, i.e. 32,400 macro-steps of 128 bytes, hence $T = \lceil 32{,}400/256 \rceil = 127$ tiles (126 full ones and one of 144 macro-steps). The fixed part of the stream is $16 + 4T + 128T = 16 + 508 + 16{,}256 = 16{,}780$ bytes. The mean CODES_CTX stream is 3,960,750 bytes: $(3{,}960{,}750 - 16{,}780)/2 \approx 1{,}971{,}986$ words remain (padding of at most 15 bytes included). Over the matrix's 8,294,400 weights, the stream costs $3{,}960{,}750 \times 8/8{,}294{,}400 = 3.8202$ bits per weight, of which $1{,}971{,}986 \times 16/8{,}294{,}400 = 3.8040$ are entropy-coded words and 0.0162 the fixed part [calc.]. A matrix's DELTA has 259,200 bytes, 8 tiles, a fixed part of 1,072 bytes, and costs $32{,}772 \times 8/8{,}294{,}400 = 0.0316$ bit per weight [calc.].

3.6.2 Normative decoding

Constants: probabilities on 12 bits ($M = 4{,}096$), state x on 32 bits, lower bound $L = 2^{16}$, 16-bit words. Tile t covers macro-steps tN to $\min((t+1)N, P/128) - 1$. Lane $l \in \{0, \ldots, 31\}$ of macro-step m produces

the 32-bit output word of index $w = 32m + l$; its sub-steps $k = 0, 1, \ldots$ produce, from low to high, byte k (geometry 1) or nibble k (geometry 2). A symbol's table is stored as a LUT of 4,096 32-bit entries (Figure 3.10): for each slot $i \in [c[s], c[s] + f[s])$, the entry carries the symbol s, its frequency and the offset $i - c[s]$, where $c[s] = \sum_{u<s} f[u]$ is the cumulative frequency.

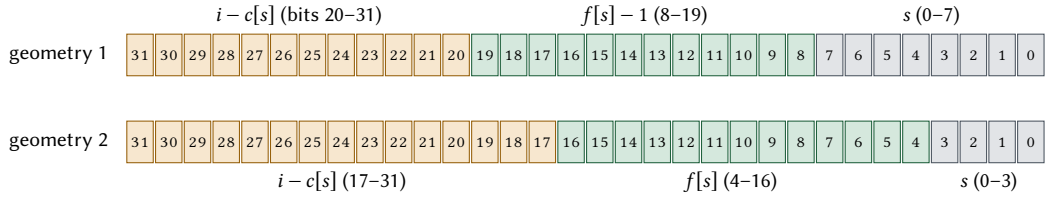


Figure 3.10: The 32-bit LUT entry, high bit on the left. In geometry 1, the frequency is stored minus one: $f = 4{,}096$ (a lone symbol) then fits in 12 bits (lot K's test `symbole_unique_a_frequence_pleine`).

Decoding a tile starts from $p \leftarrow \texttt{début}[t]$ and $x_l \leftarrow \texttt{états}[32t + l]$, then, for each macro-step, for each sub-step, **for** $l = 0$ **to 31 in order**: read the entry e of the LUT of the symbol's context at slot $x_l \bmod 4{,}096$, extract s, f and $b = i - c[s]$, set $y = f \cdot \lfloor x_l / 4{,}096 \rfloor + b$; if $y < L$, read a word ($x_l \leftarrow 2^{16} y + \texttt{mots}[p]$, $p \leftarrow p + 1$), otherwise $x_l \leftarrow y$; write s. Listing 3.1 is this core, as `trc-codec` writes it.

Listing 3.1: The core of the normative decoder (`moteur/crates/trc-codec/src/rans.rs`, `decoder_avec`, shortened; comments in French, the language of the code). A missing word reads as 0 and is counted; no stream makes the decoder panic.

```
for m in 0..nb {                          // macro-pas de la tuile
  for k in 0..spm {                       // sous-pas : 4 (geometrie 1) ou 8 (2)
    for l in 0..VOIES {                   // les 32 voies, DANS L'ORDRE
      let w = (m0 + m) * VOIES + l;       // mot de sortie de 32 bits
      let n = w * spm + k;                // indice du symbole
      let Some(c) = contexte(n, &sortie) else { continue };   // saute
      let e = luts[c as usize][(x[l] & (M - 1)) as usize];
      let (s, f, b) = (e & 0xFF, ((e >> 8) & 0xFFF) + 1, e >> 20); // geometrie 1
      let y = f * (x[l] >> 12) + b;
      if y < L { x[l] = (y << 16) | mot(p); p += 1; } else { x[l] = y; }
      sortie[n] = s as u8;
    }
  }
}
// fin de tuile : chaque voie doit valoir L, et p doit egaler debut[t + 1]
let hors = x.iter().filter(|&&v| v != L).count();
let mots_faux = deborde || p != fin;
```

The coder takes exactly the reverse path: it visits the symbols of a tile by *decreasing* macro-step, sub-step and lane, starts from $x = L$ on every lane, and, for each symbol s of frequency f and cumulative frequency c, emits the word $x \bmod 2^{16}$ then sets $x \leftarrow \lfloor x / 2^{16} \rfloor$ if $x \geq f \cdot 2^{20}$, before coding $x \leftarrow \lfloor x / f \rfloor \cdot 4{,}096 + (x \bmod f) + c$. Finally it reverses its stack of words.

3.6.3 Why it is exact

The following four statements are lot F's (`mesures/moteur/F/derivations.md` § 1 to 3), written out in full. Notation: a symbol s has a frequency $f \in [1, M]$ and a cumulative frequency c; coding $C(s, x') = \lfloor x'/f \rfloor M + (x' \bmod f) + c$; decoding from x: slot $i = x \bmod M$, symbol s such that

$c \le i < c+f$, then $D(x) = f\lfloor x/M \rfloor + i - c$. The ANS principle is Duda's [8]; interleaving several states over a single word stream, Giesen's [9].

Lemma 3.1 (Exact inverse)**.** *For every $x' \ge 0$, $D(C(s,x')) = (s,x')$.*

Proof. Let $q = \lfloor x'/f \rfloor$ and $r = x' \bmod f$, so $x' = qf + r$ with $0 \le r < f$. Then $C = qM + (r+c)$ with $0 \le r + c < c + f \le M$: the slot is $i = r + c \in [c, c+f)$, so the symbol read is s, and $\lfloor C/M \rfloor = q$. Hence $D = fq + (r+c) - c = qf + r = x'$. ∎

Lemma 3.2 (The state interval)**.** *With the coder's emission rule, if $x \in I = [L, 2^{32})$ before coding a symbol, then the reduced state x' satisfies $x' \in [f \cdot 2^4, f \cdot 2^{20})$, and $C(s,x') \in I$.*

Proof. Without emission, $L \le x < f \cdot 2^{20}$; now $f \cdot 2^4 \le M \cdot 2^4 = 2^{16} = L \le x$, so $x' = x \in [f \cdot 2^4, f \cdot 2^{20})$. With emission, $f \cdot 2^{20} \le x < 2^{32}$, so $x' = \lfloor x/2^{16} \rfloor \in [f \cdot 2^4, 2^{16})$, and $2^{16} \le f \cdot 2^{20}$ since $f \ge 1$. Finally $C(s,x') \ge \lfloor x'/f \rfloor M \ge 2^4 \cdot 2^{12} = L$ and $C(s,x') < (\lfloor x'/f \rfloor + 1)M \le 2^{20} \cdot 2^{12} = 2^{32}$. ∎

Lemma 3.3 (The decoder reads exactly when the coder emitted)**.** *At the step that decodes a symbol, the decoder reads a word if and only if the coder emitted one when coding that symbol, and a single read suffices.*

Proof. By lemma 3.1, $y = D(x)$ is the coder's x'. If the coder emitted, $x' < 2^{16} = L$: the decoder reads, and $2^{16}y + w$ restores exactly the state before the emission ($w = x \bmod 2^{16}$), which lies in I. Otherwise $x' = x \ge L$ and the decoder does not read. A second read is never needed: for $y < L$, $2^{16}y + w \ge 2^{16} = L$ as soon as $y \ge 1$, and $y \ge 16f \ge 16$ since $x \ge L$ gives $\lfloor x/M \rfloor \ge 16$. ∎

Proposition 3.4 (The two free checks of a tile)**.** *On an intact stream, the decoder (a) ends every lane on $x = L$; (b) consumes exactly the $\texttt{début}[t+1] - \texttt{début}[t]$ words of tile t.*

Proof. (a) By induction over the reversed steps, with lemma 3.3: the decoder's state after the k-th decoded symbol equals the coder's state before coding that symbol; after the last decoded symbol, which is the first coded, it is the coder's initial state, L. (b) The decoder reads a word at each step where the coder emitted one, and only there; the number of reads is therefore the number of emissions, the size of the tile's stack. The order of reads between lanes is the order of emissions reversed: the 32 lanes share a single stack because the coder's reverse order, reversed by the stack, is exactly the decoder's forward order. ∎

What the checks do not guard. Neither (a) nor (b) guards the *table*: a decoder reading a corrupted table stays within I and may end on L. Lot K saw it on the card: a corrupted LUT slot gives a wrong output with **0 lanes off** [meas.] (`moteur/K-pour-le-memoire.md` § 3). This is why the format carries the fingerprints of the tables, checked on opening, and those of the parts. Lot F's witness of witnesses checked it on the code: removing the count of lanes, or that of words, breaks one test each; its first pass had shown that a test looking only at their conjunction did not see the loss of the first (`mesures/moteur/F/casser-temoins-f1-passe1.log`). The second check (b) is not in the specification: lot F and lot K each added it on their own, without changing a byte of the format, and the judge found it true on 100 % of the decoded tiles (finding m4).

3.6.4 Thirty-two lanes at once

The rule "for $l = 0$ to 31 in order" seems to forbid decoding the lanes in parallel. It does not: only the order of *word reads* ties the lanes together (Figure 3.11).

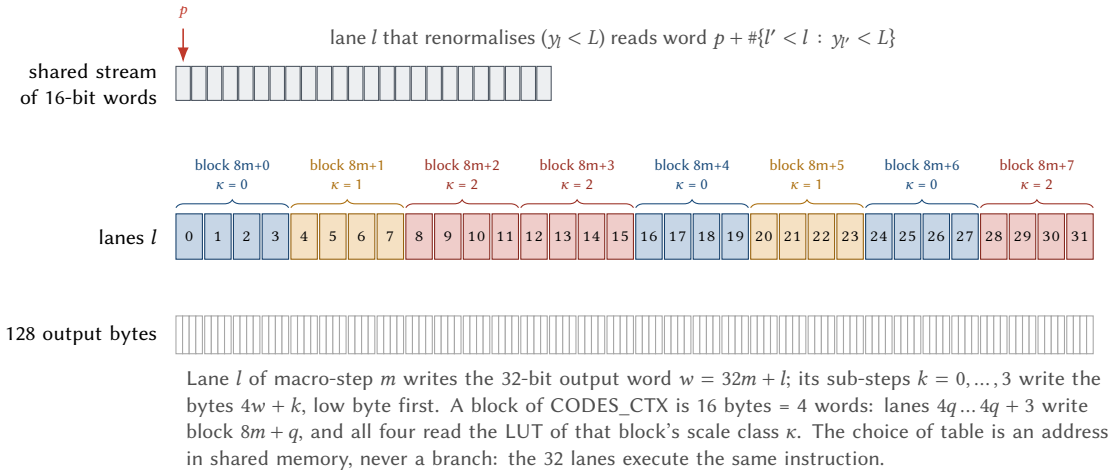


Figure 3.11: The rANS tile and the per-lane table (figure M4 of the specification). Example classes κ for the eight blocks of a macro-step.

Proposition 3.5 (Parallel decoding of the lanes)**.** *At sub-step (m, k), let $v_l = [y_l < L]$ be lane l's renormalisation indicator. A decoder that computes the 32 lanes at once and gives lane l the word $p + \sum_{l'<l} v_{l'}$, then advances p by $\sum_l v_l$, produces the same symbols and the same states as the normative decoder.*

Proof. Computing y_l depends only on x_l and the LUT: it commutes between lanes. In the normative decoder, at sub-step (m, k), the lanes that renormalise read words $p, p+1, \ldots$ in increasing lane order: lane l reads word p plus the number of lower-index lanes that also read, i.e. $p + \sum_{l'<l} v_{l'}$; after the 32 lanes, p has advanced by $\sum_l v_l$. Both decoders therefore make the same reads; by induction over sub-steps, states and pointer are equal at every step. Tiles, for their part, are independent: each has its start and its states. ∎

Two decoders use it without having read each other. On the card, a warp computes v with `ballot` and the rank with `popc` of the masked vote (`moteur/K-pour-le-memoire.md` § 2.1); in the trial, numpy computes it with a cumulative sum along the lane axis, $p + \mathrm{cumsum}(v)_l - 1$ for a lane that renormalises, over the 127 tiles of a stream at once (Listing 3.3). The format is designed for this: 32 lanes per tile is the width of a warp.

3.7 Contexts: the table chosen by the decoder

For a matrix of R rows and $B = C/32$ blocks per row, block $\beta = rB + b$ has a scale deviation $\delta[\beta]$ and a *scale class*

$$\kappa(\beta) = \begin{cases} 0 & \text{if } \delta[\beta] < 128 \text{ (scale below its row's median),} \\ 1 & \text{if } \delta[\beta] = 128 \text{ (at the median),} \\ 2 & \text{if } \delta[\beta] > 128 \text{ (above).} \end{cases}$$

For CODES_CTX, a symbol's context is chosen **per lane**: word w belongs to block $\lfloor w/4 \rfloor$ (a block is 16 bytes, hence 4 words), and its table is that of $\kappa(\lfloor w/4 \rfloor)$. The decoder always decodes DELTA before CODES_CTX: it knows κ before it needs it.

Proposition 3.6 (Context known to the decoder: exactness and rate)**.** *If the context $k(n)$ of symbol n is a function of data the decoder holds before decoding n, decoding remains exact, and the number of bits emitted is $\sum_n -\log_2(f_{k(n)}[s_n]/M)$ up to the header and the final states.*

Proof. Exactness: lemma 3.1 holds for any table; coder and decoder apply the same table to symbol n since they compute the same $k(n)$, the coder from the source, the decoder from what it has already returned, identical by induction. Rate: a coding step multiplies the state by M/f up to rounding ($x' \mapsto\approx x'M/f$), and each word emitted divides it by 2^{16}; the number of words is therefore, up to a constant per lane, $\sum_n \log_2(M/f_{k(n)}[s_n])/16$. ■

The choice of table is an *address*, not a branch: on the card, the 32 lanes execute the same instruction and differ only in the LUT they read from shared memory. Against the three class streams of study 03, the per-lane table avoids reassembly and prefix sum, for the same rate: 3.8431 against 3.8434 bits per weight on the 12 tensors of F1 [meas.] (ARCHITECTURE.md § 3.3).

Padding blocks. The specification did not say the class of a padding block, when $16RB$ is not a multiple of 128. A matrix's DELTA produces $\lceil RB/128 \rceil \cdot 128$ bytes and its codes $\lceil 16RB/128 \rceil \cdot 128$, i.e. $\lceil RB/8 \rceil \cdot 8$ blocks. Since $\lceil RB/8 \rceil \cdot 8 \leq \lceil RB/128 \rceil \cdot 128$, every padding block of the codes has a decoded δ, which is 0 (padding), hence $\kappa = 0$: the decoder has no extra rule to know (lot F's precision 6). For gpt-oss, $RB = 259{,}200 = 2{,}025 \times 128$: no padding.

3.8 The stream kinds

Code	Kind	Content, bytes produced for an $R \times C$ matrix (gpt-oss: $R = C = 2{,}880$, $B = 90$)	Level
0x01	MEDIANES	raw: R bytes, each row's E8M0 median (`tri[B/2]` of the B scales); 2,880	0
0x02	BIAIS	raw: R f32; 11,520	0
0x03	DELTA	rANS geom. 1: RB bytes, $\delta = (e - \text{med} + 128) \bmod 256$; 259,200	0
0x04	CODES_CTX	rANS geom. 1, per-lane table: $16RB$ bytes, the `qs` of the MXFP4 blocks in β order; 4,147,200	0 (v1.0)
0x05	(reserved)	three reassembled class streams: neither produced nor read	—
0x06	CODES_BRUTS	raw: $16RB$ bytes (raw file only)	0
0x07	ECHELLES_BRUTES	raw: RB E8M0 bytes (raw file only)	0
0x10	BASE_CTX	rANS geom. 1, per-lane table: each nibble = sign and magnitude class	0 (v1.1)
0x11	COMPL_CTX	rANS geom. 2, per-symbol table: rank of the magnitude within its class	1 (v1.1)
0x12	CENTRES	table kind only: the 4 reconstruction values of the base	—

Table 3.2: The stream kinds (ARCHITECTURE.md § 4.10; names as in the code). Kinds 0x10 to 0x12 are reserved to the bit for v1.1 (J2) and produced by no written file.

3.9 The exact reconstruction of an expert

3.9.1 The normative procedure

For each matrix j (R rows, C columns, $B = C/32$):

1. read $\text{MEDIANES}_j \to \text{med}[r]$ and $\text{BIAIS}_j \to \text{bias}[r]$;
2. decode $\text{DELTA}_j \to \delta[\beta]$, then $e[\beta] = (\text{med}[r] + \delta[\beta] - 128) \bmod 256$ and $\kappa(\beta)$;

3. decode CODES_CTX$_j$ with the per-lane table $\kappa \to \mathrm{qs}[16\beta \dots 16\beta + 15]$;
4. write the card's slot: codes, scales, biases (Section 3.9.3).

Listing 3.2: Reconstructing a v1.0 matrix (`moteur/crates/trc-codec/src/lib.rs`, `decoder_matrice_v10`, shortened).

```
let (delta, bd) = rans::decoder_une_table(&flux_d, &t.delta.lut)?;   // 2. les deltas
let kappa: Vec<u8> = delta.iter().map(|&d| classe_echelle(d)).collect();
let mut echelles = vec![0u8; n_blocs];
for (beta, e) in echelles.iter_mut().enumerate() {                   // e = med + delta - 128
    *e = med[beta / b].wrapping_add(delta[beta]).wrapping_sub(128);   // (mod 256)
}
let luts: [&[u32]; 3] = [&t.codes[0].lut, &t.codes[1].lut, &t.codes[2].lut];
let (mut codes, bc) = rans::decoder_par_voie(&flux_c, &kappa, &luts)?; // 3. table par voie
codes.truncate(16 * n_blocs);                                        // sans le bourrage
```

3.9.2 The original expert, and why nothing needs reordering

The original expert, in the sense of the fingerprints, is the sequence, for $j = 0, 1, 2$, of the RB 17-byte MXFP4 blocks $[e[\beta], \mathrm{qs}[16\beta \dots 16\beta + 15]]$ in β order, then, for $j = 0, 1, 2$, of the R f32 biases: $3 \times (17 \times 259{,}200 + 4 \times 2{,}880) = 13{,}253{,}760$ bytes for gpt-oss [calc.]. An MXFP4 block carries 32 E2M1 weights (4 bits each, two per byte) and a common E8M0 scale (one byte, a power of two) [10].

Lemma 3.7 (The original expert in the GGUF)**.** *In the GGUF, tensor* `blk.ℓ.ffn_*_exps.weight` *has ggml dimensions* $[C, R, E]$*, stored with* $ne_0 = C$ *fastest. Block* b *of row* r *of expert* x *is block* $(xR + r)B + b$ *of the tensor, and expert* x *occupies bytes* $[17xRB, 17(x + 1)RB)$*, in one piece; its biases (F32 tensor* $[R, E]$*) occupy* $[4xR, 4(x + 1)R)$*.*

Proof. A row of C MXFP4 weights is $B = C/32$ blocks of 17 bytes; an expert matrix, R rows; experts follow each other along the slowest dimension. The block index is therefore $((x \cdot R) + r) \cdot B + b$, and the RB blocks of x are consecutive. Same argument for the biases, at 4 bytes per value. ∎

The lemma means the original fingerprint is computed without reordering anything:

$$r(x) = \text{first 8 bytes of SHA-256}\big(\mathrm{gate}_x \,\|\, \mathrm{up}_x \,\|\, \mathrm{down}_x \,\|\, \mathrm{bias}_{\mathrm{gate},x} \,\|\, \mathrm{bias}_{\mathrm{up},x} \,\|\, \mathrm{bias}_{\mathrm{down},x}\big).$$

The coder computes it on the GGUF slices; the check recomputes it on the decoded expert *and* on the slices, independently.

Theorem 3.8 (Exact reconstruction)**.** *For every expert* x *and every v1.0 file written by the coder from a GGUF, the normative procedure applied to the level-0 part of* x *returns exactly the bytes of the original expert.*

Proof. Biases: stream BIAIS$_j$ is the copied bias slice, and is read back as is. *Scales*: the coder sets $\delta = (e - \mathrm{med} + 128) \bmod 256$, the decoder $e' = (\mathrm{med} + \delta - 128) \bmod 256$; in $\mathbb{Z}/256\mathbb{Z}$, $e' = \mathrm{med} + (e - \mathrm{med} + 128) - 128 = e$. The median read is the one the coder wrote (raw stream), whatever its value: the identity does not depend on it really being the median. *Decoded* δ: DELTA$_j$ is a single-table stream; by lemmas 3.1 and 3.3 and Proposition 3.4, the decoder returns the coded sequence. *Codes*:

CODES_CTX$_j$ is coded with the table of $\kappa(\lfloor w/4 \rfloor)$, a function of the already decoded δ, equal to the coder's; by Proposition 3.6, decoding returns the coded `qs`, padding included, which truncation to $16RB$ removes. *Assembly*: the original expert is the concatenation of the blocks $[e[\beta], \texttt{qs}[16\beta \dots]]$ in β order then of the biases, in matrix order; this is what `ExpertDecode::octets_origine` returns, and it is the GGUF layout by lemma 3.7. ■

3.9.3 The expert slot on the card

The card's decoder does not write the original expert but a "structure of arrays" (SoA) layout read by the GEMV (seam N, ARCHITECTURE.md § 5.6). For each matrix: the codes ($16RB$ bytes, β order) at offset 0; the scales (RB bytes) at offset $\text{al}_{256}(16RB)$; the biases ($4R$ bytes) at offset $\text{al}_{256}(\text{al}_{256}(16RB) + RB)$; matrix pitch $\text{al}_{256}(\text{bias offset} + 4R)$, where $\text{al}_{256}(x)$ rounds up to the next multiple of 256. For gpt-oss: codes 4,147,200 bytes at 0; scales 259,200 at 4,147,200; $4{,}147{,}200 + 259{,}200 = 4{,}406{,}400 = 17{,}212.5 \times 256$, hence biases at $17{,}213 \times 256 = 4{,}406{,}528$; $4{,}406{,}528 + 11{,}520 = 4{,}418{,}048 = 17{,}258 \times 256$, hence a pitch of 4,418,048 bytes and a **slot of 13,254,144 bytes** [calc.] (lot K's test `disposition_de_gpt_oss_egale_la_specification`). A weight's value is $k[\text{code}] \cdot 2^{e-128}$ with $k \in \{0, 1, 2, 3, 4, 6, 8, 12\}$ signed by bit 3: the E2M1 magnitudes are doubled, hence the exponent $e - 128$ instead of $e - 127$.

3.10 The fingerprints

Definition 3.9 (Fingerprint64)**.** For a byte sequence o, fingerprint64(o) is the little-endian integer formed by the first 8 bytes of SHA-256(o) [11] (`empreinte64` in the code).

The format carries them everywhere: the header (over its first 4,088 bytes), the end of META, of the RESIDENTS table, of TABLES, of DIRECTORY; each resident tensor; each chunk part (useful bytes); each original expert. The full SHA-256 of the source GGUF is in the header. On opening, the engine checks the header, the tables, the directory and the file size (a few MB hashed). In production, since the correction round, the guard compares the fingerprint of every part read with the directory's (previous chapter).

Proposition 3.10 (What an equality of fingerprints excludes)**.** *If fingerprint64 behaves as a random function on the errors considered (an error not crafted against SHA-256: an offset, a matrix order, a flipped bit), the probability that at least one wrong expert among n has its original's fingerprint is at most $n \cdot 2^{-64}$; for the 4,608 experts of the 120b,* $4{,}608/1.8447 \cdot 10^{19} \approx 2.50 \cdot 10^{-16}$ *[calc.].*

Proof. For a given distinct pair, the probability of the same 64-bit image is 2^{-64}; the union bound over the n experts gives $n \cdot 2^{-64}$. ■

This is no guarantee against an adversary: the 64-bit prefix breaks in about 2^{32} trials by the birthday paradox, if one chooses both contents. Nobody here crafts the GGUF's bytes.

What each check sees. A fingerprint does not have the same reach depending on what it hashes (`moteur/E-pour-le-memoire.md`, D14). The part's fingerprint sees any bit flipped in its useful bytes. The original expert's fingerprint, recomputed from the decoded expert, sees only what changes the output: a field the decoder does not read (the layer or expert of a part header, a reserved byte) changes nothing. The card decoder's counters see format faults and lanes off L, neither the raw streams nor the identity of the part. The break of bench E2 established it on the real path: twelve flipped bits, one per kind of region, **12 detected out of 12**, of which 12 by the part's fingerprint, 11 by the expert's, 9 by the counters; in production (before the guard), 9 [meas.]. The trial's decoder, on the

same copy, names who sees what: 6 chunks through lanes off L, 2 through a format error, 2 through the part header (their decoded bytes nonetheless equal the GGUF), and **only 2 through the expert's fingerprint**: one MEDIANES byte (layer 23, expert 4) and one BIAIS byte (layer 21, expert 24), all lanes on L [meas.] (`mesures/epreuve-J1.md` § 2.4). This is finding M1, which the guard closed.

3.11 The bounded-memory coder

3.11.1 Why two passes

At 15:10 on 2026-10-15, the machine had only 336 MB of available memory: another lot's tracer was reading gpt-oss-120b and held 8.7 GB private [meas.] (`mesures/moteur/F/00-seuils-declares.md`). The first version of the coder kept a layer's 128 prepared experts, i.e. $3 \times 128 \times 4{,}406{,}400 \approx 1.69$ GB on the 120b [calc.]: it would have made another's measurement page out. Lot F rewrote it in two passes by packets of $p = 2 \times$ threads experts (Figure 3.12); the format did not change by a byte.

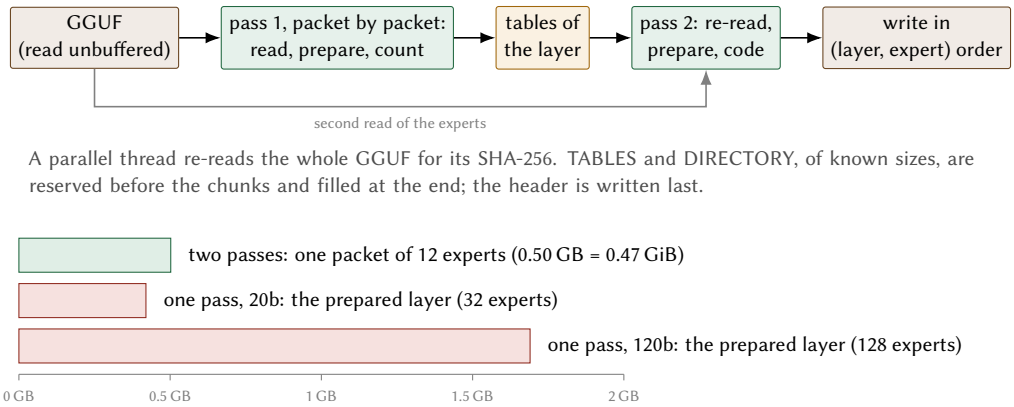


Figure 3.12: The two-pass coder of a layer, and its memory window to scale: a packet of 12 experts (≈ 0.50 GB, i.e. 0.47 GiB: 502,252,544 bytes; lot F wrote ~~0.47 GB~~) against the whole layer of the one-pass version (0.42 GB on the 20b, 1.69 GB on the 120b).

3.11.2 Same bytes, bounded memory

Notation: a layer has E experts; for matrix j, expert x gives a preparation $P_j(x)$ (medians, δ, qs, a function of its own original blocks only) and counts $N_j(x)$ (histogram of δ, histograms of the codes per class κ), integer vectors.

Proposition 3.11 (Identity of the two coders)**.** *The bytes written by the two-pass coder are those of the one-pass coder.*

Proof. (1) *Counts.* In one pass, $N_j = \sum_x N_j(x)$, summed by a parallel fold in any order; in two passes, $N_j = \sum_{\text{packets}} \sum_{x \in \text{packet}} N_j(x)$. Integer addition (u64, without overflow: at most $E \cdot RB \cdot 16 \approx 5.3 \cdot 10^8$ per bin for the 120b) is associative and commutative: both N_j are equal. (2) *Tables*: a deterministic function of N_j (rounding rule of study 03): equal. (3) *Chunks*: a chunk is a function of $(P_0(x), P_1(x), P_2(x)$, biases of x, the layer's tables); $P_j(x)$ is recomputed in the second pass from the same bytes (lemma 3.7): same chunks. (4) *Order*: (layer, expert) in both cases, a packet's results gathered in x order: same offsets, same directory, same fingerprints, same header. The original fingerprint, computed in one pass by a hasher fed matrix after matrix and in two passes by a single call on the sequence, covers the same byte sequence; SHA-256 does not depend on how calls split it. ■

The witness. The 20b recoded by the two-pass version must have the SHA-256 of the F2 file, computed by `Get-FileHash` (a path independent of our code): `e8bd6ae1…169f` on both sides; the copy with a flipped bit gives `c40f1163…7123`: the instrument falls [meas.] (`journaux/identite-20b-2passes.txt`).

Proposition 3.12 (Memory bound). *With p experts per packet, the peak of the expert loop is bounded by*

$$\underbrace{p\sum_j 17R_jB_j}_{\text{blocks read}} + \underbrace{p\sum_j 17R_jB_j}_{\text{preparations}} + \underbrace{pT_{\max}}_{\text{assembled chunks}} + \underbrace{3\cdot 4RE}_{\text{layer's biases}} + \underbrace{2\times 16\,\text{MiB}}_{\text{bounce, chunk copy}},$$

where only the biases term depends on E.

Proof. A packet is read, prepared (at most one preparation in flight per expert of the packet, $(16+1)R_jB_j$ bytes per matrix), coded and written before the next is read; a packet's chunks are held until written; the layer's biases are read once; the unbuffered read goes through an aligned bounce buffer, and the copy of residents through 16 MiB pieces. Nothing else depends on the size of a tensor. ∎

The calculation, for gpt-oss and $p = 12$. $\sum_j 17R_jB_j = 3\times 17\times 259{,}200 = 13{,}219{,}200$ bytes. Blocks read and preparations: $2\times 12\times 13{,}219{,}200 = 317{,}260{,}800$; chunks: $12\times 12{,}251{,}136 = 147{,}013{,}632$; biases: 34,560 E, i.e. 1,105,920 (20b) and 4,423,680 (120b); bounce and piece: 33,554,432. Total on the 120b: 502,252,544 bytes [calc.]. Lot F wrote ~~≈ 0.47 GB~~: that is 0.47 *GiB*; in bytes, the bound is 0.50 GB (the writer's correction, 2026-10-16: unit). The model-dependent term weighs only 3.3 MB of difference between the two. The one-pass version additionally kept $3E\times 4{,}406{,}400$ bytes of preparations: 0.42 GB (20b), 1.69 GB (120b). Hence the threshold, declared before launching F3: 120b peak ≤ 1.25× 20b peak, the 25 % margin covering the working-set noise (allocator, threads), declared and not measured.

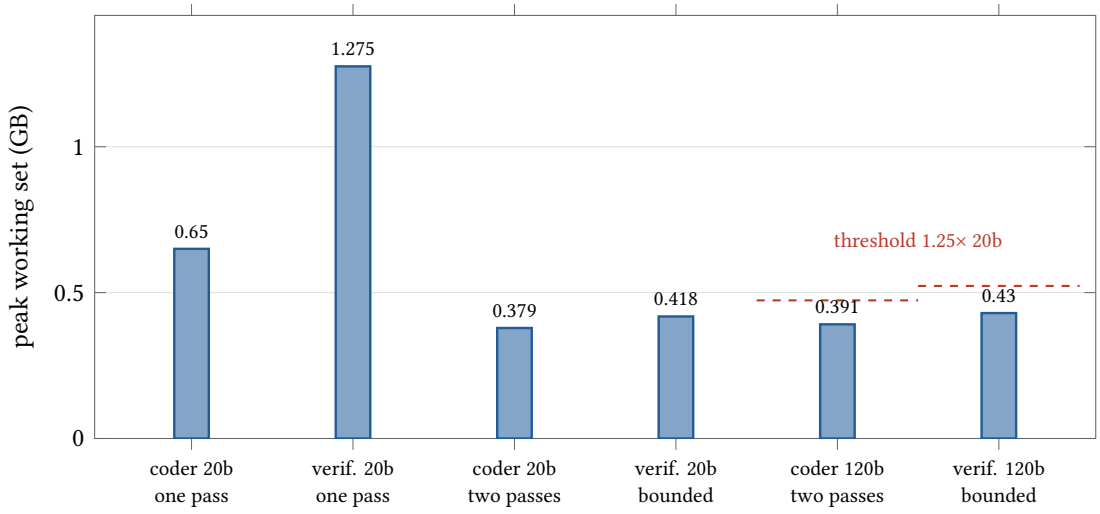


Figure 3.13: Peak working set of the coder and the check, one pass against two passes (sampled every 0.5 s by the launcher). The 120b, five times larger, is coded and checked within the same window as the 20b: ratios 1.033 (coder) and 1.028 (check), below the 1.25 threshold [meas.]. Data: `mesures/moteur/F/csv/pointes-memoire.csv`.

The price, and a unit wrong twice. The experts are read twice: 2 × 61.0 GB on the 120b instead of 61.0 GB. At the disk's throughput for random 4 MiB reads at queue depth 4 (2.58 GB/s, study 01; ~~sequential throughput~~: sequential reads of 4 MiB blocks run there at 1.04 to 1.63 GB/s), the second read

costs 61.0/2.58 = 23.6 seconds [calc.]. Lot F had written ~~≈ 24 more minutes~~ in its derivations, and struck it itself ("the unit was wrong"); the head comment of `moteur/bins/trc-codeur/src/coder.rs` still says "~~on the order of 25 more minutes of disk~~": the same error, found by the writer on 2026-10-16 and reported to lot F (the file is not his). Coding the 120b took 1,153 s in all: the second read weighs about 2 % of it (23.6/1,153 = 2.05 % [calc.]), the processor's rANS does the rest.

> **What the witness had not covered: error 87**
>
> The first two-pass version computed the GGUF's SHA-256 in 16 MiB pieces read unbuffered, and read again after the short read at the end of the file, at an offset no longer aligned: Windows refuses (error 87, invalid parameter). The defect was seen on the 20b, corrected (a short read only happens at the end of the file; the caller compares the total with the size), and a fourth break of the unbuffered-read witness now covers it: four breaks out of four fall, the restored test holds [meas.] (`journaux/casser-lecture.log`).

3.12 C1 on 768 and 4,608 experts

3.12.1 The first path: five checks per expert

The check (`trc-codeur verifier`) does not trust the coder: it re-reads the `.trc` through `trc-format` (all fingerprints), decodes each expert through `trc-codec`, and compares with the slices read from the GGUF. Five checks per expert: (1) the fingerprint of the part's useful bytes equals the directory's; (2) decoding succeeds (lanes on L, words consumed); (3) the reconstructed original bytes equal the GGUF slices byte for byte; (4) the decoded expert's fingerprint equals the directory's original fingerprint; (5) the fingerprint of the GGUF slices equals it too.

In each break, the checks "part", "bytes" and "decoded" fall; "decoding" holds (a corrupted median changes $e = \text{med} + \delta - 128$, not the rANS streams); "GGUF" holds (the source is untouched). This is the expected pattern, and it says that the five checks are not redundant.

The rate, word by word. On the 120b, study 03's accounting (original MXFP4 blocks against the MEDIANES, DELTA and CODES_CTX streams as stored) gives codes 3.820169, δ 0.031609, medians 0.002778, i.e. **3.854555 bits per weight**; with the tables (224,656 bytes), 3.854571 [meas.]. The medians cost exactly one byte per row of 2,880 weights: 8/2,880 = 0.0027778 [calc.]. The ratio r_{03} reads directly: 17 original bytes for 32 weights make 17 × 8/32 = 4.25 bits per weight, and 4.25/3.854555 = 1.102592, the published r_{03} [calc.]. The ratio of whole chunks (headers, biases, padding included) is 13,253,760/12,026,060.4 = 1.102087 [calc.]. The twelve F1 tensors of the 20b equal the arbiter's measurement to the millionth, which makes the production coder a second path of F1 (`csv/f1-contre-trc-20b.csv`).

3.12.2 The second path: no line in common

The first path reads the GGUF with the coder's own function (`gguf::lire_dans`): a systematic read error (reading another expert's slice, a wrong offset) would be common to the coder and to the check, and C1 would hold wrongly. Lot F therefore wrote a second path (`mesures/moteur/F/second_chemin_c1.py`): Python, standard library only (`hashlib` on OpenSSL 3.0.18, not the `sha2` crate), written from § 4 alone and the GGUF v3 specification, without re-reading `trc-format`, `trc-codec` or the coder's reader; the GGUF read unbuffered; the tensor offsets redone a third time

Quantity (threshold declared beforehand)	gpt-oss-20b	gpt-oss-120b
Exact experts, five checks each (0 wrong)	768 of 768	**4,608 of 4,608**
Failed checks: part, decoding, bytes, decoded, GGUF	0, 0, 0, 0, 0	0, 0, 0, 0, 0
Residents equal, bytes and fingerprint	315 of 315	471 of 471
GGUF SHA-256, recomputed by the coder then the check	`27cd6c43…5901`	`582bd40f…622d`
r_{03} (within 0.1 % of study 03: 1.1037 and 1.1026)	1.103732	1.102592
Bits per weight: codes, δ, medians, total	3.8135; 0.0343; 0.0028; 3.8506	3.8202; 0.0316; 0.0028; 3.8546
Chunks: min, mean, max (bytes)	11,837,440; 12,013,664; 12,144,640	11,763,712; 12,026,060; 12,251,136
Size of the `.trc` (arbiter's mark at 1 %)	11,157,405,696 (+0.52 %)	57,730,662,400 (+0.14 %)
In-memory break (one median corrupted)	only (7, 13) wrong	only (29, 101) wrong

Table 3.3: Criterion C1 and the rates, against the thresholds declared before the runs (`mesures/moteur/F/verdicts.md`; logs `verifier-20b.json`, `verifier-120b.json`, `comparer-03-120b.json`). On a real copy of the 20b with a single byte changed (offset 8,818,286,848, 120 → 121, seen by `cmp -l`), exactly expert (17, 29) falls, exit code 1.

by llama.cpp's own GGUF reader (`gguf_init_from_file` of `ggml-base.dll` b11515, MIT) [12]. Figure 3.16 shows how the two paths meet.

Proposition 3.13 (The second path closes the gap of a shared reading)**.** *Let $G(x)$ be the bytes of the original expert, $D(x)$ the decoded expert, $r(x)$ the directory's fingerprint, h fingerprint64. If $h(D(x)) = r(x)$ (first path) and $h(G_2(x)) = r(x)$ (second path, G_2 read by another program), then $D(x) \neq G_2(x)$ would require a collision of h; and a read error of the coder passes only if G_2 errs in exactly the same way.*

Proof. The two equalities give $h(D(x)) = h(G_2(x))$; Proposition 3.10 bounds the probability of a collision by $4{,}608 \cdot 2^{-64}$ over the model. A systematic error of `gguf::lire_dans` would make $G_1(x) \neq G(x)$; the coder would have stored $r(x) = h(G_1(x))$, and the check would have found $D = G_1$, wrongly. The second path compares $r(x)$ with $h(G_2(x))$: two programs without a common line, one of which (llama.cpp) is the reference for the GGUF format, would have to err in the same way. ■

What it measured. On both models, on mains power: 0 discrepancies between the Python reader and llama.cpp (459 and 687 tensors); data zone tiled without gaps; SHA-256 by `hashlib` equal to the download register (third computation, the first outside `sha2`); header, META (36 pairs byte for byte, plus 3), TABLES (288 and 432), DIRECTORY without discrepancy; **0 wrong expert fingerprints out of 768, then out of 4,608**; BIAIS and MEDIANES equal byte for byte to their recomputed sources [meas.] (`journaux/second-chemin-{20b,120b}.json`). The medians, recomputed: for $B = 90$, the median is `tri[45]`, the 46th smallest of a row's 90 scale bytes.

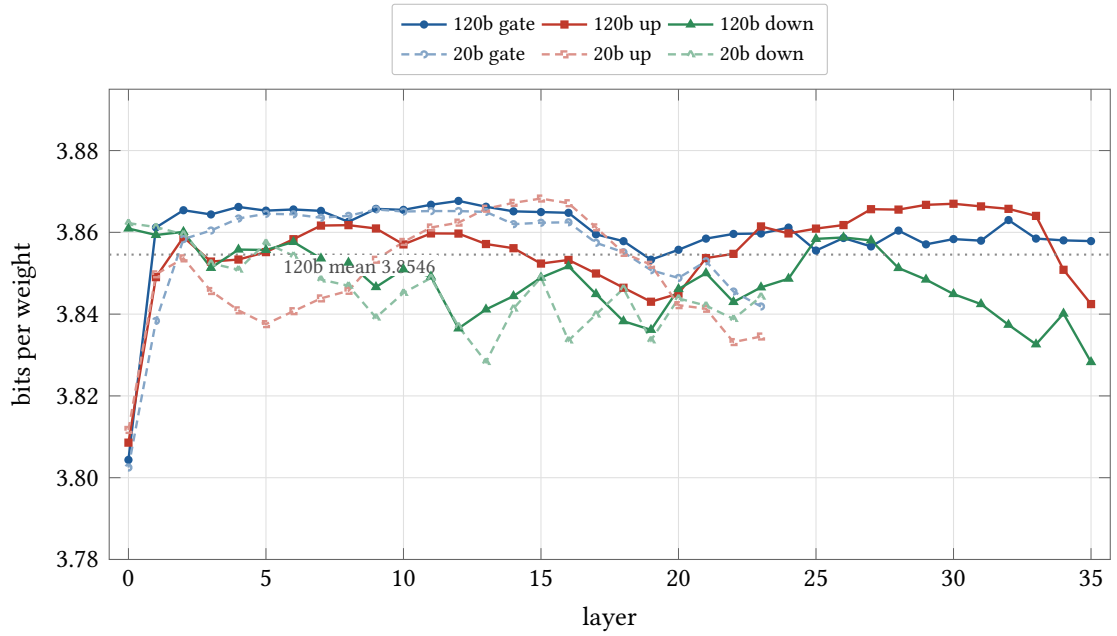


Figure 3.14: Bits per weight of each expert tensor (codes, δ and medians), layer by layer, for both models. The rate is regular across layers and across gate, up and down, within 0.05 bit, except layer 0 of the 120b (gate and up near 3.80). Data: `mesures/moteur/F/csv/taux-par-tenseur-{120b,20b}.csv`.

> **What the witness caught in its author**
>
> The first version of the second path read the GGUF version number on the magic (cursor at 0 instead of 4). Its growing re-read of the head, made for long metadata, then grew without bound up to the whole file: 9.6 GB of RAM on a 16 GB machine, before failing. Corrected (cursor at 4, re-read bounded to 256 MiB). Lesson: an unbounded fallback loop turns a parsing error into a load for the whole machine (`moteur/REPRISE-F.md`).

One link was still missing: "chunk bytes → decoded expert" had only a processor decoder on the 120b. Lot K's card decoder redid it: **4,608 of 4,608** experts of the real file equal to `trc-codec` and to the directory (`mesures/moteur/K/k1-trc-gptoss120b.json`) [meas.].

3.13 The trial's independent decoder

The specification foresaw a gesture to know whether it was true (`ARCHITECTURE.md` § 8.1): a judge who did not write it codes a Python decoder *from § 4 alone*, without reading `trc-format` or `trc-codec`, and must give back, to the bit, chunks drawn at random. A failure would have been a defect of the specification or of the coder.

What the tool does. `mesures/moteur/epreuve-J1/decodeur_s4.py` (Python 3.13, numpy) contains its own GGUF v3 reader, checks the header field by field, META, RESIDENTS, TABLES and DIRECTORY, then decodes each drawn chunk with the rANS of § 4.9.2, *vectorised over the tiles* (127 tiles × 32 lanes in parallel, 1,024 sequential steps per stream, 0.8 s per expert), the per-lane table and the reconstruction. The expert is compared with the directory's original fingerprint and with the GGUF bytes at the offset of lemma 3.7; two checks the decoder does not require are added (median

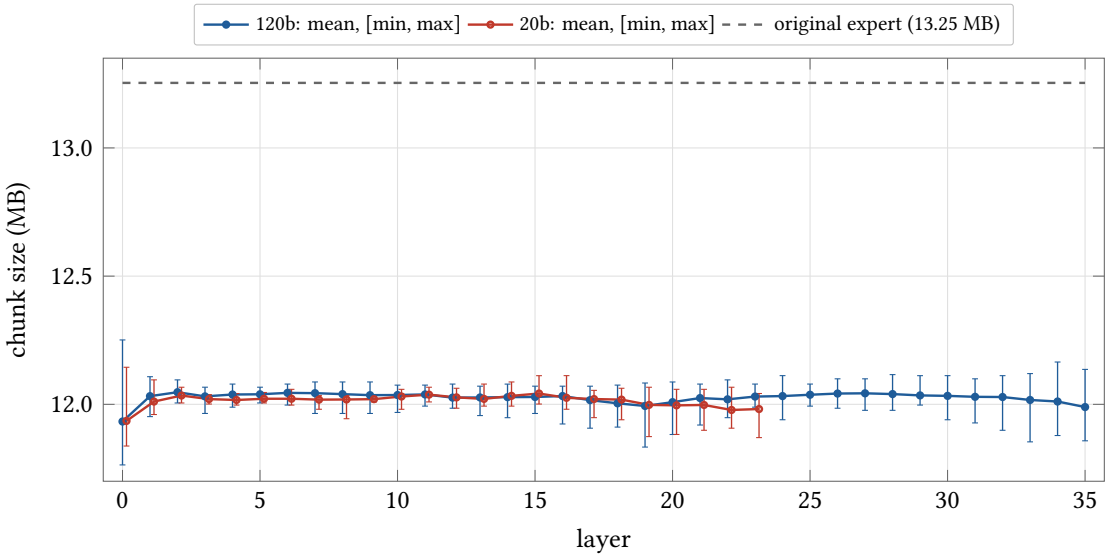


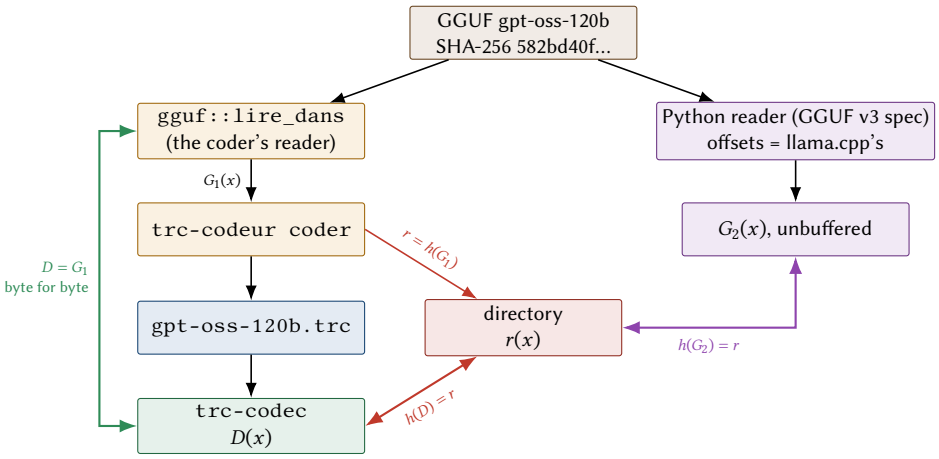
Figure 3.15: Chunk size per layer (mean, minimum and maximum over the layer's experts), against the original MXFP4 expert (13,253,760 bytes). The largest chunk of the 120b (12,251,136 bytes, layer 0) sets the size of the engine's arrival buffers. Data: `mesures/moteur/F/csv/troncons-par-couche-{120b,20b}.csv`.

equal to `tri[B/2]`, end of each tile equal to the start of the next).

Listing 3.3: The core of the judge's vectorised rANS (`mesures/moteur/epreuve-J1/decodeur_s4.py`, `decoder_flux`, geometry 1, shortened; comments in French): the rule "for $l = 0$ to 31 in order" becomes a cumulative sum along the lane axis (Proposition 3.5).

```
for ml in range(N):                          # macro-pas local, toutes les tuiles ensemble
    m = tuiles * N + ml
    act = np.nonzero(m < total_m)[0]         # tuiles encore actives
    w = m[act, None] * 32 + voies[None, :] # mot de sortie de chaque voie
    ctx = kappa[w >> 2]                      # table par voie : bloc = w / 4
    xa, pa = x[act], p[act]
    for k in range(4):                       # sous-pas
        e = luts[ctx, (xa & 4095).astype(np.int64)].astype(np.uint64)
        s, f, b = e & 0xFF, ((e >> 8) & 0xFFF) + 1, e >> 20
        y = f * (xa >> 12) + b
        besoin = y < L_RANS                  # voies qui renormalisent
        cs = np.cumsum(besoin, axis=1)       # rang de chaque lecture dans la tuile
        idx = np.clip(pa[:, None] + cs - 1, 0, max(W - 1, 0))
        xa = np.where(besoin, (y << 16) | mots[idx], y)
        pa = pa + cs[:, -1]
        sortie[4 * w + k] = s.astype(np.uint8)
    x[act], p[act] = xa, pa
```

The results. **20 chunks out of 20** drawn in `gpt-oss-120b.trc` (seed 20261016) equal to the GGUF to the bit, and 20 out of 20 in the 20b; 0 lanes off L, 0 tiles whose end differs from the start of the next; 172,800 rows out of 172,800 whose median equals `tri[B/2]`; the 471 residents of the 120b and the 315 of the 20b equal to the GGUF byte for byte; header, META (39 = 36 + 3 pairs), TABLES (432 tables summing to 4,096), DIRECTORY (4,608 contiguous entries from the start of CHUNKS to the end of the file) and `.vocab.gguf` companions conforming [meas.] (`mesures/epreuve-J1.md`



Left: the first path (`trc-codeur verifier`, 4 608 experts, five checks each). Right: the second path (`second_chemin_c1.py`), which shares no line with lot F and whose tensor offsets agree with llama.cpp's own GGUF reader. The two paths meet only in the directory; together they give $h(D(x)) = h(G_2(x))$ for every expert (Proposition 3.13). The decoding link alone was redone by two other decoders: lot K's card decoder (4 608/4 608) and the trial's Python decoder (20 drawn chunks).

Figure 3.16: The two paths of C1 on the 120b.

§ 2.2; `journaux/s4-gptoss120b-20.json`).

3.13.1 What writing the decoder taught about the text

"Nothing was missing to decode to the bit", the judge writes. But a second implementer had to guess three things, which lot F had settled on its side among nine precisions (Table 3.6). None changes a byte of the written files; all await the arbiter carrying them into the specification.

3.14 Versions, and what is reserved for J2

Version 1.0 (J1) has one level and the CODES_CTX kind; a 1.x reader reads any version 1.y lower than or equal to its own, and refuses 2.x. Version 1.1 (J2) is reserved to the bit: two levels, BASE_CTX (sign and magnitude class, in a nibble, per-lane table) at level 0, COMPL_CTX (rank of the magnitude within its class, per-symbol table, up to 192 contexts, skips for the single-magnitude class) at level 1, and the CENTRES tables. The arbiter's measurement F1, reported in Volume I, fixed its cost (Table 3.7); the switch will require recoding the file once.

What could be wrong

- **The trial's decoder decoded only 40 chunks** (20 per model) and 13 named ones; C1 over the 4,608 experts rests on lot F's two paths and on lot K's card decoder.
- **The second path is written by lot F**: its author knows the coder's choices. Four of its structure checks depend on them (precisions 1, 2, 4, 5); none touches the expert fingerprints, which depend only on § 4.7, 4.11, 4.12 and the GGUF specification.
- **The Python reader and the coder share one reading of § 4.11** (blocks then biases, gate, up, down order; `ne[0]` = columns): were it wrong with respect to llama.cpp, both would be wrong together. The witness that validates it is lot R's f64 reference, read from the GGUF and judged against llama.cpp.

Break	Expected	20b	120b
code bit	that expert, fingerprint only	(3, 7): fingerprint only	(3, 7): fingerprint only
recomputed median	that expert, MEDIANES only	(5, 9): MEDIANES only	(29, 101): MEDIANES only
bias bit	that expert, fingerprint and BIAIS	(11, 13)	(35, 127)
shifted reader	1 discrepancy against llama.cpp	1 (`output.weight`)	1
SHA-256	different	different	different
slice of $x + 1$	all experts read wrong	768 of 768 (and MEDIANES 768)	256 of 256 (layers 0 and 1, declared)

Table 3.4: The six breaks of the second path: each falls exactly where it aims, and the other experts stay correct (`verdicts.md`; `csv/second-chemin-c1.csv`). The shift also brings down the medians (they come from the same slices) and leaves the biases correct (they are not shifted): each check sees its own byte.

Break (chunk layer 12, expert 54)	Lanes off L	Wrong bytes	The second holds
one bit flipped in the words of CODES_CTX	32 (one tile)	31,122	yes
two frequencies of a table swapped	4,064	3,624,195	yes
one median changed by one unit	**0**	90 (one row of scales)	yes
one LUT slot corrupted	4,032	3,262,607	yes

Table 3.5: The breaks of the trial's decoder (`mesures/epreuve-J1.md` § 2.3). The corrupted median leaves all lanes on L and changes exactly the 90 scales of one row: only the expert's fingerprint sees it. The flipped bit corrupts a single tile: tiles are independent, and the error does not cross the boundary.

- **The coder's memory peaks** are working sets sampled every 0.5 s: a shorter peak may escape; the machine was shared.
- **The collision bound** assumes SHA-256 behaves as a random function on errors that are not crafted; it is worth nothing against an adversary who chooses the bytes.
- **The rate of v1.1** is measured on 12 tensors of the 20b by the arbiter's tool, not by the production coder, which does not write it yet; if that coder exceeds 1.004× the single level on the 120b, the arbiter decides again.
- **The copy break** was done only on the 20b; on the 120b, the break is in memory (same check code, one bit flipped after reading).

	Precision (delegated answer of 2026-10-15, reversible)	Seen by the judge too
1	a section's length is useful: fingerprint included, padding excluded	yes (m4.1)
2	the residents table's fingerprint is in the last 8 bytes of the section	yes (m4.2)
3	a part's useful bytes end with its last stream	—
4	fixed context: 0 for DELTA, 0xFF otherwise	—
5	bytes produced by a rANS stream = its P field (padding included); by a raw stream, its length	—
6	a padding block has $\kappa = 0$ (Section 3.7)	—
7	structures of § 5.3 extended (fields and functions added, nothing changed)	—
8	table rounding rule: study 03's (not normative, the frequencies are stored)	—
9	order of the raw file's streams: for each j, its three streams in a row; empty TABLES	—
m4.3	"the end of tile t is the start of tile $t + 1$": true on 100 % of tiles, not normative	second free check (Proposition 3.4)
T2	size of the LUTs: § 4.6 writes 6.9 MB; it is 7,077,888 bytes, 7.08 MB or 6.75 MiB	found by the judge of Volume II (m17)

Table 3.6: The precisions of the format's text awaiting the arbiter: lot F's nine (moteur/REPRISE-F.md), the trial judge's three imprecisions (finding m4, mesures/epreuve-J1.md § 2.5) and the one of the judge of Volume II (T2, memoire/tome2/JUGEMENT.md, m17).

Variant (12 tensors of gpt-oss-20b)	Total	Codes or base	Complement	Ratio to A_ctx
A_flux (three class streams, study 03)	3.8434	3.8059	—	1.00008
A_ctx (per-lane table, v1.0)	**3.8431**	3.8056	—	1
B_cond (complement by κ and class)	3.8591	2.6066	1.2149	1.00414
B_mag (complement by class only)	3.8733	2.6066	1.2291	1.00785
B_riche (v1.1)	**3.8550**	2.6066	1.2107	**1.00308**

Table 3.7: Bits per weight of the arbiter's measurement F1 (ARCHITECTURE.md § 3.3; decisions/journaux/arbitre/decoupe-gptoss20b.json), all streams decoded and compared byte for byte. The single format of J2 will cost exact mode 0.31 % of bytes, below the declared threshold of 0.4 %.

Chapter 4

The path of the bytes

Where we start, what this chapter establishes, what remains

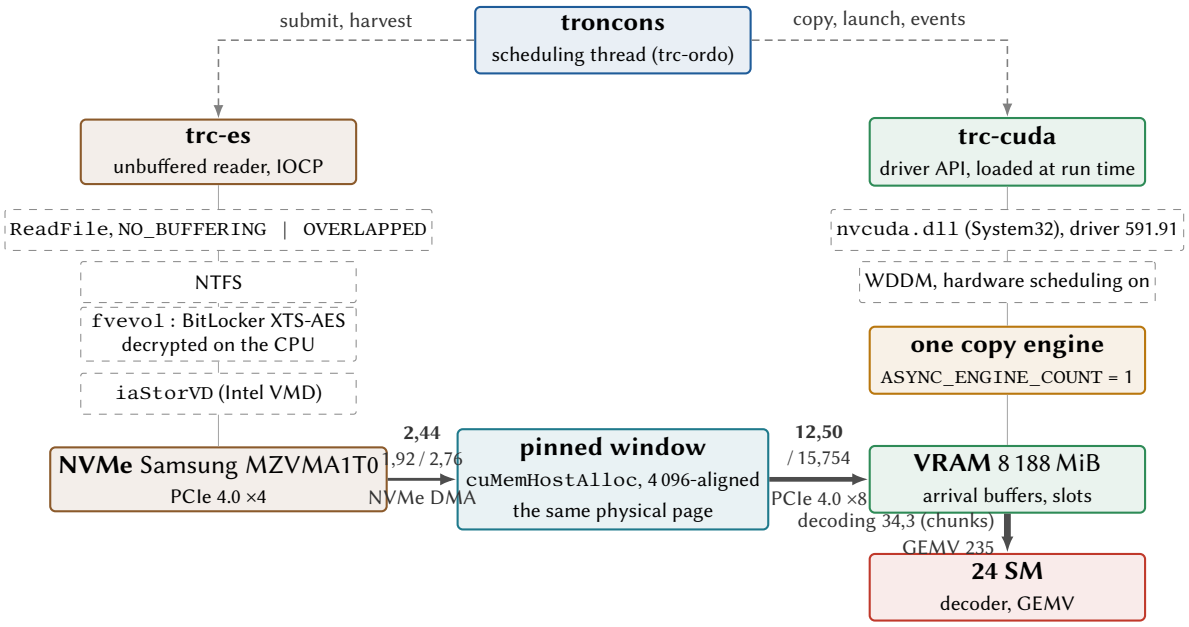
At the start, the architecture decision held the byte path to be feasible under CUDA on the strength of three measurements made by other means: a DMA at 12.2 GB/s from a Vulkan host buffer, pinned memory 21 % faster and a copy–compute overlap measured through llama.cpp's CUDA runtime called from Python, and a disk at 2.58 GB/s in 4 MiB blocks. Three things had been measured nowhere: pinning several gigabytes under WDDM on a shared 16 GB machine; reading the disk without buffering *directly* into the memory that the DMA reads; and what it costs, at every layer, for the host to read the router. **This chapter establishes** two libraries of our own, with no CUDA library linked (`trc-cuda`, the driver API loaded at run time; `trc-es`, unbuffered reading through a completion port); the three trials that answer the three questions, each with its witness, its break and its declared threshold; a fact about the machine nobody had noted (the card has a single copy engine under WDDM); the end-to-end bench that carries the 768 experts of gpt-oss-20b and the 4 608 of gpt-oss-120b bit for bit; and a discovery: **on this machine, the throughput of an unbuffered read depends on its length alone**, in two regimes, and every chunk length falls in the slow one. Reading a fixed fast length (12 MiB) yields ×1.335. **What remains** is the arbiter's decision on the read length, the mechanism of the two regimes, pinning 11 GB on a quiet machine, and the length map of the 120b file. Sources: `moteur/E-pour-le-memoire.md`, `moteur/REPRISE-E.md`, `mesures/moteur/E/journaux/`, `mesures/moteur/E/csv/`; for the engine, `moteur/O-pour-le-memoire.md` and `mesures/moteur/O/csv/`.

4.1 What the path must do

A cache miss, in the engine, is a gpt-oss expert missing from VRAM at the moment the layer's router asks for it. Its *chunk* (the coded part of its three matrices and their biases, 12 013 664 bytes on average for the 20b) must then be brought from the `.trc` file to a slot on the card, decoded, before the layer's matrix–vector products read it. In between lie four stages crossed in order: the disk, host memory, the PCIe link, and the VRAM with its compute units (Figure 4.1).

The budget is read from the token equation of Volume I: in the calibrated LRU scenario, the disk time of a 120b token is 157 ms (07 § 3.3). Every stage must therefore either hide behind the disk or cost little compared with it. Three questions had no measured answer, and lot E begins with them:

1. **E0 (a)**: can several gigabytes of host memory be pinned under WDDM, on a 16 GB machine shared by five lots?
2. **E0 (b)**: can an unbuffered read (`FILE_FLAG_NO_BUFFERING`) write directly into pinned memory, and the DMA then read it at full speed, bytes equal?
3. **E0 (c)**: what does it cost the token for the host to read the router's four indices once per layer, 36



GB/s, measured here. Disk: 12 MiB reads at random offsets, QD 4, quiet machine (E1) / whole chunks read at their own length (E2, disk alone, slow regime) / 12 MiB reads at the chunk offsets (E2-V12, disk alone). Link: DMA of 12 MiB from pinned memory (E0 b) / computed ceiling of PCIe 4.0 ×8. Decoding: one expert at a time (E2, stages). GEMV: batch of 12 MXFP4 matrices (K2).

Figure 4.1: The stack of the byte path, as built. On the left, reading (`trc-es`, then the Windows layers an unbuffered read goes through: the file system, the BitLocker filter that decrypts every byte on the CPU, the VMD controller driver); on the right, the card (`trc-cuda`, the driver, the single copy engine). In the middle, the pinned window: the same physical page is written by the NVMe's DMA and read by the card's, with no copy by the CPU.

Data: `mesures/moteur/E/journaux/e1-2026-10-15-seance3/resume.json`, `e2-2026-10-15-seance1/resume.json`, `e2-2026-10-15-v12/resume.json`, `2026-10-15-seance3/resume.json`; GEMV: `mesures/moteur/K/csv/k2-debit-passes.csv`.

times per token on the 120b?

4.2 `trc-cuda`: the driver API, with no library linked

The choice. The engine ships no CUDA runtime DLL: the installed driver (591.91, CUDA 13.1) is enough. `trc-cuda` loads `nvcuda.dll` at run time, through `LoadLibraryExW` restricted to the `System32` folder (no DLL dropped in the current folder can pass itself off as the driver), then reads each entry point by its exact exported name. The names are those that CUDA 13.1's `cuda.h` selects through its `#defines`: `cuMemAlloc` is exported as `cuMemAlloc_v2`, `cuMemcpyHtoDAsync` as `cuMemcpyHtoDAsync_v2` [13]. A missing symbol makes loading fail *with its name*: a driver that is too old says so, instead of crashing three calls later (Listing 4.1).

The primary context. `cuCtxCreate` became `cuCtxCreate_v4` in CUDA 13; the primary context has kept its interface [14]. `trc-cuda` therefore retains the primary context, and every object it creates (stream, event, device memory, pinned memory, module) holds a counted reference to an internal object that releases the context only after the last of them. The context is made current when

Listing 4.1: Loading the table of entry points (`moteur/crates/trc-cuda/src/api.rs`, lines 60 to 76, commit `014bd11`). The `table!` macro generates one field per entry point, typed after `cuda.h`; `symbole` returns an error that names the missing symbol. Comments are in French in the source.

```
            unsafe fn charger() -> Result<Api, String> {
                let nom_dll: Vec<u16> = "nvcuda.dll\0".encode_utf16().collect();
                // SAFETY : chaîne terminée par un zéro ; recherche limitée à
    System32, où le
                // pilote d'affichage installe nvcuda.dll (pas de DLL plantée
    dans le dossier courant).
                let module = unsafe {
                    LoadLibraryExW(nom_dll.as_ptr(), std::ptr::null_mut(),
    LOAD_LIBRARY_SEARCH_SYSTEM32)
                };
                if module.is_null() {
                    return Err("nvcuda.dll introuvable dans System32 : aucun
    pilote NVIDIA \
                                installé, ou pilote trop ancien".into());
                }
                let symbole = |nom: &str| -> Result<unsafe extern "system" fn()
    -> isize, String> {
                    let c = format!("{nom}\0");
                    // SAFETY : module valide, nom terminé par un zéro.
                    unsafe { GetProcAddress(module, c.as_ptr()) }
                        .ok_or_else(|| format!("symbole {nom} absent de nvcuda.
    dll (pilote trop ancien ?)"))
                };
```

needed, thread by thread, and a *generation* tells a context released and retaken from a context that stayed current. The witness `duree_de_vie_du_contexte` exercises this order.

A fact about the machine. The attribute CU_DEVICE_ATTRIBUTE_ASYNC_ENGINE_COUNT is **1** on the RTX 4060 Laptop under WDDM [meas.]: the card has a single copy engine for both directions [15]. The first ordering witness of `trc-cuda` showed it unwittingly: it looked for a race between a host-to-device copy and a device-to-host copy placed on two streams without an event, and never saw it in 20 trials. Both copies went through the same queue. Redone with a kernel that reads the VRAM during the copy, it sees the race at the first trial: 66 393 069 bytes out of 67 108 864 read before they arrived [meas.]. Section 4.5 draws the consequences.

4.3 `trc-es`: reading without buffering, through a completion port

An unbuffered read bypasses the Windows cache: the controller writes straight into the caller's buffer. It requires an offset, a length and an address that are multiples of the sector [6]; the `.trc` format aligns everything on 4 096 bytes for that reason. With FILE_FLAG_OVERLAPPED, each read is handed to the system without waiting, and its completion arrives as a packet in an *I/O completion port* (IOCP), which a thread harvests [16].

4.3.1 The structures, bit for bit

The reader keeps at most q reads entrusted to the system (its *depth*); beyond that, requests wait in an internal queue, and each harvest entrusts new ones. Figure 4.2 gives the automaton of a request,

Table 4.1: A `trc-es` read in memory. `Vol` is `#[repr(C)]`, heap-allocated, at a stable address until harvested: the address of the OVERLAPPED returned by the port is that of the `Vol`, and serves as the key into the table of entrusted reads.

structure	bytes	field	content
`Vol`	0–7	`Internal` (u64)	NTSTATUS of the read, written by the system
	8–15	`InternalHigh` (u64)	bytes transferred
	16–19	`Offset` (u32)	offset, low 32 bits
	20–23	`OffsetHigh` (u32)	offset, high 32 bits
	24–31	`hEvent`	null (the port replaces the event)
	32–39	`jeton` (u64)	identifier returned to the caller
	40–	`depart`	instant of hand-over to the system
`Requete`		token, offset, length, destination	u64, u64, u32, pointer; the last three multiples of 4 096
`Achevement`		token, bytes, error, duration	u64, u32, Windows code or none, u32 in µs (from hand-over to harvest)

errors included.

4.3.2 Mutual exclusion, proved

The danger of a completion port is the race between the thread that entrusts a read and the one that harvests it: if the packet arrives before the launcher has recorded the read in its table, the harvest looks up a missing key.

Lemma 4.1 (Every key returned by the port is in the table)**.** *In `trc-es`, every OVERLAPPED address returned by `GetQueuedCompletionStatusEx` is in the table of entrusted reads when the harvest looks it up.*

Proof. The call to `ReadFile` and the insertion of the key into the table happen under the same lock (`lancer` is called with the lock held, by `soumettre` or by `recolter`). The harvest waits on the port *outside* the lock, then takes the lock before any lookup. Take a packet that arrives before its key is inserted: at that moment the launcher holds the lock (it is between `ReadFile` and the insertion); the harvest, once woken, therefore waits for that lock, which the launcher releases only after the insertion. In every case the lookup follows the insertion. A key is removed only by the harvest of its own packet, and an OVERLAPPED produces only one packet. ∎

The same reasoning underpins dropping the reader: an OVERLAPPED belongs to the system from `ReadFile` until its packet is dequeued [16]. The destructor therefore cancels everything (`CancelIoEx`), then waits for each packet before freeing anything; if no packet comes within 10 s, it prefers to *leak* the OVERLAPPEDs rather than free memory the system might still write.

Listing 4.2: `soumettre`, `lancer` and `recolter` (`moteur/crates/trc-es/src/lib.rs`, lines 170 to 263, commit `014bd11`). The lock covers `ReadFile` and the insertion; the wait on the port happens outside the lock (lemma 4.1).

```
    /// Soumet une lecture. Refuse (sans rien lancer) un décalage, une longueur
    ou une destination
    /// qui ne sont pas multiples de 4 096, ou une longueur nulle.
```

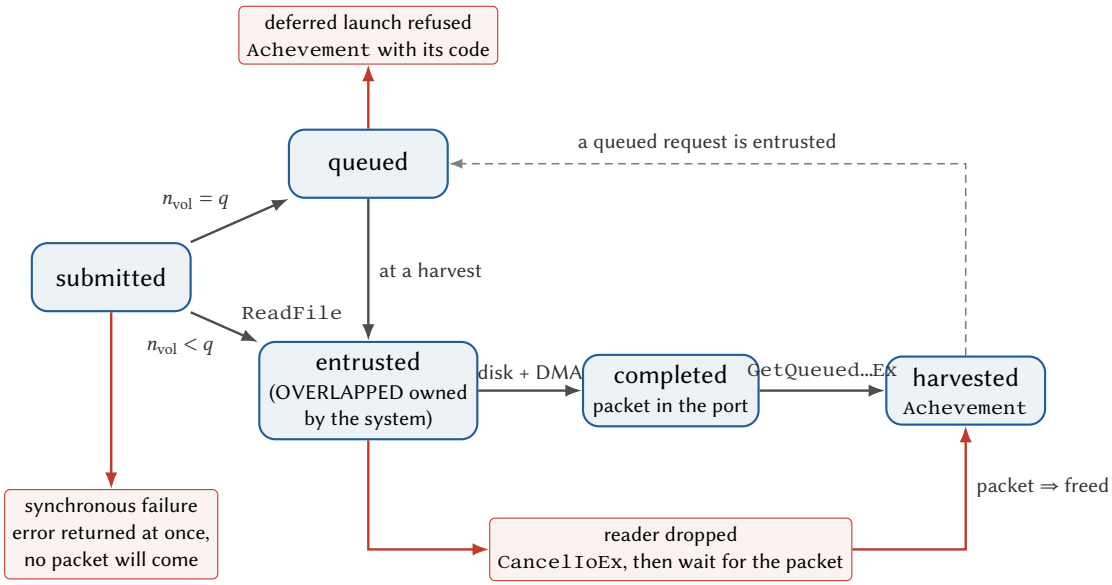


Figure 4.2: The automaton of a `trc-es` request. A synchronous failure of `ReadFile` (a code other than ERROR_IO_PENDING) will never produce a packet: the error is returned at once. An immediate success still produces a packet (`FILE_SKIP_COMPLETION_PORT_ON_SUCCESS` is not set), so everything entrusted ends up in the port. When the reader is dropped, the OVERLAPPED is freed only after its packet.

```
    pub fn soumettre(&self, r: Requete) -> Result<()> {
        if r.decalage % ALIGNEMENT != 0 || r.longueur as u64 % ALIGNEMENT != 0 ||
     r.longueur == 0 || r.destination as u64 % ALIGNEMENT != 0 {
            return Err(Erreur::Usage(format!(
                "requête {} non alignée sur 4 096 : décalage {}, longueur {},
    destination {:p}",
                r.jeton, r.decalage, r.longueur, r.destination
            )));
        }
        let mut etat = self.etat.lock().unwrap();
        if etat.vols.len() < self.profondeur && etat.attente.is_empty() {
            self.lancer(&mut etat, r)
        } else {
            etat.attente.push_back(r);
            Ok(())
        }
    }

    /// Confie une requête au système. Sous le verrou.
    fn lancer(&self, etat: &mut Etat, r: Requete) -> Result<()> {
        let mut vol = Box::new(Vol { ov: unsafe { std::mem::zeroed() }, jeton: r.
    jeton, depart: Instant::now() });
        vol.ov.Anonymous.Anonymous.Offset = r.decalage as u32;
        vol.ov.Anonymous.Anonymous.OffsetHigh = (r.decalage >> 32) as u32;
        let cle = &mut vol.ov as *mut OVERLAPPED;
        // SAFETY : OVERLAPPED à adresse stable (boîte gardée jusqu'à la récolte)
     ; destination selon
        // le contrat de l'appelant.
        let ok = unsafe { ReadFile(self.fichier, r.destination, r.longueur, std::
    ptr::null_mut(), cle) };
        if ok == 0 {
```

```
            let code = unsafe { GetLastError() };
            if code != ERROR_IO_PENDING {
                // Échec immédiat : aucun paquet de complétion ne viendra.
                return Err(Erreur::Systeme { appel: "ReadFile", code });
            }
        }
        // Succès immédiat ou en attente : dans les deux cas un paquet arrivera sur le port (le
        // drapeau FILE_SKIP_COMPLETION_PORT_ON_SUCCESS n'est pas posé).
        etat.vols.insert(cle as usize, vol);
        Ok(())
    }

    /// Récolte les lectures achevées, en attendant au plus `attente_ms` la première (0 : sans
    /// attendre ; `u32::MAX` : sans limite). Ajoute les achèvements à `sortie` et relance la file.
    /// Rend tout de suite si rien n'est en vol.
    pub fn recolter(&self, attente_ms: u32, sortie: &mut Vec<Achevement>) -> Result<()> {
        {
            let mut etat = self.etat.lock().unwrap();
            sortie.append(&mut etat.refus);
            if etat.vols.is_empty() {
                return Ok(());
            }
        }
        let mut entrees: Vec<OVERLAPPED_ENTRY> = vec![unsafe { std::mem::zeroed() }; self.profondeur];
        let mut n: u32 = 0;
        // Attente hors du verrou : un autre fil peut soumettre pendant ce temps.
        let ok = unsafe { GetQueuedCompletionStatusEx(self.port, entrees.as_mut_ptr(), entrees.len() as u32, &mut n, attente_ms, 0) };
        if ok == 0 {
            let code = unsafe { GetLastError() };
            if code == WAIT_TIMEOUT {
                return Ok(());
            }
            return Err(Erreur::Systeme { appel: "GetQueuedCompletionStatusEx", code });
        }
        let mut etat = self.etat.lock().unwrap();
        for e in &entrees[..n as usize] {
            let cle = e.lpOverlapped as usize;
            let Some(vol) = etat.vols.remove(&cle) else {
                return Err(Erreur::Usage(format!("paquet de complétion inconnu ({cle:#x})")));
            };
            let duree_us = vol.depart.elapsed().as_micros().min(u32::MAX as u128) as u32;
            let mut erreur = None;
            // `Internal` porte le NTSTATUS de la lecture ; s'il n'est pas nul, on demande au
            // système le code Windows correspondant.
            if vol.ov.Internal != 0 {
                let mut lus: u32 = 0;
                let r = unsafe { GetOverlappedResult(self.fichier, &vol.ov, &mut lus, 0) };
```

```
                erreur = Some(if r == 0 { unsafe { GetLastError() } } else { vol.
ov.Internal as u32 });
            }
            let octets = e.dwNumberOfBytesTransferred;
            etat.octets_lus += octets as u64;
            sortie.push(Achevement { jeton: vol.jeton, octets, erreur, duree_us
});
        }
        while etat.vols.len() < self.profondeur {
            let Some(r) = etat.attente.pop_front() else { break };
            if let Err(e) = self.lancer(&mut etat, r) {
                let code = match e {
                    Erreur::Systeme { code, .. } => code,
                    Erreur::Usage(_) => u32::MAX,
                };
                sortie.push(Achevement { jeton: r.jeton, octets: 0, erreur: Some(
code), duree_us: 0 });
            }
        }
        Ok(())
    }
```

Witnesses. Ten tests hold: host → device → host round trip on four non-round sizes, ordering between two streams through an event, a hand-written PTX kernel loaded through the API, twelve 1 MiB reads compared page by page, a partial last page, rejection of unaligned requests, dropping in flight, submission and harvest on two threads. Then the *witness of the witnesses*: each witness is forced false through an environment variable (`TRC_E_CASSER`), and we check that it fails *on its own message*; 7 breaks out of 7 fail, the restored suite holds [meas.] (`mesures/moteur/E/journaux/e1-temoins/`).

4.4 Pinning: trial E0 (a)

`cuMemHostAlloc` locks pages in physical memory and makes them visible to the card. An 11 GB pin on a 16 GB machine would have made the other four lots page; the protocol therefore declares a margin: a size is attempted only if available RAM exceeds the size plus 1.5 GB (answer by delegation of 2026-10-15, reversible).

Table 4.2: E0 (a), session of 2026-10-15 at 16:46, on mains power, card free [meas.]. The DMA is a 64 MiB copy made right after allocation, without warm-up.

size	allocation	release	RAM before	DMA after	verdict
0.25 GB	96 ms	46 ms	8.57 GB	10.36 GB/s	success, 0 differences
1 GB	343 ms	133 ms	8.51 GB	10.93 GB/s	success, 0 differences
4 GB	1 331 ms	594 ms	8.47 GB	11.00 GB/s	success, 0 differences
11 GB	—	—	8.29 GB	—	not attempted (8.29 < 11 + 1.5)

Data: `mesures/moteur/E/csv/2026-10-15-seance3-a.csv`.

Allocation costs about $1\,331/4 = 333$ ms per gigabyte [calc.]: an opening cost, not a path cost. The architecture's abandonment criterion K3 (no sizeable pinned memory) does not trigger. A size "not attempted" is not a size refused by the driver: the 11 GB question awaits a quiet machine.

Proposition 4.2 (The pinned window of mode A0). *In mode A0 (no warm cache in RAM), the engine pins $d \cdot k$ arrival buffers the size of the largest chunk, plus a mailbox and an embedding row. For the 20b, with $d = 3$ layers ahead and $k = 4$:*

$$3 \times 4 \times 12\,144\,640 + 4\,096 + 8\,192 = 145\,747\,968 \text{ bytes} \approx 0.146 \text{ GB}.$$

Proof. This is the definition of the window (architecture § 5.8): $3 \times 4 \times 12\,144\,640 = 145\,735\,680$ bytes of arrival buffers (12 144 640 is the largest chunk of the 20b), the mailbox 4 096, the embedding row 8 192 (a 3 060 B row read within an aligned read) [calc.]. ■

The pinned window of mode A0 is thus 27 times smaller than the largest successful pin [calc.]: measurement (a) leaves mode A (warm cache) all the room it might ask for, up to 4 GB.

4.5 The link: trial E0 (b) and the single copy engine

Correctness. 32 pieces of 12 MiB read unbuffered into pinned memory, copied to VRAM and brought back: 0 differences against an independent read of the same range (`std::fs`, through the Windows cache), 0 differences on the way back; the break (one byte corrupted in pinned memory after the read) gives exactly 1 difference [meas.]. The unbuffered read does write into, and only into, the page that the DMA reads.

Throughput. Declared threshold: 11.0 GB/s, the 12.2 of study 01 minus 10 % noise. The ABBA series of 16 copies of 12 MiB (the size of a chunk) and 16 copies of 64 MiB (the size of study 01's measurement) gives Figure 4.3.

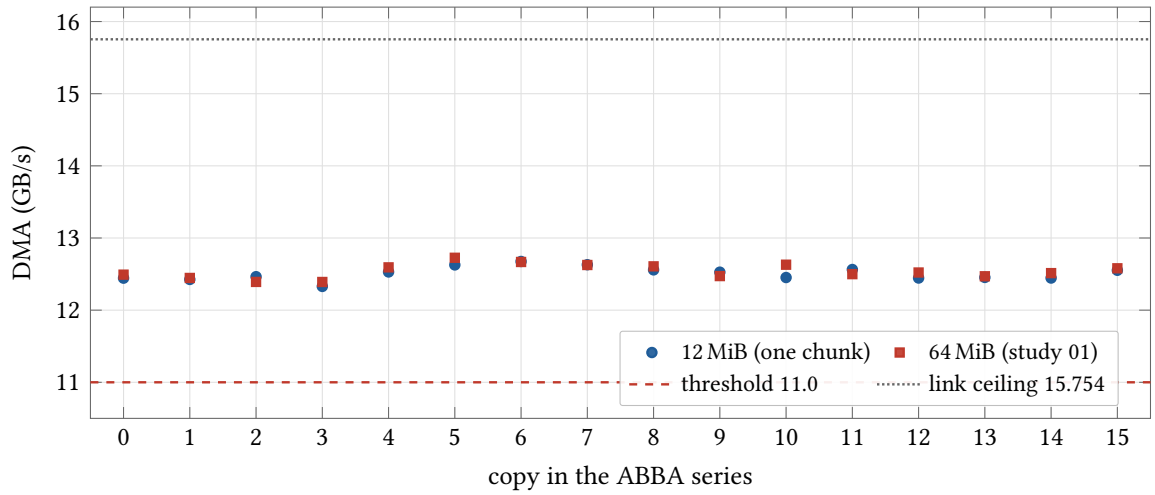


Figure 4.3: E0 (b): every host → device copy of the ABBA series [meas.]. Median at 12 MiB: **12.495 GB/s** (12.33 to 12.67, mirror noise 1.4 %); at 64 MiB: 12.517 (noise 1.6 %). The two sizes do not differ (rule of 06 § 7.5). Second path: the host clock gives 12.45 and 12.51, never above the card's events (0 out of 32). The link tops out at 15.754 GB/s (PCIe 4.0 ×8); the DMA gets 79 % of it.

Data: `mesures/moteur/E/csv/2026-10-15-seance3-b-dma.csv`, reordered by `memoire/outils/tome2_chemin_noyaux.py` (`t2c-dma.csv`).

Proposition 4.3 (The link does not limit a chunk). *Let B_d be the disk throughput and B_c the DMA throughput, both measured at the chunk size b. If $\rho = B_d/B_c < 1$, the copy of a chunk fits entirely in the shadow of the read of the next chunk, and the copy engine stays idle a fraction $1 - \rho$ of the time.*

Proof. A chunk occupies the disk for b/B_d seconds and the copy engine for b/B_c. In steady state the disk delivers a chunk every b/B_d seconds; the copy engine handles it in $b/B_c = \rho \cdot b/B_d < b/B_d$, hence before the next one arrives, and idles the rest of the time. ■

With the session's measurements, $\rho = 2.44/12.50 = 0.195$ [calc.]: copying a 12 MiB chunk takes $12\,582\,912/12.50 \cdot 10^9 = 1.01$ ms, against 5.2 ms of disk; the copy engine idles 80 % of the time. The link is not what will set the speed.

Proposition 4.4 (The single copy engine)**.** *With a single copy engine, copies in both directions form a single first-in, first-out queue. A device → host copy submitted behind k chunks in flight waits at worst $k\,b/B_c$.*

Proof. The engine runs one copy at a time, in submission order (this is what the ordering witness saw: no race between two copies in opposite directions on two streams, in 20 trials). A copy submitted behind k copies of b bytes starts only after them, that is at most $k\,b/B_c$ later. ■

For $k = 4$ chunks of 12 MiB at 11 GB/s, the wait reaches $4 \times 12\,582\,912/11 \cdot 10^9 \approx 4.6$ ms [calc.]. Three consequences for the engine (Figure 4.4): (i) the witness mode, which brings every decoded expert back to the host, slows the chunk path and is not timed; (ii) the logits of a step ($201\,088 \times 4$ B ≈ 0.8 MB, ≈ 73 µs of copy) wait behind the chunks in flight; (iii) the router's mailbox, written by a kernel directly into mapped host memory, *escapes* the queue: it carries the decision of every layer.

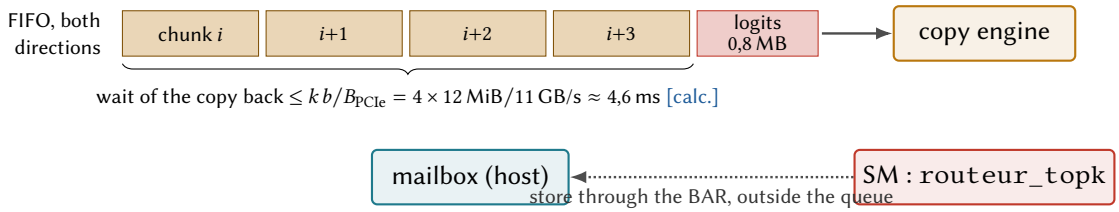


Figure 4.4: The single copy engine. Every copy, in either direction, goes through one FIFO queue; a store by a kernel into mapped host memory (the router's mailbox) crosses the bus without going through that queue.

4.6 Reading the router once per layer: trial E0 (c)

In a mixture of experts, the host knows which chunks to read only once the layer's router has been computed on the card. At every layer, then, the host must observe the end of the router, read its four indices, decide (hit or miss), and launch what follows. This is one host synchronization per layer: 36 per token on the 120b.

4.6.1 The mailbox, bit for bit

The router writes its decision into a *mailbox*: pinned and mapped host memory (`cuMemHostAlloc` with DEVICEMAP), into which the kernel writes over the bus and from which the host reads without a copy. One 64-byte slot per layer, 36 slots (2 304 bytes, within a 4 096-byte page), Figure 4.5.

The number never repeats for a given token (the token is a global counter): a slot read before it is written always carries a wrong number.

Proposition 4.5 (A host that reads the right number reads the right indices)**.** *If the kernel writes words 1 to 8, executes `__threadfence_system()`, then writes word 0, and if the host (x86-64) reads*

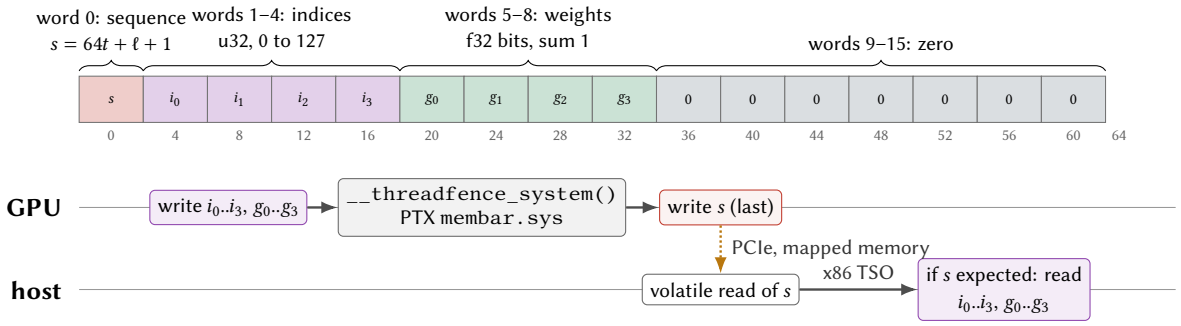


Figure 4.5: The 64-byte slot of a layer, and the order of operations. The kernel writes the indices and the weights, issues a system fence, then writes the sequence number $s = 64t + \ell + 1$ (t the token, a global counter; ℓ the layer). The host reads s with a volatile load; if it holds the expected value, it reads the indices. Lot O's engine takes this protocol over unchanged (`routeur_topk`, Chapter 5).

word 0 with a volatile load and then words 1 to 8, a host that reads the expected s reads the indices and weights written with that s.

Proof. `__threadfence_system()` compiles to PTX `membar.sys`: every write of the thread made before it becomes visible to every observer, the host included, before any write made after it [17]. So when word 0 reads s for the host, words 1 to 8, written earlier, already hold their values for it. On x86-64 (TSO ordering), two loads are not reordered with each other: the load of the indices, which follows the load of the number in program order, happens after it. The volatile load prevents the compiler from moving or removing it. ■

The witness checks it (0 wrong indices, 0 missing sequences in every synchronous variant), and the break shows it necessary: reading the mailbox without waiting yields 3 141 wrong indices over the session [meas.].

4.6.2 The threshold, and the first model

The threshold (D1). We allow reading the router to cost at most 10 % of a token's disk time: $0.1 \times 157 = 15.7$ ms per token, that is $15.7/36 = 436$ µs per synchronization [calc.]. Beyond that, the cost stops being negligible and becomes a stage of the token equation (abandonment criterion K4).

Proposition 4.6 (Model D2: the cost of a synchronization)**.** *For one layer, let t_r be the router's duration on the card and t_e that of the experts. Without the host waiting, launches are queued ahead and $T_{\text{async}} = L(t_r + t_e) + \varepsilon$. With one read of the router per layer,*

$$T_{\text{sync}} = L\,(t_r + d_{\text{sig}} + d_{\text{host}} + d_{\text{launch}} + t_e) + \varepsilon',$$

where d_{sig} is the delay between the end of the kernel and the moment the host observes it, d_{host} the host's work (reading four indices, preparing arguments), d_{launch} the delay between `cuLaunchKernel` and the start of the kernel. The cost per token $T_{\text{sync}} - T_{\text{async}} = L\,s$, $s = d_{\text{sig}} + d_{\text{host}} + d_{\text{launch}}$, depends neither on t_r nor on t_e.

Proof. Without waiting, the card chains the $2L$ kernels already queued: its time is the sum of their durations, plus the first launch. With waiting, the experts of layer ℓ can start only after the host has seen the end of the router (d_{sig}), has worked (d_{host}) and has launched (d_{launch}); these three delays add up in series at every layer, and nothing else changes as long as the card overlaps nothing. ■

The model carries its own witness: measured at two loads (0/0 and 50/250 µs), the cost must be the same. Figure 4.6 draws it to scale.

Listing 4.3: The simulated router (`moteur/bins/banc-chemin/noyaux/essais.cu`, lines 45 to 64, commit `94ac4c4`). `occuper` waits in time (register `%globaltimer`), not in cycles: the duration does not depend on the chip's current clock. The expected indices come from a 32-bit mix that the host recomputes [18].

```
// Le routeur simulé : occupe la carte `ns` nanosecondes (attention + routeur d'
    une couche), puis
// écrit les 4 indices et les 4 poids de la couche, une barrière système, et
    enfin le numéro de
// séquence. La barrière garantit qu'un hôte qui voit le numéro voit aussi les
    indices.
extern "C" __global__ void routeur_factice(uint32_t* boite, uint32_t jeton,
    uint32_t couche, uint64_t ns) {
    if (threadIdx.x != 0 || blockIdx.x != 0) return;
    occuper(ns);
    volatile uint32_t* c = boite + 16u * couche;
    float somme = 0.0f;
    float p[4];
    for (uint32_t i = 0; i < 4; ++i) {
        p[i] = 1.0f + (float)(indice_attendu(jeton, couche, i) & 7u);
        somme += p[i];
    }
    for (uint32_t i = 0; i < 4; ++i) {
        c[1 + i] = indice_attendu(jeton, couche, i);
        c[5 + i] = __float_as_uint(p[i] / somme);
    }
    __threadfence_system();
    c[0] = jeton * 64u + couche + 1u;
}
```

4.6.3 What the measurement changed in the model

Five variants are measured: `evenement` (the host waits for an event recorded after the router, then reads the mailbox), `sondage` (it polls the event), `boite` (it polls the mailbox itself), `boite_sans_vidage` (likewise, without pushing the launch through `cuStreamQuery`), `bloquant` (event with blocking wait: the thread sleeps and the system wakes it).

The threshold holds by a wide margin (Figure 4.7): under load, `evenement` costs 0.666 ms per token, `sondage` 0.692, `boite` 0.750, `bloquant` 3.12; with a context in blocking-wait mode, `evenement` costs 0.630 [meas.]. The cost is even at the trial's noise level: the synchronous pass lasts 11.779 ms against 11.114 asynchronously, and the session summary classes the gap of `evenement` as "not distinct from noise".

But the model's witness fails. The cost under load is 3.6 to 4.8 times the empty cost for four variants (5.1 against 18.5 µs for `evenement`), and 8.7 times for `boite_sans_vidage` [meas.]. ~~ratios 3.6 to 4.8 depending on the variant~~ Lot E's note wrote "3.6 to 4.8 depending on the variant": true of the four retained variants, not of the fifth (8.7, read in `modele_D2`).

Proposition 4.7 (Model D2 bis: the launch rate)**.** *Let λ be the minimum interval between two launches as seen by the card. The asynchronous baseline is $T_{\text{async}} = \max\big(L(t_r + t_e),\ 2L\lambda\big) + \varepsilon$. Empty ($t_r + t_e \ll 2\lambda$), the card already waits for launches, and synchronizing adds only the part of $d_{\text{sig}} + d_{\text{host}}$ that does not hide behind λ. Loaded ($t_r + t_e \gg 2\lambda$), the card is saturated asynchronously, and synchronizing exposes the whole chain: $s \approx d_{\text{sig}} + d_{\text{host}} + d_{\text{launch}}$, with $d_{\text{launch}} \geq \lambda$.*

Proof. A launch cannot reach the card more often than once per λ: $2L$ kernels last at least $2L\lambda$, hence

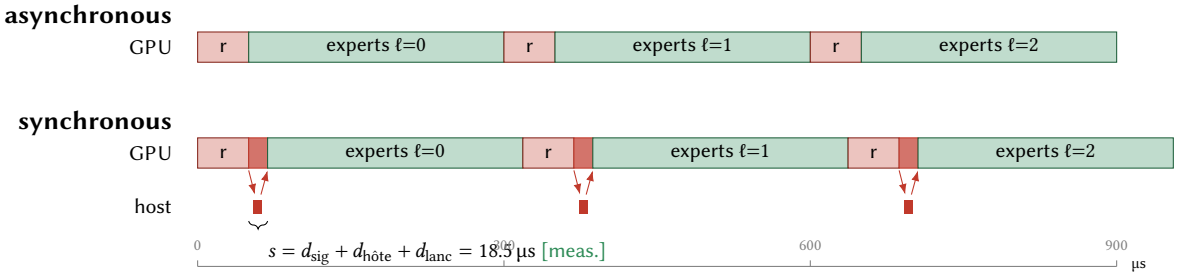


Figure 4.6: Model D2 for three layers, at the scale of the loaded trial (router 50 µs, experts 250 µs). Asynchronously, the card chains; synchronously, every layer shows the gap s between the end of the router and the start of the experts, during which the host sees the event, reads the mailbox and launches.

the maximum. Empty, the term $2L\lambda$ dominates both passes, and only the part of the host's delays that exceeds λ lengthens the synchronous pass. Loaded, the first term dominates the asynchronous pass, and every layer of the synchronous pass waits for the complete chain, the launch itself included. ■

The calculation. Empty, the asynchronous pass of 72 launches lasts 0.831 ms, hence $\lambda \approx 831/72 = 11.5$ µs [meas.], well above $t_r + t_e \approx 0$. Loaded, $L(t_r + t_e) = 36 \times 300 = 10.8$ ms and the asynchronous pass lasts 11.114 ms ($\varepsilon \approx 0.31$ ms): the card is saturated. Estimate: $s_{\text{loaded}} \approx s_{\text{empty}} + \lambda = 5.1 + 11.5 = 16.6$ µs, against 18.5 measured (10 % apart; d_{launch} is not measured alone here, it is a hypothesis) [calc.]. **The case that matters is the second**: a real expert reads 13 MB in hundreds of microseconds. This is why the threshold is judged under load. Lesson, kept in the recipes: measuring a synchronization empty only gives a cost four times too low.

4.7 The disk alone: step E1

4.7.1 The threshold, and what a clean pass is

Threshold declared before any pass: the median of the A passes (12 MiB, depth 4) must reach $0.8 \times 2.58 = 2.064$ GB/s, where 2.58 GB/s is study 01's throughput for random 4 MiB reads and $0.8 = 1 - 2 \times 10\,\%$ noise. But the machine is shared: another program reading the disk during the pass takes part of its bandwidth. We must know how many bytes the others read during each pass.

Lemma 4.8 (Foreign bytes from two raw readings)**.** *The PDH counter* `Disk Read Bytes/sec` *of the* `PhysicalDisk(_Total)` *object has type* `PERF_COUNTER_BULK_COUNT`*; its raw datum N is the cumulative number of bytes read by the whole machine. The bytes read during a pass are exactly $N_1 - N_0$, and the foreign bytes are $E = (N_1 - N_0) - b\,n$, where $b\,n$ are the bytes delivered by our reader.*

Proof. For this counter type, the displayed value is $(N_1 - N_0)/((D_1 - D_0)/F)$, with D the raw timestamp and F its frequency [19]: N is therefore a cumulative byte count, and its difference between two readings counts the bytes of the interval exactly, without integrating a sampled rate [20]. Our unbuffered reads go to the disk; they are counted in $N_1 - N_0$; the rest belongs to the others. ■

The lemma gives a free witness: $N_1 - N_0 \geq b\,n$ at every pass (a counter blind to the disk, or a read served by a cache, would make it fail); it holds at every pass of every session [meas.]. This path is independent of lot R's, which integrates the formatted rate.

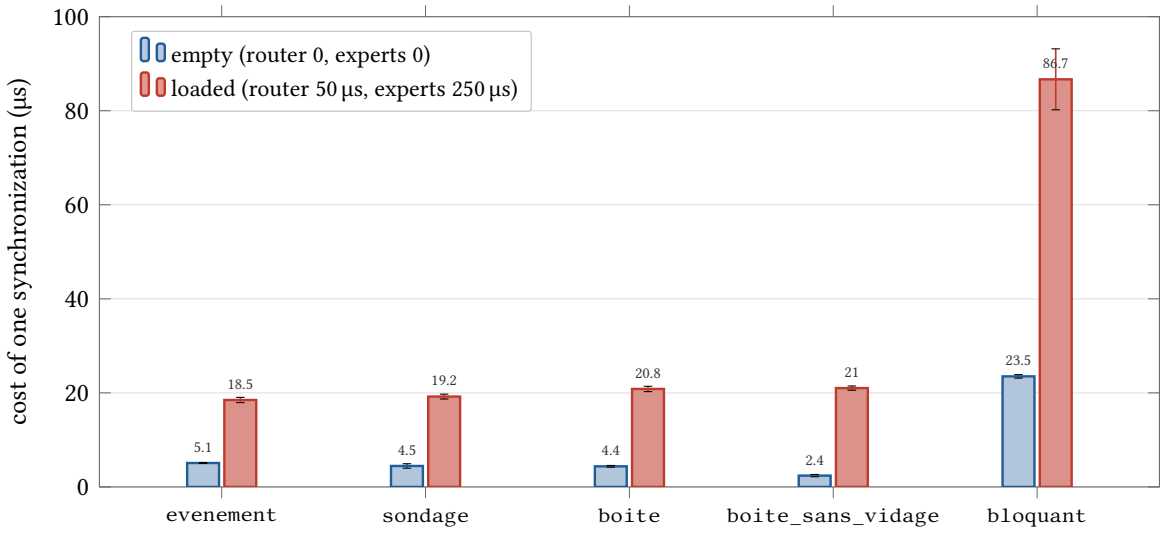


Figure 4.7: E0 (c), session of 2026-10-15 at 16:46 [meas.]: cost of one synchronization, $(T_{\text{sync}} - T_{\text{async}})/36$, per variant. Error bars: noise of the mirror passes. The threshold (436 µs) is off the chart. The retained variant, `evenement`, costs 18.5 µs under load, i.e. 0.666 ms per token, **23 times under the threshold**.

Data: `mesures/moteur/E/journaux/2026-10-15-seance3/resume.json` (`modele_D2`), reordered in `t2c-sync.csv`.

Definition 4.9 (Clean pass)**.** Rule 1: a pass is clean if $E/T \leq 0.1 \times 2.58 = 0.26$ GB/s, the foreign load staying within the noise already allowed. Rule 2, added after session 1 and before session 2: in addition, other programs occupy at most 40 % of the CPU during the pass (source: 01 § 2.3, large blocks reproducible within a few percent under 35 to 40 % occupancy). A verdict is given only on at least two clean A passes; otherwise the series is "not judged", never "failed".

This is not a correction of the throughput (nothing is subtracted): it is a sort.

4.7.2 One session under load, one session quiet

Session 1 fails under the declared rule, and its verdict stays published as is (Figure 4.8): it says that *under this load* the reader does not sustain 2.06 GB/s. What rule 1 did not look at was the CPU. Session 3, quiet, holds: 2.453 / 2.335 / 2.666 / 2.426 GB/s at QD 4, 2.328 / 2.285 / 2.224 / 2.383 at QD 1 [meas.]. The mirror noise of the A passes is 13.4 %, that of the B passes 2.7 %; QD 4 and QD 1 cannot be told apart ($5.6\,\% < 2 \times 13.4\,\%$).

4.7.3 Little's law, a second path for the throughput

Theorem 4.10 (Little, 1961)**.** *In a stable system, the mean number of customers present $\bar{N}$, their arrival rate λ and their mean sojourn time $\bar{R}$ satisfy $\bar{N} = \lambda \bar{R}$ [3].*

Corollary 4.11 (The throughput of a fixed-depth reader)**.** *A reader that permanently keeps q reads of b bytes in flight, each of mean duration $\bar{R}$ (from hand-over to the system to harvest), reads $X = q\,b/\bar{R}$ bytes per second.*

Proof. The customers are the reads in flight: $\bar{N} = q$. Their rate is $\lambda = X/b$ reads per second. Theorem 4.10 gives $q = (X/b)\,\bar{R}$. ∎

Listing 4.4: The raw cumulative counter (`moteur/bins/banc-chemin/src/disque.rs`, lines 46 to 63, commit `1cca7f3`). The counter is added by its English name (`PdhAddEnglishCounterW`): this Windows is in French.

```
    /// Octets lus sur le disque par toute la machine depuis le démarrage.
    pub fn cumul(&self) -> Result<u64, String> {
        // SAFETY : requête et compteur ouverts par `ouvrir`.
        unsafe {
            let e = PdhCollectQueryData(self.requete);
            if e != 0 {
                return Err(format!("PdhCollectQueryData : {e:#x}"));
            }
            let mut brut: PDH_RAW_COUNTER = std::mem::zeroed();
            let mut sorte = 0u32;
            let e = PdhGetRawCounterValue(self.compteur, &mut sorte, &mut brut);
            if e != 0 || brut.CStatus != 0 {
                return Err(format!("PdhGetRawCounterValue : {e:#x}, statut {:#x}"
    , brut.CStatus));
            }
            Ok(brut.FirstValue as u64)
        }
    }
}
```

The log publishes the *median* latency of a read, not its mean; a long-tailed latency has a mean above its median, and $q\,b/R_{\text{median}}$ then overestimates the throughput. Figure 4.9 sets the two paths against each other.

What Little says besides: the disk serves one 12 MiB read at a time. In session 3, the median latency is 18.7 ms at QD 4 (A passes: 18 717, 19 864, 17 563, 18 678 µs) and 4.62 ms at QD 1 (B passes: 4 664, 4 631, 4 608, 4 567 µs) [meas.]; the ratio is $18.7/4.62 = 4.05$ [calc.]. Four reads in flight thus each wait about four times as long as a single one: if the disk serves a rate μ whatever q, then $\bar{R}(q) \approx q\,b/\mu$ and the corollary gives $X = \mu$, independent of q. This is what we observe (QD 4 and QD 1 indistinguishable). [hyp.] A 12 MiB read on its own occupies all of the NVMe's internal parallelism (its channels); adding reads in flight only lengthens the queue. The engine's timeline (Section 4.10) will show the same on the real path. Conversely, at QD 1 Little overestimates by 16 to 23 %: between two reads, the disk idles during the harvest and the next hand-over.

4.8 End to end: step E2

E2 runs the whole path, without a model: every chunk of the file read unbuffered in one request of `longueur0` (its level-0 part, § 4.7 of the format) into the pinned window, copied on the copy stream into an arrival buffer in VRAM, awaited through an event, decoded by lot K's two kernels on the decode stream into a slot with SoA layout (Chapter 5). Depth 4 (gpt-oss's k); order: a permutation of the 768 experts drawn from a published seed, new at every pass.

4.8.1 The threshold, set before the measurement

Proposition 4.12 (The E2 threshold in experts served)**.** *If the disk delivers B_d bytes of chunks per second and a chunk of b bytes yields an expert of o original bytes, the throughput of experts served,*

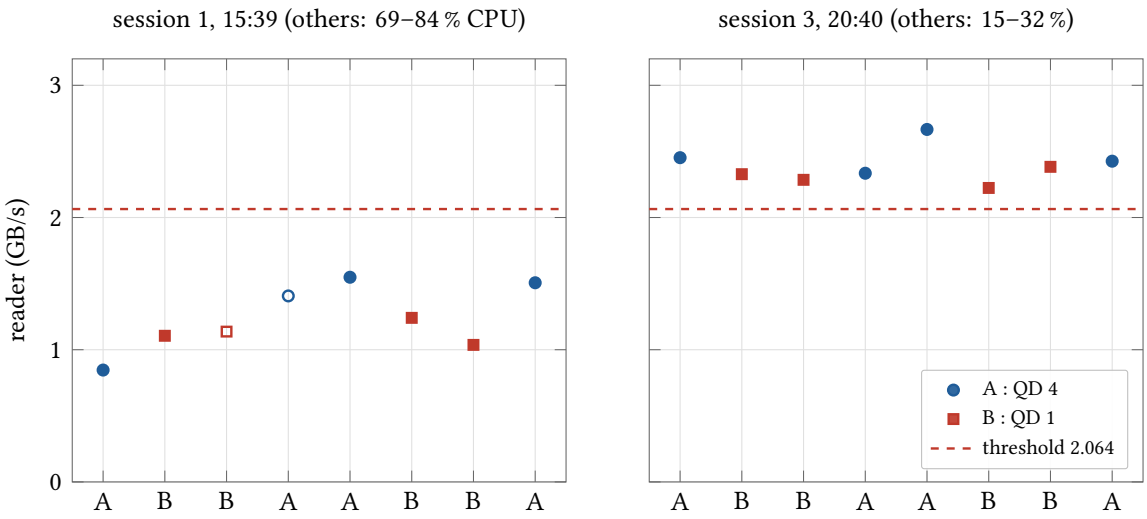


Figure 4.8: E1, every pass of the reader (128 reads of 12 MiB at drawn offsets) [meas.]. Filled: clean pass; hollow: dirty (rule 1). Left, session 1 (15:39): median of the clean A passes **1.506 GB/s**, under the threshold; the CPU was 69–84 % busy with other lots (f64 judge, tracer, antivirus), and BitLocker decrypts every byte on that same CPU. Right, session 3 (20:40, rule 2): median **2.439 GB/s**, four clean A passes out of four: **E1 holds**.

Data: `mesures/moteur/E/csv/e1-2026-10-15-seance1-lecteur-passes.csv`, `e1-2026-10-15-seance3-lecteur-passes.csv` (`t2c-e1-s1.csv`, `t2c-e1-s3.csv`).

counted at original size, is $X_{\text{served}} = B_d \cdot o/\bar{b} = B_d \cdot r$*, with* $r = o/\bar{b}$ *the mean compression ratio.*

Proof. In one second the disk delivers $B_d/\bar{b}$ chunks, each decoded into an expert of o bytes. ■

The specification takes $B_d = 2.58$ GB/s, $r(F1) = 13\,253\,760/11\,990\,368 = 1.105367$ (§ 4.14) and the rule $0.8 = 1-2\times10$ %: threshold $0.8\times2.58\times1.105367 = \mathbf{2.28148}$ GB/s of experts served [calc.]. The real file has $\bar{b} = 12\,013\,664$ B, i.e. $r = 1.103224$; the same threshold then asks for $2.28148/1.103224 = 2.068$ GB/s of chunks, 0.2 % above E1's threshold (2.064): the two steps judge almost the same disk, one alone, the other at the end of the path.

Theorem 4.13 (Throughput of a chain of stages and number of slots)**.** *Consider a chain of three stages that every chunk crosses in order: the disk (throughput* B_d *with* q *reads in flight), the copy engine (one copy at a time,* $t_c = b/B_c$*), the decode stream (one expert at a time,* t_k*). In steady state:*

1. $X \le \min(B_d,\ B_c,\ b/t_k)$ *bytes of chunks per second;*
2. *for the disk, if it is the slowest stage, to work with* q *reads in flight,* $S \ge q + \lceil N_{\text{out}} \rceil$ *slots are needed, where* $N_{\text{out}} = (X/b)\,(t_c + t_k + \delta)$ *and* δ *is the host's delay between the end of a decode and putting the slot back in service.*

Proof. (1) A stage that serves every object cannot output faster than its capacity; the chain outputs at the pace of the slowest. (2) A slot is held from the submission of its read to the end of its decode plus δ. Outside the disk, it stays $t_c + t_k + \delta$ (plus any wait); by theorem 4.10 applied to that part of the system, the mean number of slots busy outside the disk is $N_{\text{out}} = (X/b)(t_c + t_k + \delta)$. The disk keeps q reads in flight only if a free slot awaits every completion: on average $S - N_{\text{out}} \ge q$ is required. ■

The calculation. With $X/b \approx 2.44 \cdot 10^9/1.2 \cdot 10^7 \approx 203$ chunks per second, $t_c \approx 0.96$ ms, $t_k \approx 0.35$ to 0.40 ms, and δ from 0.1 to 0.5 ms (event polling, 100 µs sleep), $N_{\text{out}} \approx 203 \times 1.8 \cdot 10^{-3} \approx 0.37$ [calc.].

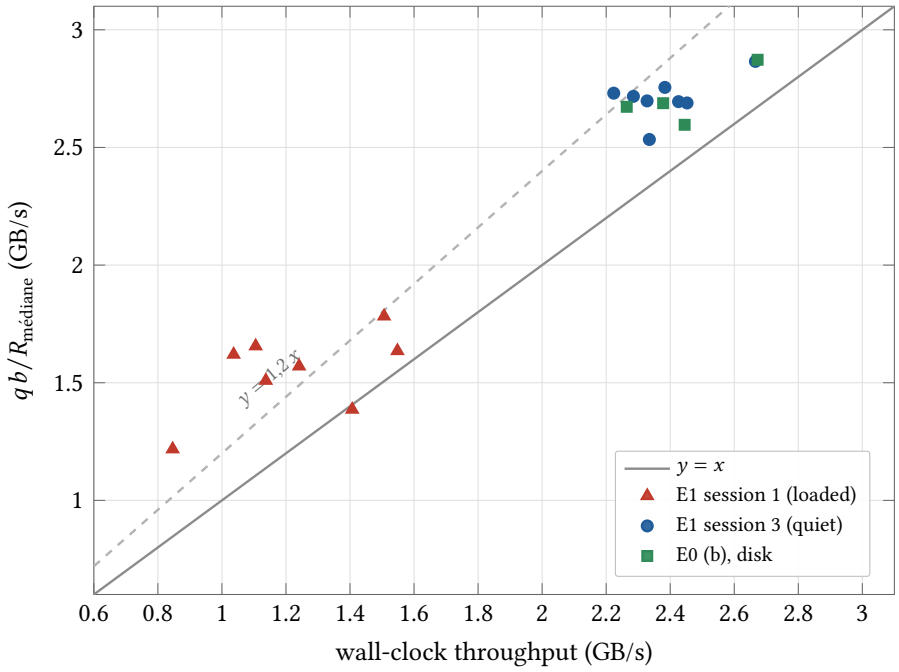


Figure 4.9: Second path for the disk throughput: $q\,b/R_{\text{median}}$ against wall-clock throughput, every pass [meas.]. On a quiet machine, Little overestimates by 8 to 23 % (ratio 1.08 to 1.23 in session 3): the median latency is not the mean, the tail is long (p90 of 27.8 ms for a median of 18.7). Under load (session 1), the scatter rises up to 1.56.

Data: `e1-*-lecteur-passes.csv` and `2026-10-15-seance3-b-disque.csv` (columns `go_s`, `go_s_little`), in `t2c-little.csv`.

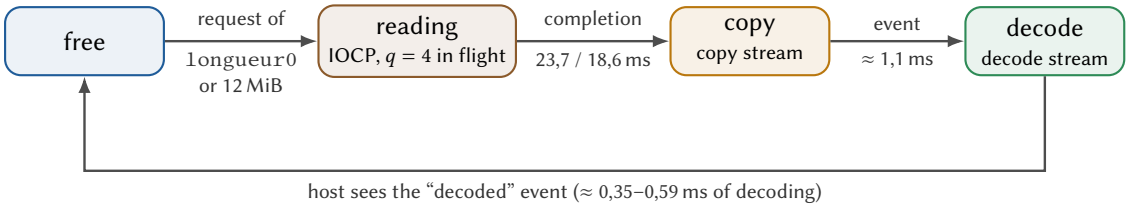
So $S \geq q + 1 = 5$ slots are needed on average; we take $q + 2 = 6$ for fluctuations, so that a slow read (the p90) does not starve the others of room. Pinned window: $6 \times 12\,144\,640 = 72.9$ MB; VRAM: $6 \times (12\,144\,640 + 13\,254\,144) = 152.4$ MB (Figure 4.10).

Point (1) of theorem 4.13 gives a witness no correct measurement can violate: **the median of the A passes must not exceed the session's smallest stage by more than 10 %**. Beyond that, either a stage is mismeasured, or the A pass does not do what it claims (a read served by a cache, a clock stopped too early, an expert counted without decoding). Its natural break is the B pass itself: same sequence of reads, same window, same reader, without copy or decode.

4.8.2 Exact transport, and what each check sees

C2. In the witness pass, every slot comes back to the host; the expert is put back into its original layout and its fingerprint compared with the directory; the fingerprint of the useful bytes that arrived in the pinned window is compared with that of the part; the card's control counters (lanes off L, format, pointers) must be zero. **768 experts out of 768**, at each of the evening's 8 witness passes; and on the 120b, 4 608 **out of** 4 608, at both witness passes [meas.].

Proposition 4.14 (What each check can see of a corruption (D14))**.** *Let one bit be flipped in the useful bytes of a part. (i) The 64-bit fingerprint of the arrived bytes differs from the directory's, barring a collision of the 64-bit prefix of SHA-256, of probability 2^{-64} per expert if SHA-256 behaves as a random function. (ii) The fingerprint of the original expert redone from the slot sees only corruptions* that change the output*: a bit of the raw streams (medians, biases) changes exactly one scale byte per affected block, or one bias; a bit of an rANS word or state changes the sequence of states, hence almost always the symbols;*



One slot = pinned slice 12 144 640 B + arrival buffer in VRAM 12 144 640 B + SoA slot 13 254 144 B + four events. Six slots ($S = q + 2$) : pinned $6 \times 12\,144\,640 = 72.9$ MB ; VRAM $6 \times (12\,144\,640 + 13\,254\,144) = 152.4$ MB [calc.]. Control counters: 768×16 B in VRAM, read back once at the end of the pass.

Figure 4.10: The E2 window and the life of a slot. Durations are the medians measured in E2 (read of `longueur0` / of 12 MiB; copy; decode) [meas.]. In the witness pass, the slot is freed only after its return to the host and its two fingerprints.

Data: `mesures/moteur/E/csv/e2-2026-10-15-seance1-passes.csv`, `e2-2026-10-15-v12-passes.csv`.

but a field the decoder does not read to produce the output (layer and expert of the part header) changes nothing. (iii) The card's counters, the only check of production mode, see format faults and lanes that do not end on L, but neither the raw streams nor the identity of the part.

Proof. (i) is the assumption on SHA-256. (ii): medians and biases are copied or added without entropy transform (a median enters $e = (\text{med} + \delta - 128) \bmod 256$ of every block of its row); a wrong state or word makes the recurrence $x \leftarrow f\,(x \gg 12) + b$ diverge; the header's identity is read by no kernel. (iii): the kernels count only what they check (magic, kinds, stream headers, return to L, final pointer). ■

The declared break places one bit in each family, one expert per bit, in an 11.16 GB copy of the file (seed 20261015). Table 4.3 is its detection table.

The last row is a real hole in the production mode of the time: a directory pointing to another expert's chunk would be decoded without error. The integrity guard added by the round of corrections after the independent trial closes it: it compares the fingerprint of the useful bytes of every chunk read with the directory's, and the part header (identity included) is among those bytes [calc.] (`moteur/O-pour-le-memoire.md`, proposition 8).

The witness pass is not timed (D15). Every 13 254 144 B slot comes back through the single copy engine (Proposition 4.4), and the host hashes about 25 MB per expert with SHA-256 on one thread, at 1.5 to 2 GB/s: 12 to 17 ms per expert, three times the disk time of a chunk [calc.]. Its throughput is published for the record, never judged; the threshold is judged on the production passes, where nothing comes back but the counters, at the end of the pass (12 288 B).

4.8.3 Throughput: the verdict fails, and the witness says why

As specified, E2 **fails**: A passes at 2.120 / 2.092 / 2.186 / 2.215 GB/s served, median **2.153 < 2.2815**, 5.6 % under the threshold; 2.135 in the N V V N series [meas.]. All eight passes were clean (others' CPU 6 to 12 %, foreign bytes ≤ 0.001 GB/s). That is the verdict.

But the chain's witness holds, and it locates the defect (Figure 4.12): the median of the A passes is 1.951 GB/s of chunks, the disk alone (B) 1.921, the DMA 12.66, decoding 34.27 (0.350 ms per expert); $A/\min = 1.016 \leq 1.10$ [meas.]. A and B do not differ (1.6 % for a noise of 1.6 %). **The path costs the disk nothing; it is the disk itself that reads this file at 1.92 GB/s**, against 2.44 for the GGUF 34 minutes earlier. The DMA has a 6.5-fold margin, decoding 18-fold.

Table 4.3: E2: the detection table, check by check [meas.]. ●: the check sees the corruption. Part: fingerprint of the useful bytes arrived in the window; original: fingerprint of the expert redone from the decoded slot; counters: the card's control counters. **12 detected out of 12, none extra** (an expert is "detected" if a check of the witness fails). Production mode sees 9: not the medians, not the biases, not the identity of the part, as Proposition 4.14 predicted.

			witness pass			
region of the bit	layer	expert	part	original	counters	production
MEDIANES	23	4	●	●		
BIAIS	21	24	●	●		
DELTA, tile starts	16	20	●	●	●	●
DELTA, states	14	28	●	●	●	●
DELTA, words	18	30	●	●	●	●
CODES_CTX, starts	4	25	●	●	●	●
CODES_CTX, states	11	20	●	●	●	●
CODES_CTX, words	2	19	●	●	●	●
CODES_CTX, header (P)	12	8	●	●	●	●
stream table (kind)	16	18	●	●	●	●
part header, magic	19	24	●	●	●	●
part header, identity	19	22	●			
total			12	11	9	9

Data: `mesures/moteur/E/csv/e2-2026-10-15-seance1-cassure.csv`, `journaux/e2-2026-10-15-seance1/resume.json`.

Figure 4.13 shows the path at work: every chunk is a line, from its submission to the disk to the end of its decode. The copy (about 1.0 ms) and the decode (0.35 ms) fit in the shadow of the four reads in flight, and the slope of the staircase is the disk's alone.

Little on the path. With `longueur0`, $q\bar{b}/R = 4 \times 12\,013\,664/23.7 \cdot 10^{-3} = 2.03$ GB/s predicted against 1.95 measured (ratio 1.04); with 12 MiB, $4 \times 12\,582\,912/18.6 \cdot 10^{-3} = 2.71$ against 2.69 (1.01) [calc.]. On the real path, latency is tighter than in the reader (p90 of 24 to 32 ms) and Little lands almost exactly.

4.9 The discovery: two regimes depending on length alone

The disk read the GGUF at 2.44 GB/s, and the same disk read the `.trc` at 1.92 at the end of the path. Why? Each hypothesis was declared before the measurement that decides it, with its rule (two files or two settings differ if the gap between medians exceeds twice the largest mirror noise and 3 %).

H3a is the only one that holds: **for fewer bytes, the read is 32 % slower**. Offsets being drawn on 4 096 in both cases, neither the start nor the end needs finer alignment: the length is what matters. The first sweep kills the idea of a grain (an 11 MiB length fast between two slow ones); the second sweep, declared beforehand in `balayage_longueurs.py`, measures 23 lengths from 8 to 13 MiB in steps of 0.25 MiB (plus 12 013 568 and 12 144 640), in two replicas with drawn orders, four passes each (Figure 4.14).

Two facts make the map credible: the two drawn orders draw it the same way (a drift in time would have drawn two different curves), and latency follows throughput (bottom panel: 1.5 times longer

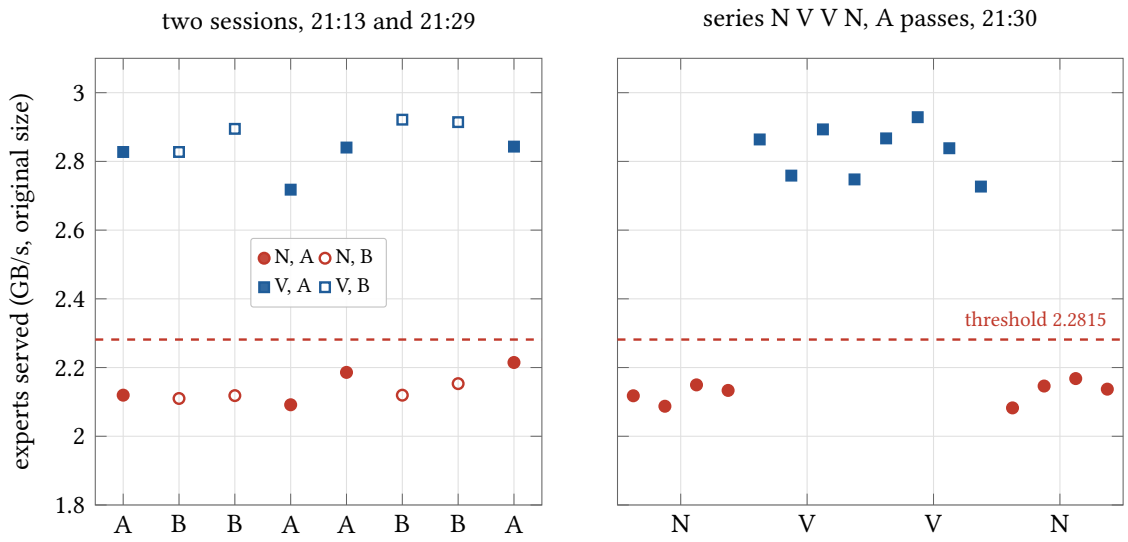


Figure 4.11: E2, experts served per pass (GB/s, counted at original size) [meas.]. Left, the 21:13 session as specified (N: reads of `longueur0`) and the 21:29 variant (V: 12 MiB reads); A: production; B: disk alone, same reads, counted the same way. Right, the N V V N series of 21:30, A passes: V / N = ×1.335 (medians of the eight A passes of each letter: 2.851 and 2.135 GB/s), i.e. +33.5 % for a mirror noise of at most 1.5 %.

Data: `mesures/moteur/E/csv/e2-2026-10-15-seance1-passes.csv`, `e2-2026-10-15-v12-passes.csv`, `e2-nvvn-2026-10-15-nvvn.csv` (`t2c-e2-n.csv`, `t2c-e2-v.csv`, `t2c-nvvn.csv`).

in the slow regime). **Every chunk length falls in a slow band; 12 MiB is fast** (3.07 GB/s in both replicas).

What the measurement does not establish: the mechanism. [hyp.] The splitting of a request into transfers by the controller driver (VMD, `iaStorVD`) or by the BitLocker filter (`fvevol`), whose last piece would fall badly for some lengths. Nothing is visible from here without kernel tracing (ETW). It is a measured and unexplained fact, published as such.

Proposition 4.15 (Experts served with a fixed read length)**.** *If every chunk is read in one request of fixed length $\ell \geq b_{\max}$ (the largest chunk), at the throughput $B(\ell)$ of that length, then the throughput of experts served is*

$$X_{\text{served}} = B(\ell) \cdot \frac{\bar{b}}{\ell} \cdot r = B(\ell) \cdot \frac{o}{\ell}.$$

Proof. The disk delivers $B(\ell)/\ell$ requests per second, each a whole chunk (since $\ell \geq b_{\max}$); the extra bytes belong to the next chunk and are neither copied nor decoded. Each request yields an expert of o bytes. ∎

For $\ell = 12$ MiB, $o/\ell = 13\,253\,760/12\,582\,912 = 1.0533$; measured in E2-V12, $B(\ell) \approx 2.69$ GB/s at the chunk offsets, hence 2.83 GB/s served predicted, **2.834 and 2.851 measured** [calc.]. The extra reading (at most 0.75 MiB per chunk, 4.7 % on average) is paid for, and repaid sevenfold by the change of regime. ~~prediction: 3.07 × 12 013 664/12 582 912 × 1.103 ≈ 3.2 GB/s served~~ The prediction written before the variant (3.2, with the reader's $B(\ell) = 3.07$) was 12 % off: at the chunk offsets, the fast regime gives only 2.7 GB/s; that gap is not explained either. ~~reading a chunk with 12 MiB: ≈ 16 ms~~ Lot E's note drew the read of a chunk with 12 MiB at about 16 ms; that is the reader's latency (S M M S), not the path's, where the median is 18.6 ms (`e2-2026-10-15-v12-passes.csv`).

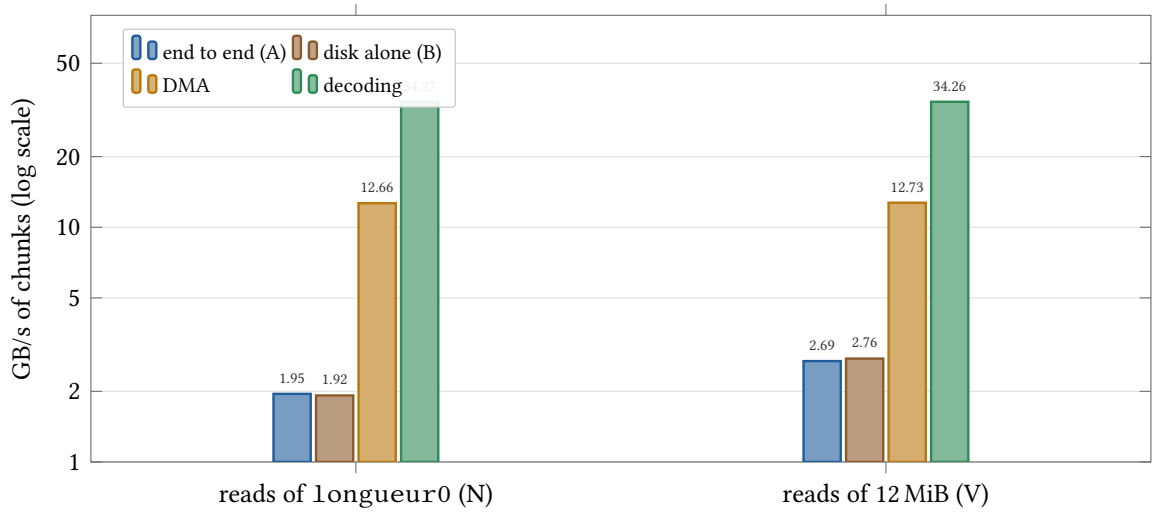


Figure 4.12: E2: end to end and its three stages measured in the same session, in GB/s of chunks read (log scale) [meas.]. The disk is the bottleneck by an order of magnitude; A sticks to the disk alone, under the bound of theorem 4.13.

Data: `e2-2026-10-15-seance1-etages.csv`, `e2-2026-10-15-v12-etages.csv` and the passes (`t2c-etages.csv`).

The gain, judged by ABBA. So as not to compare two sessions 14 minutes apart, the N V V N series was declared beforehand: V / N = ×1.335, i.e. +33.5 %: this is the ratio of the medians of the eight A passes of each letter, V 2.851 and N 2.135 GB/s served [calc.] (`e2-nvvn-2026-10-15-nvvn.csv`); the per-pass medians, N 2.126 and 2.142, V 2.811 and 2.853, give through their means 2.832/2.134 = 1.327, not the published figure. ~~A 25 % gap~~: that is 1 − 1/1.335, N under V, not the gain of V. The mirror noise is at most 1.5 %; 16 clean A passes out of 16; C2 holds in all four [meas.]. On the 120b (one session, not judged): N 2.063, V 2.877, ×1.39; decoding 0.353 ms per expert, DMA 11.07 GB/s; the declared prediction (N ≈ 2.1, V ≈ 2.8) holds [meas.] (`journaux/e2-120b-2026-10-15/`).

The proposal to the arbiter. Round every request up to a *fast length calibrated on the machine* (here 12 MiB covers every chunk of the 20b and the 120b); not one byte of the format changes; cost: a 12 MiB pinned window per buffer instead of 11.58 (+3.6 %) and at most 0.75 MiB read in excess per chunk, already counted in the ×1.335. Another way, not retained: padding the chunks to a fast length at encoding time (+4.7 % of file, and the fast length depends on the machine). Calibration (`balayage_longueurs.py`) would become an installation step of the engine. The decision had not been taken at the time this chapter was written.

4.10 The path in the engine

Lot O's engine (`troncons`) takes this path as is: same reader, same pinned memory, same mailbox, same decoder. Its `--chronogramme` option timestamps every miss (submission, read, copy, decode), the router and the end of each layer.

Figure 4.15 finds on the real path what Little said of the reader (Section 4.7): four reads entrusted together do not finish together, they finish in a queue, 3 to 6 ms apart. The engine suffers in addition from a short queue: a layer knows its misses only after its router, and has only 1.6 of them on average (59 misses per token over 36 layers, session O4) [meas.]. The disk alone, with drawn chunks at depth 4, reads 1.848 GB/s; the engine draws 0.707 GB in 457 ms per token, i.e. 1.55 GB/s useful [calc.]. What

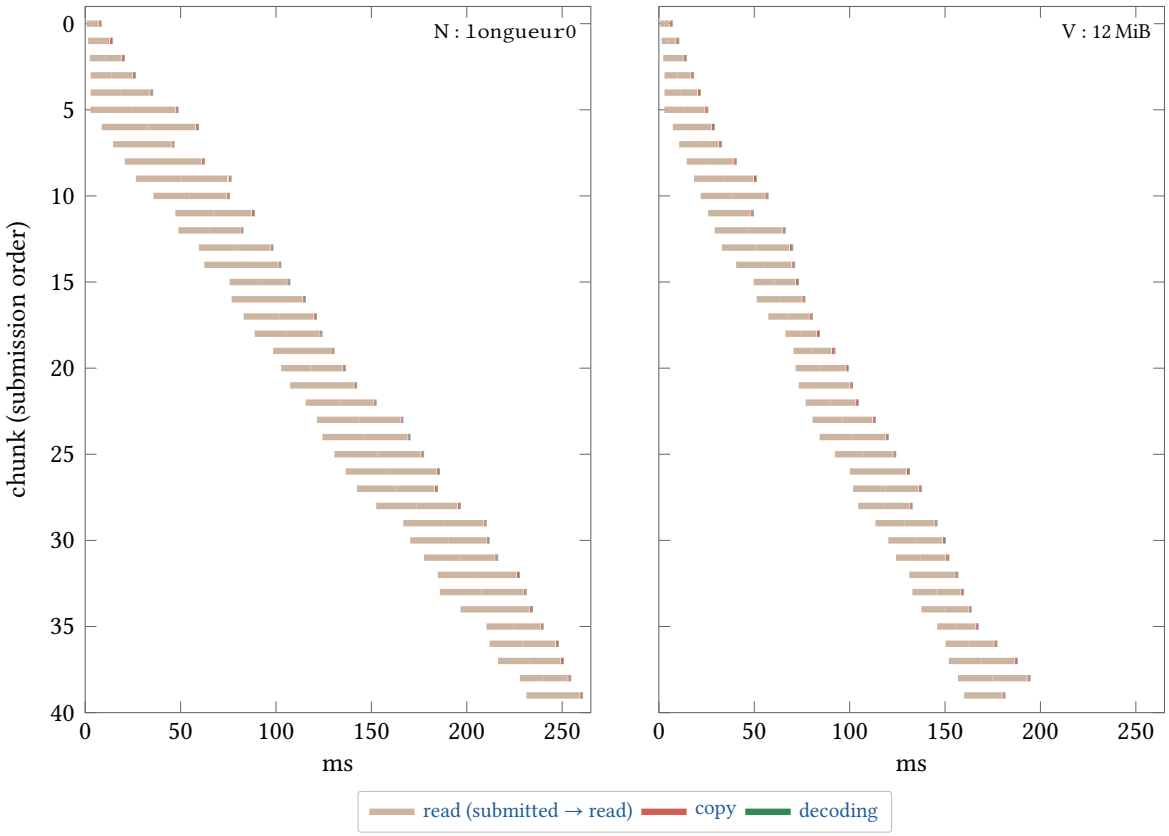


Figure 4.13: Timeline of the first 40 chunks of an A pass (second round), same drawn order in both sessions [meas.]. Reading a chunk (submitted → read) takes 23.5 ms at the median with `longueur0`, 18.6 ms with 12 MiB; the 40 chunks finish at 261 ms against 182, the 768 at 4 657 ms against 3 584. The copy and the decode are the small segments at the end of each read.

Data: `mesures/moteur/E/journaux/e2-2026-10-15-seance1/chrono-1/chrono-A-t1-r0.csv`, `e2-2026-10-15-v12/chrono-v12582912-1/chrono-A-t1-r0.csv`.

limits J1 is the queue depth per layer, which prefetching was meant to fill (killed by the measurement, −11 %: 31 % of the prefetched reads were wrong on a disk that already limits; see the engine chapter).

12 MiB reads in the engine. Lot O's L12 series (A: J1 with streamed prefill; B: the same, `--longueur-lecture 12582912`), declared beforehand, gives Figure 4.16. In generation, B gains +30 % at the median (2.179 and 2.182 against 2.613 and 3.073 tokens/s), every B pass above every A pass; but the two B passes differ by 0.46 token/s, a noise $\beta = 0.46/2.843 = 16.2\,\%$ by the definition of Definition 8.1 (~~21 %~~ according to lot O, which divided the gap by the mean of A), and the declared rule (gain > 2× noise +3 %, here $2 \times 16.2 + 3 = 35.4\,\%$ against +30.4 % [calc.]) **is not met** in four passes: L12 is neither established nor refuted. The texts of the four passes are equal (logits do not depend on the read length).

An observation, calculated here from the published passes, not declared beforehand: with streamed prefill, which reads every requested expert of a layer back to back, A gives 27.44 and 27.39 tokens/s (0.2 % apart), B 34.55 and 34.33 (0.6 %): +25.6 %, far beyond $2 \times 0.6\,\% + 3\,\% = 4.3\,\%$ [calc.]. Prefill, which keeps the disk full, shows almost all of the fast regime's gain (×1.26, against ×1.335 on E2); generation, with its short queue, shows it drowned in its variance. This is not a verdict (the criterion was not this one); it is what a longer series will have to confirm.

Table 4.4: The hypotheses on E2's slowness, in the order they were declared, on 2026-10-15 between 21:16 and 21:22 [meas.]. G: the GGUF; T: the `.trc`; L: E1's reader; C: E2's path.

hypothesis	measurement	verdict
H1, the file (fragments)	reader G T T G: G 2.571 and 2.591; **T 3.039 and 3.023** GB/s; G has 2 737 fragments, T has 33	fails, backwards
H2, disk slowed after the 11 GB copy	G stays at 2.6 ≥ 2.064, 1 min 30 s after E2	fails
H2 bis, still slowed at 21:14	L C C L: L 3.04 to 3.09; C, disk alone, 1.93 to 2.04, adjacent and clean	fails
H3, the way E2 reads	same window: C stays 36 % under L	holds
H3a, the length of the reads	S M M S, same offsets: **S (**12 582 912 **B) 3.00 to 3.09** GB/s, latency 16.3 ms; **M (**12 013 568 **B) 2.06 to 2.11**, latency 23.4 ms	**holds**: −32 %
"an alignment grain"	12 MiB −g: fast for $g \leq 128$ KiB and $g = 1$ MiB; slow for $g =$ 256 KiB, 512 KiB, 2 MiB	fails: no grain

Data: `mesures/moteur/E/journaux/e2-diag-2026-10-15/` (`lecteur-*`, `lccl-*`, `smms-*`, `grain-*`, `extents-*.txt`), `moteur/REPRISE-E.md`.

The guard on the path. Since the round of corrections, every completed read is hashed (64-bit fingerprint of its useful bytes) by four threads in parallel with the copy and the decode, and the token is returned only after all reads of the step have been checked. The fingerprint's throughput is measured at 1.07 to 1.10 GB/s per thread; on the critical path remains only the hashing of the last read, at most 12.25 MB/1.07 GB/s ≈ 11.4 ms minus the card work that follows it; measured: 5.2 to 5.4 ms of waiting per token, −1.6 % at equal disk throughput [meas.] (`mesures/moteur/O/corrections/csv/vitesse-2026-10-16-passes.csv`; lot O's note, § 13.2, wrote ~~5.2 to 5.3 ms~~ and ~~−1.7 %~~, Section 8.5.5). With 12 MiB reads, the guard hashes only the useful bytes: the extra bytes cost it nothing. The combination of L12 and the guard has not been measured.

4.11 What this chapter struck out

- ~~What remains: E2, the end-to-end bench, which awaits lot K's decoder and lot F's format.~~ E2 is done (2026-10-15, 21:13 to 21:35).
- ~~Model D2: cost independent of the load.~~ The model's witness fails; D2 bis (Proposition 4.7) replaces it.
- ~~ratios 3.6 to 4.8 depending on the variant~~: 3.6 to 4.8 for four variants, 8.7 for `boite_sans_vidage`.
- ~~E2 V12: ≈ 3.2 GB/s served predicted~~: 2.834 measured; the fast regime at the chunk offsets gives only 2.7 GB/s.
- ~~reading a chunk with 12 MiB: ≈ 16 ms~~: 18.6 ms at the median on the path (16.3 ms is the reader's latency).

What could be wrong

- **The length map is that of one machine, one file, one evening**: 23 lengths, two replicas, the 20b file. The regime of a length may depend on the controller driver, the BitLocker version, the NVMe firmware, how full the disk is; the map does not transfer, it is recalibrated. 9.5 MiB stayed between the two regimes; a length that is fast here may be slow on the 120b file (not

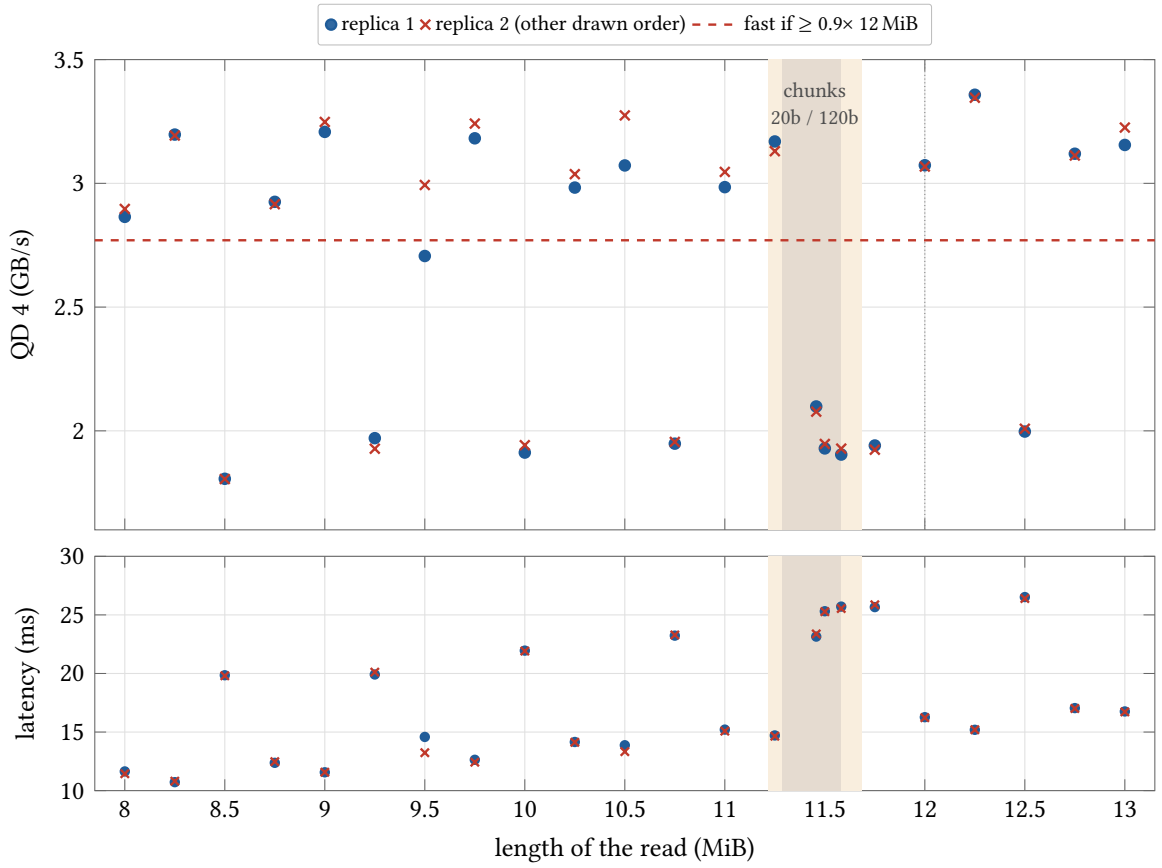


Figure 4.14: The throughput of an unbuffered read depends on its length alone, in two regimes [meas.]. Reads at QD 4 at drawn offsets on the 20b file; two replicas with drawn orders. Fast (2.87 to 3.36 GB/s): 8; 8.25; 8.75; 9; 9.75; 10.25; 10.5; 11; 11.25; 12; 12.25; 12.75; 13 MiB. Slow (1.80 to 2.10 GB/s, latency ×1.5): 8.5; 9.25; 10; 10.75; 11.457; 11.5; 11.582; 11.75; 12.5 MiB. Shaded: the chunk lengths of the 20b (11.29 to 11.58 MiB, dark) and of the 120b (11.22 to 11.68 MiB, light). The replicas agree on 22 lengths out of 23; 9.5 MiB stays between the two (2.71 then 2.99, for a threshold of 2.77).

Data: `mesures/moteur/E/csv/e2-balayage-2026-10-15-longueurs.csv`, `journaux/e2-balayage-2026-10-15/`.

measured).

- **The E2 threshold rests on 2.58 GB/s measured in 4 MiB blocks**, a length the sweep did not cover. If 4 MiB is in a fast regime, the threshold compares a fast regime with a read in a slow one: E2's "fails" verdict says that first. The same goes for C5's floor and Volume I's ceilings.
- **E2-V12 reads bytes of the next chunk**: no effect on correctness (C2 holds in the 12 MiB witness passes), but an engine reading two neighbouring chunks would read the boundary twice.
- **The witness pass checks the expert brought back**, not the slot read by the GEMV at compute time: a write by another kernel between the decode and the GEMV would not show here (the engine excludes it by pinning in the cache, lot O's lemma 4).
- **The E0 (c) trials simulate the layer**, they do not compute it: a card busy with a real GEMV might delay the next launch more than a single thread's busy wait does. The two loads bracket

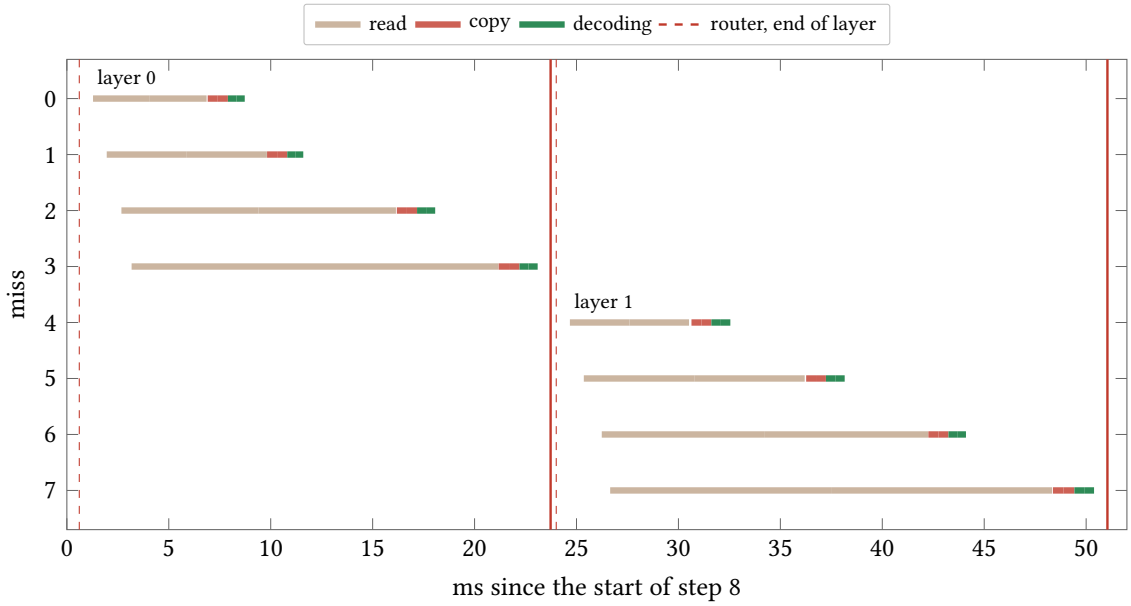


Figure 4.15: The engine on gpt-oss-120b, step 8, layers 0 and 1, where all four experts miss [meas.]. Layer 0: the router ends at 0.61 ms; the four reads, submitted at 1.28 / 1.96 / 2.68 / 3.18 ms, finish at 6.86 / 9.81 / 16.14 / 21.17 ms: **the disk serves them one after another**, about 5 ms for 12 MB each; every copy (1.0 ms) and every decode (0.85 ms) follows its read without waiting for the others; the layer ends at 23.73 ms.

Data: `mesures/moteur/0/csv/chronogramme-120b.csv` (8 steps, 1 048 timestamped misses), extracted into `t2c-couche.csv`.

it, without proving it for all.

- **The machine is shared**: the pilot refuses to measure while a `llama-*` or the tracer runs and logs the card's state before and after every pass, but a program that is neither escapes the test; foreign bytes and others' CPU bound it.
- **E0 (a) attempts only what available RAM allows**: 11 GB "not attempted" is not a refusal by the driver.
- **The `boite_sans_vidage` variant** rests on undocumented behaviour (WDDM hardware scheduling, active here [21], submits the launch without a call that pushes it); it is published, never retained.
- **The reading "the disk serves one 12 MiB read at a time"** is an interpretation of Little and of the timeline, not a measurement of the controller.
- **The L12 observation at prefill** was not declared beforehand; it rests on two pairs of passes.
- **`Lecteur::soumettre` and `copier_vers_hote` are safe functions** that entrust an asynchronous write to the system or the driver: the ordering contract is documented, not checked by the type. `Contexte::memoire` returns (0, 0) on a driver error.

Where we start, where we go. The path is built and exact, end to end, on both models; none of its stages other than the disk costs the token anything (DMA ×6.5, decoding ×18, synchronization ×23 under their thresholds). What remains lies entirely on the disk side: the decision on the read length, its mechanism, the map of the 120b, and a deeper queue per layer, which only a better predictor than the one tried could fill.

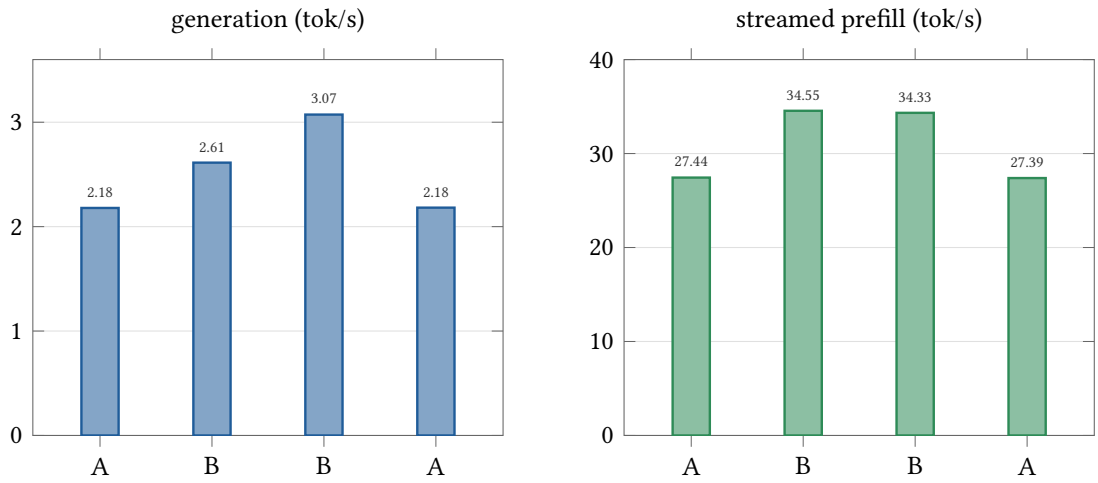


Figure 4.16: The engine on the 120b, A B B A series of 2026-10-16 (session 2) [meas.]. A: J1 and streamed prefill; B: the same with 12 MiB reads. Left, generation, the declared criterion: not met. Right, streamed prefill, which was not the declared criterion.

Data: `mesures/moteur/O/csv/o-jetons-s.csv` (passes 2026-10-16-s2-15 to 18), in `t2c-l12.csv`.

Chapter 5

The kernels on the card

Where we start, what this chapter establishes, what remains

At the start, study 03 of Volume I had an interleaved rANS decoder on the card (v4, 80.8 and 82.1 GB/s under CUDA in ABBA on a large stream) and a raw MXFP4 matrix–vector product under Vulkan (209.4 GB/s). The table chosen per lane, which removes the reassembly of the class streams, existed only on the CPU (the arbiter's measurement F1). No kernel of the gpt-oss forward pass had been written, and lot R's f64 reference had just been validated against llama.cpp. **This chapter establishes** the per-lane-table decoder on the card, proved exact (a warp step yields exactly the scalar decoding) and checked bit for bit on the 4 608 experts of gpt-oss-120b and the 768 of gpt-oss-20b, through two independent encoders; the MXFP4 GEMV, exact to the bit on integers by construction and within $3 \cdot 10^{-7}$ of an f64 reference on real weights, at 235 GB/s; and the twelve kernels of the forward pass, each with its algorithm, its proof of exactness or derived error bound, its bit-level determinism, its witness and its break. The whole chain of these kernels is at KL $3.0 \cdot 10^{-6}$ from the f64 reference when routing is imposed; with free routing, the gap comes from routing flips at margins under 10^{-3}. Finally, the gap that remained in the engine came entirely from the f16 rounding of the KV cache. **What remains** is the arbiter's decision on the decoder fallback (DELTA in tiles of 32 and batched decoding), the cause of K1's first-series effect, the chain on the 120b, tiled attention for long contexts, and the base mode of milestone J2. Sources: `moteur/K-pour-le-memoire.md`, `moteur/REPRISE-K.md`, `mesures/moteur/K/`, code `moteur/crates/trc-noyaux/`; round of corrections: `moteur/O-pour-le-memoire.md` (part III).

5.1 What the card must do

On the card, a token goes through two kinds of work. *Decoding* turns a chunk that arrived by DMA (Chapter 4) into an expert slot ready to be read; *computing* chains, layer after layer, the kernels of the gpt-oss forward pass. The kernels are written in CUDA C++, compiled at build time by `nvcc` 13.1 (`-arch=sm_89`, cubin image plus PTX), embedded in the Rust binary and loaded through the driver API: no CUDA library is shipped, no algebra library either (no GEMM in J1).

Definition 5.1 (Determinism rule, seam N)**.** No kernel uses a floating-point atomic addition; every reduction follows a fixed tree that depends only on the sizes (neither on the grid nor on the execution order of the blocks). The only atomics are integer control counters.

Without this rule, the milestone's criterion C3 (logits of the coded file and of the raw file equal to the bit) would make no sense: two runs of the same program on the same bytes could differ. Each section below says how the kernel keeps it.

Table 5.1: Kernel resources, reported by `ptxas -v` for `sm_89`, and occupancy on the RTX 4060 Laptop (Ada: 100 KiB of shared memory, 48 warps and 64 K registers per SM [22]).

kernel	registers	shared (B)	remark
`decoder_codes`	57	49 152	three 16 KiB LUTs; 2 blocks of 8 warps per SM, 384 warps on 24 SMs
`decoder_echelles`	55	24 576	one LUT and 8 192 B of medians
`decoder_*_lot`	61 / 57	same	one expert per value of `blockIdx.z`
`gemv_mxfp4` / `_lot`	71 / 79	0	four rows per warp
`copier_brut`	29	0	raw-file path (witness C3)
`attention`	97	2 304 + 4/position	at most 8 192 positions per call
`routeur_topk`	—	144 (stack)	indexed arrays

Data: `moteur/K-pour-le-memoire.md` (table T-K1), `moteur/REPRISE-K.md` (readings of 15:06 and 20:50).

5.2 The per-lane-table decoder (K1)

5.2.1 The stream, the tile, the table

The `.trc` format (§ 4.9 of the specification) codes each stream with an rANS interleaved over 32 lanes: 12-bit probabilities ($M = 4\,096$), 32-bit state x, lower bound $L = 2^{16}$, 16-bit words. A *macro-step* produces 128 contiguous bytes: lane l of macro-step m writes the 32-bit output word of index $w = 32m + l$, and its four sub-steps $k = 0..3$ write its bytes, from least to most significant. A *tile* chains $N = 256$ macro-steps; each tile has its start pointer in the shared word stream and its 32 initial states, so tiles decode independently.

For MXFP4 codes (kind CODES_CTX), a block of 32 weights carries 16 bytes, i.e. 4 words: lanes $4q$ to $4q + 3$ of macro-step m write block $8m + q = \lfloor w/4 \rfloor$, and all four choose the table of that block's context κ, its *scale class*: $\kappa = 0$, 1 or 2 according to whether the block's scale offset δ is below, at, or above the median of its row (Figure 5.1).

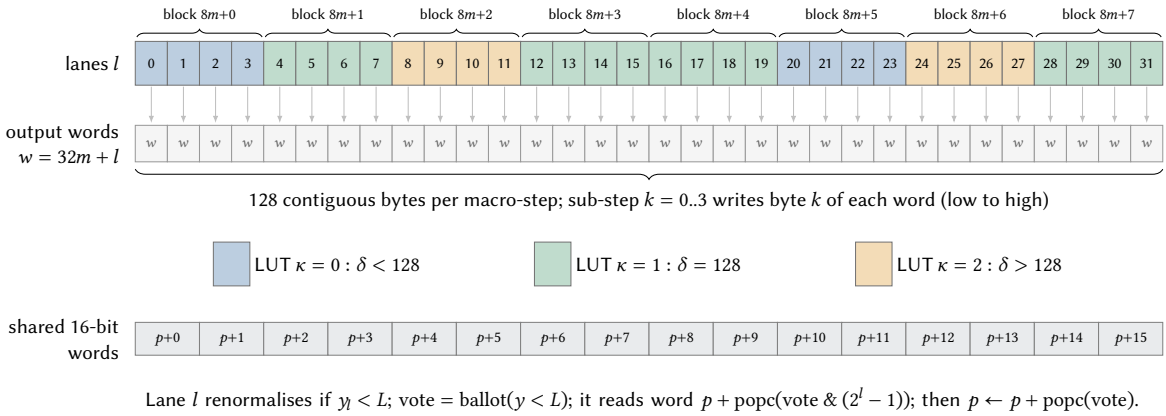


Figure 5.1: One macro-step of the tile: 32 lanes, grouped by four under the table of their block (colour: the scale class κ, an example), above the 32 output words; below, the stream of 16-bit words shared by the lanes, distributed by ballot and bit count.

A table is mounted in VRAM as a LUT of 4 096 32-bit entries, one per probability slot (Figure 5.2, top): the symbol s in bits 0 to 7, $f - 1$ in bits 8 to 19, the offset of the slot from the cumulative frequency

in bits 20 to 31. We store $f-1$ and not f: a symbol alone in its table has $f = 4\,096 = 2^{12}$, which does not fit in 12 bits (test `symbole_unique_a_frequence_pleine`).

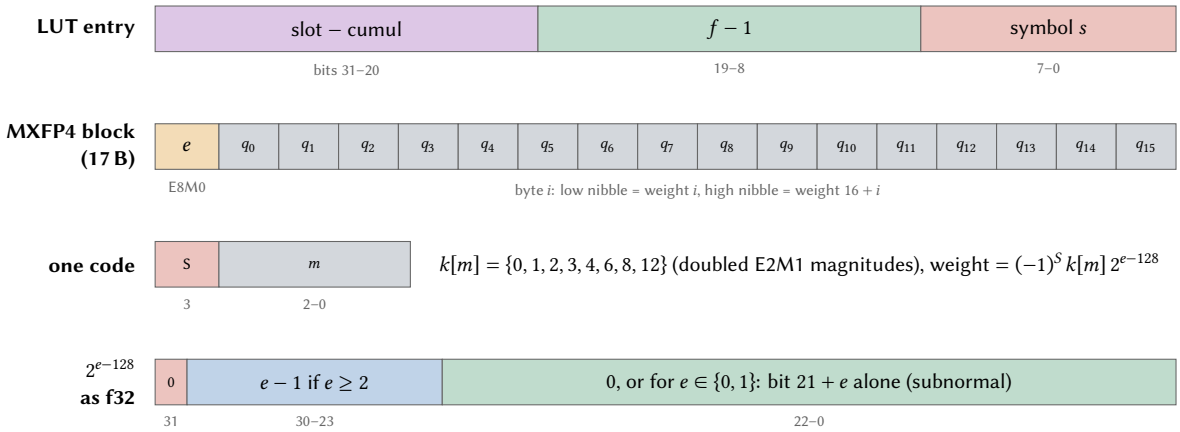


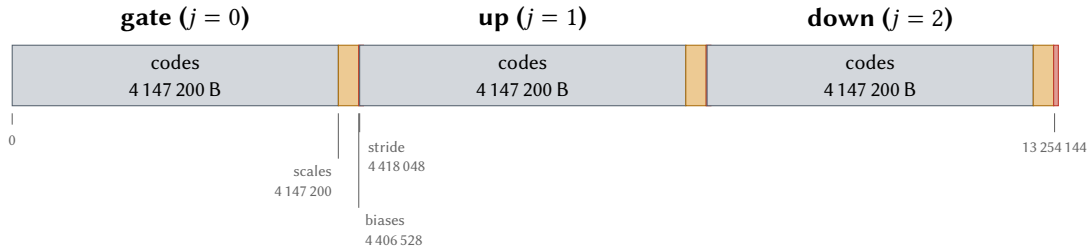
Figure 5.2: Bit for bit: the geometry-1 LUT entry; the original 17-byte MXFP4 block (one E8M0 scale, then 16 bytes of codes, byte i carrying weights i and $16+i$, the layout of ggml's `block_mxfp4` [23]); a 4-bit code, sign and doubled E2M1 magnitude [1]; and the scale 2^{e-128} written directly as f32 bits by the kernel.

5.2.2 Two kernels, one slot

The CODES_CTX table depends on κ, which DELTA gives: the decoder always decodes DELTA before CODES_CTX (§ 4.9.3). An expert is therefore decoded in two launches on the same CUDA stream:

1. `decoder_echelles`, grid $(X, 3)$, 128 threads: for each matrix j, decodes DELTA_j (a single table), rebuilds the scale of each block $e[\beta] = (\text{med}[r] + \delta[\beta] - 128) \bmod 256$, and copies the biases bit for bit (as 32-bit integers, so that no conversion touches a NaN);
2. `decoder_codes`, grid $(X, 3)$, 256 threads, 48 KiB of shared memory (the matrix's three LUTs): decodes CODES_CTX_j, the lane producing word w choosing the LUT of context $\kappa(\lfloor w/4 \rfloor)$, recomputed from the scale already written and the median.

Both kernels read the chunk part as it arrived by DMA (header, stream table, streams): the host has nothing to parse. A tile is given to a warp, and warps walk the tiles in grid strides, so any grid is correct. The output is the SoA slot that the GEMV reads (Figure 5.3).



Per matrix ($R = C = 2\,880$, $B = 90$): codes $RB \cdot 16 = 4\,147\,200$ B in block order β; scales $RB = 259\,200$ B at $\text{al}_{256}(4\,147\,200) = 4\,147\,200$; biases $4R = 11\,520$ B at $\text{al}_{256}(4\,147\,200 + 259\,200) = 4\,406\,528$; stride $\text{al}_{256}(4\,406\,528 + 11\,520) = 4\,418\,048$. Slot: $3 \times 4\,418\,048 = 13\,254\,144$ B (test `disposition_de_gpt_oss_egale_la_specification`).

Figure 5.3: The slot of a gpt-oss expert, to scale (seam N, § 5.6): for each matrix, the codes, then the scales and the biases, each aligned on 256 bytes.

5.2.3 Exactness, proved

The normative decoder of § 4.9.2 processes the 32 lanes *one by one*, in increasing order: lane l reads its LUT entry, updates its state, and, if the state falls below L, reads the next word of the shared stream. The kernel processes the 32 lanes *at once*. We must show that it produces the same symbols.

Theorem 5.2 (A warp step yields exactly the scalar decoding)**.** *Let a tile be coded by the encoder of § 4.9.2 (reverse order: macro-steps, sub-steps and lanes decreasing; each lane starts from L; word stack reversed). The kernel, which processes the 32 lanes simultaneously at each sub-step, produces the same symbols, in the same positions, as the scalar decoder processing the lanes one by one in increasing order; and at every sub-step, states and pointer are equal in both.*

Proof. By induction on the sub-steps of the tile; at the start, both have $p = \text{start}[t]$ and the same 32 states. (i) The state update of a lane, $y \leftarrow f\,(x \gg 12) + b$ with (s, f, b) read from the LUT at address $x \bmod 4\,096$, depends only on its own state and on the table: it commutes across lanes. (ii) Only the order in which words are read ties the lanes together. In the scalar decoder, at the current sub-step, the lanes that renormalize ($y < L$) read words $p, p+1, \ldots$ in increasing lane order. The kernel computes vote $=$ ballot$(y < L)$ (one bit per lane) and gives lane l the word $p + \text{popc}(\text{vote} \,\&\, (2^l - 1))$, that is p plus the number of lower-indexed lanes that also renormalize: exactly the scalar position. Then $p \leftarrow p + \text{popc}(\text{vote})$, as the scalar decoder after its 32 lanes. (iii) A lane never renormalizes twice in the same sub-step: if $x \geq L$, then $x \gg 12 \geq 16$, so $y \geq 16f \geq 16$, and for $y < L$, $(y \ll 16) \,|\, \text{word} \geq 2^{16} = L$. (iv) The encoder emitted its words in exactly the reverse order of these reads (last emitted is first read, by reversing the stack): each lane rereads the word it wrote. States and pointer being equal after the sub-step, the induction continues. ■

The listings of this chapter are verbatim excerpts of the sources, with one exception, stated here: six characters of the comments that the memoir's monospace font lacks (the floor and ceiling brackets, the summation sigma, the transpose superscript) are rendered as `floor(…)`, `ceil(…)`, `sum` and `^T` (`memoire/outils/tome2_chemin_noyaux.py`); the code itself contains none of them. Comments are in French in the sources.

Listing 5.1: A warp step (`moteur/crates/trc-noyaux/noyaux/commun.cuh`, lines 197 to 227, commit `82a84e3`). Ballot and bit count give each lane its position (theorem 5.2); two `__shfl_sync` exchanges distribute the words of the window (lemma 5.3) [17]. Comments are in French in the source.

```
__device__ __forceinline__ uint32_t pas_octet(EtatVoie& v, const uint32_t*
    __restrict__ lut,
                                              const EnteteRans& h, uint32_t voie,
     uint32_t masque_inf) {
    const uint32_t e = lut[v.x & (M - 1u)];
    v.x = (((e >> 8) & 0xFFFu) + 1u) * (v.x >> PROBA_BITS) + (e >> 20);
    const bool besoin = v.x < L;
    const uint32_t vote = __ballot_sync(0xFFFFFFFFu, besoin);
    const uint32_t pos = v.ptr + __popc(vote & masque_inf);
    const uint32_t wi = (pos >> 1) - v.base;  // < 48 : voir la démonstration
    dans REPRISE-K
    const uint32_t a = __shfl_sync(0xFFFFFFFFu, v.cour, wi & 31u);
    const uint32_t b = __shfl_sync(0xFFFFFFFFu, v.suiv, wi & 31u);
    const uint32_t w32 = wi >= 32u ? b : a;
    const uint32_t w = (w32 >> ((pos & 1u) * 16u)) & 0xFFFFu;
    v.x = besoin ? ((v.x << 16) | w) : v.x;
    v.ptr += __popc(vote);
    if ((v.ptr >> 1) - v.base >= 32u) {
```

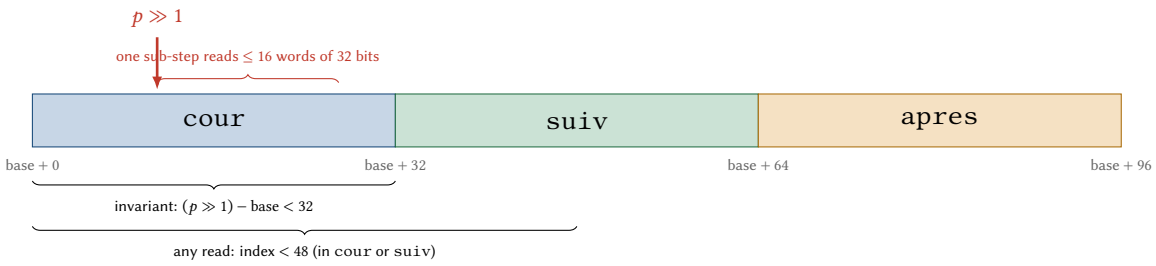
```
        v.base += 32u;
        v.cour = v.suiv;
        v.suiv = v.apres;
        v.apres = mot32(h, v.base + 64u + voie);
    }
    return e & 0xFFu;
}

__device__ __forceinline__ void debut_tuile(EtatVoie& v, const EnteteRans& h,
    uint32_t t, uint32_t voie) {
    v.x = h.etats[t * VOIES + voie];
    v.ptr = h.debut[t];
    v.base = v.ptr >> 1;
    v.cour = mot32(h, v.base + voie);
    v.suiv = mot32(h, v.base + 32u + voie);
    v.apres = mot32(h, v.base + 64u + voie);
}
```

Free checks. At the end of a tile, every lane must equal L (the encoder's initial state), and the pointer must equal the start of the next tile (or W, the number of words, for the last). The second check, added by the kernel and by lot F independently, catches an extra word the first one misses. Neither guards the *table*: a corrupted LUT slot gives a wrong output without taking a lane off L (Table 5.2); this is why the tables carry a fingerprint checked when the file is opened.

5.2.4 The word window

Reading a word from global memory at every renormalization would cost a full latency per sub-step. The warp therefore holds a window of 96 32-bit words of the stream (192 16-bit words) in three registers per lane: `cour` (word base + l), `suiv` (base + 32 + l) and `apres` (base + 64 + l), exchanged through `__shfl_sync` (Figure 5.4).



If $(p' \gg 1) - \text{base} \geq 32$: base += 32, cour ← suiv, suiv ← apres, apres ← mot32(base +64+ lane), a global load used one advance later.

Figure 5.4: The warp's word window: three registers per lane, 96 32-bit words. The invariant guarantees that no read leaves `cour` and `suiv`; `apres` is loaded one advance early, which hides its latency.

Lemma 5.3 (The window never overflows)**.** *Before each sub-step,* $(p \gg 1) - \text{base} < 32$. *Then every read of the sub-step falls in* `cour` *or* `suiv`*, and after the sub-step the invariant is restored by at most one advance of 32.*

Proof. During a sub-step, a lane reads at most word $p + 31$ (at most 32 lanes renormalize), of 32-bit word index $(p + 31) \gg 1 \leq (p \gg 1) + 16$; hence the relative index $(\text{pos} \gg 1) - \text{base} < 32 + 16 = 48$: the word is in `cour` (below 32) or in `suiv`, never beyond. After the sub-step, $p' \leq p + 32$, so

$(p' \gg 1) - \text{base} < 48$; if it reaches 32, the window advances by 32, and it falls back below 16: the invariant holds. ■

The cadence, calculated. A renormalization brings in 16 bits, and a symbol consumes its cost. A CODES_CTX byte costs about $2 \times 3.81 = 7.6$ bits (gpt-oss-120b, `bits_par_poids_entropie_tables`), hence $7.6/16 = 0.48$ reads per lane per sub-step, about 15 16-bit words per sub-step for the warp, and a window advance (64 16-bit words) every $64/15 \approx 4.2$ sub-steps [calc.]. A DELTA byte costs about $32 \times 0.034 = 1.1$ bits: an advance about every 30 sub-steps. With two registers, the word loaded at an advance is used as soon as the pointer has moved 16 more 32-bit words, about 2 CODES_CTX sub-steps; with the third, it is used only after $4.2 + 2 \approx 6$ sub-steps, and its latency is hidden: this is the reason for version 2 of the kernel. The code comment says "every ≈ 8 steps"; the calculation gives 4.2 for CODES_CTX and 30 for DELTA. The discrepancy is stated here, not measured. Guard: `mot32` returns 0 beyond $\lceil W/2 \rceil$, so a corrupted stream never makes the kernel read outside the part.

5.2.5 The per-lane table, and determinism

Proposition 5.4 (The per-lane table is correct and does not diverge)**.** *The context of word w is $\kappa(\lfloor w/4 \rfloor)$, known to the decoder before CODES_CTX is decoded. Encoder and decoder choose the same table, and theorem 5.2 applies table by table. The choice is an address, not a branch: the 32 lanes execute the same instruction.*

Proof. κ depends only on δ, decoded by the first kernel; the encoder computed it on the same δ. In theorem 5.2, point (i) holds for any table provided encoder and decoder use the same one at the same symbol; (ii) to (iv) do not look at the table. In the kernel, the table is `lut[ctx]`, a base address in shared memory; only the address read differs from lane to lane. ■

The kernel reads κ one macro-step ahead: the scale and median of the next block are loaded during the four sub-steps of the current block, so that their latency is hidden (Listing 5.2).

Listing 5.2: The loop of `corps_codes` (`moteur/crates/trc-noyaux/noyaux/decodeur.cuh`, lines 172 to 206, commit `a64740c`): one tile per warp, grid-stride walk; the class κ of the next block read ahead; the table chosen by address.

```
    for (uint32_t t = warp; t < h.T; t += nwarps) {
        EtatVoie v;
        debut_tuile(v, h, t, voie);
        const uint32_t m1 = min(t * h.N + h.N, nmacro);
        // Le mot w = m · 32 + voie appartient au bloc floor(w/4) = 8m + floor(
    voie/4) (16 octets = 4 mots).
        // Son contexte demande l'échelle du bloc et la médiane de sa ligne :
    deux lectures
        // globales, faites un macro-pas d'avance pour que leur latence se cache
    derrière les
        // quatre pas rANS du macro-pas courant (mesure du 2026-10-15). Ligne et
    reste suivis
        // sans division (le bloc avance de 8 par macro-pas).
        uint32_t bloc = (t * h.N * VOIES + voie) >> 2;
        uint32_t ligne = bloc / B, reste = bloc - ligne * B;
        bool valide = bloc < nblocs;
        uint32_t e_suiv = valide ? __ldg(ech + bloc) : 0u;
        uint32_t m_suiv = valide ? __ldg(med + ligne) : 0u;
        for (uint32_t m = t * h.N; m < m1; ++m) {
            // Au-delà des blocs réels (bourrage du flux), contexte 0 : cas que
    la
```

```
            // spécification ne dit pas (REPRISE-K), sans objet pour gpt-oss.
            const uint32_t ctx = valide ? classe_echelle(e_suiv, m_suiv) : 0u;
            const uint32_t w = m * VOIES + voie;
            bloc += 8u;
            reste += 8u;
            while (reste >= B) { reste -= B; ++ligne; }
            valide = (m + 1u < m1) && bloc < nblocs;
            if (valide) {
                e_suiv = __ldg(ech + bloc);
                m_suiv = __ldg(med + ligne);
            }
            const uint32_t* lt = lut[ctx];
            uint32_t mot = 0;
#pragma unroll
            for (uint32_t k = 0; k < 4; ++k) mot |= pas_octet(v, lt, h, voie,
    masque_inf) << (8u * k);
            if (w < mots_utiles) codes[w] = mot;
        }
        fin_tuile(v, h, t, voie, hors_l, pointeur);
    }
```

Proposition 5.5 (Determinism of the decoder)**.** *Any grid yields the same bytes, and batched decoding yields the same bytes as a single expert.*

Proof. The decoder performs only integer operations; its only atomics are integer counters (commutative). The output of each lane depends only on the tile, not on the execution order of the warps nor on the grid (one tile, one warp, grid-stride walk). The batch and the single expert call the same bodies (`corps_echelles`, `corps_codes`). ■

The test `decodeur_en_lot_egal_au_seul` checks it to the bit (three experts, one stream bit flipped in the second: it alone leaves L); its break (descriptor 0 for all) fails.

5.2.6 Witnesses and breaks

The fact to keep is in the first and fourth rows: **a corrupted LUT slot makes 8 experts wrong without taking a single lane off** L. The return to L guards the stream and the state, not the table (03 § 3.3).

5.2.7 Why a single expert is slow: the latency model

For gpt-oss, a DELTA matrix produces 259 200 bytes, i.e. 2 025 macro-steps, i.e. 8 tiles of 256 macro-steps; a CODES_CTX matrix, 4 147 200 bytes, i.e. 32 400 macro-steps, i.e. 127 tiles. An expert has 24 DELTA tiles and 381 CODES_CTX tiles. Each tile is a chain of $256 \times 4 = 1\,024$ *dependent* sub-steps: each lane's state x depends on the previous sub-step.

Proposition 5.6 (Latency model of decoding)**.** *Let τ_1 be the duration of a sub-step when a single warp occupies its SM and τ_{16} its duration when 16 warps share an SM. Decoding a single expert lasts*

$$T_{\text{expert}} \approx 1\,024\,\tau_1 + 1\,024\,\tau_{16} + 2\,t_{\text{launch}},$$

and a batch of n experts of the same layer, per expert, $T_{\text{batch}}(n)/n \approx (1\,024\,\tau_1' + n \cdot 1\,024\,\tau_{16})/n$, which tends to $1\,024\,\tau_{16}$ as n grows.

Table 5.2: K1: the decoder's witnesses and their breaks [meas.]. "Off L": lanes that do not end on L.

witness	result	break	result of the break
120b, 4 608 experts, independent bench encoder, v1 kernels	0 wrong, 0 off L	LUT $\kappa = 1$ corrupted; stream bit	8 wrong, 0 off L; 2 wrong, 64 off L, 2 pointers
120b, v2 kernels	0 wrong out of 4 608	—	—
120b, final kernels (two passes)	0 wrong out of 640 + 3 968	—	—
lot F's real file, 20b, 768 experts, against `trc-codec` and the directory, single and batched	0 wrong by both paths	LUT; stream	8 wrong, 0 off L; 8 wrong, 171 off L, 7 pointers
lot F's real file, 120b, 4 608 experts, single and batched	0 wrong, 0 off L	LUT; stream	8 wrong, 0 off L; 8 wrong, 163 off L, 8 pointers
Rust wrappers (`cargo test -p trc-noyaux`)	9 tests green	slot, bit, magic, single descriptor	fail

Data: `mesures/moteur/K/k1-justesse-*.json`, `k1-trc-gptoss20b.json`, `k1-trc-gptoss120b.json`, `k1-*casse*.json`; `mesures/moteur/K/csv/k1-justesse-par-couche.csv`, `k1-trc-par-couche.csv`.

Proof. DELTA phase: 24 tiles, so 24 warps on 24 SMs, one per SM; the phase lasts one chain of 1 024 sub-steps at τ_1 (the card is 94 % empty: 24 warps on 384 places). CODES_CTX phase: 381 warps in a single wave on 384 places (16 per SM, the shared-memory limit), one chain of 1 024 sub-steps at τ_{16}. Batched, the $24n$ DELTA warps run together (one chain, at $\tau_1' \approx \tau_1$ as long as the card is not saturated), then the n CODES_CTX waves follow one another. ■

The confrontation, with no free parameter. On the clean pass of 16:46 (120b, layer 17), DELTA takes 152 µs and CODES_CTX 195 µs, with a 2 µs gap between the launches [meas.]; hence $\tau_1 = 152/1\,024 = 148$ ns and $\tau_{16} = 195/1\,024 = 190$ ns per sub-step, about 400 and 510 cycles at 2.67 GHz [calc.]: a lone warp's sub-step is worth about ten dependent latencies (shared LUT, multiplication, ballot, two shuffles, selection, window advance). The model predicts $152 + 195 + 2 \approx 349$ µs for a single expert (measured: 343 µs, the two launches overlapping a little) and, for a batch of 4, $(152 + 4\times195)/4 = 233$ µs per expert: **233.2 µs measured** (53.35 GB/s of codes) [calc.]. Its two times are measured on A; its prediction concerns C.

The consequence. Decoding a single expert is bounded by the length of its chain (1 024 sub-steps), not by bandwidth. Only two levers: fewer sub-steps per tile (a smaller N at encoding time, a choice of the encoder that the format already carries in every stream, at the cost of 0.4 % more header per stream) or more tiles in flight (the batch). The batch bound as $n \to \infty$ is 12 441 600 B/195 µs = 63.8 GB/s of codes [calc.].

5.2.8 Throughputs, and a verdict that fails by 0.9 and 1.1 %

Threshold declared before the measurement: $0.8 \times 41.6 = 33.28$ GB/s of codes written, judged on the *worst* A pass of a series, where 41.6 GB/s is the throughput of the variant K1 replaces (decoding then reassembly of the three class streams, published by 03). Two processes per variant, 5 series A B C C B A each, card locked, mains power (Figure 5.6).

What the measurement says, in order:

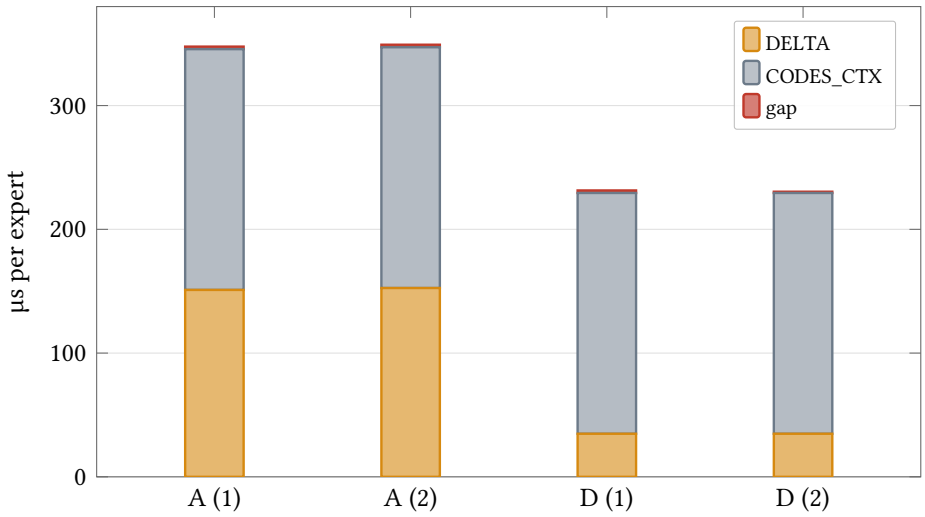


Figure 5.5: K1: breakdown of the decoding time of one expert, clean session of 20:24 [meas.]. A: DELTA 151 and 153 µs, CODES_CTX 194.6 µs, gap 2 µs. D (DELTA coded in tiles of 32 macro-steps): DELTA 34.8 µs. D's declared prediction for DELTA (19 to 22 µs) was wrong: at 8 warps per SM, the sub-step goes from 148 to 272 ns; the total holds within ±10 % because CODES_CTX dominates.

Data: `mesures/moteur/K/csv/k1-decomposition.csv`.

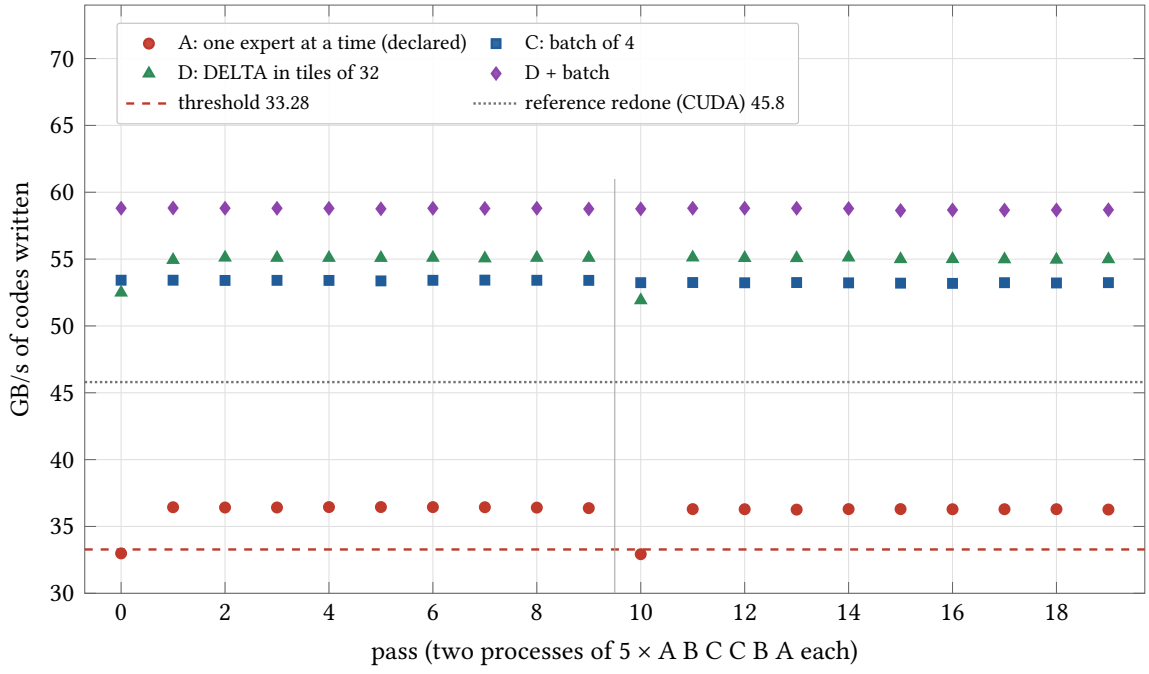


Figure 5.6: K1, decoder throughput per pass (GB/s of codes written; the time includes DELTA, scales and biases), clean session of 2026-10-15 at 20:24 [meas.]. The raw copy of the same bytes (B) runs at 105.4 GB/s, off the chart. The two worst A passes are the first series of each process (33.0 and 32.9).

Data: `mesures/moteur/K/csv/k1-debit-passes.csv`, reordered in `t2n-k1.csv`; medians and worst passes in `t2n-k1-resume.csv`.

Table 5.3: K1 and K2: throughputs (medians and worst passes of the two processes) against the declared thresholds [meas.].

variant	medians	worst	threshold	verdict
K1 A, one expert at a time (declared)	36.43 / 36.29	32.99 / 32.92	33.28	**fails by 0.9 and 1.1 %**
K1 C, batch of 4	53.41 / 53.23	53.37 / 53.18	33.28	holds
K1 D, DELTA in tiles of 32	55.09 / 55.00	52.49 / 51.92	33.28	holds (predicted 57 ± 10 %)
K1 D + batch of 4	58.80 / 58.72	58.75 / 58.64	33.28	holds
reference redone: v4 under CUDA + reassembly	45.79 / 45.75	—	—	~~41.6~~
raw copy (B)	105.45 / 105.45	—	—	—
K2, batch of 12 matrices	235.35 / 235.41	219.13 / 220.89	167.5	holds (+12 % over Vulkan)
K2, one matrix per launch	160.21 / 159.87	—	—	—

Data: `mesures/moteur/K/csv/k1-debit-passes.csv`, `k1-reference-03.csv`, `k2-debit-passes.csv`, `journaux/*-20261015-2024.*`.

1. **The declared threshold fails for the declared variant**, by 0.9 and 1.1 % (one worst pass per process: $1 - 32.99/33.28$ and $1 - 32.92/33.28$ [calc.]; ~~by 0.9 %~~, taken from `moteur/K-pour-le-memoire.md`, counted the first process only), on a first-series effect (worst pass 9 % under the others, in each process; the other nine from 36.3 to 36.5, 0.3 % apart). The rule was the worst pass: the pass is not removed. The cause is not investigated ([hyp.] the card or the cache waking up between calibration and series).
2. **The reference itself was underestimated.** ~~41.6 GB/s, the path K1 replaces~~ 03 had composed this figure with v4 *under Vulkan* (70 GB/s); redone under CUDA in the same session (v4 82.69 / 82.57 GB/s, reassembly 102.63 / 102.61), the replaced path runs at **45.8** GB/s. Against it ($0.8 \times 45.8 = 36.6$), A's median fails too.
3. **The latency model explains why** (Proposition 5.6) and predicts the two remedies.
4. **Both remedies hold against both references**: D without touching the format (the encoder chooses $N = 32$ for DELTA, +16.9 KB per expert, +0.14 % of the chunk), C without touching the encoder; together, 58.8 GB/s.
5. D's prediction was right on the total (55.0 for 57 ± 10 %) and wrong on its DELTA part (34.8 µs for 19 to 22): the model ignores contention at 8 warps per SM.

Proposal to the arbiter (not decided at the time this chapter was written): D plus batched decoding of the misses of one layer; the fallback declared by the specification (class streams, kind 0x05) is not needed, it is slower (45.8) than D (55.0). No consequence for J1: 0.34 ms per expert against 383 ms of disk per token measured at the milestone (Section 8.5.1; ~~157 ms~~, Volume I's prediction, is not a measurement); the elasticity of throughput to the decoder is zero as long as the disk limits.

5.3 The MXFP4 matrix–vector product (K2)

5.3.1 The algorithm

$y = Wx + b$ on the SoA slot. A warp handles 4 rows; lane l takes blocks $b = l, l + 32, l + 64, \ldots$ of these rows, loads the 32 values of x of a block once and applies them to the 4 rows; then a butterfly reduction over the 32 lanes (5 levels, `__shfl_xor_sync`). A block of 256 threads covers 32 rows. Codes and scales are read "streaming" (`__ldcs`): weights are not reused. The value of a weight is

$k[c] \cdot 2^{e-128}$, with $k = \{0, 1, 2, 3, 4, 6, 8, 12\}$ signed by bit 3: the doubled E2M1 magnitudes, hence -128 instead of E8M0's -127 (Figure 5.2).

Listing 5.3: The exact E8M0 scale, the value of a code and the product of a block in a fixed order (`moteur/crates/trc-noyaux/noyaux/gemv.cuh`, lines 25 to 55, commit `4473050`). The constant `0xC8643210` stores the eight doubled magnitudes, one per nibble.

```
// 2^(e - 128) en f32, exact pour tout e de 0 à 255 (e = 0 et 1 : sous-normaux).
__device__ __forceinline__ float echelle_e8m0(uint32_t e) {
    const uint32_t bits = e >= 2u ? (e - 1u) << 23 : (0x00200000u << e);
    return __uint_as_float(bits);
}

// Grandeur doublée d'un code E2M1 de 4 bits : les huit grandeurs sont rangées par
// demi-octet dans 0xC8643210 ; le bit 3 est le signe.
__device__ __forceinline__ float valeur_e2m1x2(uint32_t c) {
    const float g = float((0xC8643210u >> ((c & 7u) << 2)) & 15u);
    return (c & 8u) ? -g : g;
}

// Contribution d'un bloc : sum_i k[code_i] · x_i, dans un ordre fixe (mot t = 0..3,
// octet k = 0..3, demi-octet bas puis haut). xs[0..3] = x[0..15], xs[4..7] = x[16..31].
__device__ __forceinline__ float produit_bloc(const uint4 q, const float4 (&xs)[8]) {
    const uint32_t mots[4] = {q.x, q.y, q.z, q.w};
    float s = 0.0f;
#pragma unroll
    for (int t = 0; t < 4; ++t) {
        const uint32_t w = mots[t];
        const float xa[4] = {xs[t].x, xs[t].y, xs[t].z, xs[t].w};
        const float xb[4] = {xs[4 + t].x, xs[4 + t].y, xs[4 + t].z, xs[4 + t].w};
#pragma unroll
        for (int k = 0; k < 4; ++k) {
            s = fmaf(valeur_e2m1x2((w >> (8 * k)) & 15u), xa[k], s);
            s = fmaf(valeur_e2m1x2((w >> (8 * k + 4)) & 15u), xb[k], s);
        }
    }
    return s;
}
```

Lemma 5.7 (The E8M0 scale is exact). *For every $e \in [0, 255]$, `echelle_e8m0`(e) returns exactly 2^{e-128} in f32.*

Proof. For $e \geq 2$, the bits $(e-1) \ll 23$ give a normal f32 with biased exponent $e-1$, of value $2^{e-1-127} = 2^{e-128}$, zero mantissa; $e - 1 \leq 254$ stays below the reserved exponent 255. For $e = 0$ and $e = 1$, 2^{-128} and 2^{-127} are subnormal: a subnormal with mantissa bits m is worth $m \cdot 2^{-149}$, and $\texttt{0x00200000} = 2^{21}$, hence $2^{21+e-149} = 2^{e-128}$. The 256 values are moreover checked one by one (`e8m0_256_valeurs_fausses` $= 0$). The kernel is compiled without `--use_fast_math`, which would flush subnormals to zero. ∎

5.3.2 Exact to the bit on integers

Lemma 5.8 (Exact sums in f32). *If all products and all partial sums of a computation, in any order, are multiples of 2^{-m} of absolute value below 2^{24-m}, then the computation in f32 (round to nearest) is exact*

whatever the order.

Proof. A multiple of 2^{-m} of absolute value below 2^{24-m} is $a \cdot 2^{-m}$ with $|a| < 2^{24}$ an integer: it fits in the 24 mantissa bits of an f32, it is representable. Every operation (product, sum, `fmaf`) has an exact representable result; IEEE 754 rounding of a representable number leaves it unchanged [24]. By induction on the operations, none rounds. ■

The witness, set up. Scales $e \in [117, 123]$, inputs $x \in \{-2, \dots, 2\}$. Each term $k \cdot x \cdot 2^{e-128}$ is a multiple of 2^{-11} (since $e - 128 \geq -11$ and k, x are integers), of absolute value at most $12 \times 2 \times 2^{-5} = 0.75$; every partial sum of a row of 2 880 terms is a multiple of 2^{-11} of absolute value at most $2\,880 \times 0.75 = 2\,160 < 2^{12}$ [calc.]: it fits in $12 + 11 = 23$ bits. By lemma 5.8, the card must equal the exact sum *to the bit.* Measured: 0 wrong rows on 5 shapes (including $2\,880 \times 2\,880$, 37×96, 5×32); the break (reference ×1.01) makes 8 639 rows out of 8 640 fail [meas.]. The row that holds under the break was not examined: [hyp.] a row whose sum is zero, which ×1.01 does not change.

5.3.3 On real weights: the error bound

Proposition 5.9 (A priori bound of the GEMV)**.** *For one row, the sum is done in f32 in three stages: the 32 terms of a block (fixed order), the $\lceil B/32 \rceil$ blocks of a lane (at most 3 for $B = 90$), then the butterfly tree over 32 lanes (5 levels). The result satisfies*

$$|\hat{y} - y| \leq \gamma_{n_e} \sum |terms|, \qquad \gamma_n = \frac{nu}{1 - nu},\ u = 2^{-24},\ n_e = 32 + 3 + 5 = 40,$$

i.e. $\gamma_{40} \approx 2.4 \cdot 10^{-6}$ times the sum of absolute values.

Proof. Each term goes through at most n_e chained operations (one per term of the block, one per block of the lane, one per level of the tree), each with relative error at most u; the classical bound for recursive summation applies to every path [25]. The scale is exact (lemma 5.7) and multiplies with no error beyond its own, counted in the chain. ■

The declared threshold (10^{-5} of $\max|y|$) assumes $\sum|\text{terms}| \leq 4 \max|y|$. Measured on 24 real matrices of the 120b (8 experts × 3): $2.9 \cdot 10^{-7}$ **at worst**, 34 times under the threshold (Figure 5.7) [meas.]. Determinism follows from the fixed order: batched and single give the same bits (`lot_different_du_seul` = 0).

5.3.4 The roof

A $2\,880 \times 2\,880$ matrix reads 4 406 400 bytes of MXFP4 equivalent (codes and scales), plus 11 520 of biases and 11 520 of x. VRAM bandwidth (128-bit bus, GDDR6) bounds the throughput; K2's threshold is $0.8 \times 209.4 = 167.5$ GB/s (03's raw MXFP4 GEMV under Vulkan). Measured: **235.35 and 235.41 GB/s** for a batch of 12 matrices (one layer of 4 experts), worst passes 219.1 and 220.9; one matrix per launch runs at 160 GB/s (Figure 5.8) [meas.]. A layer takes 230 µs; the expert GEMVs of a 120b token (36 layers), about 8.3 ms [calc.], to compare with the 383 ms of disk per token measured at the milestone (157 predicted by Volume I).

5.4 The forward pass: twelve kernels (K3)

5.4.1 The kernels and their structures

The gpt-oss forward pass, as `trc-reference` (lot R) wrote it from the definition published by OpenAI and validated against llama.cpp, splits into twelve kernels: `gemv_mxfp4` (K2), `gemv_q8_0`,

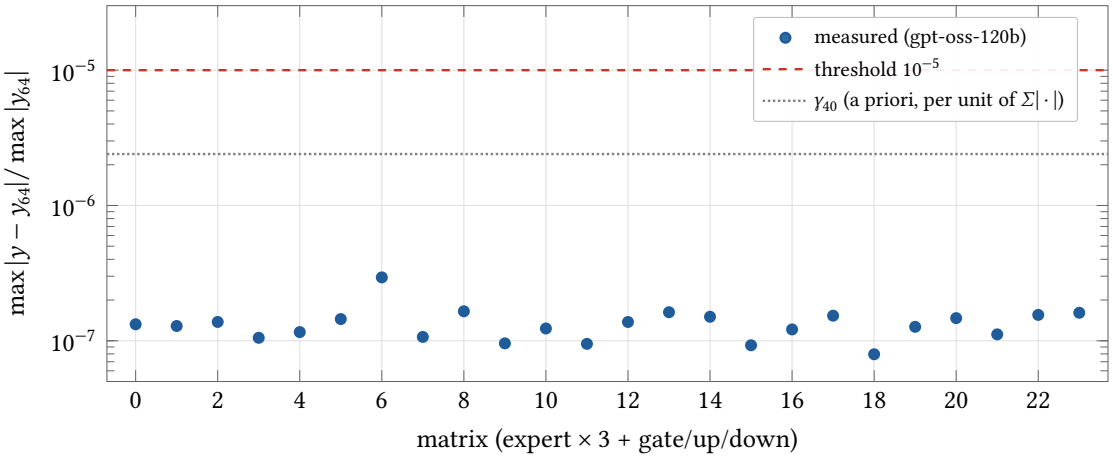


Figure 5.7: K2: relative error $\max|y - y_{64}| / \max|y_{64}|$ of the GEMV against an f64 reference, 24 real matrices of gpt-oss-120b [meas.]. Dotted, γ_{40}: the a priori bound per unit of $\sum|\text{terms}|$ (Proposition 5.9); real errors cancel out, and the measured error stays an order of magnitude below it.

Data: `mesures/moteur/K/csv/k2-ecarts-reels.csv` (from `k2-justesse-gptoss120b.json`).

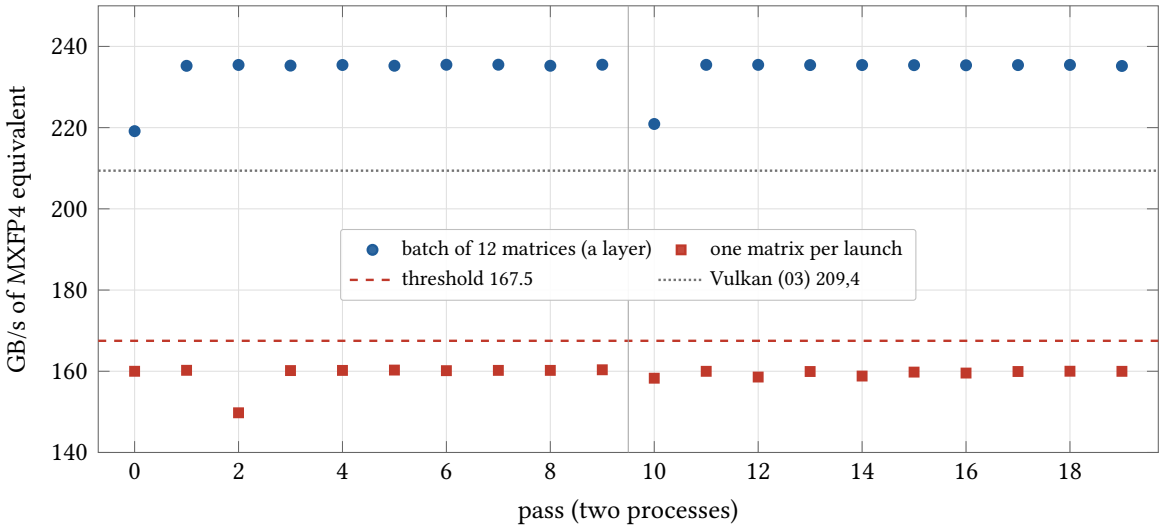


Figure 5.8: K2, throughput of the MXFP4 GEMV per pass (GB/s of MXFP4 equivalent), clean session of 20:24 [meas.]. The batch of 12 matrices exceeds 03's Vulkan reference by 12 %; one matrix per launch pays for the wave tail and the launch.

Data: `mesures/moteur/K/csv/k2-debit-passes.csv`, reordered in `t2n-k2.csv`.

`gemv_f32`, `rmsnorm` [26], `rope_yarn` [27, 28], `ecrire_kv`, `attention` (with a learned sink per head [29]), `routeur_topk`, `swiglu_oai` [30], `combiner`, `ligne_q8_0` and `argmax`. Figure 5.9 lays them out in the order of a layer.

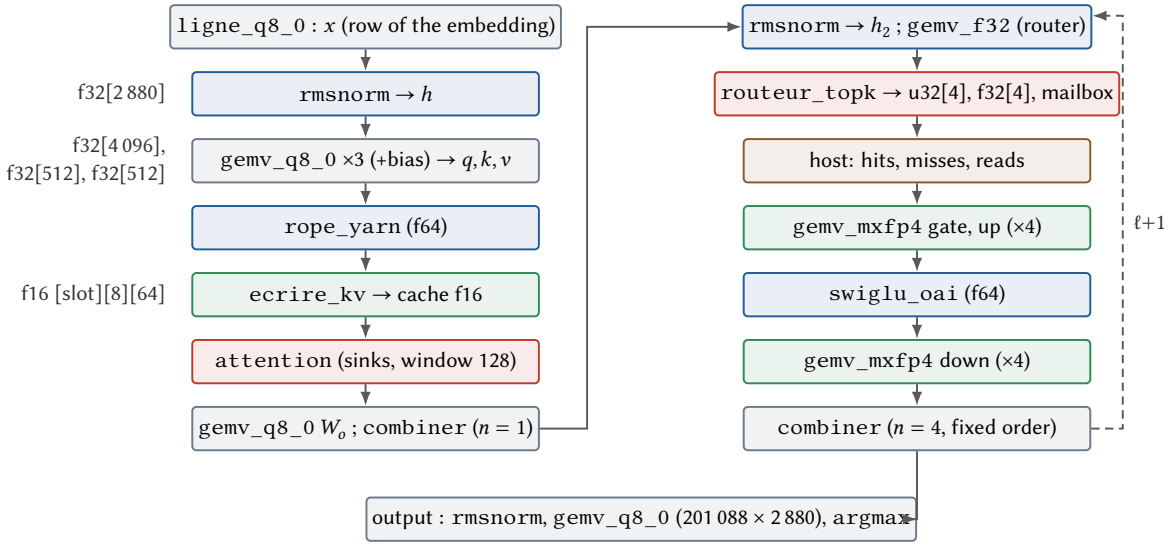


Figure 5.9: The forward pass of a gpt-oss layer, kernel by kernel ($d = 2\,880$, 64 query heads, 8 key and value heads of dimension 64). The left half precedes the router; the mailbox carries its decision to the host (Chapter 4), which decides hits and misses; the right half applies the four experts and combines them in a fixed order.

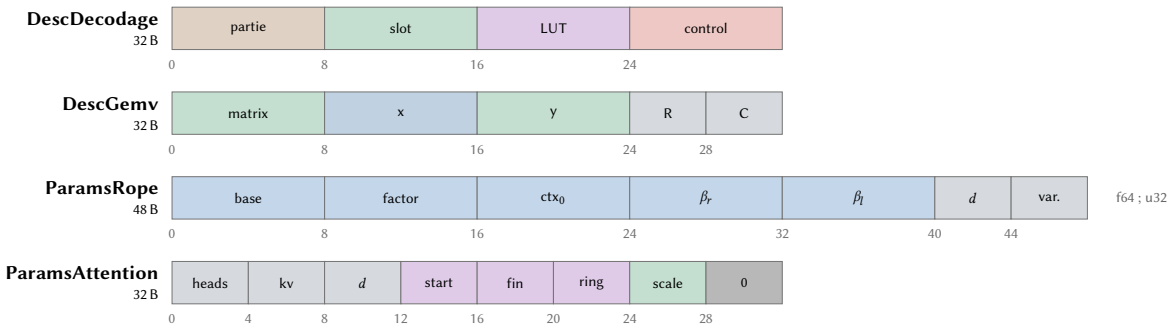


Figure 5.10: The structures passed to the kernels, byte by byte (same layout on the Rust side, `#[repr(C)]`, `moteur/crates/trc-noyaux/src/abi.rs`). `DescDecodage` and `DescGemv` describe one element of a batch; `ParamsRope` (alignment 8) carries YaRN in f64 (base 150 000, factor 32, original context 4 096, fast β 32 and slow 1) and the variant (0: OpenAI; 1: llama.cpp); `ParamsAttention` carries the range read and the ring (0, or 128 on a windowed layer).

5.4.2 The witness protocol

Each kernel is judged *alone*, on **real** weights read from the original GGUF by lot R's reader (a path independent of the `.trc`), at layers 0 (windowed) and 1 (full) of gpt-oss-20b then gpt-oss-120b. Its input is the reference's f64 output rounded to f32 (errors do not compound); its reference is the corresponding operation of `trc-reference`, applied in f64 *to the same f32 values*. The error is $e = \max_i |y_i - y_i^{64}| / \max_i |y_i^{64}|$ over the whole output vector. Each call is made twice: both outputs must be equal to the bit. Each break injects a real, plausible fault *into the reference*; only the targeted witness must fail, the other eleven must hold.

Table 5.4: The buffers of the forward pass, bit for bit.

structure	bytes	fields
mailbox slot	$4(2k+1) = 36$ useful, within 64	word 0: sequence, written last after `__threadfence_system`; words 1 to k: indices; words $k+1$ to $2k$: weights (f32 bits). Protocol of trial E0 (c), Chapter 4
KV cache of a layer	2·slots·512 per half	f16 `[slot][kv_head][64]`, for K and for V; 128 slots on an even layer (ring), otherwise the context length
Q8_0 block (ggml)	34	f16 d, then 32× i8; a block's address is even, hence reads by 16-bit words
activations	—	contiguous f32

5.4.3 A single rounding: the f64 kernels

Proposition 5.10 (A single rounding holds to 2^{-24})**.** *A kernel that computes in f64 on the card and rounds once to f32 (`rmsnorm`, `rope_yarn`, `swiglu_oai`, the router weights) satisfies* $e \leq 2^{-24} + O(2^{-50}) \approx 5.96 \cdot 10^{-8}$.

Proof. Let y be the exact value and $\hat{y}_{64}$ the f64 computation: $|\hat{y}_{64} - y| \leq c \cdot 2^{-53}|y|$, with c small (about ten operations, plus the error of CUDA's double-precision `pow`, `log`, `exp`, `sin`, `cos`, 1 to 2 ulp). Round to nearest adds $|\mathrm{fl}_{32}(\hat{y}_{64}) - \hat{y}_{64}| \leq 2^{-24}|\hat{y}_{64}|$. Hence $|\hat{y} - y| \leq (2^{-24} + c\,2^{-53})|y|(1 + O(2^{-24}))$ for each component, and e, which normalizes by the maximum, is bounded the same way. ∎

Measured (20b then 120b): `rmsnorm` 5.2 and $5.6{\cdot}10^{-8}$; `rope_yarn` 5.3 and $4.6{\cdot}10^{-8}$; `swiglu_oai` 4.9 and $3.5{\cdot}10^{-8}$; router weights 4.7 and $5.0{\cdot}10^{-8}$ [meas.]. All are under the bound, and almost at its level: the final rounding is the only visible error. Seam N's threshold (10^{-6}) leaves a 17-fold margin.

Why the rotation is computed in f64, in a single computation. The angle is $p\,\theta_i$, with $\theta_0 = 1$. In f32, the angle at position p carries an error of order $2^{-24}p$: $2.4 \cdot 10^{-4}$ rad at $p = 4\,096$, $7.8 \cdot 10^{-3}$ rad at $p = 131\,072$ [calc.]. An angle error δ gives a relative error of order δ on the rotated pair; the 10^{-6} threshold would be missed as soon as $2^{-24}p > 10^{-6}$, i.e. $p > 10^{-6} \times 2^{24} \approx 17$ [calc.]. In f64, the angle error is $2^{-53}p$, negligible at any position. The kernel therefore computes in f64 the YaRN frequency (ramp between the β bounds and concentration $0.1 \ln s + 1$), the angle, the sine and cosine, and the rotation (Listing 5.4).

Listing 5.4: `inv_freq_yarn` (`moteur/crates/trc-noyaux/noyaux/passe.cuh`, lines 145 to 167, commit `c09551e`): the frequency of pair i and the concentration, in f64. Variant 1 rounds the ramp bounds as llama.cpp does; variant 0, the model's, does not.

```
// inv_freq de la paire i et facteur de concentration, en f64 (définition : trc-reference,
// ops::Rope::yarn, depuis la configuration publiée de gpt-oss).
__device__ inline double inv_freq_yarn(const ParamsRope& p, uint32_t i, double& concentration) {
    const uint32_t demi = p.dim_tete / 2u;
    const double freq = pow(p.base, double(2u * i) / double(p.dim_tete));
    if (p.facteur <= 1.0) {
        concentration = 1.0;
        return 1.0 / freq;
    }
    concentration = 0.1 * log(p.facteur) + 1.0;
    const double deux_pi = 6.28318530717958647692528676655 9;
```

```
    double bas = double(demi) * log(p.ctx_origine / (p.beta_rapide * deux_pi)) /
    log(p.base);
    double haut = double(demi) * log(p.ctx_origine / (p.beta_lent * deux_pi)) /
    log(p.base);
    if (p.variante == 1u) {
        bas = fmax(floor(bas), 0.0);
        haut = fmin(ceil(haut), double(p.dim_tete - 1u));
    }
    const double interpolation = 1.0 / (p.facteur * freq);
    const double extrapolation = 1.0 / freq;
    const double rampe = fmin(fmax((double(i) - bas) / (haut - bas), 0.0), 1.0);
    const double masque = 1.0 - rampe;
    return interpolation * (1.0 - masque) + extrapolation * masque;
}
```

5.4.4 The exact kernels

Proposition 5.11 (The embedding is exact). *`ligne_q8_0` returns exactly $d \cdot q$ for every weight of a Q8_0 row.*

Proof. d is an f16, with 11 mantissa bits; q is an i8, $|q| \le 128$, with at most 7 significant bits ($128 = 2^7$ has only one). The product fits in 18 bits, under the 24 of an f32, and stays within the range of normals or exact subnormals ($|dq| \le 65\,504 \times 128$, and $\ge 2^{-24}$ if non-zero): it is exact. ■

Measured: 0 differences on 64 rows, the first and last included [meas.]. For `gemv_q8_0`, `gemv_f32` and `gemv_mxfp4`, lemma 5.8 gives exact witnesses: Q8_0 with $d = 2^e$, $e \in [-2, 1]$, $q \in [-127, 127]$, $x \in \{-1, 0, 1\}$, sums multiples of 2^{-2} bounded by $128 \cdot 32 \cdot 127 \cdot 2 = 1\,040\,384 < 2^{22}$ for $C = 4\,096$; f32 with integer W, $x \in [-8, 8]$, sum at most 184 320; MXFP4 with $e \in [127, 129]$, $x \in [-2, 2]$, multiples of 2^{-1} bounded by 138 240 [calc.]. Measured: 0 differences on 5 shapes, up to $2\,880 \times 4\,096$ [meas.]. On real weights, the bound of Proposition 5.9 holds with $n_e = 32 + 1 + \lceil B/32 \rceil + 5$: 41 for $C = 2\,880$, 42 for $C = 4\,096$ (`attn_output`), hence $\gamma \approx 2.5 \cdot 10^{-6}$; measured: 1.6 to $2.6 \cdot 10^{-7}$ on q, k, v, o, the router, the experts and the 201 088-row output, 39 to 62 times under 10^{-5} [meas.].

Proposition 5.12 (The combiner is bit-equal to the host). *`combiner` returns, for each component, exactly the value the host computes in f32 for $x + (((g_0 s_0 + g_1 s_1) + g_2 s_2) + g_3 s_3)$, in that order.*

Proof. IEEE 754 fixes the result of every addition and multiplication under round to nearest even: same operands, same order, same mode, same result [24]. The kernel uses `__fmul_rn` and `__fadd_rn`, which the compiler never contracts into an fma [31]; the host (Rust, x86-64, SSE) never contracts. The two sequences of operations are therefore identical, operation by operation. ■

Break made in the source: with an `fmaf` instead, the test fails from element 649 (`tests/passe.rs`) [meas.]. One rounding fewer changes bits.

Listing 5.5: `combiner` (`passe.cuh`, lines 527 to 542, commit `c09551e`): every product and every sum rounded separately; a null `poids` serves the attention residual ($n = 1$, weight 1, no multiplication).

```
// résidu[i] += ((₀w ₀s[i] + ₁w ₁s[i]) + ₂w ₂s[i]) + … dans l'ordre j = 0, 1, …,
    n − 1, chaque
// produit et chaque somme arrondis séparément (__fmul_rn, __fadd_rn : aucune
    contraction en fma),
// pour égaler au bit la même somme faite sur l'hôte en f32. s_j = sorties + j ·
    pas. `poids` nul :
// tous les poids valent 1 (s_j pris tel quel ; sert à x ← x + a Wo^T + bo).
    Grille : ceil(d / 256) × 256.
```

```
extern "C" __global__ void __launch_bounds__(256)
combiner(float* __restrict__ residu, const float* __restrict__ sorties, uint32_t pas, const float* __restrict__ poids,
         uint32_t n, uint32_t d) {
    const uint32_t i = blockIdx.x * 256u + threadIdx.x;
    if (i >= d || n == 0) return;
    float s = poids != nullptr ? __fmul_rn(poids[0], sorties[i]) : sorties[i];
    for (uint32_t j = 1; j < n; ++j) {
        const float sj = sorties[uint64_t(j) * pas + i];
        s = __fadd_rn(s, poids != nullptr ? __fmul_rn(poids[j], sj) : sj);
    }
    residu[i] = __fadd_rn(residu[i], s);
}
```

5.4.5 Attention, the smallest margin

The attention of one position, one query head per block of 256 threads, runs in three phases (Listing 5.6): (1) the scores $s_j = (q \cdot k_j)/8$, an f32 product over $d = 0..63$ in order; (2) the softmax with the head's sink in the denominator: $m = \max(\max_j s_j, \text{sink})$, $e_j = \exp(s_j - m)$, $z = \sum_j e_j + \exp(\text{sink} - m)$, both reductions through a fixed tree over 256 threads; (3) the weighted sum of the values in four groups ($j \equiv g \bmod 4$, each in order), combined $((g_0 + g_1) + (g_2 + g_3))$. The slot of a position is the position, or the position modulo 128 on a windowed layer.

Listing 5.6: The body of `attention` (`passe.cuh`, lines 325 to 371, commit `c09551e`): scores, softmax with sink, sum in four groups; no atomic, two fixed trees.

```
    // 1. scores
    for (uint32_t j = t; j < n; j += ATTENTION_FILS) {
        const uint32_t pos = p.debut + j;
        const uint32_t c = p.anneau ? pos % p.anneau : pos;
        const uint4* kr = reinterpret_cast<const uint4*>(cache_k + uint64_t(c) * dkv + uint64_t(hk) * ATTENTION_DIM);
        float s = 0.0f;
#pragma unroll
        for (int b = 0; b < 8; ++b) {
            const uint4 w = kr[b];
            const uint32_t mots[4] = {w.x, w.y, w.z, w.w};
#pragma unroll
            for (int u = 0; u < 4; ++u) {
                s = fmaf(qs[8 * b + 2 * u], f16_vers_f32(mots[u] & 0xFFFFu), s);
                s = fmaf(qs[8 * b + 2 * u + 1], f16_vers_f32(mots[u] >> 16), s);
            }
        }
        scores[j] = s * p.echelle;
    }
    __syncthreads();

    // 2. softmax avec le puits dans le dénominateur
    float m = -INFINITY;
    for (uint32_t j = t; j < n; j += ATTENTION_FILS) m = fmaxf(m, scores[j]);
    const float puits_h = puits[h];
    m = fmaxf(reduire_bloc<true>(part, m), puits_h);
    float z = 0.0f;
    for (uint32_t j = t; j < n; j += ATTENTION_FILS) {
        const float e = expf(scores[j] - m);
        scores[j] = e;
        z += e;
```

```
    }
    z = reduire_bloc<false>(part, z) + expf(puits_h - m);

    // 3. somme pondérée des valeurs
    const uint32_t dd = t & (ATTENTION_DIM - 1u);
    const uint32_t g = t / ATTENTION_DIM;  // 0..3
    float a = 0.0f;
    for (uint32_t j = g; j < n; j += 4u) {
        const uint32_t pos = p.debut + j;
        const uint32_t c = p.anneau ? pos % p.anneau : pos;
        a = fmaf(scores[j], f16_vers_f32(cache_v[uint64_t(c) * dkv + uint64_t(hk)
     * ATTENTION_DIM + dd]), a);
    }
    groupes[g][dd] = a;
    __syncthreads();
    if (t < ATTENTION_DIM)
        sortie[uint64_t(h) * ATTENTION_DIM + t] = ((groupes[0][t] + groupes[1][t
    ]) + (groupes[2][t] + groupes[3][t])) / z;
}
```

Proposition 5.13 (Bound for attention). *The relative error of the output is of order* $\max_j |\delta s_j| + \gamma_{n_a}$, *with* $|\delta s_j| \le \gamma_{64} \sum_t |q_t k_{j,t}|/8$, $\gamma_{64} \approx 3.8 \cdot 10^{-6}$, *and* $n_a \approx 1\,024$ *chained additions for* $n = 4\,096$ *positions* ($\gamma \approx 6 \cdot 10^{-5}$ *at worst).*

Proof. The score is a sum of 64 f32 products (Proposition 5.9 with $n_e = 64$). An error δs multiplies the weight by $\exp(\delta s) \approx 1 + \delta s$; the output, a weighted mean, inherits a relative error of order $\max_j |\delta s_j|$. To this add the sums of e_j and $e_j v_j$: at most $n/4$ terms per group, then a tree of 4, about 1 024 chained additions for $n = 4\,096$. These are worst-case bounds; real errors cancel out. ■

Measured: 2.2 to $6.2{\cdot}10^{-6}$ on the windowed layer, 0.46 to $1.9{\cdot}10^{-6}$ on the full layer (up to 4 096 positions), for a 10^{-5} threshold [meas.]: **the smallest margin, 1.6** on the 20b (2.5 on the 120b). If C4 required it, f64 scores would cost about 0.17 ms per layer (~~at most 0.15 ms~~) [calc.: 3.36×10^7 operations, namely 64 heads ×4 096 × 64 × 2 operations at about 0.2 Tflop/s in f64], with no effect compared with the disk.

5.4.6 Selections, and determinism as a whole

Proposition 5.14 (Selections are deterministic). *`argmax` and `routeur_topk` return the same element whatever the reduction tree; ties are broken by the smallest index.*

Proof. Both kernels take the minimum of the key $(-v, i)$ over the non-NaN values (`meilleur`: larger value, then smaller index; a NaN is never better). This is a total order; the minimum of a set under a total order is unique, and the "best of two" operation is associative and commutative: any reduction, whatever its tree, returns that element. The tree is fixed anyway. ■

Checked on vectors with ties (four equal maxima, 128 zeros, maxima at indices 64 and 127); the "largest index" break makes only those cases fail [meas.]. `routeur_topk` then computes the softmax of the k retained values in f64 and writes the mailbox slot in the order of trial E0 (c)'s protocol: indices and weights, system fence, then the sequence (Listing 5.7).

Listing 5.7: The body of `routeur_topk` (`passe.cuh`, lines 461 to 514, commit `c09551e`): k rounds of butterfly reduction on the total order, then the softmax of the k in f64 and the mailbox slot, sequence last.

```
    float v[8];
    bool pris[8];
#pragma unroll
    for (uint32_t i = 0; i < 8; ++i) {
        const uint32_t e = voie + 32u * i;
        v[i] = e < E ? logits[e] : -INFINITY;
        pris[i] = e >= E;
    }
    float choisi_v[8];
    uint32_t choisi_i[8];
    for (uint32_t r = 0; r < k; ++r) {
        float bv = -INFINITY;
        uint32_t bi = 0xFFFFFFFFu;
#pragma unroll
        for (uint32_t i = 0; i < 8; ++i)
            if (!pris[i] && meilleur(v[i], voie + 32u * i, bv, bi)) {
                bv = v[i];
                bi = voie + 32u * i;
            }
#pragma unroll
        for (int d = 16; d > 0; d >>= 1) {
            const float ov = __shfl_xor_sync(0xFFFFFFFFu, bv, d);
            const uint32_t oi = __shfl_xor_sync(0xFFFFFFFFu, bi, d);
            if (meilleur(ov, oi, bv, bi)) {
                bv = ov;
                bi = oi;
            }
        }
        if (bi != 0xFFFFFFFFu && (bi & 31u) == voie) pris[bi >> 5] = true;
        choisi_v[r] = bv;
        choisi_i[r] = bi;
    }
    if (voie != 0) return;
    double e[8];
    const double m = choisi_v[0];
    double z = 0.0;
    for (uint32_t r = 0; r < k; ++r) {
        e[r] = exp(double(choisi_v[r]) - m);
        z += e[r];
    }
    for (uint32_t r = 0; r < k; ++r) {
        const float w = float(e[r] / z);
        indices[r] = choisi_i[r];
        poids[r] = w;
        if (boite != nullptr) {
            reinterpret_cast<volatile uint32_t*>(boite)[1 + r] = choisi_i[r];
            reinterpret_cast<volatile uint32_t*>(boite)[1 + k + r] =
    __float_as_uint(w);
        }
    }
    if (boite != nullptr) {
        __threadfence_system();
        reinterpret_cast<volatile uint32_t*>(boite)[0] = sequence;
    }
}
```

The floating-point reductions of `rmsnorm` and `attention` follow a fixed tree over 256 threads,

with no atomic; those of the GEMVs, a fixed butterfly over 32 lanes. Each kernel is therefore a deterministic function of the bytes it reads (Definition 5.1), and every witness call, made twice, gives the same bits.

5.4.7 Results, kernel by kernel

Table 5.5: K3: each kernel against `trc-reference` on real weights, and its break [meas.]. Margin: threshold / worst error. The witnesses count 143 cases (20b) and 116 (120b); every call is deterministic to the bit; each break makes only its own witness fail.

kernel	threshold	worst 20b	worst 120b	margin 20b	break (in the reference): effect
`gemv_mxfp4`	10^{-5}	$2.4 \cdot 10^{-7}$	$1.6 \cdot 10^{-7}$	41	scale 2^{e-127}: error 0.66
`gemv_q8_0`	10^{-5}	$1.9 \cdot 10^{-7}$	$2.5 \cdot 10^{-7}$	53	bias omitted: 0.70, except `attn_k` (zero bias)
`gemv_f32`	10^{-5}	$2.6 \cdot 10^{-7}$	$2.2 \cdot 10^{-7}$	39	bias omitted: 0.28
`rmsnorm`	10^{-6}	$5.2 \cdot 10^{-8}$	$5.6 \cdot 10^{-8}$	19	mean over $n-1$: $1.7 \cdot 10^{-4}$
`rope_yarn`	10^{-6}	$5.3 \cdot 10^{-8}$	$4.6 \cdot 10^{-8}$	19	without the concentration: 0.35
`attention`	10^{-5}	$6.2 \cdot 10^{-6}$	$3.9 \cdot 10^{-6}$	1.6	sink omitted: 1.0
`routeur_topk` (weights)	10^{-6}	$4.7 \cdot 10^{-8}$	$5.0 \cdot 10^{-8}$	21	tie to the largest index: 18 indices differ
`swiglu_oai`	10^{-6}	$4.9 \cdot 10^{-8}$	$3.5 \cdot 10^{-8}$	20	$\alpha = 1$: 0.28
`combiner`	equality	0	0	—	reversed order: 5 762 differences
`ecrire_kv`	equality	0	0	—	round toward zero: 392 654 differences
`ligne_q8_0`	equality	0	0	—	neighbouring row: 176 448 differences
argmax, router indices	equality	0	0	—	tie to the largest index: 3 differences

Data: `mesures/moteur/K/csv/k3-par-noyau.csv` (rows `temoin` and `casse:*`), `mesures/moteur/K/k3/`.

5.4.8 The end-to-end chain

Kernels correct one by one do not yet make a correct pass: they must be chained. The witness's `chaine` mode runs the *whole* forward pass of gpt-oss-20b on the card, through the kernels of K1 to K3 alone (no host rounding between two kernels), on the first 160 real tokens of wikitext-2 window 0, against the f64 reference, layer by layer (each expert loaded once per layer).

Proposition 5.15 (Layer-by-layer order = token-by-token order)**.** *If both orders launch the same kernels with the same numerical arguments, the layer-by-layer order (all positions of a layer, then the next layer) and the token-by-token order (all layers of a position, then the next) give the same bits.*

Proof. Let x_p^ℓ be the state of position p after layer ℓ. By causality, x_p^ℓ depends only on $x_p^{\ell-1}$ and on the keys and values of positions $q \le p$ at layer ℓ, themselves functions of $x_q^{\ell-1}$. By induction on (ℓ, p) in product order, each x_p^ℓ comes out of the same kernels, on the same bytes, in the same internal order (Definition 5.1). On the ring of a windowed layer, position p is read right after it is written, and the 128 slots then hold exactly positions $p-127 \dots p$ in both orders. ∎

This is a program identity, not a measurement; lot O's engine later checked it (160 rows out of 160 equal between the chain and the engine, witness O2).

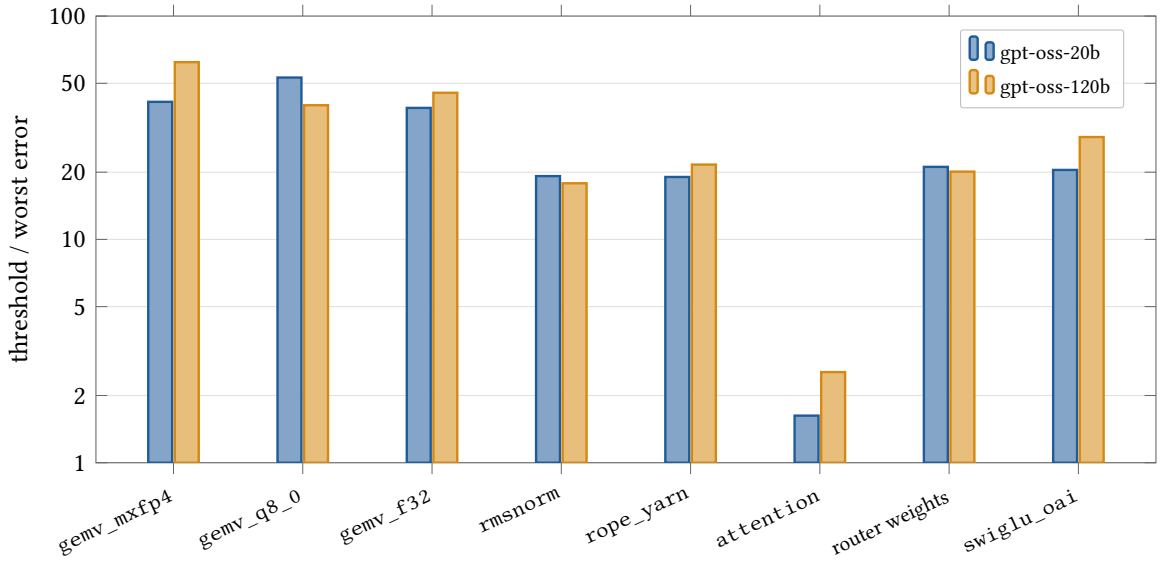


Figure 5.11: K3: the margin of each thresholded kernel (threshold / worst error, log scale), 20b and 120b [meas.]. The f64 kernels hold at 2^{-24} (margin 17 to 29 for a 10^{-6} threshold); the GEMVs, 39 to 62 times under 10^{-5}; attention is the tightest.

Data: `mesures/moteur/K/csv/k3-par-noyau.csv`, in `t2n-marges.csv`.

Table 5.6: K3: the end-to-end chain, gpt-oss-20b, 24 layers, 160 tokens [meas.]. Declared threshold: mean KL $\leq 7 \cdot 10^{-5}$.

run	mean KL	max KL	median	top-1	verdict
free routing (as declared)	$7.28 \cdot 10^{-3}$	1.02	$2.5 \cdot 10^{-6}$	158 / 160	fails
D2, imposed routing	$3.0 \cdot 10^{-6}$	$1.4 \cdot 10^{-4}$	$1.8 \cdot 10^{-7}$	160 / 160	holds
D2, break: window on odd layers	$1.6 \cdot 10^{-2}$	1.36	—	156 / 160	fails
D2, break: weights reversed	0.52	2.27	—	77 / 160	fails

Data: `mesures/moteur/K/k3/k3-chaine-*.json`, `mesures/moteur/K/csv/k3-chaine-kl.csv`.

As declared, the chain **fails** (Table 5.6): mean KL $7.28 \cdot 10^{-3}$, a hundred times the threshold. But the median is $2.5 \cdot 10^{-6}$, and the six positions that carry 99.3 % of the KL are exactly those of the first six routing flips (67, 87, 114, 129, 136, 153; Figure 5.12). The diagnosis, declared after this first result and before its code (it is written in lot K's note, 21:22), imposes the reference's experts on the chain (D2): the KL drops to $3.0 \cdot 10^{-6}$, 160 top-1 out of 160.

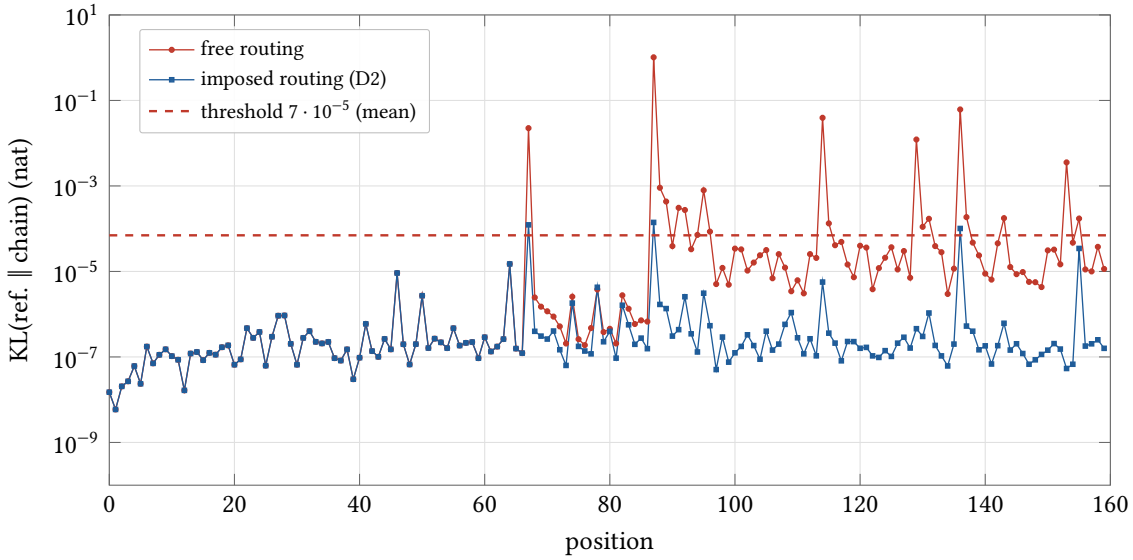


Figure 5.12: K3, the chain: KL(reference ‖ chain) per position, gpt-oss-20b [meas.]. With free routing, six peaks (positions 67, 87, 114, 129, 136, 153) carry 99.3 % of the KL; with imposed routing, they vanish and the KL stays between 10^{-9} and 10^{-4}.

Data: `mesures/moteur/K/csv/k3-chaine-kl.csv`.

Proposition 5.16 (The routing amplifier)**.** *Let μ be the reference router's margin at a decision (the gap between its 4th and 5th logits) and η the f32 chain's error on that margin. The decision flips if $\eta > \mu$ (with the right sign). A flip replaces an expert of weight g by another, and the state of the position changes by about $g\,\|s_e - s_{e'}\|$ instead of 10^{-7}: a jump, not a rounding.*

Proof. The router keeps the 4 largest logits; a perturbation η larger than the gap between the 4th and the 5th, in the direction that swaps them, exchanges the two experts. The layer's output for this position then changes by $g\,(s_{e'} - s_e)$ (up to the softmax renormalization), out of all proportion to the kernels' rounding error. The gap propagates to the following layers of the position, and a little to later positions, through their attention. ■

Measured (Figure 5.13): of 3 840 decisions, 334 have a reference margin under 10^{-2}, 34 under 10^{-3}, 4 under 10^{-4}; with free routing, 22 decisions change set (and 15 change order only). Under imposed routing, the card would have flipped 3 decisions, with margins $1.4 \cdot 10^{-4}$, $3.3 \cdot 10^{-4}$ and $7.9 \cdot 10^{-4}$; η is therefore of order 10^{-4} to 10^{-3}, and four decisions with smaller margins did not flip: the sign of η decides [meas.].

The consequence for criterion C4. The end-to-end KL of an f32 engine against the f64 reference does not measure the precision of its kernels; it counts the decisions whose margin lies under the engine's η, each at the cost of a flip. troncons (f32 activations, exact weights) must have a smaller η than llama.cpp, which brings its activations down to Q8_0. C4 will be played on that term. Lot K's

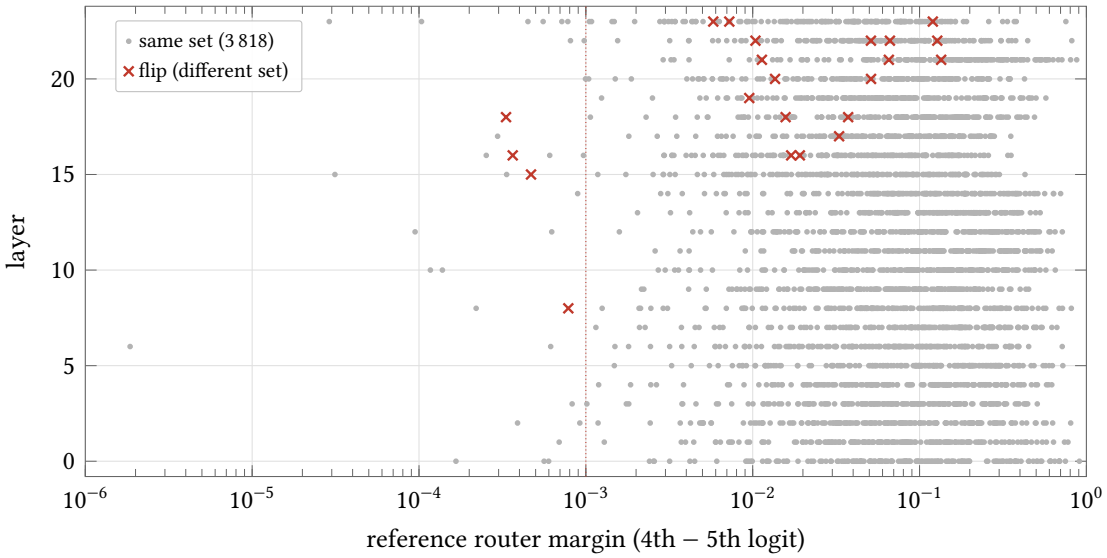


Figure 5.13: K3, diagnosis D1: every routing decision of the chain under free routing (24 layers × 160 positions), by the reference router's margin [meas.]. Crosses: the chain chooses a different set of experts. Flips concentrate at small margins; the line marks 10^{-3}.

Data: `mesures/moteur/K/k3/k3-chaine-d1-gptoss20b-marges.csv`, in `t2n-routage.csv`.

proposal to the arbiter: judge every end-to-end comparison also under imposed routing, and count the flips separately, each with its margin.

Found along the way.

- **The gpt-oss key bias is exactly 0**, in both GGUFs: 512 values out of 512, at layers 0, 1 and 5 (`k3_biais_attention.py`), whereas the biases of q and v are full ($|b|$ up to 3.9) [meas.]. Without a sink, a key bias would add $q \cdot b_k$ to all scores of a head and the softmax would cancel it; with the sink in the denominator, it would not. Its being zero is a property of the model, not a given; the "bias omitted" break revealed it by not failing on `attn_k`.
- **The window shows only after 128 positions** (Figure 5.14): under the "window" break, the KL of positions 0 to 127 is bit-equal to the sound chain's; over positions 128 to 159, it is $8.2 \cdot 10^{-2}$. An end-to-end witness shorter than the window does not see this fault.
- **The fma changes bits**: an `fmaf` put back into `combiner` breaks equality with the host (Proposition 5.12).

5.5 The round: the KV cache in f32

The independent trial found that the first token of the C12 text differed from the f64 truth's, and that the gap came from the KV cache. The round of corrections added two kernels, `ecrire_kv_f32` and `attention_kvf32`, which perform the same computations in the same order as `ecrire_kv` and `attention`, except that keys and values are kept in f32, without touching seam N's kernels (option `--kv-f32`). Witness: bit-equal to the f16 path on values representable in f16 (full cache and ring); break: non-representable values make them differ [meas.].

Proposition 5.17 (What the f16 rounding of the cache costs)**.** *The f16 rounding of a key, $\tilde{k} = k(1+\delta)$, $|\delta| \le u_{16} = 2^{-11} \approx 4.9 \cdot 10^{-4}$, perturbs the score $s = c\sum_d q_d k_d$ ($c = 1/8$, 64 terms) by at most*

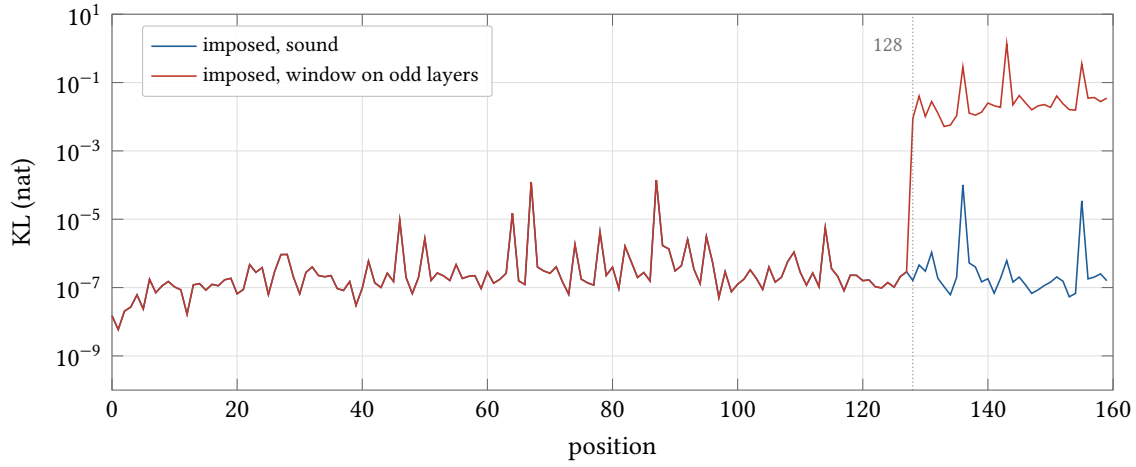


Figure 5.14: K3: the chain under imposed routing, sound and under the break "window placed on the odd layers" [meas.]. The two curves are bit-equal up to position 127; the fault appears only when the 128-slot ring starts to forget.

Data: `mesures/moteur/K/csv/k3-chaine-kl.csv` (columns `kl_impose`, `kl_impose_casse_fenetre`).

$c\,u_{16}\sum_d |q_d k_d|$ to first order, against $\gamma_{64} \approx 3.8 \cdot 10^{-6}$ for the f32 product itself. In f32, the only error left is the kernels', $u_{32}/u_{16} = 2^{-13} \approx 1/8\,192$ times smaller. The KL of a small perturbation Δz of the logits is second order, $\mathrm{KL} \approx \frac{1}{2}\mathrm{Var}_p(\Delta z)$.

Proof. The first point is the first-order expansion of s in δ. For the second, let $q_i \propto p_i e^{-\Delta z_i}$: $\mathrm{KL}(p\|q) = \mathbb{E}_p[\Delta z] + \ln \mathbb{E}_p[e^{-\Delta z}]$; expanding $\ln \mathbb{E}_p[e^{-\Delta z}] = -\mathbb{E}_p[\Delta z] + \frac{1}{2}\mathrm{Var}_p(\Delta z) + O(\Delta z^3)$ gives the result. ■

~~Divided by $8\,192^2 \approx 6.7 \cdot 10^7$, the f16 KL ($3.08{\cdot}10^{-3}$ on the 120b) would fall towards $5 \cdot 10^{-11}$ [... routing flips are not linear, but they vanish with it]~~: an estimate taken from work package O, struck out by theorem 6.9 (two regimes of the KL of a mixture of experts). The no-switch term, in u^2, is divided by $8\,192^2 \approx 6.7 \cdot 10^7$; the switch term, linear in u, does not vanish with the error: it is divided by 8 192 only. Starting from the f16 KL ($3.08{\cdot}10^{-3}$ on window 0 of the 120b), the KL in f32 is therefore at most of the order of $0.00308/8\,192 \approx 3.8 \cdot 10^{-7}$ nat [calc.] (Section 6.6.3), still far below the step of the 16-bit-per-logit format ($\approx 2.4 \cdot 10^{-4}$ nat). The only judge remains the measurement: −0.00048, the zero of the instrument (Section 6.6.4).

On the 708-token prompt, the argmaxes equal the truth's 708 out of 708 (f16: 687), routing is equal at every (layer, position), and the first token of C12 becomes the truth's [meas.]. **The f16 rounding of the KV cache carried the whole measurable gap between troncons and the exact mathematics of the model.** The cost: $18 \times (4\,096 + 128) \times 2 \times 512 \times 2\,\mathrm{B} = 155.7$ MB in f16, twice that in f32, i.e. $155.7/13.25 = 11.8$ fewer expert slots [calc.]; measured: 326 → 314 slots [meas.]. The choice for seam N (f16 or f32) lies with the arbiter.

5.6 What this chapter struck out

- ~~K1 reference: 41.6 GB/s~~: composed by 03 with v4 under Vulkan; redone under CUDA, 45.8 GB/s.
- ~~What remains: K3 (the forward pass), K4~~: K3 is done (the twelve kernels hold on both models); K4 remains.

Table 5.7: The KV cache in f16 against f32, against the judge's f64 truth [meas.]. The negative KL is that of lot R's f64 reference against the same truth ($-4.8 \cdot 10^{-4}$): a common bias of the format floor, which sets the instrument's zero.

case	mean KL	top-1	KL > 0.1	max log p gap
120b, window 0, KV f16	0.00308	95.7 %	0.39 %	3.07 nat
120b, window 0, KV f32	**−0.00048**	**100 %**	0	**0.69 steps**
20b, window 0, KV f16	0.00174	96.5 %	0.39 %	2.32 nat
20b, window 0, KV f32	**−0.00048**	**100 %**	0	2.04 steps
120b, prompt, positions 512 to 706, f16	0.00963	—	2.05 %	—
120b, prompt, positions 512 to 706, f32	**−0.00054**	—	0	—

Data: `mesures/moteur/O/corrections/journaux/verdicts-kv-2026-10-16.json`, `m5-ecart-logp-*.json`; `moteur/O-pour-le-memoire.md`, § 13.3.

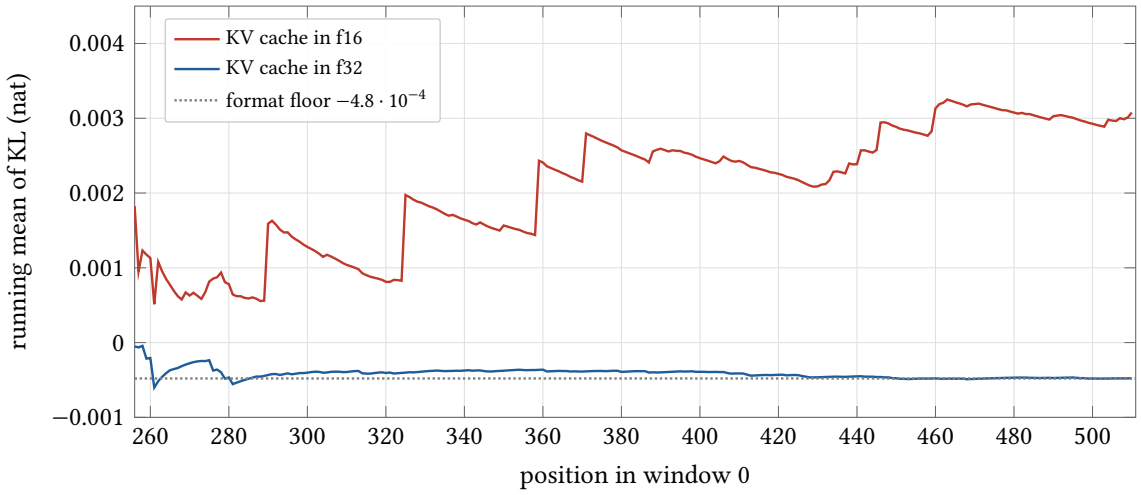


Figure 5.15: Running mean of the KL against the f64 truth, window 0 of gpt-oss-120b, scored positions 256 to 510 [meas.]. In f16, the KL climbs in jumps (routing flips) up to 0.00308; in f32, it stays at the format floor.

Data: `mesures/moteur/O/corrections/csv/kv-f16-f32-par-position.csv` (case `f0-120b`), running mean in `t2n-kv-120b.csv`.

- ~~The 120b witness rests on the bench encoder: the second encoder is proved only on the 20b~~: proved on the 120b too since 20:52 (4 608 / 4 608 against lot F's file).
- ~~D's prediction: DELTA in 19 to 22 µs~~: 34.8 µs measured (contention at 8 warps per SM); the total holds.

What could be wrong

- **K1's threshold compares two different quantities**: 41.6 GB/s is a throughput on a large stream (thousands of tiles in flight), K1's a per-expert throughput (381 tiles, one wave, plus 24 DELTA tiles alone). The comparison is cautious for us, but it is not the same quantity; C comes closer.
- **The first-series effect is not explained**; it decided K1's verdict.
- **The latency model** ignores the wave tail, the overlap of two launches and the contention of several DELTA warps on one SM: it predicted the batch right and D's DELTA part wrong.
- **The per-kernel witnesses see only layers 0 and 1**, and layer 1's input is the normalized embedding, not the real state after layer 0. A layer with sharper attention scores could bring attention closer to its threshold (margin 1.6). The chain goes through the 24 layers of the 20b and holds under imposed routing: this bounds the effect on the 20b, not on the 120b.
- **The chain ran only on the 20b**, on 160 positions of a single window; on the 120b it needs about 120 GB of reads.
- **D2 imposes the experts but keeps the card's weights**: it does not separate an error in the router weights from an error in the experts; both are bounded by the per-kernel witnesses.
- **The chain threshold ($7 \cdot 10^{-5}$) was ill-posed for an amplifying MoE**; it stayed as is and its verdict is published; the diagnosis was declared after this first result.
- **Attention reads at most** 8 192 **positions per call** (4 bytes of shared memory per position): enough for J1 (4 096), not for a long context.
- **The cadence of the word window** (4.2 sub-steps per advance for CODES_CTX) is calculated from a mean rate; the code comment says 8; neither is measured.
- **"KV f32 = truth"** is measured on one 512-token window and a 708-token prompt (120b) and one window of the 20b, not on the 10 windows.
- **Throughputs depend on the clock of a laptop GPU**; the card's state before and after each pass is logged.

Where we start, where we go. The decoder and the twelve kernels are written, proved where a proof exists, bounded where there is only a bound, and checked to the bit or under their threshold on the real weights of both models; they cost the token two orders of magnitude less than the disk. What remains is no longer correctness: the choice of the decoder fallback, that of the KV cache precision, and, next, tiled attention and J2's base mode, where the GEMV will read a table of centroids instead of the E2M1 magnitudes.

Part II

The truth and the memory

Chapter 6

The exact reference and the judge

Where we start, what this chapter establishes, what remains

At the start, the study of the models had shown that two correct engines do not return the same logits: on Qwen3.5-4B, the CPU path and the GPU path of llama.cpp were apart by a Kullback-Leibler (KL) divergence of 0.0055, while the same engine run twice was apart by nothing (at most $6.8 \cdot 10^{-5}$, the rounding of the format) [meas.]. The architecture drew a rule from it (criterion C4): troncons is not judged against another engine, it is judged against *the exact mathematics of the model*, computed in double precision. **This chapter establishes** that truth and the instrument that compares against it: a forward pass of gpt-oss in f64 that sees the weights of the file without any rounding, two computation paths equal to the bit, the YaRN ramp as OpenAI defines it (and not as llama.cpp rounds it); the theory of the KL that the judge needs, and the two defects of the instrument that its witnesses found; then the discovery of the work package: **a mixture of experts amplifies the smallest errors**, because the choice of experts is a discontinuous function of the input; 88 % of the KL disappears when routing is imposed; the f16 rounding of the KV cache carries 70 to 77 % of the routing switches of the 120b; and an f32 KV cache makes troncons equal to the f64 truth at the floor of the format. **What remains** is the f64 reference of the 120b on the ten windows (a single window and one prompt are judged), the arbiter's decision on seam N (the f32 KV cache), a criterion that judges KL distributions rather than their mean alone, and the identification of the second amplifier (the 12 % that survive imposed routing). Sources: `moteur/R-pour-le-memoire.md`, `moteur/REPRISE-R.md`, `moteur/O-pour-le-memoire.md` (part III), `mesures/epreuve-J1.md`, and the logs under `mesures/moteur/R/`, `mesures/moteur/O/corrections/`, `mesures/moteur/epreuve-J1/`.

In brief

- The reference computes exactly on the exact weights: a Q8_0 weight fits in 18 significant bits, an MXFP4 weight in 4, and an f64 offers 53 (lemma 6.1).
- Its two paths (token by token, layer by layer) return the same 4 826 112 logits to the bit (Proposition 6.2); the tiling that makes it 1 900 times thriftier in memory traffic changes no bit (Proposition 6.3).
- The KL of a perturbation Δz of the logits is exactly $\log \mathbb{E}_p e^{\Delta z} - \mathbb{E}_p \Delta z$ (Proposition 6.6): it is second order as long as the computation is smooth, and first order as soon as an expert switches (theorem 6.9).
- Measured on gpt-oss-20b: changing only the YaRN ramp (a phase difference of 10^{-4} rad per position) gives a KL of 0.0145; imposing the routing drops it to 0.00168; 1 157 positions that switch themselves carry 96.5 % of the KL.
- On the 120b, with an f32 KV cache, troncons equals the truth: KL −0.00048 (the zero of the

> instrument), 100 % top-1 agreement, routing equal on 36 layers × 708 positions; in f16: KL 0.00308.

6.1 Why judge against a mathematical truth

A lossless inference engine should return "the right logits". But which ones? Two engines that read the same bytes and are both correct do not perform the same sums in the same order, do not keep activations in the same format, do not round the cache of keys and values the same way. Their logits differ, and the measure that says so has a name: G_{moteurs}, the KL between two correct engines. Judging troncons against llama.cpp would mean demanding that it reproduce llama.cpp's approximations, which makes no sense; judging it "within G_{moteurs}" would let through any error smaller than the gap between two engines.

The architecture's answer (delegated answer of 2026-10-15, reversible; `decisions/ARCHITECTURE.md` § 12, item 9) is to judge every engine against **the function that the model defines**, computed exactly enough for its error to be invisible to the instrument. Criterion C4 then reads: the KL from the truth to troncons does not exceed the KL from the truth to llama.cpp, and troncons picks the same most likely token as the truth at least as often. The truth differs from nothing.

One must still say what "the model" is. The file being served is ggml-org's GGUF, where attention is quantised to Q8_0 while OpenAI's model carries it in bf16. The reference therefore judges *the model as stored in the file we serve*, not OpenAI's checkpoint: it is the right judge of troncons, which serves those very bytes, and it is not a judge of ggml-org's quantisation (`moteur/R-pour-le-memoire.md` § 6). For the form of the computation (attention with sinks, sliding window on every other layer, YaRN, clamped SwiGLU, router), the reference follows the implementation and configuration published by OpenAI [32, 33], not llama.cpp [34]; Section 6.2.4 shows that the difference is not academic.

6.2 The forward pass in double precision

6.2.1 Exact weights

The reference's first act is to read the weights *without rounding them.* This is not a precaution; it is an arithmetic fact that can be proved.

Lemma 6.1 (Dequantisation is exact in f64)**.** *Every weight of a Q8_0 or MXFP4 tensor of gpt-oss, and every F16 or F32 tensor, is represented exactly by an f64 number.*

Proof. Q8_0. A 34-byte block holds a scale d in f16, then 32 integers $q \in [-128, 127]$; the weight is $w = d\,q$. A normal f16 has 11 significant bits (10 stored plus the implicit bit) and an exponent from -14 to 15 (down to -24 for subnormals); $|q| \le 128$ has at most 7 significant bits ($128 = 2^7$ has only one; ~~$|q|$ fits in 7 bits plus the sign~~, false for $q = -128$, which needs 8 bits). The product of integers of 11 and 7 significant bits fits in at most 18 significant bits, and its exponent stays within $[-24 - 7, 15 + 7]$, well inside the normal range of f64 ($[-1022, 1023]$): it is exact, since f64 offers 53 significant bits. *MXFP4.* A 17-byte block holds an exponent $e \in [0, 255]$ (E8M0), then 16 bytes, two 4-bit codes per byte; the weight is $w = K[c] \cdot 2^{e-128}$, with $K = \{0, \pm1, \pm2, \pm3, \pm4, \pm6, \pm8, \pm12\}$, the doubled E2M1 values. $K[c]$ fits in at most 4 significant bits ($12 = 1100_2$) and 2^{e-128} is a power of two with exponent in $[-128, 127]$: their product is exact. *F16, F32.* Conversion to a wider format is exact. ∎

Two details of the format, which the reference had to find and which its tests guard (`moteur/crates/trc-reference/src/poids.rs`): the exponent is $e - 128$ and not $e - 127$, because K holds

the doubled E2M1 magnitudes ($0.5 \to 1$); and byte j of a block holds element j in its low nibble and element $j+16$ in its high nibble (test `mxfp4_ordre_des_demi_octets`). Table 6.1 gives the two blocks bit for bit.

Table 6.1: The two quantised blocks the reference reads, field by field, and the exact value it draws from them.

Block	Bytes	Field	Value in f64
Q8_0 (34 B, 32 weights)	0–1	d, little-endian f16	exact
	2–33	$q_0 \dots q_{31}$, i8	$w_j = d\,q_j$ (at most 18 significant bits)
MXFP4 (17 B, 32 weights)	0	e, E8M0	2^{e-128}
	$1+j$, $j<16$	bits 0–3: c_j; bits 4–7: c_{j+16}	$w_j = K[c_j]\,2^{e-128}$, $w_{j+16} = K[c_{j+16}]\,2^{e-128}$

Consequence: **all of the reference's error comes from sums and transcendental functions**, in f64, of the order of 10^{-16} relative per operation. It is invisible to an instrument that codes each logit on 16 bits (Section 6.4).

6.2.2 Two paths, one function

The reference offers two paths. The first computes one token at a time, all layers, with a KV cache, like an engine. The second computes one layer at a time for all positions of a batch of windows: it reads each expert at most once per layer. What it saves is one line of arithmetic [calc.]: token by token, ten windows of 511 positions of the 120b reread $10 \times 511 \times 36 \times 4 \times 13.254$ MB ≈ 9.75 TB of experts; layer by layer, at most $36 \times 128 \times 13.254$ MB ≈ 61.1 GB, 160 times less, and the batch fits in memory ($5\,110 \times 2\,880 \times 8$ B $= 118$ MB per activation array).

Proposition 6.2 (The two paths compute the same function, to the bit)**.** *If every scalar is computed by the same sequence of operations in both orders, the layer-by-layer pass and the token-by-token pass return, position by position, logits that are equal to the bit.*

Proof. With a causal mask, the output of position t at layer ℓ depends only on the inputs of positions $\le t$ at layer ℓ. Induction on ℓ: if the inputs of layer ℓ are equal to the bit in both orders for every position, each output of layer ℓ is a sequence of IEEE 754 operations [25] on equal operands, hence equal; this holds here because the dot product depends only on its two vectors (never on the number of positions nor on the split into threads), attention reads the same keys in the same order, and the sum of the four experts follows the order $j = 0, 1, 2, 3$. At layer 0, the inputs are the embedding rows, equal. ∎

Witness W1 turns this into an exact equality, not a tolerance: on 24 tokens of the 20b, **4 826 112 logits, 0 different**; the break (the token-by-token path with the other YaRN ramp) makes 4 625 024 of them differ, with a maximum gap of 0.124 within 24 tokens [meas.] (`mesures/moteur/R/r1/W1-chemins-20b.log`, `W1-chemins-casse-20b.log`).

6.2.3 Tiling the product, and why it changes no bit

The first version of the product of a matrix by a batch, $y[p,r] = \langle W_r, x_p \rangle$, walked the rows and, for each one, reread the n input vectors. As soon as these no longer fit in the last-level cache (24 MB), the traffic to main memory is $T = R\,n\,C \cdot 8$ bytes. With the numbers of R1 [calc.]: $n = 10 \times 511 = 5\,110$ positions, $C = 2\,880$, hence $n\,C \cdot 8 = 117.7$ MB; for the output matrix alone ($R = 201\,088$ rows), $T = 201\,088 \times 117.7$ MB ≈ 23.7 TB, which at 60 GB/s (order of magnitude of DDR5, assumption) is

about $3.9 \cdot 10^5$ s: more than four days for one matrix. Witness W1, at 24 positions (553 KB, held in cache), could not see it (`moteur/R-pour-le-memoire.md` § 2.6).

The tiling (frame: Goto and van de Geijn's anatomy of the matrix product [35]) arranges the computation in four levels: positions in packets of 256 ($256 \times 2\,880 \times 8$ B = 5.9 MB, held in L3); for each packet, one task per block of 32 rows dequantised once (737 KB, in L2); within the task, tiles of 8 positions (184 KB); within the tile, four products at a time (2 rows × 2 positions), that is eight independent FMA chains. The traffic to main memory falls to the quantised bytes of W once per packet, $\lceil n/256 \rceil\, R\, C\, b/8$ ($b = 8.5$ bits per weight for Q8_0): for the output, $20 \times 0.615 = 12.3$ GB, about **1 900 times less** [calc.].

Proposition 6.3 (The tiling is equal to the bit to the plain dot product)**.** *Let $S(a, b)$ be the sequence of operations of the reference dot product: for $i = 0, 8, 16, \ldots < C_8 = 8\lfloor C/8 \rfloor$, $\ell \leftarrow \mathrm{fma}(a_{i..i+4}, b_{i..i+4}, \ell)$ and $h \leftarrow \mathrm{fma}(a_{i+4..i+8}, b_{i+4..i+8}, h)$ on two registers of four accumulators; then $s = ((\ell_0 + h_0) + (\ell_2 + h_2)) + ((\ell_1 + h_1) + (\ell_3 + h_3))$; then $s \leftarrow \mathrm{fma}(a_j, b_j, s)$ for $j = C_8 \ldots C - 1$. Then, for any packet, block and tile size and any number of threads, the tiled product returns $y[p, r] = S(W_r, x_p)$ to the bit.*

Proof. The 2 × 2 kernel runs, for each of the four pairs (a_u, b_v), the sequence $S(a_u, b_v)$ on its own accumulators (ℓ_{uv}, h_{uv}), interleaved with the other three, and the same reduction function, written once (Listing 6.1). An IEEE 754 FMA returns the correctly rounded value of $a \cdot b + c$, a function of its three operands only [25]; two identical sequences of operations on identical operands therefore return identical results, whatever the interleaving. The tiling changes only *which* products are computed *when*; none changes its internal order. ■

The witness and its two breaks. The test `produit_egal_scalaire_au_bit` (non-dyadic reals, edges everywhere) holds. The first break tried, swapping ℓ and h for the single pair (a_1, b_1), *still holds*: the swap turns $(\ell_k + h_k)$ into $(h_k + \ell_k)$, equal by the exact commutativity of IEEE addition. This is a lesson of method that this thesis keeps: **a break must change the computed function, not merely the text of the code**. The second break (the eight terms of (a_1, b_1) on the ℓ chain alone) falls: 304 outputs out of $71 \times 19 = 1\,349$. W1 replayed after the tiling: 4 826 112 logits, 0 different, and W1's break returns exactly the same 4 625 024 differences as before [meas.] (`mesures/moteur/R/r1/W1-chemins-20b-pavage.log`).

6.2.4 The YaRN ramp: OpenAI's definition and llama.cpp's form

gpt-oss rotates its queries and keys in pairs (RoPE), with frequencies stretched by YaRN to carry a context 32 times longer than the training one [28]. The original frequencies are $\theta_i = b^{-2i/d}$, $i = 0 \ldots d/2 - 1$, with $b = 150\,000$ and $d = 64$. "NTK-by-parts" YaRN blends extrapolation θ_i and interpolation θ_i/s ($s = 32$) along a ramp over the pair index:

$$r_i = \mathrm{clamp}\Big(\frac{i - \alpha}{\beta - \alpha}, 0, 1\Big), \qquad \omega_i = \theta_i\Big((1 - r_i) + \frac{r_i}{s}\Big). \tag{6.1}$$

The bounds are the pairs whose wavelength $2\pi/\theta_i$ makes $\beta_r = 32$ and $\beta_l = 1$ turns respectively over the original context $L_0 = 4\,096$:

$$2\pi\, b^{2i/d} = \frac{L_0}{\beta} \iff i(\beta) = \frac{d}{2} \cdot \frac{\ln\big(L_0/(2\pi\beta)\big)}{\ln b}. \tag{6.2}$$

Worked out [calc.]: $\ln b = \ln 150\,000 = 11.9184$; $L_0/(2\pi \cdot 32) = 20.372$, whose logarithm is 3.0141; $L_0/(2\pi) = 651.90$, whose logarithm is 6.4798. Hence

$$\alpha = 32 \times \frac{3.0141}{11.9184} = \mathbf{8.0928}, \qquad \beta = 32 \times \frac{6.4798}{11.9184} = \mathbf{17.398}.$$

Listing 6.1: The 2×2 kernel of the reference (excerpt of `trc-reference/src/poids.rs`; comments of work package R, in French in the code: “four dot products at a time, each by exactly the sequence of operations of scalaire_avx2: eight FMA chains”; “a single writing of the reduction order, hence a single computed function”). Each product keeps its two accumulators and the common reduction; only the four sequences are interleaved.

```
// Quatre produits scalaires à la fois, (a0.b0, a0.b1, a1.b0, a1.b1), chacun par
// exactement la suite d'opérations de scalaire_avx2 : huit chaînes de FMA.
while i < n8 {
    let (x0l, x0h) = (_mm256_loadu_pd(pa0.add(i)), _mm256_loadu_pd(pa0.add(i + 4)
    ));
    let (x1l, x1h) = (_mm256_loadu_pd(pa1.add(i)), _mm256_loadu_pd(pa1.add(i + 4)
    ));
    let (y0l, y0h) = (_mm256_loadu_pd(pb0.add(i)), _mm256_loadu_pd(pb0.add(i + 4)
    ));
    let (y1l, y1h) = (_mm256_loadu_pd(pb1.add(i)), _mm256_loadu_pd(pb1.add(i + 4)
    ));
    lo00 = _mm256_fmadd_pd(x0l, y0l, lo00);  hi00 = _mm256_fmadd_pd(x0h, y0h,
    hi00);
    lo01 = _mm256_fmadd_pd(x0l, y1l, lo01);  hi01 = _mm256_fmadd_pd(x0h, y1h,
    hi01);
    lo10 = _mm256_fmadd_pd(x1l, y0l, lo10);  hi10 = _mm256_fmadd_pd(x1h, y0h,
    hi10);
    lo11 = _mm256_fmadd_pd(x1l, y1l, lo11);  hi11 = _mm256_fmadd_pd(x1h, y1h,
    hi11);
    i += 8;
}
// Une seule écriture de l'ordre de réduction, donc une seule fonction calculée.
[reduire_avx2(lo00, hi00, a0, b0, n8), reduire_avx2(lo01, hi01, a0, b1, n8),
 reduire_avx2(lo10, hi10, a1, b0, n8), reduire_avx2(lo11, hi11, a1, b1, n8)]
```

OpenAI's reference implementation keeps these real bounds, and its configuration says so: the key `truncate` is `false`; llama.cpp b11515 takes $\lfloor\alpha\rfloor = 8$ and $\lceil\beta\rceil = 18$ (function `ggml_rope_yarn_corr_dims`). The reference follows OpenAI and keeps llama.cpp's variant to measure the gap (Listing 6.2). The two ramps differ only on a few pairs (Figure 6.1): ~~for $8 \le i \le 17$~~, corrected by the second path (`rampe_yarn.py`, `mesures/moteur/R/r1/rampe-yarn-temoin.json`): **for** $9 \le i \le 17$, because at $i = 8$ both ramps are 0 ($8 < 8.0928$ on one side, $(8-8)/10 = 0$ on the other) and at $i = 18$ both are 1. The second path finds the frequencies within one ulp (0.94 at most); its break (the rounded ramp taken for OpenAI's) makes pairs 9 to 17 fall, and only them.

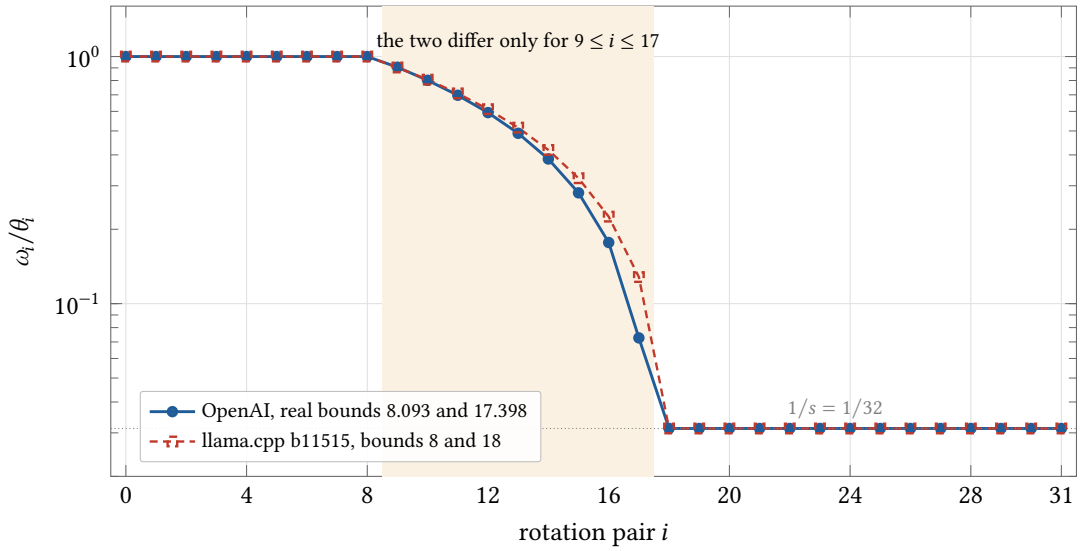


Figure 6.1: The ratio ω_i/θ_i of the stretched frequency to the original frequency, for the 32 pairs of one head of gpt-oss. Below the ramp, extrapolation (ratio 1); above it, interpolation (ratio 1/32). The two definitions part only on the coloured band.

Data: `mesures/moteur/R/r1/csv/rampe-yarn.csv` (written by `juge rampe`, the code of the reference; recomputed by `rampe_yarn.py`).

Order of magnitude of the gap. At pair $i = 17$: $\theta_{17} = b^{-34/64} = 1.78 \cdot 10^{-3}$; for OpenAI, $r = 0.957$, so $\omega/\theta = 0.043 + 0.957/32 = 0.0729$; for llama.cpp, $r = 0.9$, $\omega/\theta = 0.1 + 0.9/32 = 0.128$. Hence $\Delta\omega = 1.78 \cdot 10^{-3} \times (0.128 - 0.0729) = 9.8 \cdot 10^{-5}$ rad per position, that is 0.05 rad at position 500 [calc.]. At pair 9: $\theta_9 = b^{-18/64} = 0.0350$; for OpenAI, $r = (9 - 8.0928)/(17.3980 - 8.0928) = 0.0975$, so $\omega/\theta = 0.9025 + 0.0975/32 = 0.90555$; for llama.cpp, $r = 0.1$, $\omega/\theta = 0.9 + 0.1/32 = 0.90313$; hence $\Delta\omega = 0.0350 \times 0.00243 \approx 8.5 \cdot 10^{-5}$ rad per position [calc.]. Work package R's note (`moteur/R-pour-le-memoire.md`) wrote ~~$8.8 \cdot 10^{-5}$~~, taken over here without being redone until the judgement of Volume II. Small, but not zero, and the gap grows with position: at 131 072 tokens, 12.8 rad, an unrelated phase. What this small gap costs in KL is the subject of Section 6.5, and the answer is surprising.

The concentration factor. YaRN also multiplies the cosine and the sine by $0.1 \ln s + 1 = 1.3466$, hence the product $q \cdot k$ by $1.3466^2 = 1.8133$. llama.cpp obtains the same value by another route (`yarn_attn_factor` brought back to 1, then `mscale`). Omitting it is the error injected by R1's break (Section 6.4.3).

Listing 6.2: The YaRN frequencies in the reference (`trc-reference/src/ops.rs`). llama.cpp's variant rounds the bounds; OpenAI's keeps them real. The French comment reads: "correction dimension: the pair whose wavelength makes beta turns over L0".

```
let concentration = 0.1 * facteur.ln() + 1.0;
// Dimension de correction : la paire dont la longueur d'onde 2 pi / inv_freq
    fait beta tours sur L0.
let corr = |beta: f64| (demi as f64) * (ctx_origine / (beta * 2.0 * PI)).ln() /
    base.ln();
let (mut bas, mut haut) = (corr(beta_rapide), corr(beta_lent));
if v == VarianteYarn::LlamaCpp {
    bas = bas.floor().max(0.0);
    haut = haut.ceil().min((d - 1) as f64);
}
let inv_freq = (0..demi).map(|i| {
    let interpolation = 1.0 / (facteur * freq[i]);
    let extrapolation = 1.0 / freq[i];
    let rampe = ((i as f64 - bas) / (haut - bas)).clamp(0.0, 1.0);
    let masque = 1.0 - rampe;
    interpolation * (1.0 - masque) + extrapolation * masque
}).collect();
```

6.3 The Kullback-Leibler divergence, and what it measures

6.3.1 Definition and first properties

Let p and q be two distributions over the vocabulary V ($|V| = 201\,088$ for gpt-oss). The Kullback-Leibler divergence [2] from p to q is

$$\mathrm{KL}(p \,\|\, q) = \sum_{i \in V} p_i \log \frac{p_i}{q_i}. \tag{6.3}$$

It reads as the average number of nats lost when coding tokens drawn from p with a code that is optimal for q [36]. Two facts make it fit our use.

Theorem 6.4 (Gibbs' inequality)**.** $\mathrm{KL}(p \,\|\, q) \geq 0$*, with equality if and only if* $p = q$*.*

Proof. On the support of p, $-\log$ is strictly convex; by Jensen's inequality,

$$\sum_i p_i \log \frac{q_i}{p_i} \leq \log \sum_{i \,:\, p_i > 0} q_i \leq \log 1 = 0,$$

with equality if and only if q_i / p_i is constant on the support of p and that support carries all the mass of q, that is $q = p$. ∎

Proposition 6.5 (Cross-entropy, perplexity)**.** *For the true distribution p of the next token, the cross-entropy of an engine q decomposes as $H(p, q) = H(p) + \mathrm{KL}(p \,\|\, q)$. The average loss of the engine on a text drawn from p therefore exceeds that of the exact model by exactly its mean KL, and the ratio of perplexities is* $\exp(\overline{\mathrm{KL}})$*.*

Proof. $H(p, q) = -\sum_i p_i \log q_i = -\sum_i p_i \log p_i + \sum_i p_i \log(p_i / q_i)$. Perplexity is the exponential of the average loss. ∎

The direction of the KL matters. The judge writes $\mathrm{KL}(\text{truth} \,\|\, \text{engine})$, with the truth as base: the sum is weighted by what *the exact model* holds likely, and that is where an error costs (Proposition 6.5).

This choice is that of criterion R1; criterion C4 wrote it the other way round, and the engineer of work package R aligned it with R1 so that a single instrument judges both (delegated answer of 2026-10-15, reversible; `moteur/REPRISE-R.md`, need 2). The judge records both directions anyway.

6.3.2 The KL of a small perturbation of the logits

An approximate engine does not return an arbitrary distribution: it returns the logits of the truth, perturbed. Let $z \in \mathbb{R}^V$ be the exact logits, $p = \mathrm{softmax}(z)$, and $q = \mathrm{softmax}(z + \Delta z)$.

Proposition 6.6 (The KL is a cumulant generating function)**.**

$$\mathrm{KL}(p \,\|\, q) = \log \mathbb{E}_p\big[e^{\Delta z}\big] - \mathbb{E}_p[\Delta z] = \tfrac{1}{2}\,\mathrm{Var}_p(\Delta z) + \tfrac{1}{6}\kappa_3 + \cdots \tag{6.4}$$

where $\mathbb{E}_p$ and Var_p are over the index i drawn from p, and κ_3 is the third cumulant of Δz under p. In particular, the KL does not depend on a common shift of the logits, and it is second order *in Δz.*

Proof. $\log q_i = z_i + \Delta z_i - \log \sum_j e^{z_j + \Delta z_j}$ and $\log p_i = z_i - \log \sum_j e^{z_j}$. Hence $\log p_i - \log q_i = -\Delta z_i + \log\big(\sum_j e^{z_j} e^{\Delta z_j} / \sum_j e^{z_j}\big) = -\Delta z_i + \log \mathbb{E}_p[e^{\Delta z}]$. Average under p. The expansion is that of the cumulant generating function $K(t) = \log \mathbb{E}_p e^{t\Delta z} = t\kappa_1 + t^2\kappa_2/2 + t^3\kappa_3/6 + \cdots$ at $t = 1$, with $\kappa_1 = \mathbb{E}_p \Delta z$ and $\kappa_2 = \mathrm{Var}_p\, \Delta z$. ∎

This identity is exact; it will serve twice. First, it says that the KL of a rounding is tiny: if each logit is perturbed by a relative u, $\mathrm{Var}_p(\Delta z) = O(u^2)$ and the KL is of order u^2. Then, it will say why a mixture of experts escapes that regime (theorem 6.9).

6.3.3 Top-1 agreement, and its intervals

The judge also records, position by position, whether the engine and the truth give the same most likely token ("top-1", ties broken by the smallest index). A proportion $\hat\pi = k/n$ carries the Wilson score interval at 95 % [37], with $z = 1.96$:

$$\frac{\hat\pi + \frac{z^2}{2n}}{1 + \frac{z^2}{n}} \pm \frac{z}{1 + \frac{z^2}{n}} \sqrt{\frac{\hat\pi(1-\hat\pi)}{n} + \frac{z^2}{4n^2}}. \tag{6.5}$$

A mean KL carries the uncertainty $\sqrt{(\langle x^2\rangle - \langle x\rangle^2)/(n-1)}$, as in llama.cpp (`moteur/crates/trc-juge/src/stats.rs`).

6.3.4 Judging two engines on the same positions

C4 compares two KLs measured *at the same positions* against the same truth V: $x_i = \mathrm{KL}(V_i \,\|\, A_i)$ for llama.cpp, $y_i = \mathrm{KL}(V_i \,\|\, B_i)$ for troncons, and $d_i = y_i - x_i$. Then

$$\mathrm{Var}(\bar d) = \frac{\sigma_x^2 + \sigma_y^2 - 2\rho\,\sigma_x\sigma_y}{n}, \qquad \hat\sigma(\bar d) = \sqrt{\frac{\langle d^2\rangle - \langle d\rangle^2}{n-1}}. \tag{6.6}$$

Two engines that err at the same difficult positions have ρ close to 1, and the paired comparison sees a difference that the comparison of two independent means would drown. Work package R proposes to the arbiter a statistical verdict, $\bar d \le 2\hat\sigma(\bar d)$ and $\bar e \ge -2\hat\sigma(\bar e)$ (where $e_i \in \{-1, 0, 1\}$ compares the top-1 choices), next to the literal verdict (the two means).

The defect of the literal verdict, observed. Tried on two real engines against the truth (A = llama.cpp GPU, B = llama.cpp CPU; `mesures/moteur/R/r2/comparer-ref-carte-processeur-20b.json`): KL 0.022606 against 0.026470; top-1 89.61 % against 90.39 %; $\bar{d} = +0.00386 \pm 0.00236$ ($1.6\,\sigma$), $\bar{e} = +0.0078 \pm 0.0055$ [meas.]. The literal verdict on the KL falls, the one on top-1 holds; both statistical verdicts hold. Two engines that nothing distinguishes in correctness receive opposite literal verdicts depending on the measure. The paired standard deviation (0.00236) is hardly smaller than $\sqrt{0.0017^2 + 0.0025^2} = 0.0030$: the two engines do not err at the same positions, and Section 6.5 will say why. Break (B = the reference without the YaRN scale): the four verdicts fall ($\bar{d} = 0.220 \pm 0.008$, $\bar{e} = -0.315 \pm 0.010$).

6.4 The instrument and its witnesses

6.4.1 The logits file and its floor

So that every engine goes through the same instrument, the judge reads and writes the format of llama-perplexity (`--kl-divergence-base`; read in `tools/perplexity` of b11515, nothing copied). Per scored position, it stores two reals and one 16-bit code per vocabulary token (Figure 6.2).

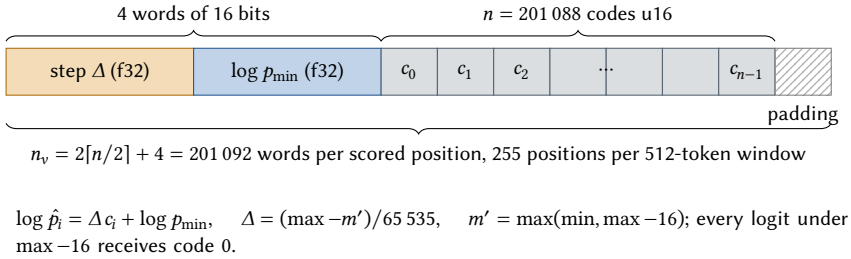


Figure 6.2: One entry of the `.kld` file, per scored position (positions 256 to 510 of each 512-token window). Token i receives code c_i; its log-probability is read back as $\Delta c_i + \log p_{\min}$.

Data: `moteur/crates/trc-juge/src/kld.rs`.

Lemma 6.7 (The floor of the instrument)**.** *The coding error of a log-probability above the cut satisfies $|\delta_i| \le \Delta/2 \le 16/131\,070 = 1.22 \cdot 10^{-4}$ nat. A distribution judged against its own coding has a KL of at most $\Delta/2$ at first order, and not $O(\Delta^2)$.*

Proof. The code is the rounding to nearest of $(\log p_i - \log p_{\min})/\Delta$, hence $|\delta_i| \le \Delta/2$, and $\Delta = (\max - m')/65\,535 \le 16/65\,535$. Against itself, $\log \hat{p}_i = \log p_i + \delta_i$ and the KL is $\sum_i \hat{p}_i(\log \hat{p}_i - \log p_i) \approx \sum_i p_i \delta_i$, bounded by $\max|\delta_i|$; it does not drop to second order because the decoded $\hat{p}$ are not renormalised (they do not sum to 1). ∎

This is the floor measured by the study of the models (at most $6.8 \cdot 10^{-5}$, i.e. $\Delta/2$ for a max − min gap of about 9) and the origin of the $7 \cdot 10^{-5}$ threshold. The judge's first unit test had a threshold set by guesswork, ~~$|\mathrm{KL}| \le 10^{-7}$~~; it fell at $5.7 \cdot 10^{-6}$ and was replaced by the derived bound, $|\mathrm{KL}| \le \Delta$ (`moteur/REPRISE-R.md`).

6.4.2 The defect the identity witness found: a coded judged file is not renormalised

When the judged Q is itself a `.kld` file (the second path of R0 and R1, and C4, where troncons goes through the same coding), its decoded values are *already* log-probabilities, normalised by the log-sum-exp of the exact logits at coding time. The first version of the judge renormalised them. But

the coding crushes every token under max −16 onto code 0, hence onto $\log p_{\min}$, instead of its true value, much smaller. Renormalising then adds at each position

$$\log \sum_i \hat{q}_i = \log\Big(\sum_{c_i>0} \hat{q}_i + N_0\, e^{\log p_{\min}} \Big) \approx N_0\, e^{\log p_{\min}}, \tag{6.7}$$

where N_0 is the number of crushed tokens. For gpt-oss, almost all tokens are under the cut: $N_0\, e^{-16} \approx 2 \cdot 10^5 \times 1.1 \cdot 10^{-7} \approx 2 \cdot 10^{-2}$ at worst [calc.]; **measured:** $1.11 \cdot 10^{-3}$ **on average** over 2 550 positions, for a file judged against itself (`mesures/moteur/R/r2/comparer-temoin-identite-renormalise-AVANT-CORRECTION.json`), sixteen times the $7 \cdot 10^{-5}$ threshold [meas.]. The second path of R0, of R1 and the C4 judgement would have been blind below 10^{-3}. Corrected (Listing 6.3): the same file against itself gives **exactly 0** (`comparer-temoin-identite.json`), and the test that guards it falls when renormalisation is put back ($> 10^{-4}$). What the correction leaves, stated: a token that P holds likely and that Q crushed receives $\log p_{\min}$, an upper bound of its true log-probability; the term is underestimated, never infinite.

Listing 6.3: The KL of a position when the judged side is a decoded entry (`trc-juge/src/stats.rs`): no renormalisation. The French comments say: coding crushes every token under max − 16 onto log_p_min; renormalising adds about $N_0 e^{\log p_{\min}}$ (1.11e-3 measured on gpt-oss-20b); without renormalisation, a file against itself gives exactly 0.

```rust
/// Le codage écrase tous les jetons sous max - 16 sur log_p_min ; renormaliser l'entrée
/// décodée ajoute log(somme des p décodés) ~ N0 * exp(log_p_min) (1,11e-3 mesuré, gpt-oss-20b).
/// Sans renormalisation, un fichier contre lui-même donne 0 exactement.
pub fn position_decodee(p: &Entree, q: &Entree, n: usize, suivant: usize) -> Position {
    position_avec(p, |i| q.log_p(i) as f64, n, suivant, false)
}
// … dans position_avec : seuls les termes de la base au-dessus de la coupe comptent.
if lp > -16.0 {
    kl += lp.exp() * (lp - (q(i) - lse_q));   // lse_q = 0 sans renormalisation
}
```

The negative KL against a file. A truth judged against a file that codes an identical computation does not give zero but a small negative value. Let $\tilde{q}$ be the distribution read back, not normalised: its tokens under the cut receive $\log q_{\min} > \log q_i$, and $\sum \tilde{q} = e^{\delta}$, with δ measured at 0.0020 on average (`mesures/moteur/epreuve-J1/journaux/normalisation-kld-20b-f0.json`). The KL then tends to $-\sum_{i \text{ under the cut}} p_i(\log q_{\min} - \log p_i) < 0$, measured at $-4.8 \cdot 10^{-4}$ (`mesures/epreuve-J1.md` § 3.3) [meas.]. This bias is common to every judged file; it sets the zero of the instrument and changes no ordering. It reappears in Section 6.6.

6.4.3 The witnesses: what the instrument sees, and what it does not

The llama.cpp GPU passes for R0 and R1 ran on 2026-10-15 from 19:59 to 20:20, on mains power, GPU idle (those of the reference from 15:35 to 17:35), on ten 512-token windows of wikitext-2 [38], i.e. 2 550 scored positions, by two independent paths: llama-perplexity, and `juge kl` (Table 6.2).

What they say:

Table 6.2: The gap between engines (R0) and the validation of the reference (R1) on gpt-oss-20b, ten windows, 2 550 positions [meas.]. "Base ‖ judged"; path 1: llama-perplexity; path 2: `juge kl`.

Measurement	KL (p. 1)	median	same top-1	p. 2	verdict
T1 floor: GPU ‖ GPU	−0.000000	−0.000000	100 %	0	holds
G_{cfg}: `--cpu-moe` ‖ `--fit`	0	0	100 %	0	published: 0
G_{moteurs}: GPU ‖ CPU	0.022741	0.0024	91.57 %	0.022735	T3 holds
T2 break: GPU ‖ $k = 2$ experts	0.4513	0.290	47.96 %	—	holds
R1: truth ‖ llama.cpp GPU	**0.022612**	0.0029	89.61 %	0.022606	holds, by 0.6 %
R1 break: truth without scale ‖ llama.cpp	0.2644	0.131	57.80 %	0.26372	falls
measurement Y: llama.cpp-ramp ref. ‖ llama.cpp	0.022603	0.0027	89.96 %	0.022581	not judged

Data: `moteur/R-pour-le-memoire.md` § 2.11; `mesures/moteur/R/r0/r0-resume.json`, `mesures/moteur/R/r1/r1-resume-20b.json`; paths 2 recomputed here by `memoire/outils/tome2_reference_cache.py`.

- **The instrument sees**: removing two experts out of four (T2) makes a KL of 0.45; taking YaRN's concentration factor away from the reference (R1's break) makes it fall by a factor of 12.
- **The engines are deterministic**: the two passes of each configuration write files identical byte for byte; so does the replayed reference (experiment D, 1 GB identical).
- $G_{\text{cfg}} = 0$ **exactly**, in batched evaluation of 512: the `--fit` and `--cpu-moe` placements are identical to the bit. Interpretation, not checked in the code: on a batch of 512, llama.cpp computes on the GPU the products whose weights are in host memory, by copying them there; whatever the placement, they are the same kernels on the same bytes. G_{cfg} is therefore zero only in evaluation, not in generation.
- G_{moteurs} **of gpt-oss is 0.0227**, four times the Qwen3.5-4B figure (0.0055) that the Q1 threshold of milestone J2 reused: it is up to the arbiter to strike it and replace it (`moteur/REPRISE-R.md`, need 6).
- **R1 holds, by 0.6 %**: the reference has no gross error. But the llama.cpp-ramp variant is as close to llama.cpp as the reference (0.022603 against 0.022612): within 0.02, every correct or nearly correct computation is at the same distance from llama.cpp. R1 therefore separates only a gross error from a correct reference, and the fine correctness of the reference rests on its own witnesses (W1, D, the tests of each operation against its definition, the second path of the ramp), and on the third computation that follows.

A third computation. The independent trial wrote its own forward pass of gpt-oss in numpy f64, from the form published by OpenAI, reading nothing of work package R (`mesures/moteur/epreuve-J1/gptoss_numpy.py`). It finds the ramp bounds (8.09278 and 17.39802). On the 255 positions of window 0 of the 20b and their 201 088 tokens each, work package R's reference equals it within **0.000123 nat, that is 0.504 quantisation step** of the format (median step $2.44 \cdot 10^{-4}$): the 16-bit rounding alone, plus 0.004 step of f32 rounding of Δ and $\log p_{\min}$; 255 positions out of 255 below one step; 100 % identical top-1. llama.cpp's ramp, by contrast, departs from it by up to 6.5 nat [meas.] (`mesures/moteur/epreuve-J1/journaux/ecart-logp-20b-f0.json`). Figure 6.3 summarises where every distribution the judge compares comes from, and what links each pair.

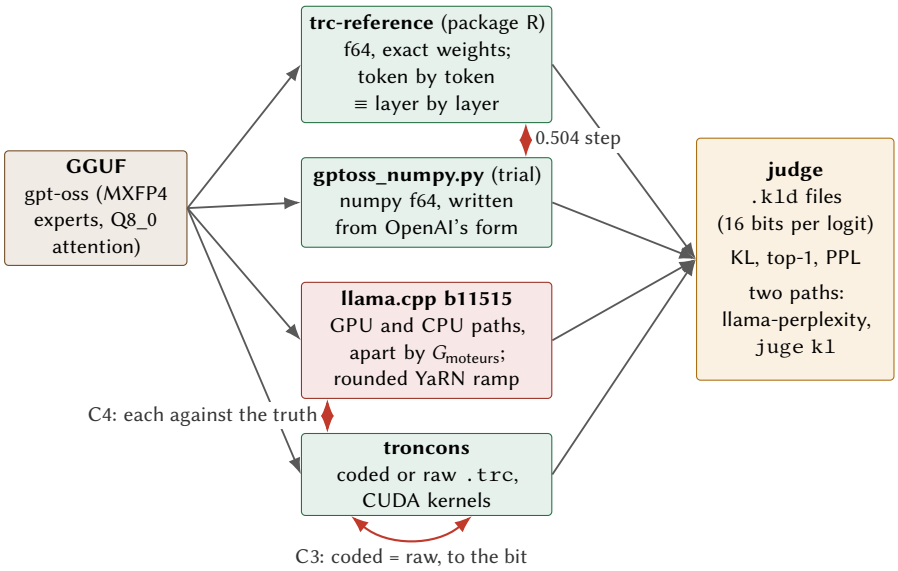


Figure 6.3: The paths of the truth. Four computations read the same GGUF; the judge compares their `.k1d` files by two paths. On each red link, the measurement that relates two computations: the reference and the trial's numpy pass within 0.504 step of the format; troncons coded and raw to the bit (C3); troncons and llama.cpp, each against the truth (C4).

6.5 The discovery: a mixture of experts amplifies the smallest errors

6.5.1 The numbers

R1 came with a series of experiments, each declared before its run (`moteur/REPRISE-R.md`), that isolate a single difference between two computations in f64 (Table 6.3).

Table 6.3: What a small difference of computation costs in KL on gpt-oss-20b (ten windows, 2 550 positions) [meas.]. The last four rows compare only two passes of the f64 reference.

Base ‖ judged	mean KL	median	99th centile	same top-1
llama.cpp CPU ‖ reference (OpenAI ramp)	0.0255	0.0027	0.47	90.4 %
llama.cpp CPU ‖ reference (llama.cpp ramp)	0.0273	0.0027	—	90.4 %
llama.cpp CPU ‖ ref. (llama.cpp ramp, Q8_0 and f16 KV imitated)	0.0220	0.0025	0.40	91.6 %
OpenAI ref. ‖ llama.cpp ref. (*only the ramp changes*)	0.0145	$9.8 \cdot 10^{-4}$	0.29	92.5 %
llama.cpp ref. ‖ the same with imitations (*only roundings change*)	0.0291	0.0030	0.54	90.3 %
OpenAI ref. ‖ llama.cpp ref., *routing imposed*	0.00168	$2.2 \cdot 10^{-4}$	0.031	97.2 %
llama.cpp CPU ‖ ref. without scale (break)	0.249	—	—	57.3 %

Data: `moteur/R-pour-le-memoire.md` § 2.8 and § 2.9; `mesures/moteur/R/r1/juge-kl-*.csv`. The imitations row was published as 0.029; recomputed here as 0.02905 (`t2-verif-reference-cache.json`).

Three facts stand out, and Figure 6.4 shows them.

1. **The KL is not a spread-out noise, it is a tail.** The median is ten times below the mean, the 99th centile a hundred times above: most positions are nearly right, and a minority diverges outright.

2. **A minute perturbation costs as much as another engine.** Changing only the ramp, whose direct effect is a phase of the order of 10^{-4} rad per position on nine pairs (Section 6.2.4), already gives 0.0145; adding only roundings of the order of 10^{-3} (Q8_0 on activations, f16 on the KV cache) gives 0.029, more than the gap between the reference and llama.cpp itself.
3. **Imitating the approximations barely brings things closer**: the reference equipped with llama.cpp's roundings is hardly closer to it (0.0220 against 0.0273). In a regime that amplifies, two computations perturbed differently are as far from each other as from the exact one.

Hypothesis I, declared before the imitations run (threshold $2.5 \cdot 10^{-3}$), assumed a linear regime; it fell, and it is the regime that it revealed.

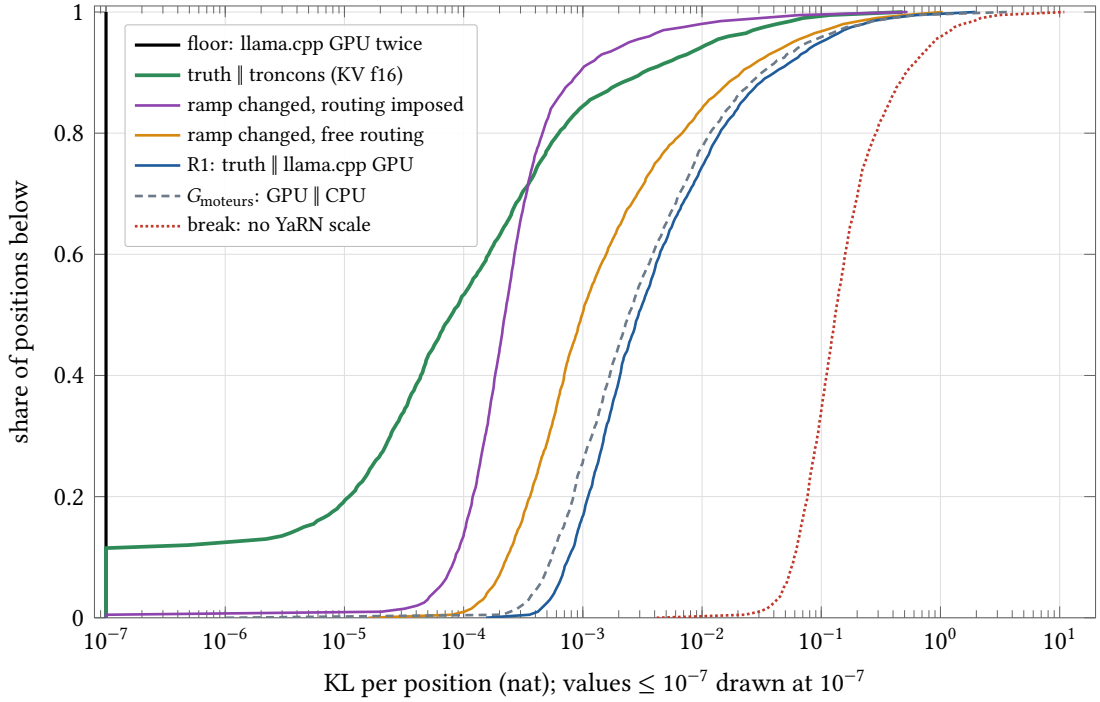


Figure 6.4: Distribution of the KL per position on gpt-oss-20b (2 550 positions, logarithmic scale). All approximate curves cross below the median and separate beyond the 90th centile. Imposing the routing (purple) tightens the tail by an order of magnitude. troncons (green, f16 KV cache) has 12 % of positions at KL ≤ 10^{-7} (the zero of the instrument is slightly negative, Section 6.4.2).

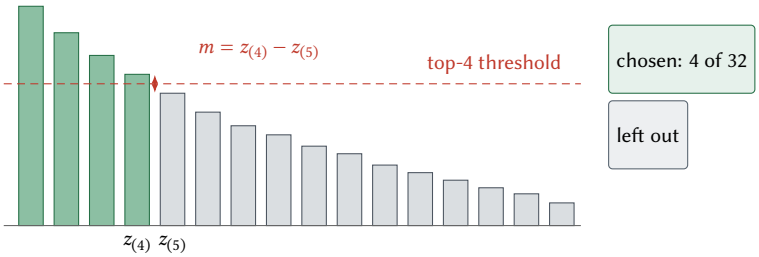
Data: `memoire/figures/tome2/donnees/t2r-kl-repartition.csv`, drawn from `mesures/moteur/R/r0/juge-kl-*.csv`, `r1/juge-kl-*.csv` and `mesures/moteur/O/csv/o-c4-20b.csv`.

6.5.2 The mechanism: the routing switch

At each layer, gpt-oss's router computes one logit per expert (32 on the 20b, 128 on the 120b), keeps the four largest and normalises their weights by a softmax over the four. The output of the layer is

$$f(h) = \sum_{j \in \text{top}_4(z(h))} g_j(h)\, E_j(h), \qquad g = \text{softmax}\left(z_{\text{top}_4}\right).$$

This function is **discontinuous** wherever the 4th and 5th logits are equal (Figure 6.5).



If a perturbation δz of the logits makes $|\delta z_{(4)} - \delta z_{(5)}| > m$, expert 4 leaves and expert 5 enters. The layer output jumps by $g_4\,(E_4(h) - E_5(h))$, a quantity of order 1, whatever the size of δz; the jump enters the residual, then the keys and values of every later position.

Figure 6.5: The routing switch. The router logits of one position, sorted; top-4 keeps the first four. The margin m between the 4th and the 5th is the distance to the discontinuity.

Proposition 6.8 (The jump of a switch does not depend on the perturbation)**.** *Let $m = z_{(4)} - z_{(5)} > 0$ be the router margin at an input h, and a perturbation that changes the logits by δz. If $\delta z_{(5)} - \delta z_{(4)} > m$, expert 4 leaves, expert 5 enters, and the output changes by*

$$\Delta f = g_4 \left(E_4(h) - E_5(h)\right) + O(\|\delta z\|),$$

whose norm does not tend to 0 as $\|\delta z\| \to 0$ (at the edge of the switch, $g_4 \to g_5 > 0$). Otherwise, $\Delta f = O(\|\delta z\| + \|\delta h\|)$.

Proof. Without a switch, the set of experts is the same and f is differentiable in h (softmax, smooth experts up to the clamped SwiGLU), hence first order. With a switch, the contribution of expert 4, of weight $g_4 > 0$, is replaced by that of expert 5, of nearby weight; the difference of two expert outputs on the same input is a quantity of the order of the outputs themselves. ■

The jump enters the residual of the position, then, through the keys and values of this layer and the following ones, every later position. With 24 layers and 5 110 positions, even a small probability of switching per (layer, position) touches a sizeable share of positions. The exact identity of Proposition 6.6 now says what that costs.

Theorem 6.9 (Two regimes of the KL of a mixture of experts)**.** *Let a perturbation of relative size u of the computations (rounding, phase) shift the router margins by δ_u, with $\mathbb{E}|\delta_u| = c\,u$ at first order; let f_m be the density of the margins, continuous and non-zero at 0. Then, for small u, the mean KL decomposes as*

$$\overline{\mathrm{KL}}(u) \;\approx\; \underbrace{a\,u^2}_{\text{no switch}} \;+\; \underbrace{f_m(0)\,c\,u \cdot \overline{\mathrm{KL}}_{jump}}_{\text{switches}}, \tag{6.8}$$

where $\overline{\mathrm{KL}}_{jump}$ is the mean KL of a switching position, of order 1 in u. The second term, linear*, dominates the first as soon as $u < f_m(0)\,c\,\overline{\mathrm{KL}}_{jump}/a$: the more precise the computation, the larger the share of switches in the KL.*

Proof. Without switches, the output logits are perturbed by $\Delta z = O(u)$ and Proposition 6.6 gives a KL of $\frac{1}{2}\,\mathrm{Var}_p\,\Delta z = O(u^2)$. A switch happens at a (layer, position) when $|\delta_u| > m$; its probability is $\int_0^\infty f_m(x)\,\Pr(|\delta_u| > x)\,dx \approx f_m(0)\,\mathbb{E}|\delta_u| = f_m(0)\,c\,u$ as long as f_m is roughly constant over the range of $|\delta_u|$. A switching position sees, by Proposition 6.8, an order-1 jump of its residual, hence of its logits, hence a KL of order 1 that does not depend on u. Summing the two contributions gives (6.8). (Switches propagated through the KV cache add a term of the same order in u; they are counted in $\overline{\mathrm{KL}}_{\mathrm{jump}}$ per touched position.) ■

The theorem is an order analysis, under assumptions (margin density continuous at 0, margin shift proportional to u before any cascade); its consequences are checked against the measurements that follow.

An order of magnitude that holds together. The margins of all (layer, position) of the 20b have a 1st centile of $1.14 \cdot 10^{-3}$ and a 10th centile of $1.24 \cdot 10^{-2}$ (`mesures/moteur/R/r1/bascules-openai-llamacpp-20b.json`): the density at 0 is therefore $f_m(0) \approx 0.01/1.14 \cdot 10^{-3} = 8.8$ (or $0.1/1.24 \cdot 10^{-2} = 8.1$) [calc.]. The measured share of switches between the two ramps is 4.15 %; the theorem draws from it a mean margin shift $\mathbb{E}|\delta| \approx 0.0415/8.5 \approx 5 \cdot 10^{-3}$, the expected order of magnitude of a phase gap of 10^{-4} rad per position, accumulated over a few hundred positions and summed in a $q \cdot k$ product [calc.] on [meas.]. And the median margin of the switching (layer, position) alone, $9.5 \cdot 10^{-3}$, is indeed of that order.

6.5.3 The imposed-routing experiment

Experiment B (declared before its run) imposes on the llama.cpp-ramp pass the experts that the OpenAI-ramp pass chose at each (layer, position), then compares the two: only the ramp differs, and no switch is possible. Declared threshold: if the KL falls below 10^{-3}, the switches carry the gap.

Measured: the KL goes from 0.0145 to 0.00168, the median from $9.8 \cdot 10^{-4}$ to $2.2 \cdot 10^{-4}$, top-1 from 92.5 % to 97.2 % [meas.]. The threshold falls ($1.68 \cdot 10^{-3} > 10^{-3}$), but what the experiment establishes is clear: the switches carry

$$1 - \frac{0.00168}{0.0145} = \mathbf{88\,\%}$$

of the KL [calc.] (recomputed: 0.884). The rest keeps a tail (maximum 0.53): a second amplifier, unidentified, or a sensitivity peculiar to a few positions. Figure 6.6 arranges all these experiments.

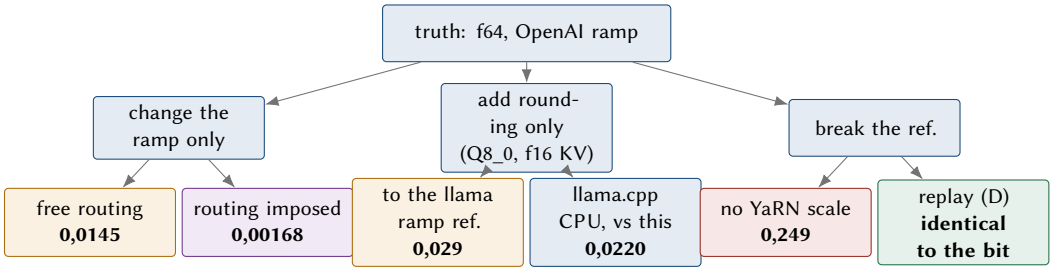


Figure 6.6: R1's experiments, each declared before its run, with the mean KL measured at each leaf (gpt-oss-20b, ten windows). The "replay (D)" leaf is the witness of the reference's determinism.

Data: `moteur/R-pour-le-memoire.md` § 2.8, § 2.9.

6.5.4 The switches, counted

The routing logs of the two passes (OpenAI ramp, llama.cpp ramp; the latter replayed identical to the bit, D holds) allow the switches to be counted one by one (`bascules.py`, `mesures/moteur/R/r1/bascules-openai-llamacpp-20b.json`) [meas.]:

- **5 087 switches out of 122 640 (layer, position), i.e. 4.15 %**;
- none of the 2 550 scored positions is free of an upstream switch in its window: from position 256 on, there has always been an earlier switch;
- **1 157 positions switch themselves (45 %) and carry 96.5 % of the KL** (mean 0.0309); the 1 393 others, touched only through the KV cache, have a mean KL of $9.4 \cdot 10^{-4}$, 33 times less;

- the router margin is 0.083 in median over all (layer, position), and **0.0095 over those that switch**; 1 % of the switches have a margin below $9.6 \cdot 10^{-5}$.

The switch is local before it propagates; propagation through the KV cache exists but weighs little (Figure 6.7).

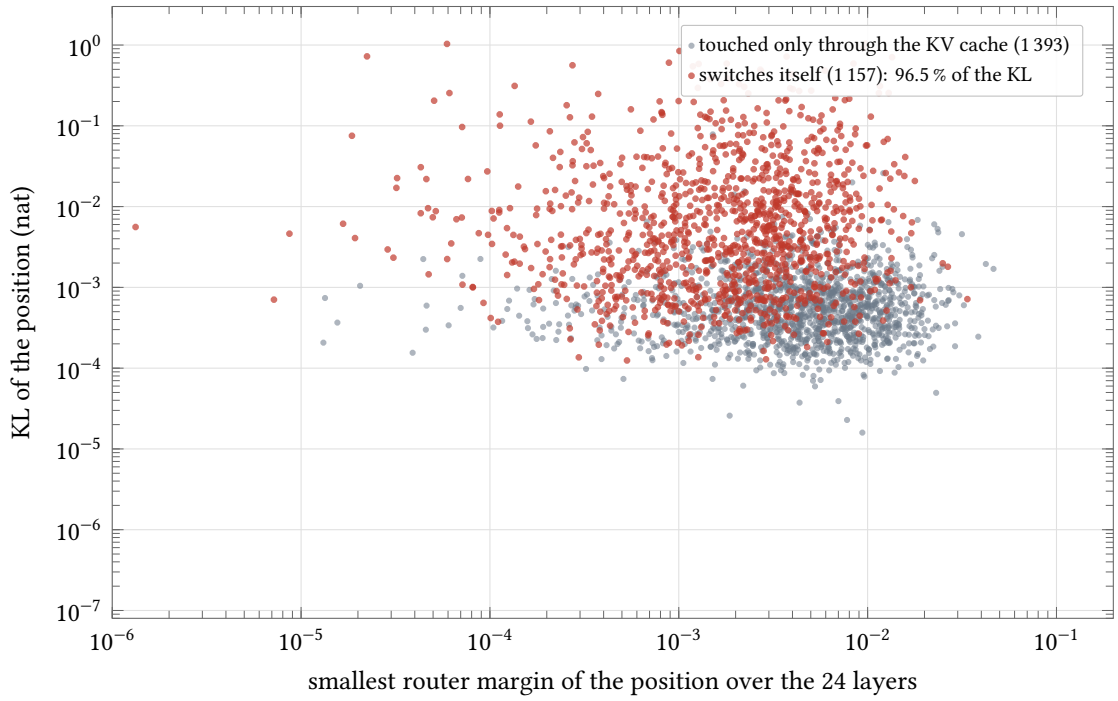


Figure 6.7: KL of each scored position against the smallest margin of its router over the 24 layers (ramp changed, free routing; gpt-oss-20b). The positions that switch themselves (red) form the top of the cloud.

Data: `memoire/figures/tome2/donnees/t2r-bascules-*.csv`, drawn from `mesures/moteur/R/r1/csv/bascules-openai-llamacpp-20b.csv`.

6.5.5 What this does to the criteria

The architecture's criterion R1, "KL(reference ‖ llama.cpp GPU) ≤ max(G_{moteurs}, $7 \cdot 10^{-5}$)", assumed that a correct reference is closer to llama.cpp than two llama.cpp engines are to each other. In a regime that amplifies, the distance from any approximate computation to the exact one is of the same order as the distance between two approximate computations: the reference may land slightly beyond G_{moteurs} without any defect, and the criterion no longer separates a correct reference from one wrong by a small error (the ramp alone makes 0.0145); it separates only a gross error (0.249). What really separates them: imposed routing, which recovers the linear regime, and a criterion that compares distributions of distances rather than a mean. Both are put to the arbiter (`moteur/REPRISE-R.md`, needs 4 and 5), who has not decided at the time this chapter is written; the literal criterion is still applied and published as is. The same amplification affects the Q1 threshold of milestone J2.

6.6 The f16 KV cache carries the switches, f32 reaches the truth

6.6.1 The first token of C12

The independent trial fed the 708 tokens of the prompt of text C12 to its numpy f64 pass of the 120b, with the argmax and the routing trace of every position, and the engine replayed the same tokens (`mesures/epreuve-J1.md` § 6.1). At position 707, that of the first generated token, the f64 truth picks token 1008 ("..", $\log p = -1.46$); troncons picks 313 ("am"), to which the truth gives $\log p = -10.03$; llama.cpp picks 1008 [meas.]. This is not a break at 512 (the first 512 tokens are window 0, equal to the bit, and both `--contexte 1024` and `--emplacements 200` leave the 708 fingerprints unchanged). It is a **cascade of switches**: at position 707, 20 layers out of 36 route differently from the truth, from layer 12 on; at position 706, none.

The experiment declared before its run (`mesures/moteur/epreuve-J1/seuils-declares.md`) replaces, in the f64 truth, the cache of keys and values by its f16-rounded version, and nothing else. That truth **picks 313 at position 707, with the same cascade of 20 layers**. The f16 rounding of the KV cache alone, which the architecture's seam N fixes for troncons (and which llama.cpp shares), is therefore a sufficient cause of the error.

6.6.2 The share of switches carried by the cache rounding

troncons routes like the truth at 96.9 % of (layer, position) before 512 and 93.1 % after; it routes like the truth *equipped with the f16 KV* at 99.3 % and 97.9 % (`mesures/epreuve-J1.md` § 6.1). The share of troncons' switches that the cache rounding explains is worked out as [calc.]:

$$\text{before 512: } 1 - \frac{1-0.993}{1-0.969} = 1 - \frac{0.7}{3.1} = \mathbf{77}\,\%, \qquad \text{after: } 1 - \frac{1-0.979}{1-0.931} = 1 - \frac{2.1}{6.9} = \mathbf{70}\,\%.$$

(Recomputed by `memoire/outils/tome2_reference_cache.py`, whose witness falls when one of the rates is falsified.) Routing agreement decreases with depth on the 120b, not on the 20b (Figure 6.8): 128 experts per layer leave tighter margins than 32, and a switch in a low layer propagates to the following ones.

6.6.3 Why f32 must reach the truth

An f16 keeps 11 significant bits: rounding to nearest gives $\tilde{k} = k(1+\delta)$ with $|\delta| \le u_{16} = 2^{-11} \approx 4.9 \cdot 10^{-4}$. For an attention score $s = c \sum_{d=1}^{64} q_d k_d$ ($c = 1/8$), the first-order error satisfies

$$|\tilde{s} - s| \le c\, u_{16} \sum_{d} |q_d k_d|,$$

and the product done in f32 adds only $\gamma_{64} = 64u_{32}/(1 - 64u_{32}) \approx 3.8 \cdot 10^{-6}$ ($u_{32} = 2^{-24}$; Higham [25]). The attention output inherits a relative error of the same order, through the softmax weights and through $\tilde{v}$. In f32, the only error left is that of the kernels, $u_{32}/u_{16} = 2^{-13} = 1/8\,192$ times smaller [calc.].

Theorem 6.9 says what becomes of the KL when u is divided by 8 192. The no-switch term, in u^2, is divided by $8\,192^2 \approx 6.7 \cdot 10^7$; the switch term, linear, by 8 192. Starting from the 0.00308 measured in f16 on window 0 of the 120b, the KL in f32 would be at most of the order of $0.00308/8\,192 \approx 3.8 \cdot 10^{-7}$ nat [calc.], well below the step of the format ($\approx 2.4 \cdot 10^{-4}$ nat): invisible. ~~The f16 KL would fall towards $5 \cdot 10^{-11}$~~: this is the estimate of the engineer of work package O (`moteur/O-pour-le-memoire.md` § 13.3), who assumed the quadratic regime for the whole KL; with switches, which are not quadratic, the honest bound is of the order of $4 \cdot 10^{-7}$. The conclusion does not change: in f32, the gap from the engine to the truth goes below the floor of the instrument.

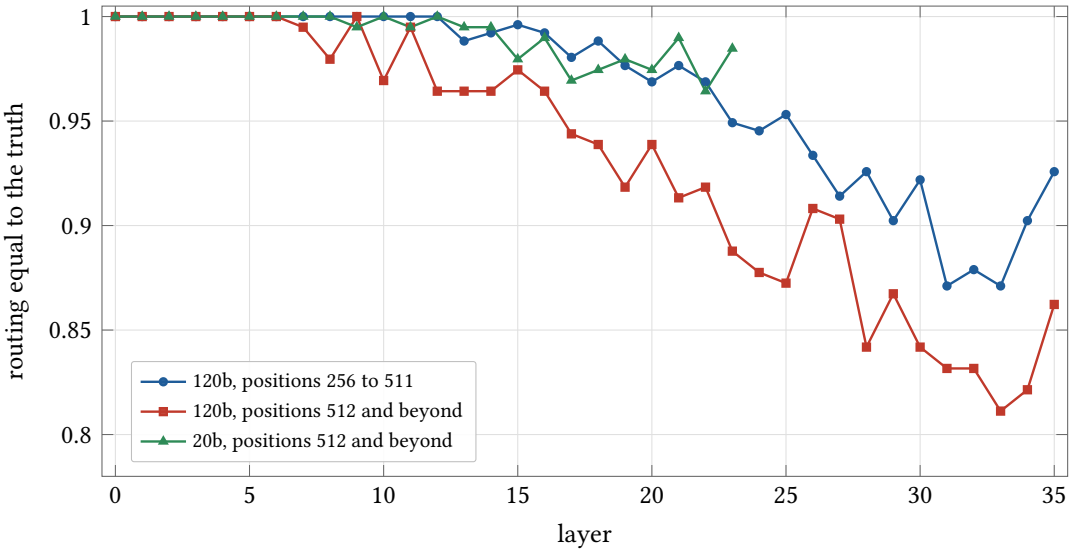


Figure 6.8: Share of positions whose routing in troncons (f16 KV cache) equals that of the f64 truth, per layer, on the 708-token prompt. The 120b drops with depth, more so beyond 512; the 20b stays above 0.96.

Data: `mesures/moteur/epreuve-J1/csv/routage-par-couche-invite.csv`.

6.6.4 Measured

The engineer of work package O added an optional f32 KV cache to the engine (`--kv-f32`, kernels `ecrire_kv_f32` and `attention_kvf32` added without touching the f16 kernels, hence outside seam N). Witness: on values representable in f16, the f32 path is equal to the bit to the f16 path (full cache and ring); break: non-representable values make them differ (`mesures/moteur/O/corrections/journaux/m5-invariants.txt`). The results against the trial's truth are in Table 6.4 and Figure 6.9.

Table 6.4: KL(f64 truth ‖ troncons) depending on the format of the KV cache [meas.]. "Max gap of log p": above the cut of the format, in nat or in quantisation steps.

Case	mean KL	top-1	KL > 0.1	KL p99	max gap of log p
120b, window 0, f16 KV	0.00308	95.7 %	0.39 %	0.0747	3.07 nat
120b, window 0, f32 KV	**−0.00048**	**100 %**	0	$2.3 \cdot 10^{-6}$	**0.69 step**
20b, window 0, f16 KV	0.00174	96.5 %	0.39 %	0.0529	2.32 nat
20b, window 0, f32 KV	**−0.00048**	**100 %**	0	≈ 0	2.04 steps
120b, prompt, positions 512 to 706, f16	0.00963	—	2.05 %	0.196	—
120b, prompt, positions 512 to 706, f32	**−0.00054**	—	0	$3.6 \cdot 10^{-6}$	—

Data: `moteur/O-pour-le-memoire.md` § 13.3; `mesures/moteur/O/corrections/journaux/verdicts-kv-2026-10-16.json`, `m5-ecart-logp-*.json`; means recomputed here from `mesures/moteur/O/corrections/csv/kv-f16-f32-par-position.csv`.

On the 708-token prompt, in f32, the argmax equals the truth **708 times out of 708** (f16: 687); routing is equal on every (layer, position), 36 × 708 (f16: 96.9 and 93.1 %); and **the first token of C12 becomes 1008, the truth's** (f16: 313) [meas.]. The negative KL is exactly that of work package R's f64 reference against the same truth ($-4.8 \cdot 10^{-4}$): the common bias of the floor of the format, the zero of

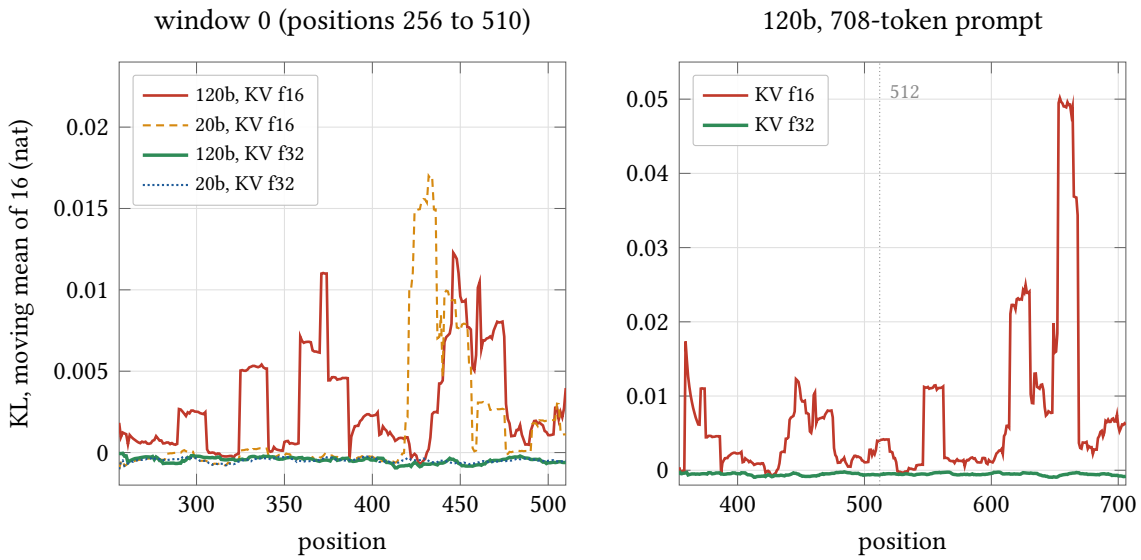


Figure 6.9: KL(f64 truth ‖ troncons) per position, as a moving mean over 16 positions. In f16, the KL comes in bursts (the switches) and rises with position on the 120b prompt; in f32, it stays at the zero of the instrument ($\approx -5 \cdot 10^{-4}$, Section 6.4.2).

Data: `memoire/figures/tome2/donnees/t2r-kv-glissant.csv`, `t2r-kv-invite-glissant.csv`, drawn from `mesures/moteur/O/corrections/csv/kv-f16-f32-par-position.csv`.

the instrument.

> **What this establishes**
>
> **The f16 rounding of the KV cache carried all the measurable gap between troncons and the exact mathematics of the model.** With an f32 KV cache, the engine, its kernels, its decoder and its expert cache return the f64 truth at the floor of the 16-bit-per-logit format, on the 120b as on the 20b. The trial had bounded it from below (70 to 77 % of the switches); work package O's measurement closes it.

What it costs. The KV cache holds 18 full layers of 4 096 positions and 18 rings of 128, each of 512 values for k and as many for v: $18 \times (4\,096 + 128) \times 2 \times 512 \times 2\,\mathrm{B} = 155.7$ MB in f16, twice that in f32. The extra 155.7 MB are worth $155.7/13.25 = 11.8$ expert slots [calc.]; **measured: 326 → 314 slots**. On the prompt's trace, the cache simulator then gives a hit rate of 0.459 → 0.442 and 77.9 → 80.4 misses per token, i.e. +3.2 % bytes read, the order of the speed cost when the disk limits [calc.] on a measured trace (`mesures/moteur/O/corrections/journaux/m5-rejouer-invite-326-314.jsonl`). The generation speed measured in f32 (1.43 and 1.56 tokens/s, `mesures/moteur/O/corrections/csv/vitesse-2026-10-16-passes.csv`) *is not comparable* to the f16 one: the text changes (f32 follows the truth, then simulated tool calls, like llama.cpp's greedy output; f16 falls into a "Here is a summary" loop that lifts its hit rate to 0.590 against 0.435); the gap measures the effect of the text, not the cost of f32. The choice of the KV cache format belongs to the arbiter (seam N); it is not decided.

"Six times closer", requalified. The C4 judgement on the 20b gave KL(truth ‖ troncons) = 0.003679 against 0.022606 for llama.cpp GPU, and 97.18 % top-1 against 89.61 % (`mesures/moteur/O/csv/o-c4-20b.csv`; recomputed here: ratio 6.15). The engineer of work package O summed it up as

"six times closer to the exact mathematics of the model than llama.cpp". It is true, **the YaRN ramp included**: 0.0145 of llama.cpp's 0.0226 come from its rounded ramp, a modelling choice, and not from the precision of its kernels (finding m5 of the trial; on window 0, 0.0143 of 0.0261). And the gap that remained for troncons came entirely from its f16 KV cache. On the 120b, the trial judged C4 for the first time, on one window: 0.00308 and 95.7 % for troncons, 0.0607 and 80.4 % for llama.cpp, paired difference -0.0576 ± 0.0120, disjoint Wilson intervals [meas.] (`mesures/epreuve-J1.md` § 3.3).

6.7 What remains

- The f64 reference of the 120b on the ten windows (about 61 GB of experts read per pass, of the order of four to six hours estimated): C4 on the 120b rests today on one window and one prompt only.
- The arbiter's decision on seam N (f32 KV: +156 MB, 12 slots, of the order of 3 % more bytes read), and on a "tail" C4 criterion (share of positions with KL > 0.1, 99th centile, judged also beyond 512 positions).
- An R1 criterion that separates a small error: the reference judged against llama.cpp with imposed routing, with its imitations, in the linear regime (R1').
- The second amplifier: the 12 % of the KL that survive imposed routing. Untested hypotheses: the kinks of the clamped SwiGLU ($\min(g, 7)$), positions whose distribution is nearly flat, attention nearly equal between two keys.

What could be wrong

- **The reference assumes that ggml-org's GGUF is the model.** It judges the model as stored (attention in Q8_0), not OpenAI's bf16 checkpoint.
- **OpenAI's ramp is taken to be the model** on the strength of two concordant primary sources (the reference implementation and `truncate: false`); training itself is not observable. The trial's numpy pass and the reference follow the same reading of the model and read the same GGUF: a shared convention error would pass. They validate each other on arithmetic and order of operations, not on convention.
- **The two-regime theorem is an order analysis**; its assumptions (margin density continuous and non-zero at 0, margin shift proportional to u) are checked only by the order of magnitude of Section 6.5.2. The $4 \cdot 10^{-7}$ bound for f32 depends on it; the measurement does not.
- **The amplifying regime is measured on gpt-oss-20b**, ten windows of wikitext-2. On the 120b, margins are tighter and the share of switches may be larger; R1's figures do not carry over without being remeasured.
- **"f32 KV = truth"** is measured on one 512-token window and one 708-token prompt for the 120b, one window for the 20b; not on the ten windows.
- **The imitations** reproduce the Q8_0 quantisation of activations and the f16 KV cache as read in the code of b11515; not the f32 sums, nor the repacked kernels of the Q8_0 weights, nor the GPU path.
- $G_{\mathbf{cfg}} = 0$ holds only for batched evaluation of 512; the interpretation (offloading of products to the GPU) is not checked in llama.cpp's code.
- **R1 holds by 0.6 %**, within the noise (± 0.0017 and ± 0.0025): another seed of windows could make it fall without the reference changing.
- **The tiling** is proved equal to the bit by construction and by two witnesses; it was not compared pass against pass with the old version on ten windows (that would have taken days).
- **The reference's passes** (15:35 to 17:35 on 2026-10-15) ran during the 120b series of the

> gpt-oss measurement agent: no effect on the logits (D holds), possible on timings.

Chapter 7

The cache, the scheduler and the engine

Where we start, what this chapter establishes, what remains

At the start, the architecture had fixed the interface of the expert cache (seam A) and a witness: the engine's simulator must equal, to the unit, those of the loading and measurement studies. troncons' speed ceiling was read off a single quantity, the hit rate h of the cache, which earlier studies computed under uniform routing for lack of a real trace. **This chapter establishes** the theory and the measurement of the cache and of what surrounds it: the optimality of Belady's algorithm, proved; LRU, its stack property, its competitive bound and its worst case; troncons' cache, its rules and two lemmas that make it equal to the simulators; **the cliff of the shared LRU**: no hit below $(L-1)k$ slots, exactly the adjacent reuse at Lk, proved and checked to the hit on 17 real traces; the measured locality of gpt-oss-20b and 120b; the on-demand scheduler and the lemmas that make it safe, exact and equal to its simulator; streamed prefill, kept; prefetching, tried then abandoned on its worked-out balance; and the engine against the best-tuned baseline measured so far: **2.22 times llama.cpp**, integrity guard included. **What remains** is a policy aware of the layer cycle, a predictor more accurate than the next layer's router, the read length (12 MiB), and a llama.cpp baseline swept beyond 16 threads. Sources: `moteur/O-pour-le-memoire.md`, `moteur/REPRISE-O.md`, `mesures/gpt-oss.md`, `mesures/epreuve-J1.md`, and the logs under `mesures/moteur/O/` and `mesures/localite-gpt-oss/`.

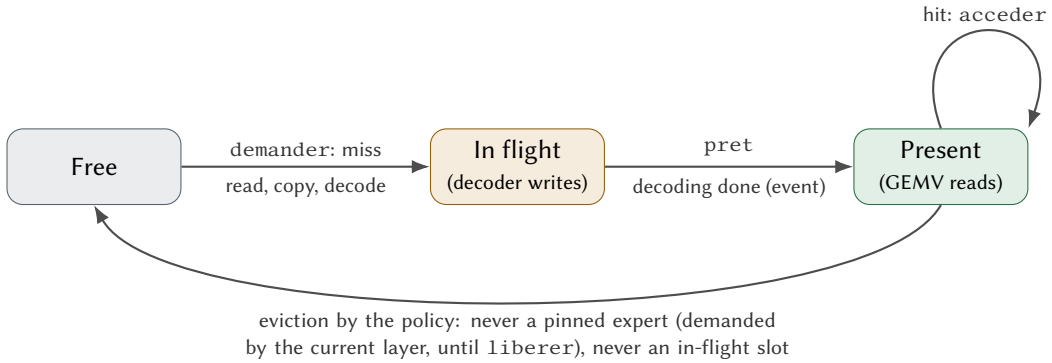
In brief

- Belady (evict what will be used latest) is optimal among all policies (theorem 7.2); LRU makes at most N times its misses, plus N (theorem 7.4), and its worst case is a cycle over $N+1$ keys.
- The expert stream of an MoE *is* that cycle: an LRU shared between the L layers has no hit below $(L-1)k$ slots, whatever the routing, and equals exactly the adjacent reuse at Lk (Proposition 7.7). On gpt-oss-20b: 0 hits out of 1 024 224 accesses at 92 slots, while Belady serves 66.4 %.
- gpt-oss is very local: a token reuses 50 % (20b) and 31 % (120b) of the previous token's experts, against 12.5 and 3.1 % at random.
- The scheduler is safe (lemma 7.9), computes to the bit what the kernels validated (Proposition 7.10) and counts its misses like its simulator, to the unit (454 210 hits, 37 310 misses on both sides).
- Prefetching by the next layer's router costs 11 %: 69 % of its reads are used, the others add 34 % bytes to a disk that already limits.
- troncons: 1.965 tokens/s on the 120b, guard included, against 0.884 for the best measured llama.cpp configuration (`fit -t 16`); ~~×3.85~~, ~~×2.8~~, ×2.22.

7.1 The expert-cache problem

A mixture of experts with L layers, E experts per layer and k active experts per token (gpt-oss-120b: $L = 36$, $E = 128$, $k = 4$; gpt-oss-20b: $L = 24$, $E = 32$, $k = 4$) requests, at every token and every layer, a **group** of k distinct identifiers (layer, expert). The request stream is the sequence of groups in compute order: $(t, 0), (t, 1), \ldots, (t, L-1), (t+1, 0), \ldots$ The GPU holds N **slots** of decoded experts (13 254 144 bytes each for gpt-oss); a present expert is a **hit**; an absent expert is a **miss**, which costs a read of about 12 MB from disk, a copy to the GPU and a decoding. The share h of accesses served by the cache decides everything: at every token, the disk must deliver $(1-h)$ times the routed bytes, $R = k\,L\,e = 1.909$ GB for the 120b at original size (`mesures/gpt-oss.md` § 1).

A **policy** chooses, at each miss of a full cache, the victim to evict. It is *online* if it sees only the past, *offline* if it knows the whole sequence. Two rules of troncons are added (`moteur/crates/trc-cache/src/lib.rs`): an expert requested by the current layer is **pinned** until its products are done (it is not evicted while it serves), and an **in-flight** slot (being written by the decoder) is never evicted. Figure 7.1 gives the automaton of a slot.



RAM tier (warm cache, mode A): an entry is born only from a disk read (a coded section cannot be rebuilt from a decoded expert); every access to an expert it holds touches it, VRAM hits included (inclusive tier). A VRAM miss found in RAM costs a copy and a decoding, not a read.

Figure 7.1: The automaton of a VRAM slot of troncons' cache (diagram M5), and the RAM tier of mode A. In J2, the states will carry the level (base or exact) of the present expert.

Data: `moteur/crates/trc-cache/src/lib.rs` (header).

7.2 The offline optimum: Belady

Let us begin with the bound that no policy can exceed. We first work in the classical model: a sequence of single requests $\sigma_1, \sigma_2, \ldots$ on a cache of N places, and a "demand" policy (which loads only on a miss, evicting a single key).

Definition 7.1 (MIN, or Belady)**.** At each miss of a full cache, MIN evicts the present key whose *next request is farthest away* (never: infinity; tie: the smallest key) [39].

Theorem 7.2 (Optimality of MIN)**.** *For every sequence σ and every demand policy A starting from the same content, MIN makes no more misses than A.*

Proof. We build a sequence of policies $A = A_0, A_1, \ldots, A_n = \text{MIN}$ such that A_i takes the same decisions as MIN on the first i requests and makes no more misses than A_{i-1}. Suppose A_{i-1} built; it follows MIN

up to request $i-1$, so both hold the same content before request i. If A_{i-1} takes MIN's decision at request i, set $A_i = A_{i-1}$. Otherwise, at a miss, A_{i-1} evicts y while MIN evicts x, with $n_x \geq n_y$, where n_z is the time of the next request of z. Define A_i: at request i it evicts x; afterwards it imitates A_{i-1}, through three phases.

Phase 1: A_i holds $D \cup \{y\}$ and A_{i-1} holds $D \cup \{x\}$, with as many misses. On any request for a key of D or a key absent from both, A_i does as A_{i-1} (same hit, or same miss and same victim), except if A_{i-1} evicts x: then A_i evicts y, and the two contents become equal. Before n_x comes $n_y \leq n_x$: at the request for y, A_i hits and A_{i-1} misses, evicting a key w. If $w = x$, the contents are equal and A_i has one miss fewer. Otherwise we go to phase 2 with the pair (w, x).

Phase 2: A_i holds $D' \cup \{a\}$, A_{i-1} holds $D' \cup \{x\}$, and A_i has *at least one miss fewer*. A request for a: hit for A_i, miss for A_{i-1}, which evicts w'; if $w' = x$ the contents become equal, otherwise the pair becomes (w', x) and A_i's lead grows. A request for x: miss for A_i, which evicts a and loads x; the contents become equal and A_i has lost at most the lead it had, so it still has no more misses. If A_{i-1} evicts x on another miss, A_i evicts a: contents equal. Any other request is handled in the same way by both.

Phase 0: equal contents; A_i imitates A_{i-1} exactly.

In all cases, A_i makes no more misses than A_{i-1} until the end, and follows MIN up to request i. It evicts only on a miss: it is a demand policy. By induction, MIN $= A_n$ makes no more misses than A. ■

What the theorem covers, and what it does not. The engine serves *groups* of k pinned experts, not single requests. troncons' simulator applies MIN among the non-pinned keys (`moteur/crates/trc-cache/src/politiques.rs`); its optimality in that pinning model is not proved here. It is checked: on every real trace and every budget, Belady serves at least as much as LRU, LFU and static pinning (witness G3 of the gpt-oss measurement, no violation; the tool stops otherwise). Belady reads the future of the trace: its figures are **bounds**, not an engine policy.

A defect the witness found. The first version of the Belady policy looked up an expert's next use from a "now" updated by each access. But the cache chooses its victim *before* inserting the miss: when the first expert of a group misses, "now" still holds the time of the previous group, and an expert served at that group seemed requested again immediately, so was never evicted. Witness O1 saw it: from 10 to 6 646 fewer hits than the reference simulators, while LRU was exact everywhere [meas.] (`mesures/moteur/O/journaux/temoin-o1-passe1-belady-faux.json`, kept). The next use is now computed at access time (Listing 7.1): a present non-pinned key has not been served since its last access, so that figure stays right.

7.3 LRU: stack, competitive bound, worst case

LRU evicts the key whose last use is oldest. It is the most common online policy, that of llama.cpp's expert cache, and troncons' in J1. Three properties describe it.

Proposition 7.3 (Stack property)**.** *At any time, the content of an LRU cache of N places is the set of the N most recently requested distinct keys. In particular, the content at capacity N is included in the content at any capacity $N' > N$, on the same sequence [40].*

Proof. Induction on requests. A request for z makes z the most recent; if z was among the N most recent, the set does not change; otherwise it is a miss, and LRU evicts the oldest present key, which is

Listing 7.1: The next use in the Belady policy of `trc-cache` (`politiques.rs`): computed at access time, by binary search in the times at which the expert is requested (t = token $\cdot L$ + layer). French comments: "the first use of id strictly after t"; "at each hit"; "victim: the non-pinned present key with the largest next use; tie: the smallest id".

```
/// Le premier usage de `id` strictement après `t` ; u64::MAX s'il n'y en a plus.
fn prochain_apres(&self, id: Id, t: u64) -> u64 {
    match self.usages.get(&id) {
        None => u64::MAX,
        Some(v) => {
            let i = v.partition_point(|&x| x <= t);
            v.get(i).copied().unwrap_or(u64::MAX)
        }
    }
}
fn acceder(&mut self, id: Id, t: u64) {        // à chaque succès
    let p = self.prochain_apres(id, t);
    self.prochain.insert(id, p);
}
// victime : le présent non épinglé de plus grand prochain usage ; égalité : le
    plus petit id.
```

the N-th most recent before the request and becomes the $(N{+}1)$-th after it: the content stays the set of the N most recent. The inclusion follows: the N most recent are among the N' most recent. ■

Theorem 7.4 (LRU is N-competitive). *For every sequence σ,* $\mathrm{LRU}_N(\sigma) \le N\left(\mathrm{OPT}_N(\sigma) + 1\right)$, *where* OPT_N *is the number of misses of MIN [41].*

Proof. Cut σ into *phases*: each phase is the longest run of requests, from the end of the previous one, that contains only N distinct keys. (i) LRU makes at most N misses per phase: a key loaded during a phase is not evicted before the end of the phase, since that would require N other distinct keys requested after its last use, and the phase contains only $N-1$ others. (ii) MIN makes at least one miss per phase, except perhaps the first: let p be the first request of phase j; the run from the second request of phase j to the first of phase $j+1$ contains the $N-1$ other keys of phase j and the new key that opens phase $j+1$, i.e. N distinct keys, all different from p. Right after serving p, MIN holds p and $N-1$ other keys; serving N keys distinct from p costs it at least one miss. These runs are disjoint from one phase to the next. With P phases, LRU $\le NP$ and OPT $\ge P-1$. ■

The worst case. The bound is reached: on the cyclic sequence $1, 2, \ldots, N+1, 1, 2, \ldots$, LRU always evicts the key that will be requested next, and misses on every request; MIN misses only once every N requests. The next section shows that **the expert stream of an MoE, served by a single cache for all layers, has exactly that shape**.

7.4 troncons' cache, and why it equals its simulators

7.4.1 The interface

The `trc-cache` crate (Rust, no dependency) holds the two-tier cache and its policy (Table 7.1): the VRAM holds decoded experts, the warm RAM tier (mode A) holds coded sections in pinned memory. The simulator `rejouer` is *the very code* of the engine, called on a routing trace, which makes the witness "engine count equal to simulator count, to the unit" possible.

Table 7.1: The interface of troncons' cache (seam A), and what the scheduler does with each decision.

Element	Role
`demander(couche, experts, t)`	pins the group, then returns one decision per expert, in router order
`Succes {emplacement}`	the expert is (or will be, if in flight) in this slot: the GEMV reads it
`DefautRam {emplacement, tiede}`	the coded section is already in pinned memory: copy, decoding
`DefautDisque {emplacement, tiede}`	disk read (into the warm entry or an arrival buffer), copy, decoding
`pret(emplacement)`	decoding finished: in flight → present
`liberer(couche)`	end of the layer's GEMVs: its experts are unpinned
`Compteurs`	`succes_vram + succes_ram + defauts_disque` = accesses; bytes, evictions
`rejouer(trace, N, tièdes, ...)`	the simulator: the same code, called on a trace

7.4.2 Two eviction rules, one result

The reference simulators do not evict at the same moment as the engine. That of the gpt-oss measurement (rule "B", like llama.cpp's cache) serves the whole group, then evicts until it is back to N; that of troncons and of the loading study (rule "P") pins the group and evicts at each miss. One must show they count the same thing. Notation: a group $G = (x_1, \dots, x_k)$ of distinct keys served in that order; S the content, ordered from oldest to most recent; $N \geq k$.

Lemma 7.5 (Two rules, one result)**.** *Starting from the same content S_0, rules B and P give the same hits, in the same order, and the same final content, in the same order.*

Proof. Hits. In B, nothing leaves during service: x_i is a hit if and only if $x_i \in S_0$. In P, no element of G leaves (pinned) and nothing enters except elements of G; the x_i being distinct, x_i was not added before step i; hence $x_i \in S$ at step i if and only if $x_i \in S_0$. *Order of the others.* Only the elements of G move; those of $S_0 \setminus G$ keep their order under both rules. *Evictions of P.* Once S is full it stays full; each miss with S full removes the oldest of $S \setminus G$: P therefore removes the m oldest of $S_0 \setminus G$, with $m = \max(0, |S_0| + d - N)$ and d the number of misses. A candidate always exists: if S were full and included in G at a miss, we would have $N = |S| \leq k - 1$ (the current miss is not yet in S), contrary to $N \geq k$. *Evictions of B.* After service, all elements of G are at the end; $|S| = |S_0| + d$; the oldest are those of $S_0 \setminus G$ in their order; B removes $m = |S_0| + d - N \leq |S_0| + d - k = |S_0 \setminus G|$ of them (since $d = k - |G \cap S_0|$): exactly the m oldest of $S_0 \setminus G$. *Order of G.* Under both rules, the elements of G end at the tail, in service order. ■

The same proof holds for Belady, replacing "the oldest" by "the largest next use": the key of an element of $S_0 \setminus G$ does not change during the group (it is not served). **Corollary**: at every group boundary, an LRU with rule P holds the N most recently served distinct identifiers.

7.4.3 The inclusive RAM tier

Lemma 7.6 (The RAM tier equals a single cache of its size)**.** *Let a VRAM tier of V slots decide alone, and a RAM tier of $R \geq V$ entries receive `acceder` for every access to a key it holds (VRAM hits included)*

and `inserer` *for every disk read, and pin the group. With LRU: the VRAM hits are those of a single LRU of V slots, and the VRAM plus RAM hits those of a single LRU of R slots.*

Proof. The VRAM evolves like a single cache (its decisions do not read the RAM). For the RAM, it suffices to show that it receives the same calls as a single LRU of R slots, which could fail only on a VRAM hit whose key would be absent from the RAM. Induction on groups: if the RAM has evolved like a single LRU up to the boundary, the corollary of lemma 7.5 gives VRAM_0 = the V most recent $\subseteq$ the R most recent = RAM_0. During the group, a VRAM hit x_i is in $\text{VRAM}_0 \subseteq \text{RAM}_0$, and the RAM does not evict elements of the group (pinned): x_i is still there, and the RAM calls `acceder` as the single LRU would. A VRAM miss is, for the RAM, a hit or a miss, as for the single LRU. ■

This is exactly the "two tiers" model of the ceilings (Figure 7.2): $h_v = h(V)$, $h_r = h(R) - h(V)$, misses $1 - h(R)$; the engine behaves like the equation that set its thresholds. For Belady, the same conclusion follows from MIN's stack property [40]; with the group rule and pinning, it is not proved here, only checked on 20 configurations (seeds 100 to 103, $(V, R) \in \{(8, 8), (8, 20), (16, 40), (30, 90), (12, 160)\}$).

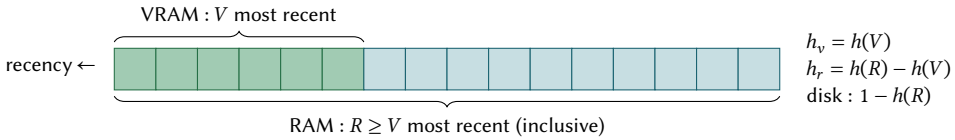


Figure 7.2: The two tiers of the cache on the same stream: at every group boundary, the VRAM holds the V most recent keys, the RAM the R most recent, and the former is included in the latter (lemma 7.6).

7.4.4 Witness O1

Four paths, each compared *to the unit* (integer hits; a rate published by a Python tool is converted back to an integer, and the tool checks that n/accesses gives back the published float to the bit): Table 7.2.

Table 7.2: Witness O1: troncons' simulator against the reference simulators [meas.]. Synthetic traces: $T = 4\,000$ tokens, $L = 4$, $E = 128$, $k = 8$, seed 20261015.

Path	Against	comparisons	gaps	under break
P1	`cache_lru`, `cache_belady` of the loading study, layer by layer; uniform, frozen, sticky; $C \in \{8, 16, 32, 64\}$	96	0	48
P2	`g_lru`, `g_belady` (global cache); $N \in \{8, 31, 32, 51, 128, 205\}$	36	0	22
P3	`g_lru` at V and at R (two tiers); $(V, R) \in \{(8, 32), (16, 64), (32, 32), (51, 205)\}$	24	0	12
P4	rates published on 12 real traces of gpt-oss-20b, budgets 5 to 40 %	352	0	308
	total	**508**	**0**	**390**

Data: `moteur/O-pour-le-memoire.md`, part I; `mesures/moteur/O/journaux/temoin-o1.json`, `temoin-o1-casse.json`.

The witness compares *rules*, not speeds: it proves that `rejouer` and the Python simulators apply the same rule. It does not say that this rule is llama.cpp's, which differs by the order of misses within a group (gaps of 4 and 5 hits, measured by the replica `g_lru_llama`, Section 7.6). Nine breaks of the unit witnesses all fall, and the restored copy holds (`mesures/moteur/O/journaux/tests-casses.txt`).

7.5 The cliff of the shared LRU

llama.cpp's cache, like most published proposals, shares a single LRU between all layers. This choice has a consequence that can be proved, and that does not depend on the text.

Proposition 7.7 (The threshold and the plateau of the shared LRU)**.** *An LRU cache of N slots serves the group stream of an MoE with L layers and k distinct experts per layer, the accesses of a group being judged against the content before any eviction. For any routing:*

1. *if $N \leq (L-1)\,k$, the shared LRU has **no hit at all**;*
2. *if $N \geq L\,k$, every adjacent reuse (an expert requested at token t and at token $t+1$, same layer) is a hit, so the hit rate over tokens $t \geq 1$ is at least the mean adjacent reuse $\bar{r}$;*
3. *if $N = L\,k$ exactly, a hit is an adjacent reuse and only that: the rate is exactly $\bar{r}$ over tokens $t \geq 1$, i.e. $\bar{r}\,(T-1)/T$ over all accesses of T tokens.*

Proof. Let key (ℓ, e) be requested at token $t+1$. The recency rank of a key is its place in the order of last uses (1 = most recent); by Proposition 7.3 and the corollary of lemma 7.5, when a group is judged the cache holds exactly the keys of rank $\leq N$. (i) The last use of (ℓ, e) is at latest at token t, layer ℓ. Between that last use and the judgement of group $(t+1, \ell)$, the stream has served at least groups $(t, \ell+1), \ldots, (t, L-1), (t+1, 0), \ldots, (t+1, \ell-1)$: $L-1$ groups of k distinct keys, all of layers other than ℓ, hence distinct from each other and from (ℓ, e). Its rank is at least $(L-1)k+1 > N$: it is no longer in the cache, and it is a miss. (ii) If (ℓ, e) was served at token t, the keys more recent than it at judgement time are at most the $(L-1)k$ keys above and at most $k-1$ keys of its own group (t, ℓ): rank $\leq Lk \leq N$, it is in the cache. (iii) If (ℓ, e) was not served at token t, its last use is at latest at token $t-1$; between it and the judgement of $(t+1, \ell)$, the stream has served the whole of token t: L groups, Lk distinct keys, all different from (ℓ, e) (those of layer ℓ at token t do not contain it, those of other layers have another layer). Its rank is at least $Lk+1 > N$: miss. With (ii), at $N = Lk$, hit is equivalent to adjacent reuse. Over all accesses (token 0 has only misses), the exact value is $\bar{r}\,(T-1)/T$. ■

Corollary 7.8 (The threshold as a fraction of the experts)**.** *With $f = N/(LE)$, the shared LRU is zero for $f \leq \frac{L-1}{L} \cdot \frac{k}{E}$ and at least $\bar{r}$ for $f \geq k/E$. gpt-oss-20b: zero up to 11.98 %, at least $\bar{r}$ from 12.5 %; gpt-oss-120b: zero up to 3.04 %, at least $\bar{r}$ from 3.125 %.*

Figure 7.3 draws the proof: a token is one turn of a ring of L sectors of k accesses; between two possible requests of the same expert, $(L-1)k$ distinct accesses pass. It is the cyclic sequence, LRU's worst case (Section 7.3), that the computation of an MoE produces by itself. Point (i) was proved twice on the same day, independently: by the engineer of work package O (his lemma 3, "the cliff", with its unit witness) and by the gpt-oss measurement agent (its proposition 1, which adds (ii) and (iii)).

Measured. On the four reading traces of gpt-oss-20b ($L = 24$, $k = 4$: $(L-1)k = 92$, $Lk = 96$), LRU returns **0 hits out of 1 024 224 accesses up to $N = 92$ included**, 9.3 % at 93, 34.8 % at 95, **50.26 % at 96**; Belady, at 92, serves 66.4 % [meas.] (Figure 7.4). The following plateau is the exact value of (iii): $\bar{r}\,(T-1)/T = 0.5026$. The witness at the exact threshold capacities (declared before the computation; the tool stops if it falls) checks, on the **17 real traces of both models**, the zero at $(L-1)k$ and equality (iii) to the hit (gap below 10^{-10} accesses, the rounding of two summation orders; equality to the bit on 11 traces) [meas.] (Figure 7.5). Its break (the zero expected at $(L-1)k+1$) makes it fall: LRU(93) = 0.0972.

Beyond the cliff, a plateau. From $N = 96$ to 120, LRU gains only 0.12 point (50.26 → 50.37 %), Belady 6.8 points. The reason is that of (iii): an expert of layer ℓ absent from token t still has before it,

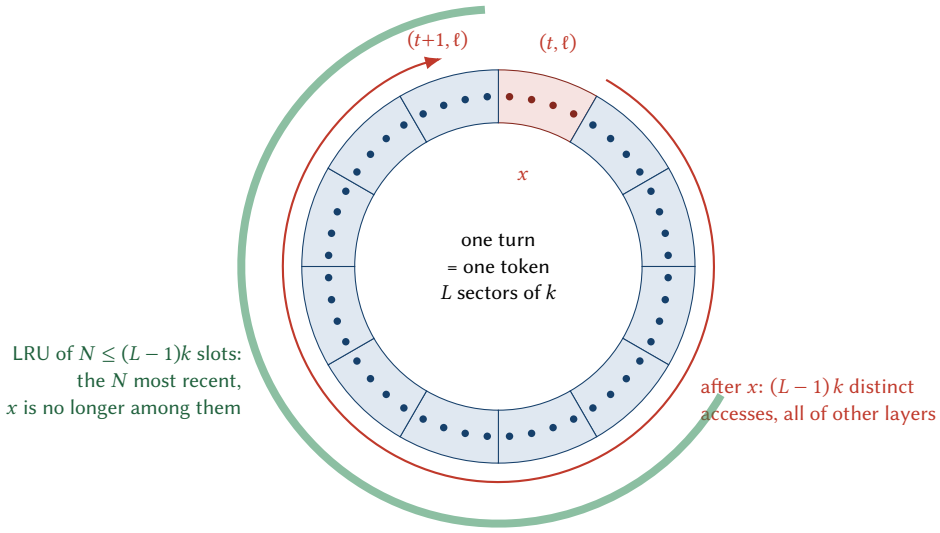


Figure 7.3: The access stream as a ring: one turn is one token, each sector one layer of k accesses. Expert x served at (t, ℓ) can come back only at $(t+1, \ell)$, after $(L-1)k$ distinct accesses; an LRU of $N \leq (L-1)k$ slots, which keeps only the N most recent, has already evicted it.

at its judgement, the whole of token t and the keys of the other layers served since its last use; the slots beyond Lk keep the non-reused experts of the layers served longest ago, which leave before their layer comes back.

What it tells the engine. For gpt-oss-120b, $(L-1)k = 140$ slots of 13 254 144 B, i.e. **1 855 580 160 B (1.73 GiB)** of VRAM below which LRU serves nothing [calc.]. J1's VRAM (326 slots, about 4.3 GB) is well above; but any "minimal window" that brought the cache below 1.86 GB with LRU would pay $h = 0$, where the offline optimum still keeps two thirds. And being above the threshold is not enough: on the real generation of the 120b, at the budget of llama.cpp's cache (323 slots), a fixed set of experts beats its LRU (0.500 against 0.392) and Belady reaches 0.685 (`mesures/localite-gpt-oss/cache-base-contre-optimum.json`): 29 points of hit rate to be taken by a better policy before any format gain.

7.6 The locality of gpt-oss

The traces are those of the gpt-oss measurement: 10 669 reading tokens per model, in four genres (English and French prose, dialogue, code), recorded by the same engine as the llama.cpp baseline (`mesures/gpt-oss.md` § 4). Two witnesses vouch for them: a second path of the top-k (our sort of the router scores) finds no disagreement on 345 336 choices (20b) and 444 276 (120b); and the replica of llama.cpp's cache rule (`g_lru_llama`: hits touched in score order, misses sorted by number, least recently used slot) finds **exactly** the tally that llama.cpp publishes in the same run: 20 373 hits out of 24 576 and 88 443 out of 98 304 (20b), 14 446 out of 36 864 (120b); with 322 slots instead of 323, the witness falls (witness G5) [meas.].

7.6.1 Reuse: popularity and recency

Reuse breaks down into two terms (Table 7.3). **Popularity** (some experts serve more than others) takes it from 0.125 to 0.31 on the 20b; **recency** (the next token takes up the experts of the previous one), from 0.31 to 0.50. Witness G4 proves it: shuffling the order of tokens brings reuse back to

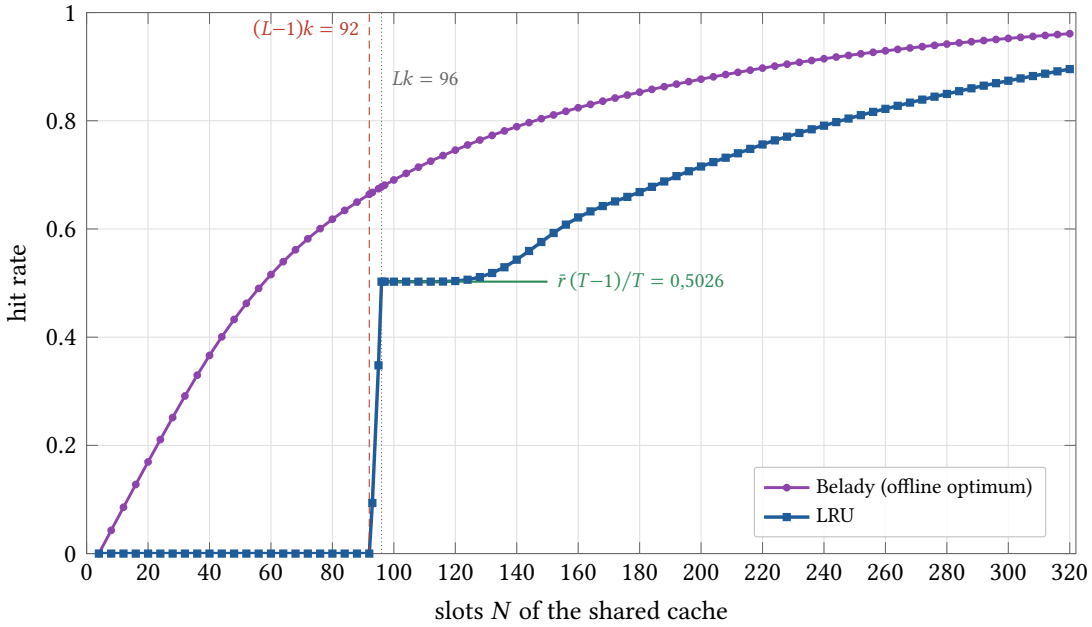


Figure 7.4: Hit rate of a shared cache versus the number of slots, LRU and Belady, on the four reading traces of gpt-oss-20b (10 669 tokens). LRU is zero up to 92, jumps to the adjacent reuse at 96, then stays on a plateau.

Data: `mesures/moteur/O/csv/falaise-20b-dense.csv` (trace `t20b.tous.lecture`).

Table 7.3: Share of the k experts of a token already chosen by the previous token, same layer, mean over layers, four genres in reading [meas.]. "Random with real popularity" draws each token independently, with the measured frequency of each expert.

Model	tokens	measured $\bar{r}$	uniform random k/E	random, real popularity	gap in σ
gpt-oss-20b	10 669	**0.503**	0.125	0.310	+422
gpt-oss-120b	10 669	**0.312**	0.031	0.131	+664

Data: `mesures/gpt-oss.md` § 6.1, § 6.2;
`mesures/localite-gpt-oss/figures/reutilisation-gpt-oss-*.csv`.

random with real popularity (0.309 against 0.310), so the second term belongs to the order of the text. On the 120b, ten times uniform random. Per layer, reuse grows with depth: from 0.14 (layer 0) to 0.53 (layer 35) on the 120b, from 0.23 to 0.66 on the 20b.

7.6.2 The policies

Five facts stand out (Figure 7.6; `mesures/gpt-oss.md` § 6):

1. **Locality is strong**: at 20 % of the experts, Belady serves 81 % (20b) and 90 % (120b) of accesses, LRU 60 and 78 %, where a routing without locality would serve 20 %. The uniform assumption of earlier studies underestimated the cache by a factor of 3 to 4 on the served share.
2. **The gap to Belady is large** (0.60 against 0.81 at 20 % on the 20b): it is the margin of a perfect predictor.
3. **Sharing the budget only helps a good policy**: on the 120b, shared Belady beats per-layer Belady at every budget (0.632 against 0.508 at 5 %), but shared LRU stays below per-layer LRU up to 15 % (0.333 against 0.384 at 5 %): the cyclic stream costs it even above the threshold.

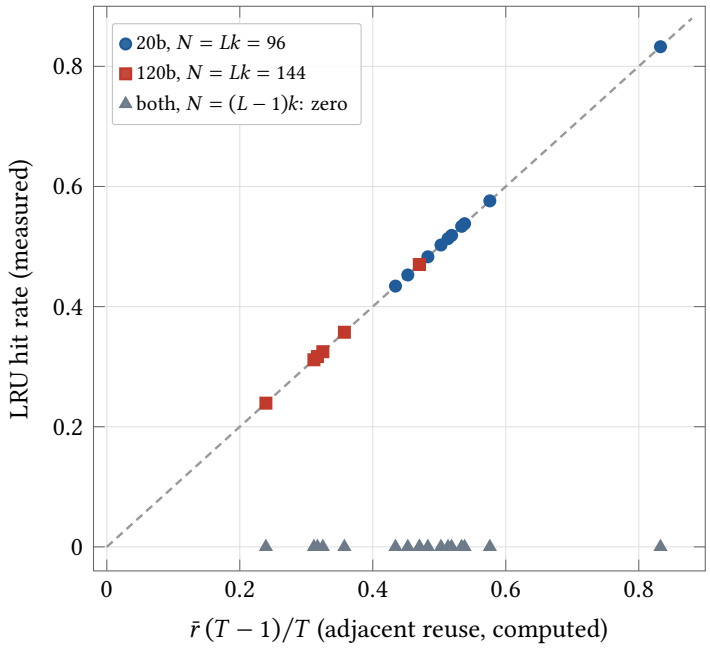


Figure 7.5: Proposition 7.7 on the 17 real traces (10 of the 20b, 7 of the 120b; reading and generation): at $N = Lk$, the measured hit rate falls on the diagonal $\bar{r}\,(T-1)/T$; at $N = (L-1)k$, it is zero everywhere.

Data: `memoire/figures/tome2/donnees/t2o-prop1.csv`, drawn from `mesures/localite-gpt-oss/20b/localite-gpt-oss-20b.json` and `120b/localite-gpt-oss-120b.json` (fields `seuil_lru_partage`, `seuil_borne_ii`).

4. **A set chosen offline does not travel**: pinning calibrated on the start of a trace collapses when the genre changes (0.147 at 10 % on the 120b, against 0.37 to 0.64 within each genre taken alone).
5. **Routing depends on the computation path**: on the 120b, GPU against CPU, only 81.1 % identical choices, and the first divergence comes at the 3rd token. A trace holds for its path, and the accuracy of an expert predictor has a ceiling set by this noise.

What locality is worth in tokens per second. With the machine's measured throughputs (VRAM 251.3 GB/s, PCIe 12.2 GB/s, disk 2.82 GB/s in 16 MiB blocks), an expert worth its original bytes, and the two-tier model of lemma 7.6 (VRAM at 7 % of the experts, $h_v = 0.449$; total budget 20 %, LRU, $h = 0.777$), the time of an overlapped token works out as [calc.]:

$$\text{VRAM: } \frac{1.686 + 0.449 \times 1.909}{251.3} = 10.1 \text{ ms}, \qquad \text{PCIe: } \frac{0.551 \times 1.909}{12.2} = 86.2 \text{ ms},$$
$$\text{disk: } \frac{0.223 \times 1.909}{2.82} = 151 \text{ ms}.$$

The maximum, 151 ms, gives 6.6 tokens/s; the disk bounds. Any compression factor c of the experts *in transit* divides the disk and PCIe terms by c: at $c = 2$, $\max(10.1; 43.1; 75.5) = 75.5$ ms, i.e. 13.2 tokens/s, as long as decoding on the GPU keeps up (`mesures/gpt-oss.md` § 8.3). This is troncons' lever, quantified.

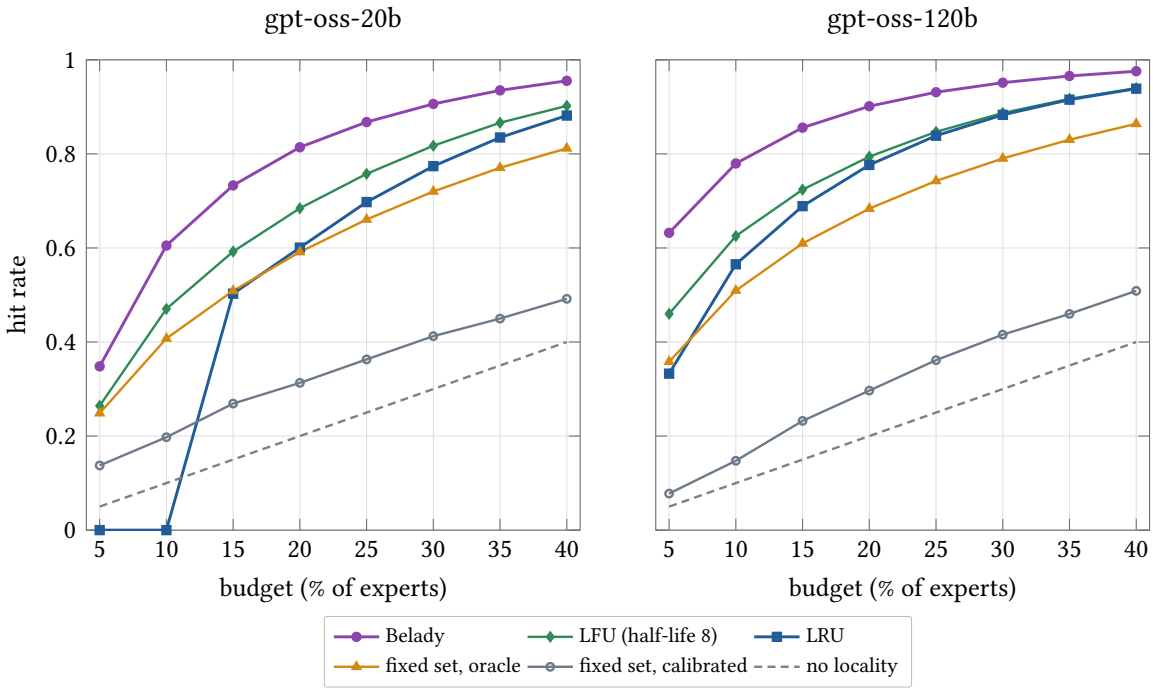


Figure 7.6: Hit rate of a shared cache versus its budget (share of the experts), four genres in reading. On the 20b, LRU is zero at 5 and 10 % (below the cliff, 11.98 %); on the 120b, the cliff is at 3.04 %, below the first plotted budget.

Data: `mesures/localite-gpt-oss/figures/succes-gpt-oss-20b-lecture.csv` and `succes-gpt-oss-120b-lecture.csv`.

7.7 The on-demand scheduler

7.7.1 The path of a token

The J1 engine (`trc-ordo`) serves each layer on demand, without prefetching (Figure 7.7). Three CUDA streams: **compute** (high priority), **copy**, **decode**. For each layer ℓ, the compute stream runs attention then the router, which writes the k indices and their weights into a **mailbox** in mapped memory (word 0: a sequence number written last, after a barrier; words 1 to k: the indices; $k+1$ to $2k$: the weights). The host waits for the "router ℓ" event, checks the sequence, reads the indices and calls `demander`. Each miss becomes an unbuffered IOCP read into a pinned arrival buffer; as soon as it completes, the copy goes on the copy stream, then the decoding on the decode stream, which writes the slot; the compute stream waits for the "ready" event of each miss, then launches the four GEMVs and the combination, in router order.

7.7.2 The order is safe

Lemma 7.9 (The order of the three streams is safe)**.** *In `Moteur::pas`, no expert slot is read by a GEMV while a decoding writes it, and no arrival buffer (pinned or in VRAM) is rewritten before the copy or the decoding that read it has finished.*

Assumptions. (H1) A CUDA stream executes its operations in queue order, each after the end of the previous one. (H2) `cuStreamWaitEvent(f, e)` makes every operation queued afterwards on f wait for the end of all the work that e captured [42]. (H3) The host decides layer ℓ (request, submit, enqueue) only after seeing the "router ℓ" event finished, recorded on the compute stream right after

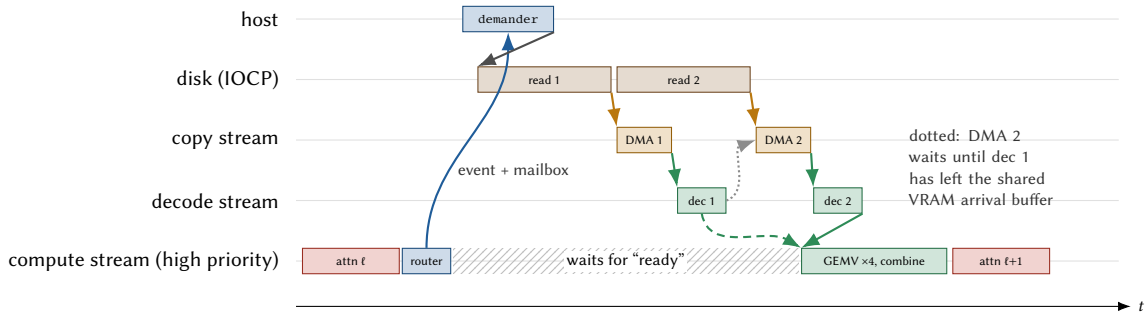


Figure 7.7: The path of a layer with two missing experts (diagram M1, as built): the streams meet only through events; the mailbox carries the router's indices to the host.

Data: `moteur/crates/trc-ordo/src/lib.rs` (header and `Moteur::pas`).

the router. (H4) An expert requested by layer ℓ is pinned until `liberer(`ℓ`)`, called after enqueueing the end of layer ℓ; an eviction victim is never pinned nor in flight.

Proof. (a) *Slots.* Let s be a slot decoded for layer ℓ' and evicted by a request of layer ℓ, later in the stream. By (H4), the eviction follows `liberer(`ℓ'`)`, hence the enqueueing of the end of ℓ' on the compute stream; the new decoding into s is enqueued after the decision of ℓ, hence, by (H3), after the host has seen the router of ℓ finished. Since that router is enqueued after the end of ℓ' on the same stream, (H1) gives: all GEMVs that read s are finished when the decoding starts. Conversely, the GEMVs of ℓ that read s are enqueued after waiting for the event of its decoding (H2): they start only after it ends. (b) *Arrival buffers.* Buffer i serves at most once per layer; its reuse at layer ℓ follows the decision of ℓ, hence (H3, H1) the end of the previous layers, which waited (H2) for the decodings of their misses, which waited for their copies. The explicit waits (copy after the previous decoding of the same buffer, read after the copy of the same pinned buffer) make this point true without (H3), which keeps it should a prefetch ever advance the misses. ■

Cost: one host synchronisation per layer, 36 per token on the 120b, 18.5 µs each under load (trial E0 c of work package E), i.e. 0.67 ms per token, 0.15 % of the 457 ms of a token [calc.].

7.7.3 The engine computes what the kernels validated, to the bit

Proposition 7.10 (Token by token equals layer by layer)**.** *For a given token sequence and given weights, the logits returned by the engine (one token at a time, all layers) are equal to the bit to those of the kernel-validation chain (one layer at a time, all positions), if both launch the same kernels with the same numerical arguments.*

Proof. By the architecture's determinism rule (no floating-point atomic addition, fixed-tree reductions that depend only on sizes), each kernel is a deterministic function of the bytes it reads and of its parameters. Order the computations by (layer ℓ, position p). Induction hypothesis: for every $(\ell', p') < (\ell, p)$ in product order, the residual entering (ℓ', p') and cell p' of the KV cache of ℓ' are equal to the bit in both orders. At (ℓ, p), the inputs are: the residual, output of $(\ell - 1, p)$, equal by induction; the cells $q \le p$ of the KV cache of ℓ, written by (ℓ, q), equal by induction, and cell p itself; for a sliding-window layer, the 128-entry ring holds at read time exactly positions $\max(0, p-127) \dots p$ in both orders; the chosen experts are those of the router of (ℓ, p), computed on equal inputs, and their slots carry the same bytes (exact format and transport). The same kernels return the same bits. ■

Listing 7.2: The decision of a layer in `Moteur::pas` (`trc-ordo/src/lib.rs`, excerpt): wait for the router, check the mailbox, ask the cache. French comments: "the event variant of trial E0 c (18.5 µs per synchronisation under load): wait for the event, then read the mailbox"; "the trace that rejouer will read back".

```
self.ev_routeur.enregistrer(&self.calcul)?;
// Variante « evenement » de l'essai E0 c (18,5 us par synchronisation sous
    charge) :
// on attend l'événement, puis on lit la boîte.
self.ev_routeur.attendre()?;
let lu = self.boite.lire_volatil_u32(0);
if lu != seq {
    bail!("couche {l} : boîte aux lettres à la séquence {lu}, {seq} attendue");
}
for i in 0..k {
    let x = self.boite.lire_volatil_u32(4 * (1 + i));
    if x as usize >= h.experts { bail!("couche {l} : expert {x} hors de [0, {})",
     h.experts); }
    experts[i] = x as u16;
}
self.trace.extend_from_slice(&experts[..k]);          // la trace que rejouer
    relira
let decisions = self.cache.demander(l as u16, &experts[..k], pas * l_n as u64 + l
     as u64);
```

Witness O2: 160 rows out of 160 equal to work package K's chain (KL, top-1, relative gap), mean KL $7.2794050122285915 \cdot 10^{-3}$ to the digit; the path that places the GGUF's experts without the decoder, 16 out of 16. The breaks (position shifted by 1; experts 0 and 1 swapped at layer 5) make every row differ [meas.] (`mesures/moteur/O/journaux/o2/`). The witness does not judge the model's correctness (that is C4, in the chapter on the reference): it judges that the engine computes *exactly* what work package K validated.

Proposition 7.11 (C3 by construction: the cache does not enter the computation)**.** *If two files (coded and raw) give, for every expert, slots equal byte for byte, the engine returns the same logits to the bit on any token sequence, whatever the number of slots, the cache policy and the arrival order of the reads.*

Proof. An expert's GEMVs read its slot and the input vector; at read time, the slot carries the decoded bytes of that expert (lemma 7.9), equal by assumption in both files. The cache chooses only *where* an expert is, never *which bytes* it carries; the order of the combination is the router's, not the slots'. The other inputs do not depend on the expert file. Proposition 7.10 concludes. ■

Measured: on the 20b, 5 120 steps, 0 different logit fingerprint, `.kld` files equal byte for byte, equal perplexity (461.978277060145); the two passes did not have the same number of slots (362 and 361) nor therefore the same sequence of misses: the "whatever the cache" part is observed, not merely deduced. On the 120b, 5 120 steps, 0 difference, perplexity 422.20558522791305 on both sides, with 325 and 326 slots [meas.].

Proposition 7.12 (The engine's count equals the simulator)**.** *The engine calls `trc-cache` in the simulator's order: for each token t and each layer ℓ, `demander(`ℓ`,` $G(t,\ell)$`,` $tL+\ell$`)`, `pret` on each miss in decision order, then `liberer(`ℓ`)`, and nothing else touches the cache in $\mathcal{J}1$. `rejouer` performs the same sequence on the trace written by the engine. Same calls on the same initial state: same counters.*

Measured: on the 20b (5 120 steps, 362 slots), **454 210 hits, 37 310 misses, 36 948 evictions on both sides** ($h = 0.924$); break (falsified LRU): 427 984 / 63 536 / 63 174. On the 120b (326 slots): 334 608 hits, 402 672 misses, 402 346 evictions on both sides ($h = 0.454$); break: 289 788 / 447 492 [meas.] (`moteur/REPRISE-O.md`, O3).

7.7.4 The serial floor, and what the engine does with it

The token equation sets the time of a token served serially as the sum of the stages. With the quantities measured in session O4 (generation, 120b, mode A0, 326 slots, $h = 0.591$), criterion C5 is computed step by step (Table 7.4).

Table 7.4: The serial floor of the 120b, stage by stage, in session O4 [calc.] on [meas.].

Stage	Formula	Value
disk	section bytes read per token / throughput of the disk alone	0.7072/1.848 GB/s = **382.7 ms**
PCIe	GPU time of the copies per token (59 misses, 1.28 ms each)	**75.8 ms**
decoding	GPU time of the decodings per token (0.92 ms per expert)	**54.2 ms**
GEMV	$(A + R)/235$ GB/s, $A = 1.686$ GB, $R = 144 \times 13.25$ MB	**15.3 ms**
T_0	fixed cost	3.5 ms
sum		**531.5 ms → 1.881 tokens/s**
C5 threshold	0.8× floor	1.505 tokens/s

Data: `moteur/O-pour-le-memoire.md` § 6.5; `mesures/moteur/O/journaux/o4/verdicts-2026-10-15-s1.json`.

Measured: 457 ms per token, 2.186 and 2.182 tokens/s, above the *serial* floor: the copies and decodings of a layer overlap with the reads of the next misses of the same layer. The timeline of a layer whose four experts are all missing (Figure 7.8) shows what limits: the router ends at 0.61 ms; the four reads, submitted at 1.28, 1.96, 2.68 and 3.18 ms, end at 6.86, 9.81, 16.14 and 21.17 ms. **The disk serves them one after the other** (about 5 ms for 12 MB, 2.4 GB/s each); depth 4 does not make them progress together. Each copy (about 1.0 ms) and each decoding (0.85 ms) follow the read of their section without waiting for the others, and the end of the layer (23.73 ms) follows the last decoding by 0.6 ms. With 59 misses over 36 layers, 1.6 per layer on average, the engine draws 0.707 GB in 457 ms, 1.55 GB/s of useful throughput, against 1.85 GB/s for the disk alone at depth 4: **what limits J1 is the queue depth per layer** ($\leq k$), which only a prefetch could fill.

7.8 Streamed prefill

In the step loop, a prompt of P positions reads $B_{\text{loop}} = \sum_p \sum_\ell (1 - h_{p,\ell})\, k\, t$ bytes, with $t \approx 12.03$ MB per section. Streamed, each layer passes all positions through attention and the router, then reads once each expert requested by at least one position: $B_{\text{stream}} = \sum_\ell |U_\ell \setminus V_\ell|\, t$, with U_ℓ the set of chosen experts and V_ℓ those already in VRAM. Under uniform routing, $\mathbb{E}|U_\ell| = E\,(1 - (1 - k/E)^P)$; for $E = 128$, $k = 4$, $P = 708$, $(1 - 1/32)^{708} = e^{708 \ln 0.96875} = e^{-22.5}$: U_ℓ is the whole set, and $B_{\text{stream}} \leq 36 \times 128 \times 12.03$ MB $= 55.4$ GB, against $B_{\text{loop}} \approx 708 \times 144 \times 0.55 \times 12.03$ MB ≈ 675 GB [calc.]: a ratio of about 12. **Measured**: 26 s instead of 7 min 36 s, ×17.5, at the mean throughputs of passes `s2-11` to `s2-14` (27.15 against 1.554 tokens/s, `mesures/moteur/O/csv/o-jetons-s.csv`; lot O's note wrote ~~7 min 50 s~~, which does not come from these passes); on top of the bytes, the loop pays one synchronisation per layer and per position, which the stream does once per layer. Exactness

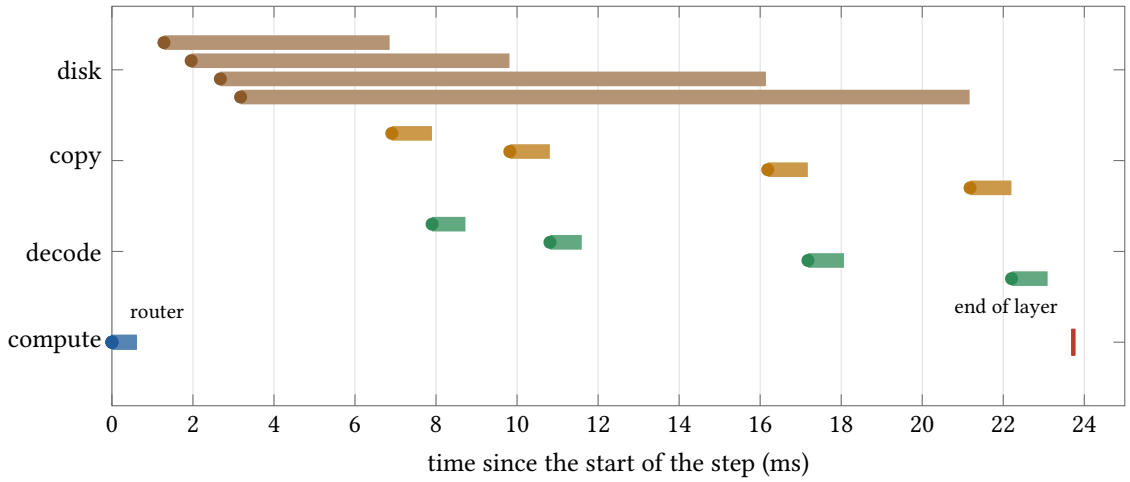


Figure 7.8: The measured timeline of a layer of the 120b whose four experts are missing (step 8, layer 0): from submission to the end of each read, then its copy and its decoding; at the bottom, the router and the end of the layer.

Data: `memoire/figures/tome2/donnees/t2o-chrono-couche.csv`, drawn from `mesures/moteur/O/csv/chronogramme-120b.csv` (8 tokens, 1 048 timed misses).

follows from Proposition 7.10 (layer-by-layer order is that of the kernel chain); witness F: 5 120 fingerprints equal to the loop on the 20b, and, redone by the independent trial on the 120b, `.kld` files equal byte for byte from one build to another [meas.].

A defect proved, then corrected. Streamed prefill hands up to eight reads at a time to the system, in a round robin of 12 arrival buffers. Its first rule took buffer $s \bmod 12$ for the s-th read. Counter-example: let read $s-12$ stay in the system (completions arrive out of order) while $s-11, \ldots, s-1$ pass and are reaped; the list of in-flight reads then holds only $s-12$, and s is submitted into buffer $s \bmod 12 = (s-12) \bmod 12$, which the disk is still writing. A delay of about $11 \times 6.6\ \text{ms}/3 \approx 24\ \text{ms}$ of one read over the others suffices [calc.], and the trial recorded a 90th centile of 172 ms for a cold read.

Lemma 7.13 (The arrival buffers of streamed prefill)**.** *With the corrected rule (the round-robin buffer if it carries no in-flight read, otherwise the next one that carries none), no read writes into an arrival buffer while another read writes there, while the copy that read it has not finished, or while a guard hash still reads it.*

Proof. Existence: at choice time, at most $\min(dk, 2 \cdot \text{depth}) - 1 = 7 < 12$ reads are in flight, so a free buffer exists. The copy: we wait for the event of the previous copy of that buffer before submitting (it was enqueued at its reaping, hence before). The guard: we wait for the end of its hash before any submission. The VRAM buffer is rewritten only after waiting for the decoding that read it. ∎

Under the trial's load (its numpy f64 pass of the 120b running at the same time), the ten windows of the raw 120b finish in 361 s with 0 conflict avoided: the case did not occur in that pass. The rule is proved; whether this defect caused the hang observed by the trial (finding G1) remains a hypothesis, the old binary having also finished under the same load (`moteur/O-pour-le-memoire.md` § 14.1).

7.9 Prefetching, tried and abandoned

The mechanism. After the attention of layer ℓ, the engine applies to the current residual the norm and the router of $\ell + 1$; the predicted experts absent from the VRAM are read ahead into $2k$ pinned buffers, outside the cache (whose count stays that of `rejouer`; the four texts of the series are equal). The loading study cited 96 % correct predictions one layer ahead, *literature, not reproduced.*

The balance, set out before the measurement. A correct prediction removes a read from the critical path; a wrong one adds a read to a disk that already limits. Let p be the share of predictions used and f the share of a token's time that overlap can hide (the off-disk computation); the relative gain is, at first order,

$$\text{gain} \approx f - \frac{1-p}{p} \cdot (\text{disk share}). \tag{7.1}$$

The declared prediction (`moteur/REPRISE-O.md`) was at most +19 % (the $457 - 383 = 74$ ms of off-disk computation, relative to the 383 ms of the disk) plus what the deeper queue would give back to the disk; that prediction did not count wrong reads.

Measured (session `2026-10-16-s3`, A B B A, 120b, mains power): 69.2 % of the prefetched reads are used (11 529 out of 16 670); the others add 34 % bytes read (953 against 710 MB per token); the computation they can overlap is worth about 80 ms per token, $f \approx 80/460 = 0.17$. With $p = 0.692$ and a disk that carries about 80 % of the time, the negative term of (7.1) is $(0.308/0.692) \times 0.8 \approx 0.36$ and outweighs f: the first order predicts a loss, of the order of $0.17 - 0.36 \approx -0.19$ [calc.]. **Generation: A 2.158 and 2.178, B 1.935 and 1.924 tokens/s, i.e.** −11.0 %, for a noise of 0.9 % [meas.] (recomputed here from `mesures/moteur/O/csv/o-jetons-s.csv`). The gap passes the decision rule ($2 \times 0.9\,\% + 3\,\%$): it is real, in the wrong direction, and smaller than the first order (the deeper queue gives back to the disk part of what the wrong reads take from it, which (7.1) does not count).

> **What is killed, and what is not**
>
> Prefetching by the next layer's router is **abandoned for this predictor and this disk** (BitLocker-encrypted, 1.8 GB/s), not in general. By (7.1), it would come back only with p above about 0.9, or a disk that does not limit. A defect was found on the way and corrected (the completion of the current layer reaped while draining the prefetches).

12 MiB reads, not established. Reading each section in one 12 MiB request (instead of its exact length, about 12.0 MB) gives +30 % in median (A 2.179 and 2.182, B 2.613 and 3.073 tokens/s), every B pass above every A pass; but the noise of B, $\beta = 16.2\,\%$ (~~21 %~~ in lot O's note, the gap divided by the mean of A), raises the rule's threshold to $2 \times 16.2 + 3 = 35.4\,\%$, above the gain: the rule does not conclude in four passes. The supposed mechanism is the map of the disk's two regimes drawn by work package E; the decision belongs to the arbiter.

7.10 The integrity guard, and what it costs

The independent trial showed that in production the engine did not check the raw bytes of a section (medians and biases, 0.36 % of the bytes, not rANS-coded): a corrupted median gave a wrong expert without error, and the produced text was even identical (finding M1). The round of corrections added a **guard**: each completed read is hashed by four threads (empreinte64, the 64-bit prefix of

SHA-256 [43]) and compared with the directory before the token leaves; each embedding row read is compared with a table built at open time.

Proposition 7.14 (No output depends on an unchecked byte)**.** *With the guard active, the token and logits returned by a step depend only on bytes read whose fingerprint equals the directory's, up to a collision of the 64-bit prefix (probability 2^{-64} per check, if SHA-256 behaves like a random function).*

Proof. The decoder and the copy read only the useful bytes of a part; each completed read is handed to the guard before any other operation on its buffer; no read writes that buffer before the hash ends (lemma 7.13), so the fingerprint covers the bytes the copy read; the step waits for all hashes before reading the token or the logits, and a different fingerprint is a named fatal error. A warm entry was hashed at its read; a later corruption of the RAM is not covered. ■

The cost, set out then measured. A step of the 120b reads about 0.71 GB of sections; the fingerprint runs at 1.07 to 1.10 GB/s per thread, measured (`mesures/moteur/O/corrections/journaux/debit-empreinte.jsonl`; ~~2 GB/s per core~~, estimated by the trial). It therefore takes about 0.65 s of core time per step, spread over four threads (0.17 s of wall time, under the 0.5 s of a step); only the hash of the last read remains on the critical path, at most 12.25 MB/1.07 GB/s ≈ 11.4 ms, minus the GPU work that follows it [calc.]. **Measured**: 5.2 to 5.4 ms of waiting per token over the four guarded passes; at equal disk throughput (passes 04 and 05, at 1.715 and 1.708 GB/s), $1.900152/1.931729 - 1 = -1.6\,\%$ [meas.] (`mesures/moteur/O/corrections/csv/vitesse-2026-10-16-passes.csv`). Lot O's note wrote ~~5.2 to 5.3 ms~~ (pass 07, at 5.42 ms, was not in it) and ~~−1.7 %~~, computed on the rounded values 1.900 and 1.932 (Section 8.5.5). The declared series says −12.3 % under a threshold of 26.8 % (noise 11.9 %), not real: the disk varied from 1.51 to 1.95 GB/s during the session. Opening costs 0.95 s more (the row table: 0.615 GB read and hashed), the working set 1.9 MB, the same for both models.

7.11 The engine against the baseline: criterion C6

C6 asks that troncons beat the best measured llama.cpp configuration on the 120b, same prompt, 256 greedy tokens, by more than twice the largest noise and 3 % (decision rule of the study of the models). All the difficulty lies in "the best". Its history reads in bars (Figure 7.9).

- ~~×3.85~~: J1 (2.184 tokens/s) against `fit` at 6 threads (0.568), the baseline published by the gpt-oss measurement. Struck: the baseline was under-tuned.
- ~~×2.79 to 2.89~~: the independent trial, against `fit` at 10 threads (0.702 in its session). Struck in turn: the trial had not tried beyond 10 threads.
- ×2.22: the round of corrections swept the baseline (session of 2026-10-16 from 08:13 to 10:06, palindrome, one warm-up pass discarded, mains power at every pass, no foreign process): `fit` `-t 6` 0.574; `-t 10` 0.665; `-t 12` 0.836 (noise 18.3 %); **`-t 16` 0.884** (0.900 and 0.867); `-ngl 99` `--n-cpu-moe 34` `--fit off` `-t 10` 0.640. The best, `fit` `-t 16`, beats `-t 6` by 54 %: the published baseline underestimated llama.cpp by 54 %. The mechanism is the one the trial had declared: under mmap, each thread in a page fault is a read in flight; 16 threads make a deeper queue on the disk. troncons, **integrity guard included**, makes 1.965 tokens/s (four passes of the speed session, noise 10.97 %): $1.965/0.884 = \mathbf{2.22}$, beyond the threshold $2 \times 10.97\,\% + 3\,\% = 24.9\,\%$ [meas.] (recomputed here: 1.9649 and 0.8838). **C6 holds.**

Without the guard, troncons makes 2.054 tokens/s in the same session (two passes); with the guard and an f32 KV cache, 1.496, but with a different text (chapter on the reference), hence no comparison is possible. Streamed prefill runs at 26.6 tokens/s, against 3.1 to 4.0 for llama.cpp (batched): a factor

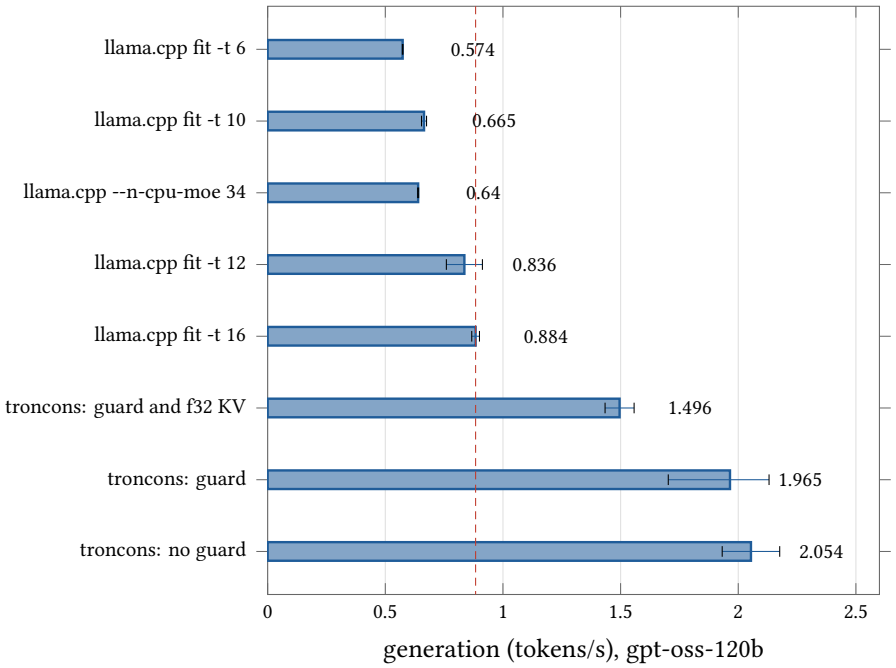


Figure 7.9: Generation on gpt-oss-120b (256 greedy tokens, 708-token wikitext prompt): the llama.cpp baseline swept by the round of corrections and troncons in the speed session of the same round, on the same day, on mains power. Mean of the passes, bars from the smallest to the largest; dashed line: the best baseline, `fit -t 16`.

Data: `memoire/figures/tome2/donnees/t2o-c6.csv`, drawn from `mesures/moteur/0/corrections/csv/base-2026-10-16-passes.csv` and `vitesse-2026-10-16-passes.csv`.

of 7. And the engine's peak memory is 384 MB, against 10.9 GB for llama.cpp on the same model (`moteur/0-pour-le-memoire.md` § 8). **The baseline may still be better**: neither more than 16 threads, nor `--mlock`, nor other `--n-cpu-moe` values were swept.

7.12 What remains

- **A policy aware of the layer cycle.** The cliff and the plateau come from LRU ignoring that the stream returns to each layer once per token; a per-layer LRU, an SLRU, or an LFU (0.47 at 10 % on the 20b, where LRU makes 0) do better below the threshold, and 29 points remain between llama.cpp's LRU and Belady on the 120b's generation.
- **A more accurate predictor**: prefetching comes back only above $p \approx 0.9$; routing that depends on the computation path (81.1 % agreement GPU against CPU) bounds what can be expected from a predictor learnt elsewhere.
- **The read length** (12 MiB, +30 % in median, not established) and the queue depth per layer, which is what really limits J1.
- **The baseline** beyond 16 threads and with `--mlock`.

What could be wrong

- **The optimality of Belady with group pinning** is not proved, only checked (G3); likewise lemma 7.6 for Belady. Belady is an oracle: its figures are bounds.

- **Witness O1 compares rules**, not speed figures; the engine's rule differs from llama.cpp's by the order of misses within a group (gaps of 4 and 5 hits).
- **The cliff depends on group pinning and on** $N \geq k$; a cache that served a token's experts in another order (prefetching) moves the threshold.
- **Locality comes from 10 669 tokens of four texts**, without a conversation template; the recorded generations are often degenerate and inflate recency. The traces are those of llama.cpp's computation path; the engine produces its own.
- **The C5 floor counts the GEMV at the throughput of another session** (235 GB/s); if the real GEMV is slower, the floor is optimistic and the threshold too high: the verdict stays valid in that direction.
- **The prefetch balance** (7.1) is first order; the measurement (A B B A, one session) is what decides, and it holds only for this disk.
- **C6 compares two sessions of the same day** (baseline and speed); the speed session saw the disk vary from 1.51 to 1.95 GB/s, and troncons' noise (11 %) is large. The baseline may be better than 0.884.
- **The cause of the G1 hang** is not established; the arrival-buffer defect is proved possible, not observed (0 conflicts under the replayed load).
- **The guard assumes SHA-256 without a useful collision** on 64 bits; it covers neither the warm RAM after its read, nor the VRAM. Its cost (−1.6 %) rests on one pair of passes at equal disk throughput.

Part III

The judgement

Chapter 8

Results of milestone J1

Where we start, what this chapter establishes, what remains

Where we start: a program built by five lots, each proved on its own by its control case, and twelve criteria declared on 15 October 2026 before the first line of code (`decisions/ARCHITECTURE.md`, § 7). Volume I had predicted what such a program could do; it had never seen it run. **This chapter establishes** what troncons really does on Stone's machine with gpt-oss-120b: logits identical to the bit between the coded and the raw file over 5 120 steps; 384 MB of peak memory for a 63.4 GB model, the same to within 3.7 MB as for a model five times smaller; 2.18 tokens per second in generation, 2.22 times the best llama.cpp baseline measured once the integrity guard is counted; and, with an f32 KV cache, outputs equal to the exact mathematics of the model up to the file format. Every number that a later measurement contradicted is struck out where it stood, with what replaced it. **What remains**: the decisions the round of corrections handed to the arbiter (default KV cache, criteria C4, C7 and C8), and what the trial could not redo (Chapter 9). Sources: `moteur/O-pour-le-memoire.md` (parts II and III), `moteur/REPRISE-O.md`, `mesures/epreuve-J1.md`, `mesures/gpt-oss.md`, and the CSV files and logs under `mesures/moteur/O/` and `mesures/moteur/epreuve-J1/`.

8.1 What is measured, and how

The model and the task. OpenAI's gpt-oss-120b in its native MXFP4 format: a GGUF file of 63 387 346 208 bytes, 36 layers, 128 experts per layer of which 4 serve each token, 13.25 MB per expert; 4 608 experts in all, 61.07 GB, and 1.69 GB of weights read at every token (attention, routers, output head) [meas.] (`mesures/gpt-oss.md`, § 1). Its development twin, gpt-oss-20b (12 109 566 624 bytes, 24 layers, 32 experts), is already larger than the graphics card. The speed task has been fixed since Volume I: the 708-token wikitext prompt (SHA-256 `eadbe87e`…), then 256 greedy tokens, in mode A0 (no expert cache in RAM: every miss is read from disk).

The sessions. Six sessions produced the numbers of this chapter; all on mains power, checked before and after every pass (`Win32_Battery`, status 2), with a free card and the card lock held (Table 8.1).

The decision rule. It is Volume I's, declared before any measurement (study 06, § 7.5) and taken over unchanged by the architecture (§ 7.2).

Definition 8.1 (Noise, real difference)**.** Let a configuration be measured by two passes X_1, X_2 of one palindrome. Its *noise* is $\beta = |X_1 - X_2|/\bar{X}$; when a configuration has several pairs, its noise is the

Table 8.1: The measurement sessions of milestone J1, of the trial and of the round of corrections.

Session	Date	Content and file
O4 `s1` (lot O)	15 Oct., 22:40–23:03	A120 D Y20 Y20 D A120: C5, C7, C8, C10; `mesures/moteur/O/csv/o4-2026-10-15-s1-passes.csv`
O3 120b (lot O)	15–16 Oct., 23:03–00:53	C3, C11, cache count over the 10 windows
J1+ `s2`, `s3` (lot O)	16 Oct., 00:54–02:08	stream prefill, prefetch, 12 MiB reads; `csv/o-jetons-s.csv`
trial `s1`	16 Oct., from 02:27	12 passes in a palindrome: troncons, baseline at 6 and 10 threads, 20b, warm cache; `mesures/moteur/epreuve-J1/csv/s1-2026-10-16-passes.csv`
round, speed	16 Oct., 06:41–07:19	guard, no guard, f32 KV, in ABBA; `mesures/moteur/O/corrections/csv/vitesse-2026-10-16-passes.csv`
round, baseline	16 Oct., 08:13–10:06	llama.cpp swept: 5 configurations in a palindrome; `corrections/csv/base-2026-10-16-passes.csv`

largest noise of a pair. A configuration B differs *really* from a configuration A if

$$\left|\frac{\bar{X}_B}{\bar{X}_A} - 1\right| \;>\; 2\,\max(\beta_A, \beta_B) + 3\,\%.$$

The noise of a program against itself is the yardstick: a difference smaller than twice a program's distance to itself is not a difference. The 3 % margin protects against a series too short to see its own noise. The trial wrote it as "twice the noise and 3 %"; the verdicts of the round apply it in the additive form above, which is stricter. No verdict of this chapter changes from one reading to the other [calc.] (`memoire/outils/tome2_resultats.py`).

A second path for every number. The numbers of this chapter are not copied from the reports: a committed tool (`memoire/outils/tome2_resultats.py`) rereads the passes themselves, recomputes means, noises, ratios, intervals and floors, compares them with the verdicts the lots published, and stops with an error at the first discrepancy. It found one, which was not a mistake of the lots but an ambiguity of reading (the cost of the guard, Section 8.5.5).

8.2 The criteria at a glance

Table 8.2 follows every criterion of J1 through the three successive judges: the lot that measured it, the independent trial that redid it, and the round of corrections that answered the trial.

> **In brief**
>
> ~~Ten criteria out of twelve hold~~: these were ten rows out of twelve, C7 counting twice and C9 missing. Of the twelve declared criteria, eleven are judged and nine hold entirely; C9, the physical efficiency, is published without being judged (0.84 for J1, Section 8.5.1). C7 holds on its ceiling, with a margin of five, but its tolerance term (0.5 MB) was ill posed: the engine's memory grows by 2.9 MB–3.7 MB when the model grows by 51 GB, and the bound that names every term explains only 2.05 MB of it. C8 measured the machine rather than the engine; the process instrument shows that the engine counts its bytes to within 20 MB. Both criteria await

Table 8.2: Milestone J1, criterion by criterion (gpt-oss-120b unless stated). The thresholds are the architecture's, § 7.1, declared on 15 October before any construction.

Criterion	Threshold	Lot (15 Oct.)	Trial and round (16 Oct.)	Verdict
C1 format	0 wrong out of 4 608	0 wrong (lot F, two paths)	40 drawn chunks + 13 named, 0 wrong, by a 3rd decoder	**holds**
C2 transport	0 mismatch; 100 % of corruptions seen	0 out of 4 608 (lot E)	3 394 decodes equal to the digest; in production, raw streams unguarded (M1), then guard built	**holds** (guard)
C3 exact	0 bit over 5 120 steps	0 (20b and 120b)	0, other build, stream = loop byte for byte	**holds**
C4 correct	KL and top choice ≥ llama.cpp	20b: 0.003 68 vs 0.022 6	120b: 0.003 08 vs 0.060 7; f32 KV: at the floor of the format	**holds**
C5 speed	≥ 0.8× serial floor	2.186 and 2.182 ≥ 1.505	1.70–2.18 ≥ 1.43–1.52, guard included	**holds**
C6 (J1+)	$> 2\beta + 3$ % over the best baseline	~~×3.85~~	~~×2.79~~; ×2.22 against `fit -t 16`	**holds**
C7 ceiling	≤ 1.9 GB	384.2 MB	384.4 MB; 386.8 MB guard included	**holds**
C7 gap	$\leq 2\beta + 0.5$ MB	3.70 MB	2.94 MB; bound with named terms 2.05 MB	**fails**
C8 bytes	PDH ≤ idle × duration + 100 MB	fails on 1 pass of 4	PDH fails; process counters: constant 19.92 MB	instrument to change
C9 efficiency (J1+)	published, not judged	not computed	0.84 (J1); 0.64–0.82 (round), computed by the reprise of the judgement	published
C10 VRAM	≤ 7 279 MiB	6 520–6 537	6 574; 6 811 (round)	**holds**
C11 lossless	equal PPL	422.205 585 227 913 05	the same, coded and raw	**holds**
C12 text	256 equal tokens	`e4c6dbac`… over 17 passes	the same; but 1st token ≠ f64 truth (f16 KV)	**holds**

Data: `moteur/O-pour-le-memoire.md` § 8 and § 14, `mesures/epreuve-J1.md` § 3.1, `mesures/moteur/O/corrections/journaux/verdicts-*-2026-10-16.json`.

> a decision of the arbiter. Neither failure touches the thesis: memory does not follow the size of the model.

8.3 Exactness: coded equals raw, to the bit, over 5 120 steps

What is compared. Lot F writes two files of the same model: the *coded* `.trc`, whose every expert is compressed losslessly by the rANS coder with per-lane tables, and the *raw* `.trc`, with the same expert bytes copied as they are. The engine reads the first by decoding on the card, the second by copying. If the coding is truly lossless and the engine deterministic, both must return the same logits, to the bit; the smallest difference would be a fault.

Proposition 8.2 (C3 by construction; the cache does not enter the computation)**.** *If the coded and raw files give, for every expert, a VRAM slot equal byte for byte, the engine returns the same logits to the bit on every token sequence, whatever the number of slots, the cache policy and the arrival order of reads.*

Proof. The products of an expert read its slot and the activation h_2. When they read, the slot holds the decoded bytes of that expert: the order of the card's three streams guarantees it (lemma 4 of lot O, `moteur/O-pour-le-memoire.md` § 6.1), and those bytes are equal in both files by hypothesis (C1 for the coded file, a copy for the raw one, C2 for the transport). The cache only chooses *where* an expert sits, never *which bytes* it holds; the weighted sum of the k experts runs in the router's order $i = 0, \dots, k-1$, not in the slots' order. The other inputs (resident weights, KV cache) do not depend on the expert file. Every kernel being a deterministic function of the bytes it reads (no floating-point atomic addition, fixed-tree reductions), the logits are equal. ■

How to compare 5 120 **distributions without keeping them.** An output distribution has 201 088 f32 logits: keeping them for the 10 windows of 512 positions would take $5\,120 \times 201\,088 \times 4 = 4.12$ GB [calc.]. The engine keeps instead the 64-bit digest (prefix of SHA-256) of the logits of each step, 40 960 bytes in all; two steps equal to the bit have the same digest, two different steps a different one except for a collision, of probability 2^{-64} per pair. A second path, independent of the digest, compares the `.kld` files written by both passes byte for byte.

Table 8.3: C3 and C11: the coded file against the raw file, over the 10 windows of 512 tokens (5 120 steps) [meas.].

	Steps differing	`.kld`	PPL of scored positions	Slots
20b, lot O	0 / 5 120	equal byte for byte	461.978 277 060 145 (both)	362 and 361
120b, lot O (step loop)	0 / 5 120	SHA-256 `5b212fe9`… (both)	422.205 585 227 913 05 (both)	326 and 325
120b, trial (stream, other build)	0 / 5 120	`5b212fe9`…, = lot O	422.205 585 227 913 05	—

Data: `mesures/moteur/O/journaux/o3/`, `mesures/moteur/epreuve-J1/journaux/c3-120b-10fenetres.json`.

Three facts make this equality stronger than a mere "0 mismatch". (i) The coded and raw passes did not have the same number of slots (362 and 361 on the 20b, 326 and 325 on the 120b, because the raw file's arrival buffers are larger): they did not serve the same misses in the same order, and the "whatever the cache" part of Proposition 8.2 is observed, not only deduced. (ii) The trial redid both passes from a clean clone, with its own build, through stream prefill (one layer at a time for all positions) instead of the step loop (one token at a time through all layers): its `.kld` files equal those

of lot O byte for byte. Two builds, two orders of computation, two days, one result to the bit. (iii) The greedy 256-token text (C12) has the same digest, `e4c6dbacb63a`..., over the 17 passes of lot O (coded, raw, stream, 12 MiB reads, prefetch), over the four passes of the trial and over the six f16-KV passes of the round, with or without the guard.

8.4 Memory: 384 MB for 63 GB

The number and its slope. In mode A0, the peak working set of the process (pinned memory included, read by the system on the handle after exit) is 384.19 MB for gpt-oss-120b and 380.50 MB for gpt-oss-20b [meas.] (`o4-2026-10-15-s1-passes.csv`). llama.cpp, in its best published configuration, holds 10.93 GB and 9.42 GB [meas.] (`mesures/base-gpt-oss/*/resume.csv`). On the 120b, C6's baseline, `fit -t 16`, swept by the round, holds more: 11 355 and 11 483 MB, 11.42 GB on average [meas.] (`base-2026-10-16-passes.csv`, column `pointe_et_mo`). The slopes below keep the published baseline, measured on both models; it is the most favourable to llama.cpp. Figure 8.1 draws both lines.

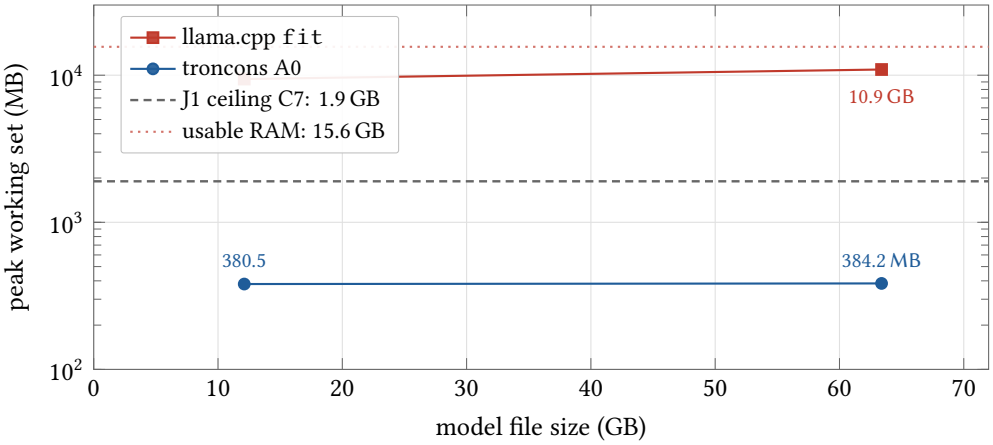


Figure 8.1: Peak memory against the size of the model file (logarithmic scale). troncons gains 3.7 MB when the model gains 51.3 GB; llama.cpp gains 1.5 GB and approaches the machine's RAM. The declared ceiling of J1 (C7) is dashed.

Data: `memoire/figures/tome2/donnees/res-memoire.csv`, drawn from `o4-2026-10-15-s1-passes.csv` and `mesures/base-gpt-oss/\{20b,120b\}/resume.csv`.

The slopes, step by step [calc.]:

$$\frac{\Delta M_{\text{troncons}}}{\Delta P} = \frac{384\,192\,512 - 380\,502\,016}{63\,387\,346\,208 - 12\,109\,566\,624} = \frac{3\,690\,496}{51\,277\,779\,584} \approx 7.2 \times 10^{-5},$$

$$\frac{\Delta M_{\text{llama}}}{\Delta P} = \frac{(10.932\,8 - 9.418\,5) \times 10^9}{51\,277\,779\,584} \approx 0.029\,5.$$

One more byte in the model costs troncons 7.2×10^{-5} byte of memory, llama.cpp 0.03, four hundred times more. The 120b model is 165 times larger than troncons's window, and llama.cpp holds 28.5 times more memory to serve it (29.7 times in C6's configuration). This is, measured, the reading of "without RAM or anything" that Volume I retained: *the memory in use no longer depends on the size of the model.*

What the window is made of. Proposition A.1 of A's advocate wrote the engine's host memory as

$$M = M_{\text{fixed}} + d \cdot k \cdot \ell_{\max} + M_{\text{warm}} + M_{\text{directory}},$$

with $d \cdot k = 12$ pinned arrival buffers of the size $\ell_{\max}$ of the largest chunk, $M_{\text{warm}} = 0$ in mode A0, and only the directory growing with the model. Lot O's diagnostic, which records the working set after every step of opening, showed that the proposition forgot terms (Table 8.4).

Table 8.4: The terms of the engine's memory that grow with the model: counts of the 120b minus those of the 20b, and measured unit costs [calc.] on [meas.].

Term	Δ count × unit cost	Δ bytes
window: $d \cdot k$ buffers of the size of the largest chunk	$12 \times (12\,251\,136 - 12\,144\,640)$	1 277 952
timed events (4 per possible miss, 2 per layer)	216 × 657 B–660 B	≈ 142 000
card allocations (residents, KV caches)	180 × ≤ 290 B	≤ 52 000
directory held twice (file, cache sizes)	2 × 3 840 × 56 B	430 080
table of residents, rANS tables	156 × 88; 144 × ≈ 560	≈ 94 000
routing trace (256 steps)	96 B per step	≤ 49 000
sum of named terms		≈ 2.05 MB
measured (trial; lot O)		2.94 MB; 3.30 MB–3.70 MB

Data: `moteur/O-pour-le-memoire.md` § 13.6, `mesures/moteur/O/corrections/journaux/m4-couts-unitaires.jsonl`.

The first term disappears if the window has a fixed size: with reads of 12 582 912 bytes for both models, the 120b − 20b gap drops from 3.30 MB to 0.99 MB [meas.]. The others are proportional to the number of experts $L \cdot E$ and stay under 1 MB up to about 10 000 experts [calc.]. There remain 0.9 MB these terms do not explain (page granularity of small allocations, heap): the bound with named terms is not closed, and even with it, $2.05 + 2 \times 0.14 + 0.5 = 2.83$ MB leaves the trial's measurement failing by 0.11 MB. The verdict of C7-gap therefore stays "fails"; what was wrong is the tolerance term the architecture posed, not the property it meant to judge.

What the machine loses, and committed memory. Two new measurements of the trial complete C7 (Figure 9.2 in the next chapter). The available physical memory of the *whole* machine drops by 0.46 GB–0.73 GB during a troncons pass, against 6.3 GB–11.6 GB during a llama.cpp pass [meas.]. On the other hand, troncons's private *committed* memory is 6.93 GB: the order of its VRAM allocations and pinned memory, reserved against Windows's commit limit (RAM plus page file) without occupying physical RAM. It is not judged; it is published (finding m3).

The window is visible. Giving the engine 1 GB of warm cache raises its peak by 994.2 MB [meas.]: the instrument does see the window, and C7 is not measuring a floor it could not leave.

VRAM (C10). The peak read by `nvidia-smi` every 250 ms is 6 520–6 537 MiB at lot O, 6 574 in the trial, 6 811 in the round's session (same occupation by the engine, 6 317 303 600 bytes; the difference is elsewhere on the card), always under the threshold of 7 279 MiB (8 188 − 909, the VRAM minus what the display and Stone's applications occupy at rest).

8.5 Speed

8.5.1 The serial floor (C5)

The formula. A token served *without any overlap* would pay every stage of its path one after the other. Its time is then the sum

$$t_{\text{serial}} = t_{\text{disk}} + t_{\text{PCIe}} + t_{\text{dec}} + t_{\text{GEMV}} + T_0,$$

with t_{disk} the chunk bytes read per token (engine counter) divided by the throughput of a disk-only pass in the same session, t_{PCIe} and t_{dec} the card times of the copies and decodes per token (the engine's CUDA events), $t_{\text{GEMV}} = (A + R)/235\,\text{GB/s}$ with A the resident weights read and $R = k \cdot L$ expert slots, and $T_0 = 3.5\,\text{ms}$ Volume I's fixed cost. C5 requires at least $0.8/t_{\text{serial}}$ tokens per second; $0.8 = 1 - 2 \times 10\,\%$, the noise of a 4 MiB read at queue depth 4 measured in Volume I.

The calculation, step by step (session O4, pass 1, [meas.] for every term):

$$t_{\text{disk}} = \frac{0.707\,3\,\text{GB}}{1.848\,\text{GB/s}} = 382.7\,\text{ms}, \qquad t_{\text{PCIe}} = 59\ \text{misses} \times 1.28\,\text{ms} = 75.8\,\text{ms},$$

$$t_{\text{dec}} = 59 \times 0.92\,\text{ms} = 54.2\,\text{ms}, \qquad t_{\text{GEMV}} = \frac{1.686\,\text{GB} + 144 \times 13.25\,\text{MB}}{235\,\text{GB/s}} = 15.3\,\text{ms},$$

hence $t_{\text{serial}} = 382.7 + 75.8 + 54.2 + 15.3 + 3.5 = 531.5\,\text{ms}$, a floor of 1.881 token/s and a threshold of 1.505. The engine measures 457 ms per token, 2.186 **tokens/s**: it goes *under* the sum, because the copies and decodes of a layer overlap the reads of the following misses of the same layer. Figure 8.2 redoes this calculation for every pass of the round.

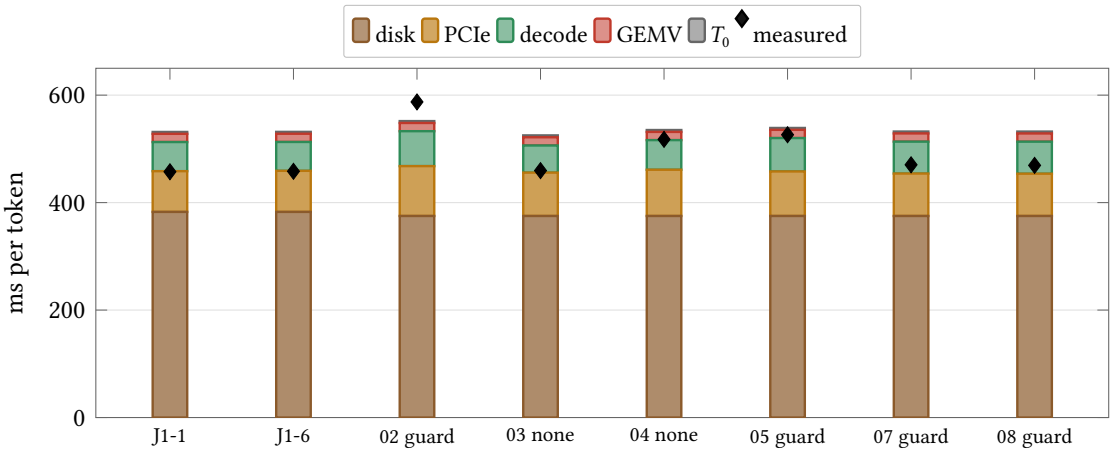


Figure 8.2: The serial floor, stage by stage (stacked bars), against the measured time per token (diamonds), gpt-oss-120b in mode A0 [calc.] on [meas.]. The disk takes 68 %–72 % of the sum. The diamond goes under the bar as soon as the disk queue overlaps the card; pass 02 of the round, above it, is the slowest of the session (1.70 token/s), guard included.

Data: `memoire/figures/tome2/donnees/res-plancher.csv`, drawn from `o4-2026-10-15-s1-passes.csv` and `vitesse-2026-10-16-passes.csv`; every recomputed floor equals the log's to within 10^{-6}.

The physical efficiency (C9). The architecture (§ 7.2) declared a criterion published and not judged: the measured throughput divided by the ceiling computed at the measured h. Neither lot

O nor the trial had computed it; the reprise of the judgement does it here, on the terms of the floor [calc.] from [meas.]. The ceiling is Volume I's token equation with no speculation and no remaining latency ($\mathscr{E} = 1$, $t_{\text{lat}} = 0$), every stage overlapped:

$$t_{\text{ceiling}} = \max\left(t_{\text{disk}},\ t_{\text{PCIe}},\ t_{\text{dec}} + t_{\text{GEMV}}\right) + T_0.$$

Pass 1 of J1: max(382.7; 75.8; 54.2 + 15.3) + 3.5 = 386.2 ms, a ceiling of 2.589 tokens/s; measured 457.4 ms, hence an efficiency of 386.2/457.4 = **0.84** (pass 6: 0.84). Over the round's six f16-KV passes, from 0.64 (pass 02, the slowest) to 0.82 (pass 03). The engine thus holds 84 % of the throughput it would have if the link and the card hid entirely behind the disk; the missing 16 % are the t_{lat} term of the equation, the latency that the disk's short queue lets show, the one prefetching aimed at (Section 8.5.6). Redone by `memoire/outils/tome2_reprise.py` from `res-plancher.csv`.

What limits J1, and what Volume I had predicted. The disk reads on average 1.6 chunks per layer (59 misses over 36 layers): the queue seldom has its four requests in flight, and the engine draws only 1.55 GB/s of useful bytes from the disk where a disk-only pass at depth 4 draws 1.85 [meas.]. The timeline of one layer shows it: four reads submitted at 1.28, 1.96, 2.68, and 3.18 ms finish at 6.86, 9.81, 16.14, and 21.17 ms, served one after the other (`mesures/moteur/O/csv/chronogramme-120b.csv`). Volume I had fitted an "LRU" locality scenario on a single published point; it predicted there, for a 4.8 GB VRAM cache, a hit rate $h_v = 0.590$ and, in mode A0, a speed marker of 2.15 tokens/s (the 326 slots of J1 make 4.3 GB). The engine counts $h = 0.591$ over its 326 slots and measures 2.18 [meas.]. The agreement is to be taken for what it is: a scenario fitted on one point, whose shape the 20b locality measurement had already contradicted, lands right at J1's operating point.

8.5.2 The baseline, and the history of its numbers

C6 compares troncons with the *best* measured configuration of llama.cpp on the same model, the same prompt and the same 256 tokens. The baseline number changed twice; the first two ratios are struck out where they were written, and gathered here.

1. **15 October** (`mesures/gpt-oss.md`, § 3.2): three configurations, all at 6 threads; the best, `fit`, does 0.568 token/s. Against lot O's J1: ~~×3.85~~.
2. **16 October, trial**: the judge adds a configuration of his choice, `fit -t 10`, by an argument declared in advance (under `mmap`, every thread in a page fault is a read in flight): 0.702 against 0.627 at 6 threads in the same session, a real difference. Against troncons: ~~×2.79 to 2.89~~.
3. **16 October, round**: the baseline is swept (Table 8.5). The best is `fit -t 16`, 0.884 **token/s**, 54 % above the published baseline.

Why the number of threads changes everything. llama.cpp maps the file into memory (`mmap`); an expert missing from the page cache is read through page faults, and a faulting thread waits for its page before continuing. With n compute threads there are thus at most n reads in flight. Little's law [3] ties the throughput X of a system to its mean number of requests in flight N and their residence time R: $X = N/R$. As long as the disk is not saturated, R varies little and throughput grows with N; Volume I measured this growth of disk throughput with queue depth. From 6 to 16 threads the baseline gains 54 %; from 12 to 16, nothing discernible: the plateau begins. This reading is a hypothesis consistent with the measurement, not a measurement of the queue depth itself.

Table 8.5: The llama.cpp b11515 baseline swept by the round (gpt-oss-120b, 256 greedy tokens; palindrome t6 t10 t12 t16 m m t16 t12 t10 t6 after a discarded warm-up pass) [meas.]. "m": `-ngl 99 --n-cpu-moe 34 --fit off -t 10`.

Configuration	Passes (tok/s)	Mean	Noise β	Gap of `t16`	Real? (threshold)
`fit -t 6` (published)	0.575; 0.573	0.574	0.4 %	+54.0 %	yes (10.4 %)
`fit -t 10` (trial)	0.675; 0.654	0.665	3.2 %	+33.0 %	yes (10.4 %)
m (`--n-cpu-moe 34`)	0.642; 0.638	0.640	0.6 %	+38.2 %	yes (10.4 %)
`fit -t 12`	0.913; 0.760	0.836	18.3 %	+5.7 %	no (39.6 %)
`fit -t 16`	0.900; 0.867	0.884	3.7 %	—	—

Data: `mesures/moteur/O/corrections/csv/base-2026-10-16-passes.csv`, `journaux/verdicts-base-2026-10-16.json`; recomputed by `tome2_resultats.py`.

8.5.3 C6 redone: guard included

The round added an integrity guard to the engine (Section 9.9.3), active by default. The honest number for C6 is therefore that of the guarded engine against the best baseline [calc.] on [meas.]:

$$\frac{\bar{X}_{\text{guard}}}{\bar{X}_{\texttt{t16}}} = \frac{(1.703 + 1.900 + 2.126 + 2.131)/4}{(0.900 + 0.867)/2} = \frac{1.965}{0.884} = \mathbf{2.22}.$$

The noise of troncons, by Definition 8.1, is the largest noise of a pair: 10.97 % (passes 02 and 05); the threshold is $2 \times 10.97 + 3 = 24.9$ % and the gap, +122 %, exceeds it by far. A stricter reading of the noise, the range of the four guarded passes (21.8 %), brings the threshold to 46.6 %: C6 still holds. Without the guard, the ratio would be $2.054/0.884 = 2.32$. Figure 8.3 gathers all the series.

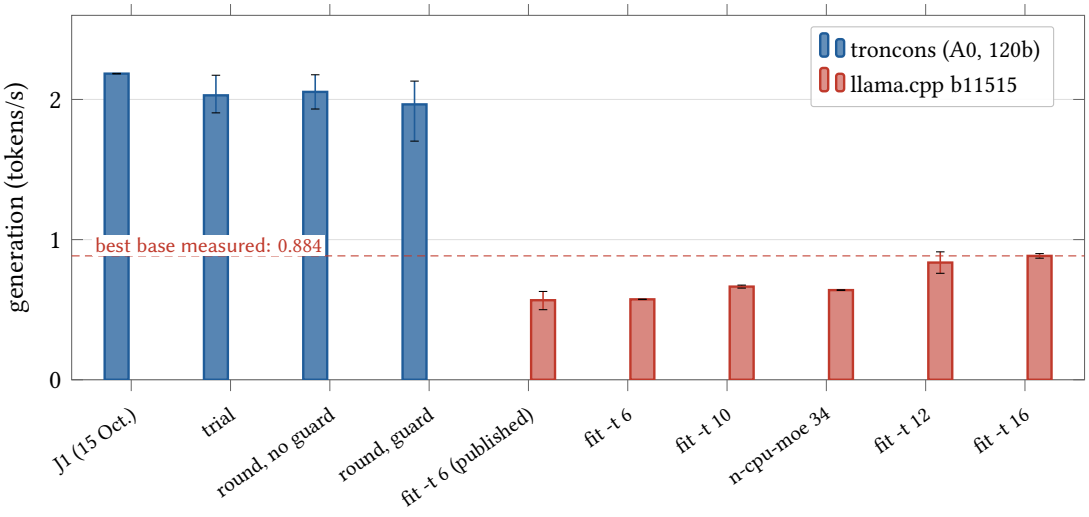


Figure 8.3: Generation on gpt-oss-120b (mean, and range of the passes as an error bar): troncons in mode A0 in four sessions, llama.cpp in its published baseline and in the five configurations of the round's sweep [meas.]. The dashed line is the best baseline measured (`fit -t 16`). The published baseline was the slowest of the swept configurations.

Data: `memoire/figures/tome2/donnees/res-vitesse-120b.csv`, drawn from `o4-2026-10-15-s1-passes.csv`, `s1-2026-10-16-passes.csv`, `vitesse-2026-10-16-passes.csv`, `base-2026-10-16-passes.csv` and `mesures/base-gpt-oss/120b/resume.csv`.

Remark 8.3 (Two sessions, not one). The troncons passes and those of the swept baseline come from two distinct sessions of the same morning (speed between 06:41 and 07:19, baseline from 08:13 to 10:06), and the disk of this machine varies from one session to the next (from 1.51 GB/s–1.95 GB/s within the speed session alone; 20 % slower in session 2 than in session 1 at lot O, cause not measured). The ×2.22 ratio therefore carries a between-session noise the rule does not see. Two safeguards bound it: the trial measured troncons and llama.cpp in the same session, and found ×2.89 against `fit -t 10` (2.03 against 0.702); and the gap (122 %) far exceeds any disk variation observed. A single series, troncons and `fit -t 16` in one palindrome, remains to be run; it is in the future work.

8.5.4 Stream prefill

The cost, posed. In the step loop, a prompt of P positions reads, layer by layer, the missing experts of every position:

$$B_{\text{loop}} = \sum_{p=1}^{P} \sum_{\ell=1}^{L} (1 - h_{p,\ell})\, k\, t,$$

with $t \approx 12.03$ MB per chunk. By stream, each layer reads once every expert requested by at least one position: $B_{\text{stream}} = \sum_{\ell} |U_\ell \setminus V_\ell|\, t$, where U_ℓ is the set of chosen experts and V_ℓ those already in VRAM. Under uniform routing, $\mathbb{E}|U_\ell| = E\,(1 - (1 - k/E)^P)$; for $E = 128$, $k = 4$, $P = 708$: $(1 - 1/32)^{708} = e^{708 \ln 0.96875} = e^{-22.5}$, and U_ℓ is the whole set. Hence $B_{\text{stream}} \leq 36 \times 128 \times 12.03$ MB $= 55.4$ GB, against $B_{\text{loop}} \approx 708 \times 144 \times 0.55 \times 12.03$ MB ≈ 675 GB [calc.], a ratio of about 12.

Measured. 26 s instead of 7 min 36 s for the 708 tokens, ×17.5, the ratio of the mean throughputs of passes `s2-11` to `s2-14` (27.15 against 1.554 tokens/s; per pass, 26.9–27.4 against 1.46–1.65) [meas.]. Lot O's note wrote ~~7 min 50 s~~, a duration that does not come from these passes: on top of the bytes, the loop pays one synchronisation per layer and per position, which the stream pays once per layer. llama.cpp prefills at 3.1–4.0 tokens/s: troncons does it seven times faster. Exactness holds by the same Proposition 8.2 and by control F (5 120 digests equal to the loop's on the 20b; equal `.kld` files on the 120b in the trial).

8.5.5 The cost of the integrity guard

Predicted. The guard hashes every chunk read (0.71 GB per token) on four threads, at 1.07 to 1.10 GB/s per thread [meas.], in parallel with the copy and the decode; the step waits for all its hashes before returning its token. The work, $0.71/1.07 \approx 0.65$ s of core time per step, is spread over four threads (0.17 s of wall time) under a step of 0.46 s; only the hash of the last chunk stays on the critical path, at most

$$12.25\ \text{MB}/1.07\ \text{GB/s} \approx 11.4\ \text{ms},$$

i.e. 2.5 % of a 457 ms step [calc.]. The round's declared prediction wrote 2.4 %, with 1.1 GB/s and 460 ms (`mesures/moteur/O/corrections/seuils-declares.md`); at the slowest measured throughput, 1.07 GB/s, it is 2.5 %.

Measured. The guard's wait is 5.2 ms–5.4 ms per token over the four guarded passes [meas.]. The declared ABBA series (passes 02 and 05 guarded, 03 and 04 unguarded) gives a gap of −12.3 % for a threshold of 26.8 %: *not real*; the disk varied from 1.51 GB/s–1.95 GB/s from one pass to the next, and that is what the gap measures. At equal disk throughput (passes 05 and 04, 1.708 and 1.715 GB/s), the gap is −1.6 % (published −1.7 % on the rounded values 1.900 and 1.932). The second-path tool first found −4.3 % by counting the four guarded passes: passes 07 and 08 are guarded too, but belong to

the f32-KV series; this is no fault of the lot, it is the number of another series. Opening costs 0.95 s more (the table of digests of the embedding rows: 0.615 GB read and hashed), and memory 1.9 MB, the same for both models.

8.5.6 What J1+ kept, killed, left open

Kept: the stream (×17.5 at prefill, Section 8.5.4).
Killed: prefetch by the next layer's router. After the attention of ℓ, the engine applies the norm and router of $\ell+1$ to the residual and reads the predicted chunks ahead. A right prediction removes a read from the critical path; a wrong one adds a read on a disk that already limits. With p the share of predictions served and f the share of a token's time that overlap can hide, the gain is about $f - \frac{1-p}{p}\, s_{\text{disk}}$; here $p = 0.692$, the bytes read rise by 34 % (953 against 710 MB per token) and $f \approx 80/460 = 0.17$: the loss dominates. Measured: −11 % (noise 1 %) [meas.]. The mechanism would come back with $p > 0.9$ or so, or a disk that does not limit.
Open: 12 MiB reads. Reading every chunk as one request of 12 582 912 bytes moves the read into the fast regime of the disk measured by lot E: +30 % in median (2.18 → 2.84 tokens/s), every B pass above every A pass, but a noise of 16.2 % between the two B passes (Definition 8.1; ~~21 %~~, the gap divided by the mean of A): the threshold is $2 \times 16.2 + 3 = 35.4$ % against +30.4 %, the rule is not crossed in four passes. Neither established nor refuted.
One pass, to be confirmed: the warm cache. 1 GB of warm cache divides stream prefill by 2.2 (27.1 → 12.5 tokens/s) without giving anything to generation (1.90) [meas.]. Nobody had measured it; one pass does not make a series.

8.6 Fidelity: how far from the exact mathematics of the model

8.6.1 The instrument

The divergence. For a position, let p be the next-token distribution of the truth (the model's forward pass in double precision, on the same weights) and q that of an engine. We measure

$$\mathrm{KL}(p \,\|\, q) = \sum_{i=1}^{V} p_i \,(\log p_i - \log q_i) \;\geq 0,$$

the Kullback–Leibler divergence [2], in nats, over a vocabulary of $V = 201\,088$ tokens, and the share of positions where the engine's top choice is the truth's, with its 95 % Wilson interval [37]. For a small perturbation Δz of the logits, $\mathrm{KL} \approx \frac{1}{2}\mathrm{Var}_p(\Delta z)$: the divergence is second order in the error.

Why against the truth, and not against llama.cpp. Two correct engines diverge: on Qwen3.5-4B, by 0.005 5 (Volume I); one engine cannot be the judge of another. The architecture therefore posed C4 against the exact mathematics: troncons must be at least as close to them as llama.cpp, both measured by the same instrument.

The file format, and what it costs. All engines write their logits in llama.cpp's `.kld` format: per position, $\log q$ quantized on 16 bits between a floor and the maximum, with a step of about 2.44×10^{-4} nat. Two consequences, proved in Section 9.4: a computation identical to the truth departs from it by at most half a step per token above the floor; and the tokens under the floor receive excess mass, so that the KL of an exact computation is *negative*, of the order of -4.8×10^{-4}. This bias is common to all files and sets the zero of the instrument.

8.6.2 C4 on both models

Table 8.6: C4: troncons and llama.cpp against the double-precision truth [meas.]. Window 0 has 255 scored positions. $\bar{d}$: paired mean of $\mathrm{KL}_{\text{troncons}} - \mathrm{KL}_{\text{llama}}$ per position, ± one standard error.

Model, truth	Engine	Mean KL	Same top choice [Wilson 95 %]	$\bar{d}$
20b, lot R ref., 10 windows	troncons	0.003 68	97.18 % [96.46; 97.75]	
	llama.cpp GPU	0.022 6	89.61 % [88.36; 90.73]	
20b, numpy f64, window 0	troncons	0.001 74	246/255 = 96.5 % [93.4; 98.1]	−0.024 3 ± 0.005 3
	llama.cpp GPU	0.026 1	222/255 = 87.1 % [82.4; 90.6]	
120b, numpy f64, window 0	**troncons**	0.003 08	244/255 = 95.7 % [92.4; 97.6]	−0.057 6 ± 0.012 0
	llama.cpp GPU	0.060 7	205/255 = 80.4 % [75.1; 84.8]	
120b, numpy f64, window 0	troncons, **f32 KV**	−0.000 48	255/255 = 100 %	

Data: `mesures/moteur/O/journaux/o3/c4-20b-comparer.*`, `mesures/moteur/epreuve-J1/csv/c4-numpy-\{20b,120b\}-f0.csv`, `mesures/moteur/O/corrections/journaux/verdicts-kv-2026-10-16.json`; Wilson and $\bar{d}$ recomputed by `tome2_resultats.py`.

The paired test. Comparing two KL means position by position removes the common variance of difficult positions: $d_i = \mathrm{KL}_{t,i} - \mathrm{KL}_{l,i}$, $\bar{d} \pm s_d/\sqrt{n}$. On the 120b, $\bar{d} = -0.057\,6$ for a standard error of 0.012 0: nearly five standard errors below zero, and the Wilson intervals of the top choices are disjoint. C4 holds on the 120b for the first time.

"Six times closer", requalified. Lot O had written that troncons was "six times closer" to the truth than llama.cpp (0.003 7 against 0.022 6 on the 20b). It is true, but the trial and lot R found its main reason: llama.cpp rounds the bounds of the YaRN ramp [28], where OpenAI's reference implementation [32] does not (`"truncate": false`). On window 0 of the 20b, the ramp alone explains 0.014 3 of llama.cpp's 0.026 1 (55 %); over the 10 windows, 0.014 5 of 0.022 6 (lot R). The gap thus comes mostly from a modelling choice, OpenAI's, validated by an independent computation; the precision of the kernels is not its main cause.

8.6.3 The f16 KV cache carried the whole gap

What the trial saw. Beyond the means, the first token of C12's text (position 707 of the prompt) differs: the truth picks token 1008 ("…", $\log p = -1.46$), troncons token 313 ("am", to which the truth gives only $\log p = -10.03$), after a cascade of routing flips over 20 layers out of 36, from layer 12 on. The truth recomputed with only the KV cache rounded to f16, and nothing else, picks 313 too, with the same cascade [meas.] (`premier-jeton-kvf16-120b.json`). Figure 8.4 shows routing agreement per layer: on the 120b it drops with depth, down to 0.81 beyond position 512; on the 20b it does not go below 0.96.

Why f16 rounding suffices, posed. f16 has 11 bits of significand: rounding to nearest gives $\tilde{k} = k(1+\delta)$ with $|\delta| \le u_{16} = 2^{-11} \approx 4.9 \times 10^{-4}$ (the binary16 format of IEEE 754). For an attention

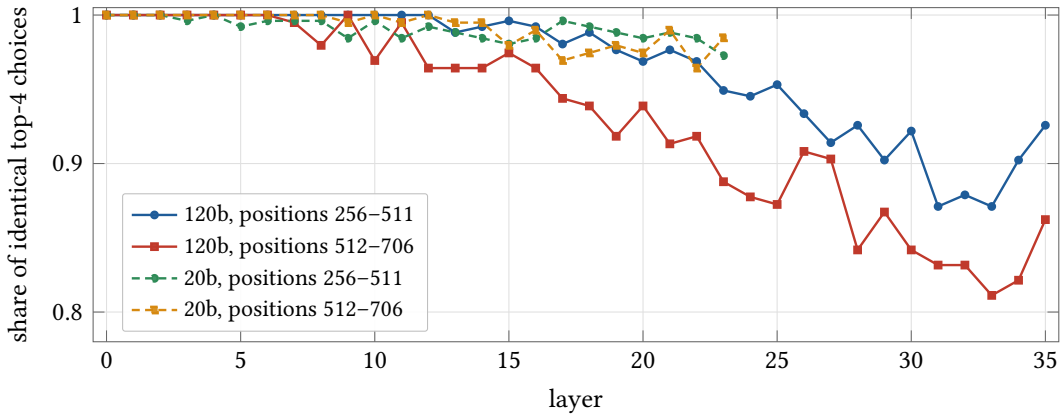


Figure 8.4: Share of identical top-4 router choices between troncons (f16 KV) and the f64 truth, per layer, on C12's prompt [meas.]. The 120b drops with depth, and more after position 512; the 20b does not. With an f32 KV cache, agreement is 1 over the 36 layers and the 708 positions.

Data: `memoire/figures/tome2/donnees/res-routage.csv`, drawn from `mesures/moteur/epreuve-J1/csv/routage-par-couche-invite.csv`.

score $s = c\sum_{j=1}^{64} q_j k_j$ ($c = 1/8$),

$$|\tilde{s} - s| \;\le\; c\,u_{16}\sum_j |q_j k_j| \;+\; \gamma_{64}\,c\sum_j |q_j \tilde{k}_j|, \qquad \gamma_{64} = \frac{64\,u_{32}}{1 - 64\,u_{32}} \approx 3.8\times 10^{-6},$$

to first order, the second term being the f32 dot product ($u_{32} = 2^{-24}$, [25]). The attention output inherits a relative error of the same order, through the softmax weights and through $\tilde{v}$. With an f32 cache, only the kernels' error remains, $u_{32}/u_{16} = 2^{-13} \approx 1/8\,192$ times smaller; ~~the KL being second order, it would be divided by about $8\,192^2 \approx 6.7\times 10^7$: from 3.08×10^{-3} towards 5×10^{-11}, well under the step of the format. This order of magnitude assumes a linear error; routing flips are not linear, but they vanish with it.~~ This estimate of work package O is struck out by theorem 6.9: the no-switch term is divided by $8\,192^2 \approx 6.7\times 10^7$, but the switch term, linear, only by 8 192; the KL in f32 is therefore at most of the order of $3.08\times 10^{-3}/8\,192 \approx 3.8\times 10^{-7}$ nat [calc.] (Section 6.6.3), well under the step of the format ($\approx 2.4\times 10^{-4}$ nat). The measurement below remains the only judge. A router that picks 4 experts out of 128 turns a small continuous error into a discrete one when two scores are close: this is why the 120b, with 128 experts, flips more than the 20b, with 32.

Measured. The round added an option `--kv-f32` to the engine (kernels added, no existing kernel touched; control: equal to the bit to the f16 path on values representable in f16, and different on values that are not). Table 8.7 and Figure 8.5 give the result.

On the 708-token prompt with f32 KV, the top choices equal the truth's at 708 positions out of 708 (687 with f16), routing is equal on all 36×708 (layer, position) pairs, and **the first token of C12 is 1008, the truth's**. The f16 rounding of the KV cache thus carried *the whole* measurable gap between troncons and the exact mathematics of the model; the trial had bounded it from below (70 %–77 % of routing flips).

The price. The KV cache of gpt-oss-120b at a context of 4 096 has 18 full layers and 18 layers with a sliding window of 128 positions:

$$18\times(4\,096 + 128)\times 2\times 512\times 2\,\mathrm{B} = 155.7\,\mathrm{MB}\ \text{in f16},$$

Table 8.7: f16 against f32 KV cache, against the f64 truth [meas.]. "Steps": quantization steps of the `.kld` format.

	Mean KL	Top choice	KL > 0.1	KL p99	Max. gap of log p
120b, window 0, f16	0.003 08	95.7 %	0.39 %	0.074 7	3.07 nat
120b, window 0, f32	−0.000 48	100 %	0	2.3×10^{-6}	0.69 steps
20b, window 0, f16	0.001 74	96.5 %	0.39 %	0.052 9	2.32 nat
20b, window 0, f32	−0.000 48	100 %	0	≈ 0	2.04 steps
120b, prompt, positions 512–706, f16	0.009 63	—	2.05 %	0.196	—
120b, prompt, positions 512–706, f32	−0.000 54	—	0	3.6×10^{-6}	—

Data: `moteur/O-pour-le-memoire.md` § 13.3; `mesures/moteur/O/corrections/journaux/verdicts-kv-2026-10-16.json`, `m5-ecart-logp-*.json`; means recomputed from `kv-f16-f32-par-position.csv`.

twice as much in f32: +155.7 MB of VRAM, i.e. 155.7/13.25 = 11.8 expert slots [calc.]; measured: 326 → 314 slots. Replayed by the cache simulator on the prompt's trace, this loss moves h from 0.459 to 0.442 and misses from 77.9 to 80.4 per token: +3.2 % of bytes read, the order of the speed cost when the disk limits [calc.] on a measured trace. The generation measured in f32 (1.50 against 2.13 tokens/s) *is not* that cost: the text changes (`72304d85`…, which starts with "…" like the truth, then simulates tool calls like llama.cpp's greedy output), and the loop of the f16 text raised its cache hit rate to 0.590 against 0.435. Comparing two speeds on two different texts measures the texts. The default stays f16 until the arbiter decides seam N; the round proposes f32.

8.7 What the results say together

1. **Exact**: the coded path equals the raw path to the bit, over 5 120 steps and from one build to another; Volume I's lossless coding costs the output nothing.
2. **Bounded**: 384 MB for 63.4 GB, +3.7 MB when the model takes 51 GB more; llama.cpp holds 28.5 times more memory (29.7 times in C6's configuration).
3. **Faster than the baseline**: 2.22 times the best baseline measured, guard included; seven times at prefill.
4. **Correct**: closer to the truth than llama.cpp on both models; and, with an f32 KV cache, equal to the truth up to the format.
5. **Limited by the disk**: 68 %–72 % of the serial floor; the next gain is in the disk queue (longer reads, or more reads in flight) and in the disk itself (Chapter 10).

What could be wrong

- **C6 crosses two sessions**: troncons and the swept baseline did not run in the same palindrome; the ratio carries a between-session noise the rule does not measure. The same session, in the trial, gave ×2.89 against a weaker baseline.
- **The baseline may be better still**: neither more than 16 threads, nor `--mlock`, nor other values of `--n-cpu-moe` were swept. The ×2.22 is a ratio to the best baseline *measured*.
- **C4 on the 120b rests on one window** of 255 positions and on the prompt; one window is not ten. The numpy truth and lot R's reference read the same GGUF and the same reading of the model: a common convention error would pass (Section 9.10).
- **"f32 KV = truth"** is measured on one window of 512 tokens and one prompt of 708 (120b),

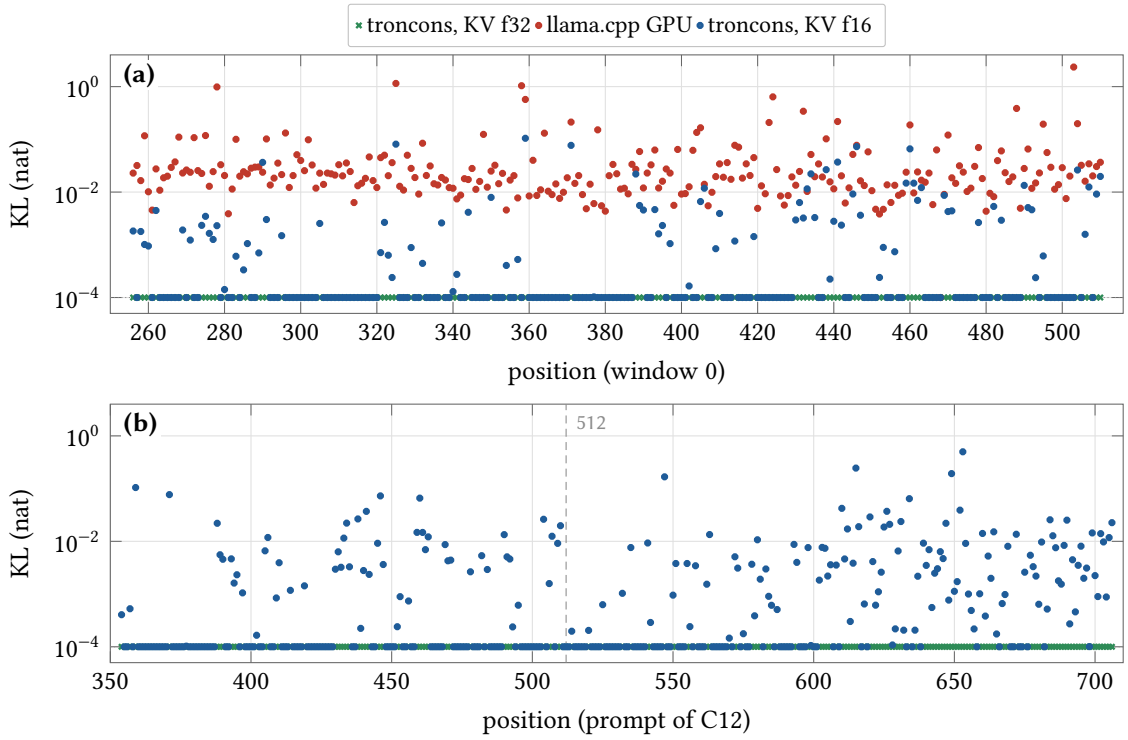


Figure 8.5: KL per position against the f64 truth, logarithmic scale [meas.]. (a) Window 0 of the 120b: llama.cpp (red), troncons with f16 KV (blue), troncons with f32 KV (green). (b) C12's prompt beyond window 0: the f16 KL rises after position 512; f32 stays at the floor. A KL below 10^{-4}, including every negative KL (bias of the format), is drawn on the dotted line: this is the case of 165 positions out of 255 for f16 in (a), 255 out of 255 for f32, 353 out of 353 in (b).

Data: `memoire/figures/tome2/donnees/res-kl-120b-f0.csv` and `res-kl-invite.csv`, drawn from `c4-numpy-120b-f0.csv` and `kv-f16-f32-par-position.csv`.

one window of the 20b; not on the ten windows.

- **The cost of the guard** rests on one pair at equal disk (−1.6 %); the declared series says −12.3 % under a threshold of 26.8 % (noise 11.9 %), not real either way.
- **The agreement with Volume I** ($h = 0.591$ against 0.590 predicted) may be a coincidence of a scenario fitted on one point.
- **C12's text and the harmony conversation** are greedy: they show that the engine produces coherent text, not its quality; the judge is C4.

Chapter 9

The independent trial and the round of corrections

Where we start, what this chapter establishes, what remains

Where we start: a milestone declared held by the lots that built it, on their own instruments. The SOCLE demands more: "the work is reviewed by a judge who did not do it". **This chapter establishes** what an independent judge found in one night (16 October 2026, from 02:12 to 06:10): the specification decodes to the bit under a third implementer; the f64 reference is confirmed by a third computation, up to rounding; C3 holds from one build to another; C4 holds on the 120b; and eleven things nobody had seen, two of them serious. It then establishes what the round of corrections made of them the same morning: every finding with its correction, its control case, the break that makes the control fail, and its verdict. **What remains**: the root cause of the G1 hang (not reproduced), and the decisions handed to the arbiter. Sources: `mesures/epreuve-J1.md`, `moteur/epreuve-J1-pour-le-memoire.md`, `moteur/O-pour-le-memoire.md` (part III), `moteur/REPRISE-O.md`, `mesures/moteur/epreuve-J1/` and `mesures/moteur/O/corrections/`.

9.1 The protocol

The architecture wrote the trial's protocol before any construction (`decisions/ARCHITECTURE.md`, § 8). The judge is an agent who wrote neither the specification nor a line of the judged code; he works from a clean clone, on mains power, with a free card, and reads only the specification, the code and the logs. He follows seven steps: (1) write, *from § 4 of the specification alone*, a decoder of the format and reproduce to the bit chunks drawn at random; (2) replay the breaks and check that every control fails, then holds; (3) redo the numbers in ABBA series; (4) check that the baseline is not badly tuned; (5) check that the memory instrument sees the window; (6) read the text; (7) write his verdict, with his own "What could be wrong" section.

The conditions, recorded. Mains power at every pass; free card at the start (346 MiB); card lock taken at 02:26 and released at 04:44. Foreign load recorded and left alone: a `find` from another agent (one core), an `nvidia-smi` monitor running since the day before, and an orphan `nvidia-smi` born at 01:38 (it would become finding G2). Build from a clone at commit `527dff5`, in 21 s, *without* the SOCLE's recipe (which broke: finding m2). The judge opened the format crates only *after* step 1, and his decoder took nothing from them. 128 GB were left free on the disk.

9.2 Step 1: is the specification true?

The tool. `decodeur_s4.py` (Python 3.13, numpy): a GGUF v3 reader written from the public specification; the `.trc` header field by field; the META, RESIDENTS, TABLES and DIRECTORY sections compared with the GGUF; and, per chunk, the part header, the stream table, the rANS decoding of § 4.9.2, the per-lane table of § 4.9.3 and the exact reconstruction of § 4.11. Every rebuilt expert is compared with the original digest of the directory and with the GGUF bytes; two checks the decoder does not require are added (median equal to `sort[B/2]`, end of every tile equal to the start of the next).

The vectorised decoder. An rANS stream of the specification is cut into independent tiles of 32 lanes; an expert has 127 of them, and each stream decodes in 1 024 sequential steps. The judge decodes the 127 tiles at once, lane by lane in parallel: 0.8 s per expert instead of minutes. One must then prove that this vectorised form is indeed the normative decoding, which reads the words "for $l = 0$ to 31, in order".

Lemma 9.1 (Vectorised rANS is the rANS of § 4.9.2)**.** *Within a tile, let k be the sub-step of macro-step m, p the current position in the stream of 16-bit words, and for each lane l the intermediate state y_l after the decoding step. Normative decoding makes lane l renormalise if and only if $y_l < L = 2^{16}$, reading one word. It is equivalent to compute $b_l = [y_l < L]$ for all 32 lanes at once, to make every renormalising lane read the word of index $p + \sum_{l' \le l} b_{l'} - 1$, and then to set $p \leftarrow p + \sum_l b_l$.*

Proof. In the normative order, lane l reads, if it renormalises, the word following those read by lanes $l' < l$ in the same sub-step, i.e. word $p + \#\{l' < l \,:\, y_{l'} < L\} = p + \sum_{l' \le l} b_{l'} - 1$ when $b_l = 1$. It remains to see that no lane renormalises twice in the same sub-step, otherwise the normative order would read two words for one lane. Before the step the state satisfies $x \ge L$; the step gives $y = f_s\,(x \gg 12) + (x \bmod 2^{12}) - c_s$ with $f_s \ge 1$, hence $y \ge x \gg 12 \ge 2^{16} \gg 12 = 16$. If $y < L$, renormalisation gives $(y \ll 16) \mid w \ge 16 \cdot 2^{16} \ge L$: one word always suffices. Each tile has its own start and states, so tiles are independent. ■

The results (Table 9.1). Nothing was missing from § 4 to decode to the bit.

Table 9.1: Step 1: the judge's decoder, written from § 4 alone, on the production files [meas.].

	gpt-oss-120b	gpt-oss-20b
Chunks drawn (seed 20261016), equal to the GGUF to the bit	**20 / 20**	**20 / 20**
Lanes off L; tiles whose end ≠ start of the next	0; 0	0; 0
Medians equal to `sort[B/2]`	172 800 / 172 800 rows	same
Residents byte for byte (and digest)	**471 / 471**	**315 / 315**
META: GGUF pairs byte for byte, plus 3 coder keys	yes (39 = 36 + 3)	yes
TABLES: all summing to 4 096; digest	432 tables; yes	288; yes
DIRECTORY: contiguous entries up to the end of the file	4 608; yes	768; yes
Header: digest, size, source SHA-256 = download register	yes (`582bd40f`…)	yes (`27cd6c43`…)

Data: `mesures/moteur/epreuve-J1/journaux/s4-gptoss120b-20.json`, `s4-gptoss20b-20.json`.

Table 9.2: The breaks of the judge's decoder: each fails on the targeted chunk only [meas.].

Break	Lanes off L	Wrong bytes	Second holds
one bit flipped in the CODES_CTX words	32 (one tile)	31 122	yes
two frequencies of a table swapped	4 064	3 624 195	yes
one median changed by one unit	**0**	90 (one row of scales)	yes
one LUT slot corrupted	4 032	3 262 607	yes

Data: `journaux/s4-gptoss120b-casse-\{mot,table,mediane,lut\}.json`.

Does the decoder's control fail? Each break is applied in memory to the first drawn chunk (layer 12, expert 54); the second (layer 25, expert 109) must hold (Table 9.2).

The wrong median leaves every lane on L: only the expert's digest sees it. This is the seed of finding M1.

The lots' corrupted copies. On lot E's copy (12 bits flipped at random in the 20b), the 12 touched chunks are detected, and the detail says who sees what: 6 through lanes off L, 2 through a format error, 2 through the part header (whose decoded bytes nevertheless equal the GGUF), and **only 2 through the expert's digest**: one byte of MEDIANS (layer 23, expert 4) and one byte of BIASES (layer 21, expert 24), which keep all their lanes on L. On lot O's copy with a wrong median, the expert is wrong and the lanes are on L.

What writing it taught about § 4. Three imprecisions, which fooled nobody but which a second implementer would have to guess (finding m4): the section lengths of the header are *useful* lengths, without padding; the digest of the residents table is in the last 8 bytes of the section, not "after the data area"; and "the end of tile t is the start of tile $t + 1$", true on 100 % of decoded tiles, is a free check that § 4 does not make normative.

9.3 Do the controls fail?

Table 9.3: The breaks replayed by the judge [meas.].

Break	Expected	Observed
judge's decoder: word, table, median, LUT slot	targeted chunk fails, the other holds	yes, 4 of 4
judge's decoder on copies E2 (12 bits) and O3 (median)	13 of 13 detected	yes
engine, production, copy E2	named fatal error	"layer 2, expert 19: wrong decoding (32 lanes off L, 0 format errors, 1 wrong tile pointer)"
engine, production, copy O3	nothing in production	the engine runs to the end, and its 32-token text **equals that of the sound file** (`48f70a45`…)
engine, control mode, copy O3	named digest	"control: layer 0, expert 30: digest `2e3544c4`… instead of `c9f614cc`…"
C7: 1 GB of warm cache	≈ +1 GB	+994 MB

Data: `mesures/epreuve-J1.md` § 4; `journaux/seances/complements-2026-10-16.json`.

The judge did not replay the breaks of the kernels (K1 to K3), the reference (R0, R1), the cache (O1) and the byte path (E0, E2): their logs exist at the lots.

9.4 A third double-precision forward pass

Why a third computation. C4 judges troncons against a "truth": lot R's f64 forward pass. Were that truth wrong, C4 would be worth nothing. The judge wrote another one, in numpy f64, from the model form published by OpenAI (reference implementation under Apache-2.0, [32]; nothing copied), without reading anything of lot R. It comprises the Q8_0 embedding, RMSNorm, the projections and their biases, half-split RoPE with the real YaRN ramp and the concentration $0.1 \ln 32 + 1$, attention with 64 query heads and 8 key heads, the $1/8$ scale, the per-head sink in the denominator and a 128-key window on even layers, the top-4 router then the softmax over the four, the SwiGLU clamped at 7, the MXFP4 experts dequantised exactly, and the output head. The ramp bounds it finds, 8.092 78 and 17.398 02, equal lot R's. Duration: 6 min for the 20b, 17 min for the 120b, over 511 positions.

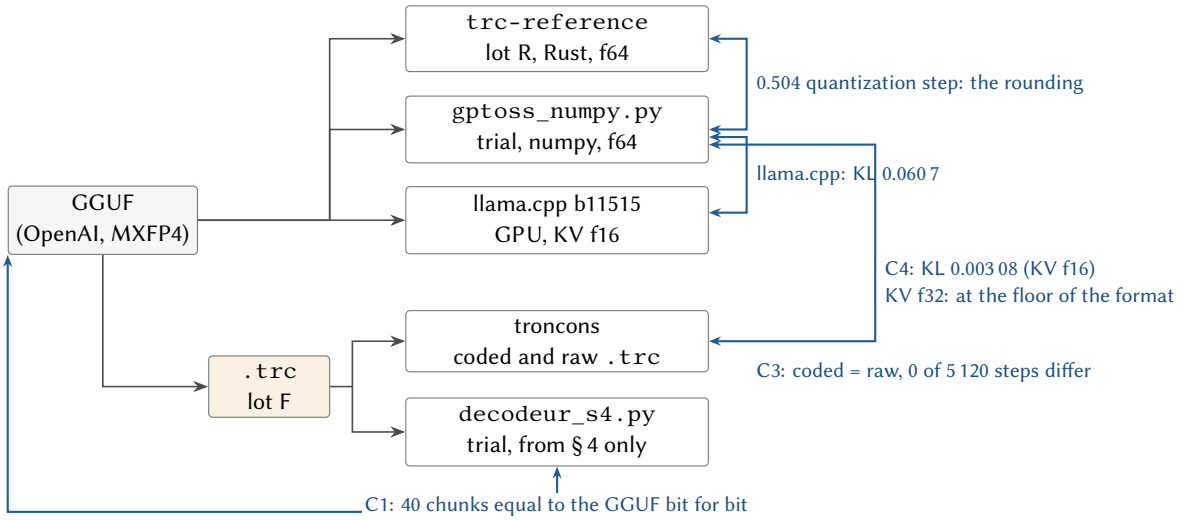


Figure 9.1: The paths of the truth: five programs, three independent implementers (lot R, the judge, llama.cpp) for the forward pass, two (lot F, the judge) for the format, and the measurement that ties each pair.

The reference against the judge's pass. Over the 255 scored positions of window 0 of the 20b, and the 201 088 tokens of each, the maximum gap of log-probabilities above the format's floor is 0.000 123 nat, i.e. 0.504 **quantization step** (median step 2.44×10^{-4}); 255 positions out of 255 under one step; 100 % identical top choices [meas.]. This number has a theoretical bound.

Proposition 9.2 (The gap due to the format alone). *A* `.k1d` *file codes, per position,* $\log q_i = \mathit{step} \cdot \mathit{code}_i + \log q_{\min}$ *with an integer code rounded to nearest, and gives code 0 to every logit under the floor. If two computations return the same real* $\log q_i$*, the values read from their files differ by at most step/2 on every token above the floor, plus the f32 rounding of step and* $\log q_{\min}$ *(of the order of* 10^{-6} *relative).*

Proof. Rounding to nearest moves a value by at most half a step; the value read is $\text{step} \cdot \text{code} + \log q_{\min}$, computed from two f32 numbers stored in the file, whose rounding adds its own term. ■

Measured: 0.504 step, i.e. step/2 plus 0.004 step. Lot R's reference thus equals the judge's pass *up to the rounding of the format, and nothing else.* llama.cpp's ramp, for its part, departs from it by up to 6.5 nat on some tokens.

Why an exact KL is negative. The tokens under the format's floor (about 85 000 out of 201 088) all receive $\log \tilde{q} = \log q_{\min}$, larger than their true value: the distribution read, $\tilde{q}$, is not normalised, $\sum_i \tilde{q}_i = e^{\delta}$, with $\delta = 0.002\,0$ on average [meas.].

Lemma 9.3 (The KL of an exact computation against a file)**.** *For a computation identical to the truth,* $\mathrm{KL}(p \,\|\, \tilde{q}) = \sum_i p_i(\log p_i - \log \tilde{q}_i)$ *tends to* $-\sum_{i \text{ under the floor}} p_i \,(\log q_{\min} - \log p_i) < 0$.

Proof. Above the floor, $\log \tilde{q}_i = \log p_i$ to within half a step (Proposition 9.2), and these gaps, of mixed signs, cancel on average. Under the floor, $\log \tilde{q}_i = \log q_{\min} > \log p_i$, and every term $p_i(\log p_i - \log q_{\min})$ is negative. ∎

Measured: -4.8×10^{-4} for lot R's f64 reference against the judge's truth. This bias, common to all files, changes no ordering; it sets the zero of the instrument, and it is exactly the KL the f32 KV cache would later reach (Section 8.6.3).

C4, at last judged on the 120b. Against this truth, troncons does 0.003 08 and 95.7 % of top choices on window 0 of the 120b, llama.cpp 0.060 7 and 80.4 %; paired difference $-0.057\,6 \pm 0.012\,0$, disjoint Wilson intervals. The detail is in Table 8.6. On the same window, the YaRN ramp alone explains 55 % of llama.cpp's gap on the 20b: lot O's "six times closer" is true, but it bears mostly on a modelling choice (finding m5).

9.5 The speed and memory session

The judge's driver is independent of the lots': fresh processes, palindrome order declared before any pass (commit `4832184`), peaks held by the system and read on the handle after exit, available memory of the whole machine every 100 ms, VRAM every 250 ms, power state before and after. Figure 9.2 draws the twelve passes.

Reproduction. troncons does 2.03 tokens/s on average against 2.18 at lot O: a gap of 7.4 %, under 2×5.5 % of noise, *reproduced.* llama.cpp `fit -t 6` does 0.627 against 0.568 published: 10.4 %, under 2×9.5 %, *reproduced.* The texts equal those of lot O and of the baseline.

C5, and the trap of a cold disk (m1). The first disk-only pass read 0.725 GB/s (90th percentile of 172 ms for a 12 MB read), the second 1.822 GB/s; the engine was reading at 1.79 GB/s the next minute. Lot O's formula took the median of the two D passes: it moves the threshold by 25 % (1.10–1.13 instead of 1.43–1.48). With the warm pass alone, for pass 01 [calc.]:

$$t_{\text{serial}} = \underbrace{\frac{710.12\ \text{MB}}{1.822\ \text{GB/s}}}_{389.7\ \text{ms}} + \underbrace{79.2}_{\text{PCIe}} + \underbrace{57.0}_{\text{dec.}} + \underbrace{\frac{1.686\ \text{GB} + 4 \cdot 36 \cdot 13\,254\,144\ \text{B}}{235\ \text{GB/s}}}_{15.3\ \text{ms}} + 3.5 = 544.7\ \text{ms},$$

a floor of 1.836 token/s and a threshold of 1.469. The three T120 passes hold in both cases. The round corrected the formula: one discarded warm-up D pass.

What the machine loses. During a troncons pass, the available memory of the whole machine drops by 0.46 GB–0.73 GB; during a llama.cpp pass, by 6.3 GB–11.6 GB [meas.]. The declared threshold for a finding (twice the working set plus 200 MB, 0.97 GB) is not crossed. troncons's *committed* memory, 6.93 GB, is eighteen times its working set: C7 does not see it (finding m3).

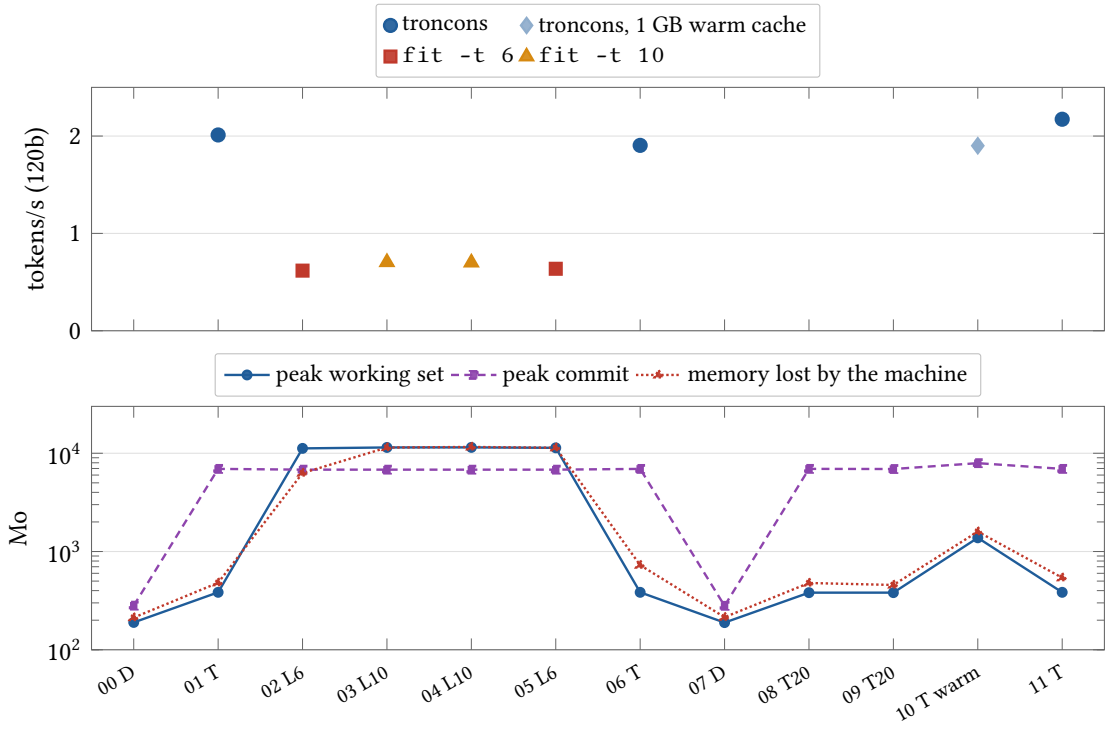


Figure 9.2: The trial's session `s1-2026-10-16`, in palindrome order [meas.]. Top, generation on the 120b: troncons (circle), with 1 GB of warm cache (diamond), llama.cpp at 6 threads (square) and at 10 threads (triangle). Bottom, per pass: peak working set, peak committed memory, and loss of available memory of the whole machine (logarithmic scale). The D (disk only) and T20 (20b) passes have no 120b generation.

Data: `memoire/figures/tome2/donnees/res-epreuve-seance.csv`, drawn from `mesures/moteur/epreuve-J1/csv/s1-2026-10-16-passes.csv`.

C8, by two instruments. The criterion compared the bytes read according to the system's PDH counter with those the engine counts. The judge added the *process* input-output counters (`GetProcessIoCounters`), checked by unbuffered reads of 4 KiB to 64 MiB counted to the byte (Table 9.4).

The engine counts right: the process reads 22.8 MB more, the buffered reads at opening (META, tables, directory, ≈ 13.5 MB) and about 9 MB of others. The large PDH gaps are outside the process: on a cold first pass, +2.37 GB, very probably the images of the libraries (CUDA, llama) loaded through page faults, which PDH sees and `ReadFile` does not count [hyp.]. Finding M3: C8 as written measures the machine.

9.6 Is the baseline honest?

The judge ran the best published configuration (`fit`, 6 threads) and another of his choice, `fit -t 10`, in the same session; his reason, declared beforehand: under `mmap`, every thread in a page fault is a read in flight, so more threads make a deeper queue on the disk. Result: 0.627 token/s at 6 threads (noise 3.0 %), 0.702 at 10 threads (noise 0.6 %); a gap of 11.9 %, real; same texts. The published baseline underestimated llama.cpp, and lot O's ×3.85 had to be struck out (finding M2). The judge himself wrote what he had not done: neither 12 nor 16 threads, nor `--n-cpu-moe`. The round did it; the result is in Section 8.5.2: at 16 threads, 0.884, and the gap was 54 %, not 12.

Table 9.4: C8 by two instruments [meas.]: the system's global counter (PDH) and the process counters, against the engine's count.

Pass	PDH − engine	Declared bound	PDH	Process − engine
s1 01 T120 (first 120b pass)	+2 372.8 MB	100.8	fails	—
s1 06 T120	+314.9	244.2	fails	—
s1 08, 09 T20	+96.5; +39.7	103.5; 121.3	holds	—
s1 10 T120 warm	+241.9	2 689.4	holds	—
s1 11 T120	+125.4	154.4	holds	—
c3 (coded; raw, 3 windows)	—	—	—	+22.8; +22.6 MB

Data: `mesures/moteur/epreuve-J1/journaux/c8-juge.json`, `temoin-compteurs-es.json`.

9.7 Is the text right?

C12. The 256 greedy tokens of the 120b start with "am", then a coherent and accurate summary of the prompt text (Robert Boulter, *The Bill* in 2000, *Herons* at the Royal Court in 2001), before falling, after about a hundred tokens, into a loop ("Here is a summary" shortened down to a repeated "Here"). llama.cpp's greedy output on the same prompt loops too, on simulated tool calls: this is the behaviour of greedy decoding on this model, not a visible defect of the engine.

The first token, settled by the truth. The two engines differ from the first token: troncons says "am" (313), llama.cpp "..." (1008). The judge passed the 708 prompt tokens through his f64 pass of the 120b; the engine replayed them by stream and by loop (equal to the bit), and position 707 indeed returns token 313.

Table 9.5: The first token of C12 (position 707 of the prompt) according to each computation [meas.].

Computation	argmax	true log p of 313 ("am")	true log p of 1008 ("...")
f64 truth	**1008**	−10.03	−1.46
llama.cpp	1008		
troncons (f16 KV)	**313**		
f64 truth with only the KV rounded to f16	**313**	−1.47 (under the imitation)	−3.84 (under the imitation)
troncons (f32 KV, round)	**1008**		

Data: `journaux/premier-jeton-c12-120b.json`, `premier-jeton-kvf16-120b.json`; `mesures/moteur/O/corrections/journaux/m5-invite-120b-f32.json`.

It is not a break at 512: the PPL of positions 354 to 706 is 1 200.3 for troncons and 1 221.8 for the truth, this passage is hard for the model itself; and `--contexte 1024` like `--emplacements 200` leave the 708 digests unchanged. It is a cascade of routing flips: at position 707, 20 layers out of 36 route differently from the truth, from layer 12 on; at position 706, none. The f16 KV cache is the sufficient cause, and the main one: in an experiment declared before its pass (commit `549b38b`), the truth with only the KV rounded to f16 makes the same error, and troncons is far closer to that truth (routing equal at 99.3 and 97.9 % against 96.9 and 93.1 %): rounding carries about 70 %–77 % of the flips. Finding M5. What followed, the f32 cache that makes troncons equal to the truth, is in Section 8.6.3.

The harmony conversation (published, not judged): an answer in French, structured in tables, on inference of a mixture of experts with offloading; the `analysis` channel precedes `final`, as the format requires [44]. The substance contains approximations of the model, not of the engine; characters split between two tokens come out as "?" (a text assembly defect, known to lot O).

9.8 The findings

The judge ranked his findings by severity; each says where, what is found, the evidence and a proposed correction (Table 9.6). Every correction is a proposal; the seams change only through the arbiter.

9.9 The round of corrections

The round declared its first threshold on 16 October at 06:12 (commit `ca5d338`), seven minutes after the last commit of the trial's report; its final state is dated 10:15 (`moteur/REPRISE-O.md`, commit `30d8165`). Its rules were the project's, applied to every finding: *reproduce first* the defect with the previous binary (kept, a frozen copy of commit `7b288a6`), declare the threshold before the pass (`mesures/moteur/O/corrections/seuils-declares.md`, one block committed per finding), correct, then show that the control fails with the old binary and holds with the new one. The break of the control is the old binary.

9.9.1 G1: the hung pass

What the judge saw. The `evaluer --flux --brut` pass over the ten windows of the 120b, launched at 03:33 while the judge's numpy pass was reading 61 GB and occupying the processor, never finished. After 35 min: no further read in 10 s (1 863 reads, 18.06 GB), card at 0 %, main thread at 100 %, and not one line on the error output. The judge sampled the main thread's instruction pointer 300 times (300 out of 300 in `nvcuda64.dll`) then named its stack through the symbols: `troncons::principal` → destruction of the `Moteur` → `Arc<Interne>::drop_slow` → destruction of the CUDA events → `nvcuda64.dll`. The engine had left its loop, very probably on an error, and its destruction was waiting for work left queued on the card; the error itself was never written. Relaunched alone, the same command finishes in 287 s.

The corrected exit (Figure 9.3). On any error, the engine first writes the error (on the error output and in a `<journal>.erreur.json`), then ends its own process with `TerminateProcess` [45], without destroying the engine or the CUDA context: the driver frees everything at the end of the process. A heartbeat per prefill layer dates any future hang.

A probable cause, found by reading the code. In stream prefill, the reads of a batch took their pinned arrival buffers round robin, $s \bmod (d \cdot k)$ with $d \cdot k = 12$, with at most $\min(d \cdot k, 2 \cdot \text{depth}) = 8$ reads in flight, handed to the system four at a time (Figure 9.4).

Lemma 9.4 (The arrival buffers of stream prefill)**.** *(i) With the rule from before the round, a read could write into a buffer that another read was still writing. (ii) With the corrected rule (take the round-robin buffer if it holds no read in flight, otherwise the next that holds none), no read writes into a buffer i while (a) another read is writing it, (b) the copy to the card that was reading it has not finished, (c) a hash of the guard is still reading it.*

Table 9.6: The trial's findings, and what the round of corrections made of them.

	Trial's finding	Round's correction and control	Verdict
G1 serious	a pass under load hangs forever without a message: the engine's destruction spins in `nvcuda64.dll`	error written before any destruction, exit without destroying, heartbeat per layer; stream arrival buffers corrected (lemma 9.4)	corrected; root cause not reproduced
G2 serious	after an error, an orphan `nvidia-smi` holds the driver's pipes, which waits forever	`Drop` that kills the child, "kill on close" job, null input, non-inheritable handles; 3 cases, 0 orphans	corrected
M1	production does not guard raw streams: a wrong median gives a text identical to the sound one	production guard (Proposition 9.5); copy O3: named error	corrected, −1.6 % at equal disk
M2	baseline under-tuned; ×3.85 to strike out	baseline swept: `fit -t 16`, 0.884; C6 ×2.22	done
M3	C8 measures the machine	process counters in the log: constant gap of 19.92 MB	instrument done; criterion to the arbiter
M4	C7-gap still fails	bound with named terms (Table 8.4)	not closed; to the arbiter
M5	the f16 KV decides the first token of C12	option `--kv-f32`: equal to the truth up to the format	measured; seam N to the arbiter
m1	C5's threshold depends on a cold D pass	discarded warm-up D pass	done
m2	the SOCLE's recipe breaks the build	`build.rs` calls `nvcc` without `cmd`	corrected
m3	C7 does not see committed memory	published: 6.92 GB–6.97 GB; machine loss 0.47 GB–0.63 GB	published, not judged
m4, m6	three imprecisions of § 4; where the verdict goes	sentences proposed	to the arbiter
m5	"six times closer" needs its reason	requalified: including the YaRN ramp	done

Data: `mesures/epreuve-J1.md` § 8; `moteur/O-pour-le-memoire.md` § 14; `moteur/REPRISE-O.md`, final state of the round.

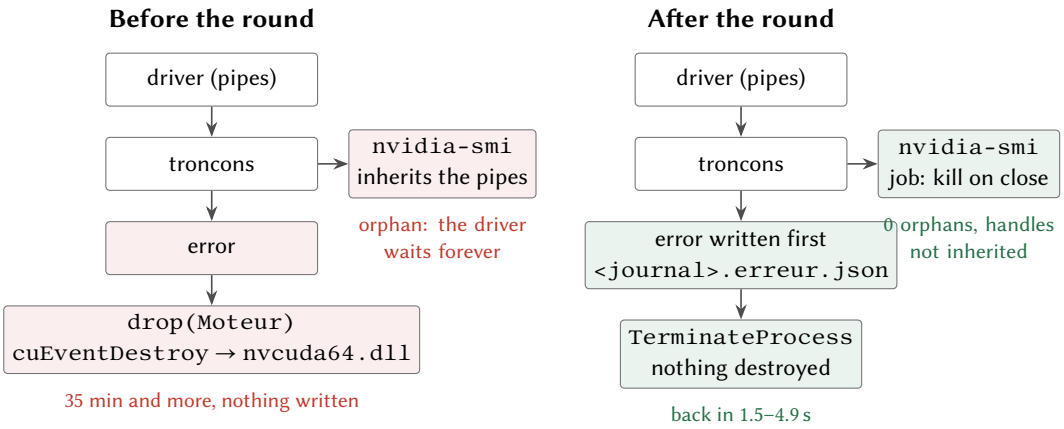


Figure 9.3: The exit on error, before and after the round (findings G1 and G2). Before: the engine's destruction waits for the card forever, and the `nvidia-smi` child, which inherited the driver's pipes, keeps them open. After: the error is written first, nothing is destroyed, and the child dies with its parent.

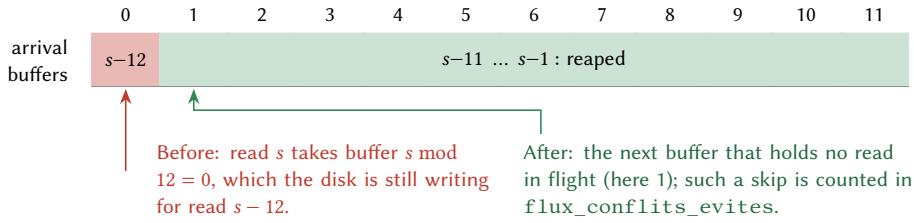


Figure 9.4: The round robin of arrival buffers in stream prefill, and the counterexample of lemma 9.4: read $s-12$ stays in flight while $s-11,\dots,s-1$ go through and are reaped; read s would take its buffer.

Proof. (i) Counterexample. Completions arrive out of order. Let read $s-12$ stay in the system while $s-11,\dots,s-1$ go through the three other places and are reaped: the local list of reads in flight then holds only $s-12$, of length $1<8$, and read s is submitted into buffer $s \bmod 12 = (s-12) \bmod 12$, which the disk is still writing. The copy launched when $s-12$ is reaped then reads a mixture: wrong lane check (fatal error, followed by G1's destruction) or, if only raw bytes are touched, a wrong expert without an error. It suffices that one read lags the others by about $11 \times 6.6\,\text{ms}/3 \approx 24\,\text{ms}$ (12 MB at 1.8 GB/s, three places) [calc.]; the 90th percentile of a cold read recorded by the judge, 172 ms, exceeds it. (ii) Existence: at the moment of the choice, at most $\min(d\cdot k, 2\cdot \text{depth}) - 1 < d\cdot k$ reads are in flight, so at least one buffer holds none. (a) follows from the rule. (b): submission waits for the event of the buffer's previous copy, queued when it was reaped. (c): submission first waits for the end of the zone's hashes. The corresponding VRAM buffer is rewritten only after the end of the decode that was reading it. ■

Listing 9.1: The choice of an arrival buffer in stream prefill, after the round (`moteur/crates/trc-ordo/src/lib.rs`, excerpt; the code's comments are in French).

```rust
// Le tampon du tourniquet, sauf s'il porte encore une lecture en vol : les
// lectures s'achèvent dans le désordre ; avant la ronde de corrections, la
// nouvelle lecture écrasait alors l'ancienne dans le même tampon (cause
// probable de G1). Il y a toujours un tampon libre : au plus
// min(d · k, 2 · profondeur) − 1 < d · k lectures sont en vol ici.
let candidat = prochaine_arrivee % self.n_arrivees;
let mut i_arr = candidat;
while en_vol.iter().any(|&(_, b)| b == i_arr) {
```

```
    i_arr = (i_arr + 1) % self.n_arrivees;
}
if i_arr != candidat {
    self.compteurs.flux_conflits_evites += 1;   // le cas du lemme, compté
}
prochaine_arrivee = i_arr + 1;
self.ev_copie[i_arr].attendre()?;               // (b) la copie précédente est
    finie
```

Replayed under the same load. Block 4 of the declared thresholds replayed the hung pass under the trial's load (the f64 numpy pass of the 120b, 61 GB read, launched first). The new binary finishes in 361 s, with 360 heartbeats, a `.kld` equal to lot O's and **0 arrival conflicts**: the lemma's case did not occur. The old binary, under the same load, **also finishes** (338 s, equal `.kld`). The trial's hang is thus not reproduced: it depends on an order of completions or a machine state that two replays did not recover. What is known: the destruction could wait forever (named stack); the new binary destroys nothing on an error and writes it first; the round robin could overwrite a read in flight (proved). What is not known: the original error.

9.9.2 G2: the orphan that holds the pipes

The engine's VRAM sampler launched `nvidia-smi -lms 250` and killed it only in its normal stop; no destruction did. Any error left the child alive. And it inherited the engine's handles, hence the driver's output pipes: the driver, which waits for the outputs to close, waited as long as the orphan lived, that is, forever. In the trial, two passes finished in a few seconds "gave back control" only after 518 and 630 s, when the judge killed their orphans; an orphan born at 01:38 from a lot O pass was still running at 04:40.

The correction, in four moves. (1) A destruction of the sampler that kills and waits for the child; (2) the child placed in a Windows "kill on close" job [46], which kills it even if the parent dies by `TerminateProcess`, which the destruction does not see; (3) a null standard input; (4) the parent's standard handles made non-inheritable, because a true `bInheritHandles` gives the child every inheritable handle [47], the driver's pipes among them.

The control. `corrections/temoin_sortie.py` launches three cases as the drivers do (missing file, corrupted file, killed process), outputs in pipes, and waits for them to close. Declared threshold: return within 30 s, code 1, error written, no `nvidia-smi` alive 2 s later. The old binary **fails** on all three (return at the 90 s timeout, one orphan each); the new one **holds** (1.5, 4.9 and 4.9 s, 0 orphans). The control itself had two defects before it was valid, published: the first pass killed the process before the child was born (it proved nothing); the second did not drain the pipes, and the child blocked on its full error output (two reader threads corrected it).

9.9.3 M1: the production guard

The hole. In production, the only check on a chunk was the return of the rANS lanes to L. It guards only the *coded* streams: the raw streams of a part (MEDIANS, BIASES: about 43 kB out of 12 MB, 0.36 %) and, in mode A0, the embedding row read at every token were guarded by nothing. On the copy with a wrong median, the text produced was *identical* to that of the sound file: a corruption invisible down to the output.

The guard (Figure 9.5). At every completed read, the useful bytes of the chunk are hashed by a worker thread and compared with the digest the directory already holds (no seam touched); hashing runs at the same time as the copy and the decode, which only read the buffer; the step waits for all its hashes before returning its token. Embedding rows are compared with a table of digests built at opening by reading `token_embd` once, its whole digest checked on the way.

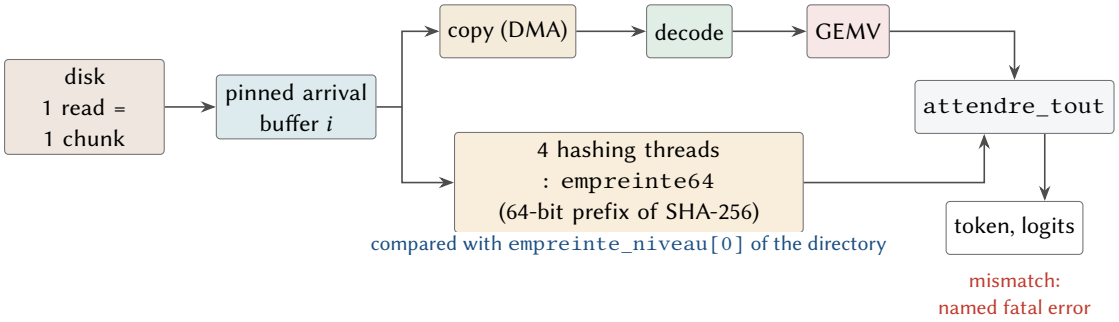


Figure 9.5: The production guard: hashing runs in parallel with the copy and the decode, on four threads; a barrier joins it before the step returns its token.

Proposition 9.5 (No output depends on an unchecked byte)**.** *With the guard active, the token and logits returned by a step, or a prefill, depend only on bytes read whose 64-bit digest equals the directory's (chunks: useful bytes of level 0; rows: table built at opening), up to a collision of the 64-bit prefix of SHA-256 [43].*

Proof. (1) The decoder and the copy read only the useful bytes of a part; padding is read by no kernel. (2) Every completed read is handed to the guard before any other operation on its buffer. (3) No read writes that buffer before the hash ends (lemma 9.4, (c)): the digest covers the bytes the copy read. (4) The step and the prefill wait for all hashes before reading the token or the logits; a different digest is a named fatal error. (5) A warm entry served without a read was hashed when it was read; a later corruption of RAM is not covered. (6) Every row read is compared with its slot before it is copied to the card. Wrong content passes with probability 2^{-64} per check, if SHA-256 behaves as a random function. ■

Listing 9.2: The guard turns a wrong chunk into a named fatal error (`moteur/crates/trc-ordo/src/garde.rs`, excerpt; the message is in French).

```
fn recevoir(&mut self, f: Fait) -> Result<()> {
    let t = self.en_cours.remove(&f.ticket).expect("ticket de la garde");
    if f.empreinte != t.attendu {                // l'empreinte du répertoire
        bail!(
            "garde de production : couche {}, expert {} : empreinte des {} octets utiles lus {:016x} au lieu de {:016x} (répertoire)",
            t.couche, t.expert, t.octets, f.empreinte, t.attendu
        );
    }
    ...
```

The controls, and their breaks (on the 20b). Copy with a wrong median: named fatal error "production guard: layer 0, expert 30: digest of the 11 836 992 useful bytes read `17d8974e`… instead of `9aa7c5fb`…"; without the guard (the break), the pass runs to the end, with the sound file's text, as in the trial. Chunk broken after reading (`--casser-garde troncon:5`): named error; without

the guard, text changed without an error. Broken row (`ligne:3`): named error; without the guard, text unchanged (one bit of a Q8_0 row does not change these 32 tokens: invisible). Sound file: same text to the bit with and without the guard, 920 chunks checked out of 920 read, 739 rows out of 739. Lot E's copy: the guard names layer 2, expert 19, before the lane check. The cost is measured in Section 8.5.5: −1.6 % at equal disk, 5.2 ms–5.4 ms of waiting per token. C2 becomes true in production.

9.9.4 M2 to M5, and the minor findings

M2, the baseline. Swept in a palindrome (Table 8.5): `fit -t 16`, 0.884 token/s; C6 recomputed, guard included, ×2.22; ~~×3.85~~ struck out in lot O's notes, its resumption note and `mesures/gpt-oss.md`.
M3, C8. The process counters, read on the handle after exit, are in the engine's log. Against the engine's count (reader, residents included, plus the buffered reads at opening, 13.57 MB), the gap is 19.92 MB**, constant to the byte** over the six f16-KV 120b passes of the session, including the vocabulary read by `llama.dll` (12.98 MB). Proposal to the arbiter: judge C8 by this instrument, bound equal to the vocabulary size plus 100 MB, PDH published without being judged. On the way, the round found that the first version of the count added the loading of the residents twice (+1.69 GB): corrected.
M4, C7-gap. The bound with named terms (Table 8.4) is not closed; proposal: judge the gap at a fixed window size, against the terms outside the window (≈ 0.77 MB) plus two noises plus 0.5 MB. The verdict stays "fails".
M5, the KV cache. Option `--kv-f32`, outside the seam: equal to the truth up to the format, 12 fewer slots (Section 8.6.3).
m1. One discarded warm-up D pass at the head of the session: the kept pass reads 1.893 GB/s, the discarded one 1.874, without a transient this time (90th percentile 32.9 ms); C5 thresholds 1.45–1.52, and the eight 120b passes hold.
m2. The kernels' `build.rs` called `nvcc` through `cmd /C` with quotes that `Command::arg` protects with backslashes, which `cmd` does not read; it now calls `nvcc` directly. With and without vcvars the build succeeds, and the three `noyaux.fatbin` images are equal byte for byte (SHA-256 `aeae5bdc`..., 295 864 bytes).
m3. Committed memory (6.92 GB–6.97 GB) and machine loss (0.47 GB–0.63 GB) published next to C7.
m4, m6. Three sentences proposed for § 4; where the verdict goes, to the arbiter.
m5. "Six times closer" requalified everywhere: including the YaRN ramp.

9.9.5 What remains for the arbiter

The round handed twelve points to the arbiter, each marked "answer by delegation of 2026-10-16, reversible" (`moteur/REPRISE-O.md`, points 21 to 32). The heaviest: seam N (f32 KV cache by default, f16 as an option: C4 and C12 would become "equal to the truth up to the format", for 155.7 MB of VRAM and 12 slots); a tail criterion for C4 (share of positions with KL > 0.1, KL at the 99th percentile, judged also beyond 512 positions; numbers to set it: f16 0.39 % and 0.075 on window 0, 2.05 % and 0.196 on positions 512 to 706; f32: 0); the greedy sequence of the truth to publish next to C12 (not done: the numpy pass recomputes the whole prefix at each token, about 20 min per position on the 120b, days for 256 tokens); C8 and C7 as above; the guard written into § 4.12 of the specification; warning the lots that any child launched under a driver with pipes must have non-inheritable handles or live in a job killed on close; and a lot O orphan `nvidia-smi` (PID 13652), left alive because neither the judge nor the round had launched it.

9.10 What the trial could not check

- **C1 on the** 4 608 **experts** by the third decoder: 40 drawn and 13 named, plus 3 394 decodes checked by the engine's control; the rest rests on lot F and its two paths.
- **C2 end to end** by lot E's bench: not rerun.
- **C4 on the ten windows of the 120b**: neither lot R's f64 reference nor a ten-window llama.cpp baseline exists on the 120b; the judgement bears on one window of 255 positions, plus the 353 scored positions of the prompt.
- **The cause of G1**: the original error was never written.
- The breaks of lots K, R, O1 and E.
- **The speed session is one session**: three troncons passes, two of each baseline; the rule is judged only within it.

What could be wrong

- **The judge's numpy pass and lot R's reference read the same GGUF.** A dequantisation error common to both (nibble order, E8M0 scale) would pass. Safeguards: the judge's MXFP4 decoder follows ggml's convention (low nibbles for elements 0 to 15), and C3 like the texts tie everything else together to the bit; the convention of the format is judged only by agreement with llama.cpp, not by OpenAI.
- **They follow the same reading of the model** (real ramp, sinks, 128-wide window). Had OpenAI trained with another convention, both would be wrong together; OpenAI's `config.json` and reference implementation are the only sources of this reading.
- **C4 on the 120b rests on one window**; the intervals are disjoint, but one window is not ten.
- **The machine's memory loss is global**: other processes move it. It is clean in session `s1`; it was soiled (10.5 GB, the numpy pass at the same time) in another pass the judge did not use.
- **The judge's instrument had two defects**, found and corrected before any verdict: a process handle passed as a 32-bit integer (a counter control that returned 0), and a first C8 calculation that counted the loading of the residents twice ("the engine counts 1.4 GB too many"), redone correctly and never published.
- **The cause of G1 is not established**: the buffer defect is proved possible, not observed (0 conflicts in two replays).
- **The guard assumes SHA-256 without a useful collision** on 64 bits; it covers neither warm RAM after its read, nor VRAM.
- **The time of one threshold was wrong** in the judge's file: ~~02:40~~ 02:26, the commit's time, before the driver's trial run (02:26:35) and the first pass (02:27:40). The thresholds were indeed declared before any pass; only the written time was wrong.

Chapter 10

Discussion, limits and future work

Where we start, what this chapter establishes, what remains

Where we start: a program that holds nine whole criteria out of the eleven judged (C7 holds on its ceiling and fails on its gap, C8 awaits another instrument, C9 is published without being judged), an independent trial and a round of corrections (Chapters 8 and 9). **This chapter establishes** what Volume II says in sum; what Volume I's predictions were worth against the measurements; the struck-out numbers and the ideas killed by numbers, gathered; the limits of the method; then the work to come, every step with the measurement that will judge it: the base + complement split (J2), the `D:` disk without BitLocker, the compatible gate for relais, and Stone's biomimetic study, the weight genome. **What remains** is to do it, in the order decided with Stone.

10.1 What Volume II establishes

1. **A** 63.4 GB **model runs with** 384 MB **of memory** on a machine that has 16, and its memory does not follow its size: +3.7 MB for +51.3 GB of model (Section 8.4). The reading of "without RAM or anything" that Volume I retained is verified by measurement.
2. **Lossless coding costs the output nothing**: logits equal to the bit between the coded and the raw file over 5 120 steps, from one build and one order of computation to another (Section 8.3).
3. **troncons is faster than the best baseline measured**: ×2.22 in generation, integrity guard included, and ×7 at prefill (Section 8.5).
4. **It is closer to the exact mathematics than llama.cpp**, and, with an f32 KV cache, equal to them up to the format: the whole measurable gap came from the f16 rounding of the cache (Section 8.6.3).
5. **The method worked**: the independent trial found two serious and five medium defects the lots had not seen, and the round corrected or measured them in one morning, every correction with a control that fails on the old binary (Section 9.9).

10.2 Volume I's predictions, against the measurements

Volume I computed before building. Table 10.1 confronts its numbers with the program's; a prediction that landed right is not a proof, a prediction that landed wrong is information.

Two lessons. First, the only number of Volume I that really moved is the one it had not measured itself: the baseline, estimated then measured too fast. Second, the disk throughput in the engine is ~~a third of what Volume I measured in large sequential blocks~~ a little more than half (1.55 GB/s useful) to two thirds (1.85 GB/s, disk alone at the chunk length) of what Volume I measured in random 16 MiB reads (2.82 GB/s) [calc.]: not because the disk changed, but because the engine reads chunks in the disk's slow regime and does not have enough reads in flight (Section 8.5.1). This is the nearest lever.

Table 10.1: Volume I's predictions and Volume II's measurements (gpt-oss-120b, mode A0 unless stated).

Quantity	Volume I (computed or estimated)	Volume II (measured)
VRAM cache hit rate	$h_v = 0.590$ (LRU scenario fitted on one point, 4.8 GB cache)	$h = 0.591$ (326 slots, 4.3 GB)
speed in A0	marker 2.15 tokens/s (fitted LRU); 0.99 (uniform); 3.32 (Belady)	2.18–2.19 (lot O); 1.965 guard included
host memory	A0: 1.9 GB; minimal window of A's advocate: 0.64 GB	0.384 GB, under the minimal window
limiting stage	the disk (157 against 58 and 36 ms, scenario with warm cache)	the disk: 68 %–72 % of the serial floor
disk throughput	2.82 GB/s in random 16 MiB reads (depth 1); 2.58 GB/s in random 4 MiB reads (depth 4)	1.85 GB/s–1.89 GB/s at the chunk length (lot E's slow regime); 1.55 GB/s useful in the engine
lossless coding rate	$r = 1.1026$ on the experts	a 12.03 MB chunk for a 13.25 MB expert: $r \approx 1.10$
decoder on the card	82 GB/s under CUDA, on a large stream	0.85 ms–0.92 ms per single expert, i.e. 14 GB/s–16 GB/s of output: a single expert does not fill the card [hyp.]; 10 % of a token's time
llama.cpp baseline	estimated; exploration 0.49	~~0.568~~; 0.884 (`fit -t 16`)

Data: Volume I (abstract, chapters "The machine", "Formulation", "Architecture"); Chapter 8.

10.3 The struck-out numbers

When a measurement contradicted a published number, even our own, the sentence was struck out where it stood, and what replaced it was said (Table 10.2). It is the trace that the method worked.

10.4 The ideas killed by numbers

The SOCLE asks that an idea be killed by numbers, not by opinion, and that failures be kept with their reason (Table 10.3).

10.5 The limits

- **One machine, shared.** Everything is measured on Stone's laptop, the target; but other agents were measuring at the same time, and the disk varied by 20 % from one session to another without a measured cause (the NVMe temperature cannot be read without administrator rights).
- **An encrypted disk.** Every speed number carries BitLocker, which decrypts every byte on the processor and refuses the I/O bypass; the unencrypted `D:` disk (Section 10.6.2) will tell what it costs on a real model.
- **One target model**, gpt-oss, in native MXFP4. The format and the engine are written for a mixture of experts whose expert is the slowest dimension of each tensor; another model will need another coder, and perhaps other kernels.
- **One speed task** (a 708-token prompt, 256 greedy tokens): the cache hit rate, hence the speed, depends on the text (Section 8.6.3).
- **An NVIDIA card required**: choosing CUDA for J1 closes the door to other cards until the Vulkan port.

Table 10.2: The numbers struck out during construction, the trial and the round, and what replaced them.

Struck out	Replaced by, and why	Where
~~C6: ×3.85 over the baseline~~	×2.79 to 2.89 (trial, `fit -t 10`), then ×2.22 (round, `fit -t 16`, guard included): the baseline ran at 6 threads	Section 8.5.2
~~×2.79 to 2.89~~	×2.22: the baseline had not been swept beyond 10 threads	Section 8.5.3
~~best 120b baseline: fit, 0.568~~	`fit -t 16`, 0.884: +54 %, a real gap	Table 8.5
"six times closer to the truth" (not struck, requalified)	including llama.cpp's YaRN ramp, 55 % of its gap: a modelling choice	Section 8.6
~~C2: 100 % of corruptions detected~~ (in production)	true in control mode only; true in production since the guard	Section 9.9.3
~~digest: 2 GB/s per core~~ (estimated)	1.07 to 1.10 GB/s per thread, measured	Section 8.5.5
~~f32 KV: 604 MB instead of 302, −7 %~~	+156 MB, −3.6 % of slots: the first calculation forgot the ring of the windowed layers	Section 8.6.3
~~C5 threshold on the median of the two D passes~~	one discarded warm-up D pass: the first, cold, moved the threshold by 25 %	Section 9.5
~~engine byte count~~ (residents counted twice, +1.69 GB)	corrected in the code; gap to the process constant, 19.92 MB	Section 9.9.4
~~time of the trial's threshold: 02:40~~	02:26, the commit's time	Section 9.10

- **A short judging window on the 120b**: C4 bears on 255 positions and the prompt; lot R's reference over the ten windows of the 120b is missing.
- **Two pending decisions**: until the arbiter decides seam N, C7 and C8, two criteria out of twelve stay "fails" or "instrument to change".

10.6 Future work

The order was decided with Stone on 15 October, then taken up again on the morning of the 16th ("on fini une tache proprement puis ln enchaine l'autre", *we finish one task cleanly then chain the next*): the milestone and its trial, this Volume II, then a full pause and a restart by Stone (the `D:` disk), then the study of the weight genome, then the compatible gate for relais (`REPRISE.md`). Figure 10.1 places each piece of work with respect to the milestones.

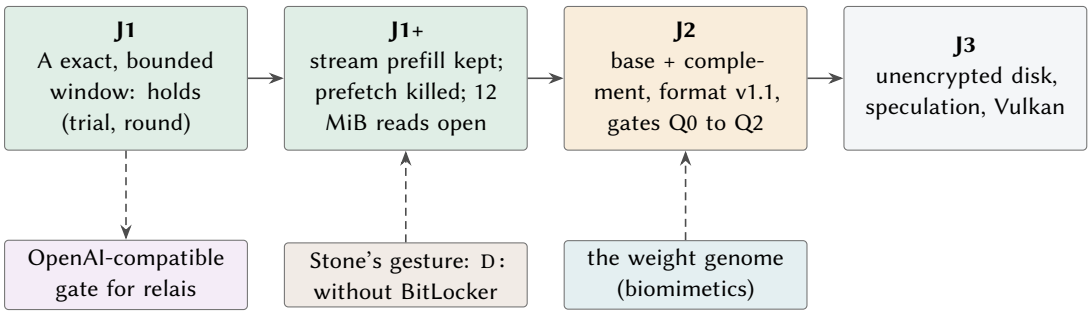


Figure 10.1: The milestones of troncons and the work to come. In green, what holds; in orange, the milestone of the contribution; dashed, what feeds a milestone.

10.6.1 First of all: a single series, and the arbiter's decisions

Three short moves would close the weaknesses of this volume: one palindrome series, guarded troncons against `fit -t 16` (and 20 threads, and `--mlock`), for a C6 without between-session

Table 10.3: The ideas tried during J1, J1+ and the round, and the number that judged them.

Idea	The number that judges it
Prefetch by the next layer's router	−11 %: 69 % of predictions right; the 31 % wrong ones cost more disk than overlap gives back [meas.]
12 MiB reads	+30 % in median, but a noise of 16.2 % between the B passes (threshold 35.4 %; ~~21 %~~ dispersion): neither established nor killed [meas.]
A randomly corrupted copy as the only break of C3	insufficient: a wrong median, invisible to the decoder, was needed to show wrong logits [meas.]
Comparing two speeds on two different texts	not to be done: the f32-KV text does not loop, h 0.435 against 0.590; the cost of the KV is measured on one trace [calc.]
Draining prefetches while refusing any foreign completion	failure, corrected: the current layer's reads finish during the drain
Assembling the text by decoding each token alone	display failure: UTF-8 characters split between two tokens come out wrong
Building the binary while a driver launches it twice	not to be done: the two C3 passes must run on the same binary
Calling `nvcc` through `cmd /C` from `Command::arg`	failure: quotes protected by backslashes, which `cmd` does not read
Checking the guard before every product (instead of the end of the step)	not tried: computed at about 6 ms × 30 layers per step, −40 % [calc.]

Data: `moteur/REPRISE-O.md` (recipes of J1, J1+ and the round).

noise; lot R's f64 reference over the ten windows of the 120b, for a C4 over 5 120 positions; and the arbiter's decisions (f32 KV by default, tail criterion for C4, C7 at a fixed window, C8 by the process instrument), which would make the milestone clean.

10.6.2 The `D:` disk without BitLocker

The gesture. It is the only one that falls to Stone, on his machine: a tool already written and committed (`outils/preparer-disque-D.ps1`, launched by a desktop shortcut that asks for administrator rights) checks that no agent is running, turns off the page file of `C:`, restarts, shrinks `C:` by 200 GB (at least 150), creates an unencrypted `D:` volume "Modeles", checks that `C:` stays encrypted and `D:` is not, puts the page file back in every case, and restarts. Model weights are public files; nothing personal needs to be encrypted there. `C:` stays encrypted, and the BitLocker recovery key has been saved since 15 October.

What it could give back, posed. The disk holds 68 %–72 % of the serial floor. On pass 03 of the round (no guard), the engine reads 0.710 GB of chunks per token, and the other stages cost together 80.7 + 50.5 + 15.3 + 3.5 = 150.0 ms. If the bytes read stay the same and only the disk throughput W changes [calc.]:

$$t_{\text{serial}}(W) = \frac{0.710\ \text{GB}}{W} + 150.0\ \text{ms},$$

$$\begin{aligned}
W &= 1.893\ \text{GB/s (measured, C:)} && \Rightarrow 525\ \text{ms}, && 1.90\ \text{token/s},\\
W &= 2.82\ \text{GB/s (Volume I, large blocks)} && \Rightarrow 402\ \text{ms}, && 2.49\ \text{tokens/s},\\
W &= 6.0\ \text{GB/s (manufacturer's rating)} && \Rightarrow 268\ \text{ms}, && 3.73\ \text{tokens/s}.
\end{aligned}$$

With perfect overlap, the bound would be the slowest stage: at 6.0 GB/s, the disk would take 118 ms and the link 81 ms, so at most 8.5 tokens/s; the link would again be nearly the maximum, as Volume I

announced. These are bounds, not predictions: Intel RST VMD stays on the path, its share of the 53 % lost is not separated, and the measurement will decide.

What will be redone on `D:`. The models and the `.trc` files will be moved there (digests checked), then the disk and milestone benches redone identically: the difference between `C:` and `D:`, same sessions, same files, will give for the first time the cost of BitLocker on a real model. The throughput of an unbuffered read depends on its length, in two regimes (lot E): the read length will be judged again on `D:`.

10.6.3 J2: the base + complement split

What is ready. The `.trc` format already stores the streams of a chunk by refinement level, every level prefix being decodable alone; the stream kinds of the base and the complement are specified to the bit; and the arbiter's F1 measurement showed that the split, with a rich context for the complement, costs only +0.31 % of bytes against the single level (`decisions/ARCHITECTURE.md`, § 3). The cache already knows the levels (`Base`, `Exact`), unused before J2. Volume I proved that the exact refinement of a native 4-bit code costs nothing in entropy.

What it can give. Serve the base alone, without its complement, at the misses where a byte is expensive: B3 promised ×1.74 in throughput (×1.78 with the bits measured by F1), at the price of 48.5 % of expert uses served by the base, for an estimated KL of 0.05–0.12 [calc.].

The gates, taken over without loosening them (§ 7.3 of the architecture). *Q-instrument*: the exact re-expression must fall back to KL $\leq 7 \cdot 10^{-5}$ and 100 % of top choices, otherwise the bench is wrong. *Q1*: KL of B against the exact mode of the same engine $\leq 0.005\,5$, a threshold inherited from Qwen3.5-4B that will be struck out and replaced in the open by the gap between engines measured on gpt-oss. *Q2*: the (KL, bytes read per token) point of B must pass under the frontier of uniform requantisations by at least two standard errors. *Speed*: a B mode is worth it if it beats the exact mode of the same engine by more than two noises and 3 %. What this volume changes: B's judge will be the exact mode of the same engine, which Proposition 8.2 makes reproducible to the bit, and, if the arbiter adopts the f32 KV, an exact mode itself equal to the truth up to the format. The Q0 bench (the quality of B without an engine) and the base mode of the kernels remain to be done.

10.6.4 The compatible gate for relais

relais is Stone's automation tool, driven by an agent running on free local models; it is the mission troncons interrupted ("on vas continuer la suite relais plus tard on doit faire tomber se levier d'abord", *we will continue relais later, we must bring down this lever first*). The gate would be a local service speaking the OpenAI-compatible chat completions protocol, on top of `troncons generer` and the harmony format the engine already produces, so that relais can call gpt-oss-120b like any other model. What it would cost in use, posed with the measured speeds [calc.]: a 1 000-token prompt prefills in $1000/27 \approx 37$ s; a 300-token answer is generated in $300/1.965 \approx 153$ s. It is slow for a conversation, acceptable for an automation task running on its own, which is relais's use case. Two requirements follow: stream the tokens as they come (and assemble the UTF-8 bytes split between two tokens, a known defect), and keep the engine open between requests, opening costing 2 s–3 s.

10.6.5 The weight genome: Stone's biomimetic study

The idea, in his words. On 15 October, Stone wrote (in French, typing errors included):

> *J'ai penser à du biomimetisme informatique par exemple simjler une forme de molecule d'adn virtuel qui peut stocker une grande qua tutoriel d'informations en quelque sorte creer une nouvelle convention differentes du bit une transcription et implémentation*

(*I thought of computer biomimetics, for instance simulating a kind of virtual DNA molecule that can store a large quantity of information, in a way creating a new convention different from the bit, a transcription and implementation*), then, the same day: "j'ai la certitude que ce qu'on cherche se trouve dans du biomimetisme" (*I am certain that what we are looking for lies in biomimetics*). This conviction is the working hypothesis of the study; measurement will judge it (`idees/biomimetisme-genome-des-poids.md`).

What does not work: changing the alphabet. DNA writes with four letters, exactly $\log_2 4 = 2$ bits per letter. Writing with four symbols rather than two creates no room: every message has a minimum size, its entropy, and no alphabet goes below it (Shannon [48]). On a computer, a "virtual molecule" is stored in bits. The strength of DNA is not its alphabet; it is in its *mechanisms*.

What may work: copying the mechanisms. Several are already in troncons without bearing the name: expression on demand (the cell transcribes only the genes it needs: experts are decoded only at the moment of computation, then freed); epigenetics (marks say which genes are active: the cache keeps frequently called experts near the computation); families before detail (purines and pyrimidines before exact identity: the base before the complement, J2). The study adds four, each with a measurable hypothesis on gpt-oss.

Table 10.4: The mechanisms of the living retained for the study, and the hypothesis each poses on the weights of gpt-oss.

Mechanism	In the living	Measurable hypothesis on gpt-oss
The recipe rather than the result	DNA carries the program that builds the body, not its shape	an expert is represented by a pseudo-random generator seed and a few coefficients, which the card "transcribes" at the moment of computation; SeedLM does it with shift registers [49], hypernetworks with a network that generates another's weights [50]
Gene duplication	a new gene is born from a copy of an old one, then mutates [51]	the experts of a layer, or of neighbouring layers, are related: a "parent" expert and, for each descendant, a short "mutation" cost fewer bits than whole experts
Predictive coding	the cortex transmits only the gap between what it expected and what arrives [52]	predict each weight from its neighbours, its parent or its row, and store only the gap: fewer bits when the prediction is good
Synaptic pruning	the young brain removes useless connections [53]	a share of an expert's weights can be removed without changing the output beyond the judge's threshold [54, 55]

Data: `idees/biomimetisme-genome-des-poids.md`, § 3 and § 5 bis; sources reread on 2026-10-16.

Why this machine lends itself to it, posed. Volume I's roof theorem says that speed is bounded by the throughput of the slowest stage divided by the bytes it must pass; this volume measured that this stage is the disk. A weight genome *spends computation to save bytes*, exactly the trade the machine asks for: the card waits for the disk, with a budget of about 926 operations per compressed byte at

the disk's throughput (Volume I; ~~214~~, the budget at the link's throughput, 12.2 GB/s, which does not apply when the disk limits). Let us put a number on what it would give if it held. Today an expert is read at 3.843 1 bits per weight losslessly (F1 measurement). If a representation reached b' bits per weight at equal quality, the bytes read from disk and copied over the link would be multiplied by $b'/3.8431$, and the rest unchanged. For $b' = 2$, on pass 03 of the round [calc.]:

$t_{\text{serial}} = 0.520\times(375.1+80.7)+50.5+15.3+3.5 = 306.5\,\text{ms}$, i.e. a floor of 3.26 tokens/s, against a floor of 1.90

Both numbers are serial floors, with no overlap (Section 8.5.1): pass 03 measures 2.18 tokens/s, above its floor, and only a measured engine will judge the genome. It is an "if", and the study must judge it.

The plan of the study (`idees/biomimetisme-genome-des-poids.md`, § 5). (1) Read the state of the art at the source (generator seeds, hypernetworks, trellis codes, computer biomimetics). (2) Measure on the real weights of gpt-oss-20b, on the processor, the bits per weight obtained against the reconstruction error, for each mechanism. (3) Measure transcription: a kernel that regenerates an expert from its recipe, and its throughput, against the disk's 1.89 GB/s. (4) Judge the quality of the text against the exact reference, threshold declared beforehand. (5) Write it, in a chapter of its own. The success criterion, to be declared before measuring: fewer bits per weight read from disk than today's lossless coding, a quality loss under the judge's threshold, and a transcription faster than the read it replaces. An understood failure says where to look: every failure will be analysed (which share of the weights is noise, which has structure) instead of stopping at the verdict.

What could be wrong

- **The gains of the `D:` disk** are bounds computed under the assumption that only the disk bytes change; Intel RST VMD and the read-length regime may take back a share.
- **The gain of a 2-bit genome** is a calculation under an "if": the weights of a trained model may look like noise, and noise does not summarise; the recipe could then only approximate the weights, with a loss.
- **Transcription may cost more than it gives back**: if regenerating an expert takes longer than reading it, the gain disappears; today's decoder already takes 10 % of a token's time.
- **The gate for relais** is costed with the speeds of a single task; a long prompt or another text change the cache hit rate.
- **The gains of J2** are the architecture's, estimated under a linearity valid for small errors; Q0 has not yet measured the quality of B.
- **The summary inherits the weaknesses of the two previous chapters**, first C6 across two sessions and C4 on one window.

Conclusion

On 15 October 2026, Stone asked for “une vrai solution pour prendre un modele ultra lourd quelcoque d’une ai le faire tourner proprement sans ram ni rien” (*a real solution to take any ultra-heavy AI model and run it cleanly without RAM or anything*). Volume I had measured the machine, proved the roof of the problem and chosen, by calculation, an exact architecture that prepares the contribution. This Volume II built the program and submitted it to a judge who had not written it.

The result fits in four numbers, each reread by a second path. A 63.4 GB model, four times the RAM and eight times the card’s memory, runs with 384 MB of memory, the same to within 3.7 MB as for a model five times smaller. Its outputs are identical to the bit, over 5 120 steps, whether its experts are read compressed or raw. It produces 1.965 tokens per second with its integrity guard, 2.22 times the best measured configuration of llama.cpp, and prefills seven times faster. And, with an f32 KV cache, it computes what the model computes in exact mathematics, up to the file format.

What this volume has not yet done is said as clearly: the research contribution, the base + complement split, awaits its quality bench and milestone J2; two criteria await a decision of the arbiter; the ratio to the baseline crosses two sessions; and the root cause of the only hang observed is unknown. About ten numbers were struck out on the way (Table 10.2), among them the first speed ratio, which compared troncons with a badly tuned baseline. The method found them, which was its job.

What comes next is decided: a pause, the `D:` disk without BitLocker, which will tell what encryption costs on a real model; then the study of the weight genome, Stone’s intuition that the living already knows how to store a lot in little, to be judged by numbers; then the gate that will give this engine back to relais, the mission it had interrupted. The lever Stone wanted brought down first has come down: the memory of an AI no longer depends on its size.

Appendix A

Measurement protocols of the engine

Where we start, what this chapter establishes, what remains

Where we start. The measurement rules of Volume I (its appendix A, “Measurement protocols”): the provenance label of every number, the witness and its break, the threshold declared beforehand, ABBA series and the decision rule, the second path, power and the shared machine, the quality judge. They all hold here, as they are. **What this appendix establishes.** What construction added to them: an end criterion for every step of every lot, with its witness, its break and the derivation of its threshold; the proof that a palindrome cancels a linear drift; the lock of the card; the time of the commit as the proof that a threshold precedes its pass; the protocol of the trial and that of the round of corrections, block by block. **What remains.** Two protocols the machine has not yet allowed: a single palindromic series containing both guarded troncons and the best baseline, and a baseline beyond 16 threads and with `--mlock` (Chapter 10).

A.1 What is inherited from Volume I

Every number carries its provenance ([meas.], [calc.], [lit.]). Every measurement has its witness, a case whose answer is known in advance, and every new witness must be able to fail: its condition is forced to false, the run is repeated, the instrument is checked to fall, then restored. A threshold derives from a named measured quantity, and is declared before the run. A cost is never founded on a single pass. Every published number we depend on is redone by an independent path. Only passes on mains power are reference throughputs. The decision rule is that of study 06, § 7.5, recalled in Definition 8.1: a configuration B really differs from A if

$$\left|\frac{\bar{X}_B}{\bar{X}_A} - 1\right| \;>\; 2\,\max(\beta_A, \beta_B) + 3\,\%, \qquad \beta = \frac{|X_1 - X_2|}{\bar{X}}, \tag{A.1}$$

where β is the noise of a configuration, the gap between its two passes of one palindrome relative to their mean.

A.2 The end criteria of each step

The architecture wrote, before the first line of code, what would make each step hold (`decisions/ARCHITECTURE.md`, § 6.5). Table A.1 restates them, with the chapter where each is judged. The factor 0.8 recurs everywhere; it is not a margin chosen by guesswork: it is $1 - 2 \times 10\,\%$, where $10\,\%$ is the noise measured in Volume I of a 4 MiB read at queue depth 4 (study 01). A throughput measured above 0.8 times its reference therefore cannot be declared slower than it through the noise of the two measurements alone.

Table A.1: The end criteria of the steps of the five lots, declared on 15 October 2026 before construction (`decisions/ARCHITECTURE.md`, § 6.5), and the chapter that judges them.

Step	Witness	Break	Threshold (and its source)	Judged
F1	write then read round trip identical; processor decoder equal to the arbiter's streams	one header byte falsified: refusal; a falsified table: wrong decoding	0 gap	Chapter 3
F2, F3	C1: all experts decoded by the processor equal to the original (fingerprint and byte for byte); rates within 0.1 % of F1	a falsified median in a copy: inexactness detected	0 wrong expert	Section 3.12
E0	(a) pin 0.25, 1, 4 and 11 GB; (b) unbuffered read into that memory then DMA, bytes equal; (c) 36 synchronisations per token	a falsified byte in the source	(b) DMA $\geq$ 11.0 GB/s; (c) $\leq$ 15.7 ms per token	Chapter 4
E2	C2: fingerprint of every expert of the 20b, disk $\rightarrow$ VRAM decoded, equal to the directory	a bit flipped in a copy of the file	$\geq 0.8 \times 2.58 \times r$(F1) GB/s of experts served	Section 4.8
K1	output equal to `trc-codec` on the 108 tensors of the 120b; lanes ending on L	a corrupted table slot	$\geq 0.8 \times 41.6 = 33.28$ GB/s (reference later struck out for 45.8)	Section 5.2
K2	MXFP4 GEMV exact to the bit on integers, 10^{-5} on reals	reference shifted by 1 %	$\geq 0.8 \times 209.4 = 167.5$ GB/s	Section 5.3
K3	each kernel of the forward pass against `trc-reference`	each witness forced to false	thresholds of the kernel interface (§ 5.6)	Section 5.4
R0	floor of the instrument (same configuration twice)	another model: clear KL	published: G_{cfg}, G_{moteurs}	Chapter 6
R1	KL(reference $\|$ llama.cpp) $\leq$ max($G_{\text{moteurs}}, 7 \cdot 10^{-5}$) on 10 windows of 512	YaRN without its scale	same	Section 6.4
O1	`rejouer` equal to the simulator of study 04, to the unit	falsified policy	0 gap	Section 7.4
O3	C3, C4 on the 20b; cache count equal to `rejouer` on the engine's trace	corrupted chunk	milestone criteria	Chapter 8
O4	milestone J1	milestone criteria	milestone criteria	Chapter 8

The thresholds, step by step. Each is redone from named numbers [calc.]:

- **E0 (b)**: the host $\rightarrow$ card DMA measured in Volume I is 12.2 GB/s; minus its 10 % noise, $12.2 \times 0.9 = 10.98 \approx 11.0$ GB/s.
- **E0 (c)**: in Volume I's calibrated LRU scenario, the disk takes 157 ms per token of the 120b; a synchronisation costing at most a tenth of it, 15.7 ms, does not change the slowest stage.
- **E2**: 2.58 GB/s is the disk throughput for 4 MiB reads measured in Volume I (study 01); served as chunks compressed at rate r, these bytes are worth $2.58\,r$ GB/s of decoded experts; the factor 0.8

covers the noise.

- **K1**: $0.8 \times 41.6 = 33.28$ GB/s, where 41.6 GB/s was the throughput published by study 03 for the path K1 replaces. That number was itself composed under Vulkan; redone under CUDA it is 45.8 GB/s, and the corrected threshold $0.8 \times 45.8 = 36.6$ GB/s. The kernels chapter strikes out the first and judges against both (Section 5.2).
- **K2**: $0.8 \times 209.4 = 167.5$ GB/s, where 209.4 GB/s is study 03's raw MXFP4 GEMV.
- **C5**: 0.8 times the serial floor $t_{\text{série}}$ recomputed *in the same session*, with the h counted by the engine and the throughputs the `troncons disque` pass has just measured (Section 8.5).
- **C7**: 1.9 GB is the peak RAM of mode A0 published by study 07, which the minimal window (0.64 GB) had to beat; 0.5 MB is the directory, the only term believed to grow with the model (finding M4 of the trial found others, Section 2.7).
- **C10**: 8,188−909 = 7,279 MiB, the card's memory minus the 909 MiB that the display and applications held in it at the reading of 15 October at 10:15, card free (`mesures/REPRISE-gpt-oss.md`).

A.3 A threshold precedes its pass: the time of the commit is proof

A threshold declared "beforehand" proves nothing if nothing dates the declaration. Every block of thresholds is therefore committed *before* the first pass it judges, and it is the time of the commit, not the one written in the text, that is proof. Table A.2 gives, for every threshold file, the commit that created it and the number of commits that touched it afterwards (blocks added, each before its pass; never a threshold changed after its own).

Table A.2: The declared threshold files and the time of the commit that created them (generated by `memoire/outils/tome2_annexes.py` from `git log`).

Declared thresholds	Created by commit (local time)	Commits
`mesures/moteur/F/00-seuils-declares.md`	`c0df2b9`, 2026-10-15 13:05:21	3
`mesures/moteur/O/corrections/seuils-declares.md`	`ca5d338`, 2026-10-16 06:12:40	6
`mesures/moteur/epreuve-J1/seuils-declares.md`	`4832184`, 2026-10-16 02:26:05	4

The rule served three times against its own authors: the times written by hand were wrong, and they were struck out for the time of the commit. The trial wrote its declared thresholds at ~~02:40~~; commit `4832184` is from 02:26:05. The first token of C12, at ~~05:50~~; commit `549b38b`, from 05:31:09. Block 1 of the round, ~~around 06:25~~; commit `ca5d338`, from 06:12:40 (`mesures/moteur/epreuve-J1/seuils-declares.md`, `mesures/moteur/O/corrections/seuils-declares.md`). The three errors go the same way, the one that would have made the declaration later than it was: none could have favoured a verdict.

A.4 ABBA series and palindromes

The sessions of this volume are not A B pairs but *palindromes*: the order of the passes reads the same both ways. The trial's session begins with D120 T120 L120 L120t10 L120t10 L120 T120 D120 (troncons disque, troncons, llama.cpp with 6 then 10 threads); the swept baseline of the round, after a discarded

warm-up pass, with t6 t10 t12 t16 m m t16 t12 t10 t6 (`mesures/moteur/O/corrections/seuils-declares.md`, block 6). The reason is a lemma.

Lemma A.1 (A palindrome cancels a linear drift)**.** *Let a session have n passes at times $1, \dots, n$, and s_i the configuration of pass i, with $s_i = s_{n+1-i}$ for all i. Assume pass i measures $X_i = \mu_{s_i} + a + b\,i + \varepsilon_i$, where μ_c is the true value of configuration c, $a + b\,i$ a linear drift of the machine (warm-up, foreign load rising or falling) and ε_i a zero-mean noise. Then, for two configurations c and c', the expectation of $\bar{X}_c - \bar{X}_{c'}$ is exactly $\mu_c - \mu_{c'}$: the drift vanishes, whatever a and b.*

Proof. Let $I_c = \{\, i : s_i = c \,\}$, of cardinality n_c. Since $s_i = s_{n+1-i}$, the map $i \mapsto n+1-i$ sends I_c into itself; it is an involution, hence a bijection of I_c. Then

$$\sum_{i \in I_c} i \;=\; \sum_{i \in I_c} (n+1-i) \quad\Longrightarrow\quad \frac{1}{n_c} \sum_{i \in I_c} i \;=\; \frac{n+1}{2}.$$

The expectation of $\bar{X}_c = \frac{1}{n_c} \sum_{i \in I_c} X_i$ is therefore $\mu_c + a + b\,\frac{n+1}{2}$. The drift term, $a + b\,(n+1)/2$, is the same for all configurations, and cancels in the difference. ∎

Remark A.2. The lemma holds only for a linear drift: a curved drift (a disk that cools quickly, then slowly) leaves a second-order term. It holds only for the part of the session that is a palindrome: the last four passes of the trial (T20 T20 T120tiede1 T120) do not form a palindrome with the rest, and the ratio comparing them to it is protected only by rule (A.1). The noise β of a configuration measures the gap between two *symmetric* passes: it contains the drift between them, so the rule is stricter when the machine drifts. This is intended.

The rule applies to the means of each configuration, never to a single pass. A warm-up pass is discarded when it precedes the series. Finding m1 of the trial gives the reason in numbers: the first `troncons disque` pass of its session read at 0.725 GB/s (90th percentile: 172 ms per read), the second at 1.822 GB/s, and the median of the two moved the threshold of C5 by 25 % (1.12 instead of 1.47 tokens/s; C5 held in both cases) [meas.] (`mesures/epreuve-J1.md`, m1). The round of corrections has since discarded a warm-up D pass before its series (block 3); the general rule awaits the arbiter's decision (Chapter 9).

A.5 The lock of the card and the shared machine

The card is a single resource, and other agents were working on the machine. Before any session that measures it, the driver script waits until the file `~/dev/verrou-carte.txt` is absent or older than two hours, then writes it in its own name (lot, date, tool, process number) and gives it back at the end (`mesures/moteur/O/pilote_o4.py`, and the driver scripts of lots E and K). Before *every* pass it reads and logs the power state (`Win32_Battery.BatteryStatus`, 2: mains), the live `llama-*` processes and the occupied VRAM; it stops if the machine is not on mains. The lock of the round of corrections was taken at 06:41, renewed at 07:21, 08:13 and 09:14, and given back at 10:06 (`moteur/REPRISE-O.md`); the trial's, taken at 02:26 and given back at 04:44 (Section 9.1).

The lock protects the card, not the rest. Foreign processor and disk load is recorded and left, never killed: no process one did not start is killed (SOCLE). "Best" throughputs therefore remain lower bounds of the idle machine, as in Volume I. The tools of lot F, which do not need the card, run at "below normal" priority (`mesures/moteur/F/lancer.ps1`), and builds each go to their own target, with four jobs (`-j 4`), because five lots were compiling at once.

A.6 The witness of the witnesses

A witness that cannot fall proves nothing. Every lot therefore wrote, next to its witnesses, a tool that breaks them one by one and checks that they fall, then hold once restored; Table A.4 gathers them (their full descriptions are in Chapter B).

Table A.4: The tools that break the witnesses, by lot.

Tool	What it breaks
`mesures/moteur/F/casser_temoins_f1.py`	the witnesses of F1 (header, table)
`mesures/moteur/F/casser_copie.py`	C1 on a real copy: a falsified median must be detected
`mesures/moteur/F/casser_lecture.py`	the coder's unbuffered read
`mesures/moteur/E/temoins_des_temoins.py`	every witness of `trc-cuda` and `trc-es`
`mesures/moteur/O/casser_mediane.py`	C3: a single median of a single expert changes in a copy of the 20b
`mesures/moteur/O/casser_tests.py`	every unit witness of `trc-cache`, in a throw-away copy of the crate
`mesures/moteur/O/corrections/temoin_garde.py`	the production guard, through `--casser-garde` and `--sans-garde`
`memoire/outils/tome2_reference_cache.py`	`--casser`: the redone numbers of the reference and cache chapters
`memoire/outils/tome2_annexes.py`	`--casser`: the witness of the options of Chapter B

The round of corrections pushed the rule one notch further: the break of a correction's witness is *the old binary*. Before correcting a finding, lot O replayed it on a frozen copy of the earlier program (commit `7b288a6`); the witness had to fall on the old one and hold on the new one (Section 9.9).

A.7 The independent trial

The protocol is written in § 8 of the architecture, before any construction, and followed in Chapter 9. The judge wrote neither the specification nor the code; he works from a clean clone (commit `527dff5`), on mains, card free, and reads only the specification, the code and the logs. His seven gestures: (1) a decoder written from § 4 alone, on 20 chunks drawn at random (seed published); (2) every break replayed; (3) C3, C4, C5, C7, C8, C10 redone in ABBA series with his own commands, judged by rule (A.1); (4) the baseline rerun, and another configuration of his choice; (5) the window: 1 GiB of warm cache must appear in the peak memory, between 0.9 and 1.1 times; (6) the text read; (7) the verdict, with his own "What could be wrong" section. His thresholds are in `mesures/moteur/epreuve-J1/seuils-declares.md`, committed before his first pass (Table A.2).

A.8 The round of corrections, block by block

Every finding of the trial received a block of thresholds, committed before the first pass it judges (`mesures/moteur/O/corrections/seuils-declares.md`). Table A.5 gives their content and outcome; the details are in Section 9.9.

Table A.5: The six threshold blocks of the round of corrections (16 October 2026) and their outcome (`mesures/moteur/0/corrections/seuils-declares.md`, `moteur/REPRISE-0.md`).

Block	Declared threshold	Outcome
1: G1, G2	on the old binary, the three cases (file absent, corrupted, process killed) must fall; on the new one, return within 30 s, code 1, error written, and 0 live `nvidia-smi` child 2 s later	falls on the old one (return at 90 s, one orphan each); holds on the new one (1.5, 4.9 and 4.9 s, 0 orphan), after two defects of the witness itself, stated and corrected
2: m2	with and without the SOCLE's `vcvars` recipe, building `trc-noyaux` succeeds and the three `noyaux.fatbin` images are equal byte for byte	holds: same SHA-256 (`aeae5bdc…`, 295 864 bytes)
3: M1	the production guard stays on by default if C5 holds with it; its cost is real according to rule (A.1); prediction [calc.]: not real (at most 2.4 %, declared with 1.1 GB/s and 460 ms; 2.5 % at the slowest measured throughput, 1.07 GB/s, Section 8.5.5)	the correctness witnesses hold (G-XO3, G-T, G-L: named fatal error, and none with `--sans-garde`; G-=: same text to the bit with and without the guard); cost not real in the ABBA series (−12.3 % against a threshold of 26.8 %), −1.6 % at equal disk (Section 8.5.5); guard on by default
4: G1 under load	the trial's hung pass, replayed under the same load, finishes or writes its error within 30 min, never a silent wait	both binaries, new and old, finish; 0 arrival conflict observed: the root cause of G1 is not established
5: M5	hypotheses H5a to H5c on the f32 KV cache (first token of C12, routing agreement, KL); cost in an ABBA series	measured: the f32 KV reaches the truth up to the format (Section 6.6); the decision on seam N left to the arbiter; default unchanged (f16)
6: M2	llama.cpp baseline swept in a palindrome; C6 recomputed against the best; ~~×3.85~~ struck out everywhere	best baseline `fit -t 16`, 0.884 token/s; C6 ×2.22 guard included, holds (Section 7.11)

A.9 Reproducing the builds

A published number is reproducible only if the program that produced it rebuilds. Three rules ensure it. *One target per lot and per judge*: each build writes to its own output folder, outside the repository (`mesures/moteur/E/construire.bat`, `mesures/moteur/epreuve-J1/construire.bat`), so that two lots do not overwrite each other. *The judge starts from a clean clone*, at a named commit (`527dff5`, built in 21 s). *The machine's recipe is tested by a witness*: the trial found that the SOCLE's recipe (`vcvars64.bat` first) broke the build of the kernels (finding m2); since the correction, block 2 of the round requires that building with and without it gives the same kernel image to the byte (`mesures/moteur/0/corrections/construire-avec-vcvars.bat`).

What could be wrong

- **The reference noise of 10 %** comes from a Volume I measurement (a 4 MiB read at queue depth 4); the chunks of the 120b, about 12 MB each, are read in a single request each. The factor 0.8 inherits it without having been remeasured for that length; Chapter 4 shows that the length changes the regime of the disk.
- **The palindrome lemma assumes a linear drift.** The machine's encrypted disk varies from 1.51 to 1.95 GB/s from one pass to the next within a single session (Section 8.5.5): this is not a

drift, it is a dispersion, which the palindrome does not remove and which only the rule covers.

- **The lock is a convention, not a mechanism of the system**: a program that does not read it can take the card. The driver scripts read the VRAM before every pass to see it, but a brief use between two readings would go unnoticed.
- **C6 is judged across two sessions** (guarded troncons in the round's session, the baseline in the next); the lemma does not protect a comparison between sessions. The single series remains to be done (Chapter 10).

Appendix B

Tools of the engine

Where we start, what this chapter establishes, what remains

Where we start. Appendix B of Volume I (“Tools”): every tool of the studies, sorted by study, with the first sentence of its own description, and the chain that builds the thesis. **What this appendix establishes.** The same for the system: the chain that builds this volume and checks its numbers; the `troncons` program, its four subcommands and every one of its options, kept up to date by a witness; every committed tool of the five lots, of the trial and of the round; every figure and the files it reads; every folder of logs. **What remains.** The trial options (`--kv-f32`, `--longueur-lecture`) await the arbiter’s decision to become defaults or disappear.

B.1 The chain of Volume II

Volume II is built like the first, by the same script and the same `latexmk` configuration (`memoire/construire.sh`, `memoire/latexmkrc`):

- `./construire.sh tome2` (or `tome2-en`, `tome2-fr`) regenerates the data of Volume II’s figures, redoes its numbers by a second path, regenerates the tables of this appendix, then builds the two PDFs with `latexmk` (lualatex, biber) in `memoire/tome2/en/` and `memoire/tome2/fr/`; `./construire.sh` without argument builds both volumes. The script fails at the first error, at any missing reference or citation, at any entry that biber does not find or finds twice, and at any data tool that exits with an error.
- `memoire/outils/tome2_format.py`: the CSV files of the “Architecture” and “Format” chapters (lines of code and witnesses per member, rates per tensor, chunks per layer, memory peaks of the coder), drawn from the `Cargo.toml` files and lot F’s logs, and the calculations of the format chapter.
- `memoire/outils/tome2_chemin_noyaux.py`: the CSV files of the chapters “The path of the bytes” and “The kernels”, drawn from the logs of lots E, K and O.
- `memoire/outils/tome2_reference_cache.py`: the CSV files of the “Reference” and “Cache” chapters, and 31 published numbers redone from the logs; it stops at the first gap, and `--casser` must make it fall.
- `memoire/outils/tome2_resultats.py`: the CSV files of the “Results”, “Trial” and “Discussion” chapters; it rereads every pass, recomputes means, noises, ratios, Wilson intervals and floors, and exits with an error if a recomputation departs from a published number.
- `memoire/outils/tome2_annexes.py`: the tables of this appendix and the threshold table of Chapter A, from `git ls-files` and `git log`; witness: the options of Table B.1 against those the code reads.

B.2 The `troncons` program

The program (`moteur/bins/troncons/src/main.rs`, lot O) has four subcommands. `generer` prefills the prompt, then produces n greedy tokens, and writes the text and a JSON log (command, date, power, engine counters, tokens/s, peak memory and VRAM, bytes read from disk by the system counter and by the engine, fingerprint of the text). `mesurer` is `generer` with ten seconds of rest before the pass, to quantify the idle disk. `evaluer` runs the windows of a llama-perplexity `.kld` base file (same tokens, each from position 0), passes the scored logits through the same 16-bit quantisation as llama.cpp (criterion C4) and writes the fingerprint64 of the logits of every step (criterion C3). `disque` measures the disk stage alone, in the same session as the engine: n chunks drawn without replacement with a published seed, read unbuffered at the engine's length, with `profondeur` reads in flight. Table B.1 gives every option.

Table B.1: The options of the `troncons` program, as `moteur/bins/troncons/src/main.rs` reads them (checked by `memoire/outils/tome2_annexes.py`: none too many, none too few). The names are French, as in the code.

Option	Default	Effect
The engine (every subcommand that opens it)		
`--modele` *f.trc*	required	the `.trc` file served
`--ram-tiede` *GB*	0 (mode A0)	warm cache: coded chunks kept in pinned memory
`--enchassement-en-ram`	no	keep the embedding table (0.6 GB) in pinned memory instead of reading one row per token
`--profondeur` *n*	4	reads entrusted to the system at once
`--couches-d-avance` *d*	3	layers whose arrival buffers rotate (window of $d \cdot k$ buffers)
`--contexte` *n*	4 096	positions at most (KV cache of the full layers)
`--temoin`	no	every decoded expert goes back to the host, its fingerprint compared to the directory (C2)
`--brut`	no	the expected file is raw (flag bit 2); it must be
`--emplacements` *n*	from the VRAM	expert slots in video memory
`--vram-plafond` *MiB*	7 179	ceiling of the total VRAM used to compute the slots: $8{,}188 - 909 - 100$
`--longueur-lecture` *B*	the format's	length of every chunk read, a multiple of 4 096 (trial: Section 4.9)
`--chronogramme` *f.csv*	none	dates of every stage of every miss
`--prechargement`	no	J1+: anticipated reads of the experts predicted by the next layer's router (tried, abandoned)
`--sans-battement`	heartbeat	removes the line written at every layer of the streamed prefill (finding G1)
`--sans-garde`	guard	removes the production guard; serves only to quantify its cost
`--casser-garde` *troncon:N* | *ligne:N*	none	witness of the guard: flips one bit of the N-th chunk or N-th row read
`--kv-f32`	f16	trial: f32 KV cache (finding M5; seam N with the arbiter)
`generer`, `mesurer`		
`--vocab` *f.vocab.gguf*	required	the tokenizer companion

Option	Default	Effect
`--invite` *f.txt*	required without `--harmonie`	the prompt, in UTF-8 (its SHA-256 goes to the log)
`--harmonie` *"question"*	none	a prompt in gpt-oss's harmony conversation format
`--systeme` *text*	fixed message	the system message of the harmony prompt
`--n` *n*	256	tokens produced (`disque`: chunks read, 1 000)
`--repos` *s*	0; 10 (`mesurer`)	seconds of rest before the pass, to quantify the idle disk
`--dll` *f.dll*	llama.cpp b11515	the tokenizer library
`--arret-fin`	no	stop at the end token of the harmony conversation
`--sans-vram`	VRAM sampled	do not sample the VRAM
`--flux`	step loop	streamed prefill (J1+): every expert read once for all positions
`--journal` *f.json*	none	the log of the pass
`--sortie` *f*	none; required by `evaluer`	the text produced; for `evaluer`, the `.kld` written
`--trace` *f.trace*	none	the cache trace (for `rejouer`)
`evaluer`		
`--base` *f.kld*	required	llama-perplexity's base file (tokens and logits)
`--fenetres` *n*	all	windows evaluated
`--positions` *n*	all (512)	positions evaluated per window
`--empreintes` *f.csv*	none	the fingerprint64 of the f32 logits of every step (C3)
`disque`		
`--graine` *g*	1	seed of the draw of chunks (xorshift64*, that of lot E's experiments)

B.3 The committed tools

Every [meas.] or [calc.] number of this volume comes from a committed tool. Table B.2 lists them, sorted by lot, each with the first line of its own description, quoted as written (in French), and its number of lines. The table is generated by `memoire/outils/tome2_annexes.py` from `git ls-files`: it follows the repository.

Table B.2: The tools of the engine and of its measurements, by lot.

Tool	Its own description (first line, quoted as written, in French)	Lines
O: the troncons program		
`moteur/bins/troncons/src/main.rs`	troncons : le programme (ARCHITECTURE.md § 5.10).	683
F: the coder		
`moteur/bins/trc-codeur/src/main.rs`	trc-codeur : le codeur GGUF → .trc de troncons et la vérification C1 (lot F).	125
E: the path bench		
`moteur/bins/banc-chemin/src/main.rs`	banc-chemin : le banc du lot E.	223
K: the kernel bench		

Tool	Its own description (first line, quoted as written, in French)	Lines
`moteur/crates/trc-noyaux/banc/banc_k.cu`	Banc du lot K (étapes K1 et K2) : témoins, cassures et débits des noyaux du moteur, sur les poids réels de gpt-oss, par un chemin indépendant des lots F et E.	1 376
`moteur/crates/trc-noyaux/banc/construire.bat`	Construit le banc du lot K (banc_k.exe) et l'image des noyaux seule, hors du depot. CUDA 13.1.2 depaquete sans installation (SOCLE, section La machine). …	12
O: the cache simulator		
`moteur/crates/trc-cache/src/bin/rejouer.rs`	rejouer : le simulateur du cache en ligne de commande (témoin O1, courbes de succès).	119
O: engine examples		
`moteur/crates/trc-ordo/examples/debit_empreinte.rs`	Débit de empreinte64 (préfixe de SHA-256, crate sha2 0.10.9) sur des tampons de la taille d'un tronçon, sur un fil puis sur n fils …	47
`moteur/crates/trc-ordo/examples/memoire_unitaire.rs`	Coûts unitaires en ensemble de travail (WS) des structures du moteur qui croissent avec le modèle (constat M4 de l'épreuve J1 : le terme de 0,5 Mo de C7 ne comptait que le répertoire). …	37
R: the judge program		
`moteur/crates/trc-juge/src/bin/juge.rs`	juge : le programme du lot R.	572
E: byte-path measurements		
`mesures/moteur/E/analyse_e0.py`	Analyse d'une séance E0 (journaux de banc-chemin) : verdicts contre les seuils déclarés, CSV.	216
`mesures/moteur/E/analyse_e1.py`	Analyse de la mesure E1 du lot E (débit du lecteur, disque seul) : verdicts contre les seuils déclarés avant la première passe mesurée (moteur/REPRISE-E.md, « E1, débit du lecteur sur le …	117
`mesures/moteur/E/analyse_e2.py`	Analyse de l'étape E2 du lot E : verdicts contre les critères déclarés avant toute ligne de code d'E2 (moteur/REPRISE-E.md, « E2 », commit 6b59ff3), CSV des figures du mémoire.	211
`mesures/moteur/E/balayage_longueurs.py`	Balayage du débit du lecteur sans tampon contre la longueur d'une lecture (lot E, diagnostic d'E2).	115
`mesures/moteur/E/construire.bat`	Construit banc-chemin (lot E) en release, cible propre au lot, -j 4 (cinq lots compilent en meme temps). Argument 1 (facultatif) : (rien) …	32
`mesures/moteur/E/pilote_e0.py`	Pilote des essais E0 du lot E (ARCHITECTURE.md § 6.5) : une séance complète, journalisée.	192
`mesures/moteur/E/pilote_e1.py`	Pilote de la mesure E1 du lot E : débit du lecteur sans tampon, disque seul (banc-chemin lecteur).	191
`mesures/moteur/E/pilote_e2.py`	Pilote de l'étape E2 du lot E : le chemin disque → VRAM décodé sur gpt-oss-20b (banc-chemin chemin).	184
`mesures/moteur/E/serie_nvvn.py`	Série N V V N du lot E (déclarée dans moteur/REPRISE-E.md avant la mesure) : banc-chemin chemin tel que spécifié (N, lecture de longueur0) contre la variante E2-V (V, –longueur-lecture), …	84

Tool	Its own description (first line, quoted as written, in French)	Lines
mesures/moteur/E/temoins_des_temoins.py	Témoin des témoins de l'étape E1 (lot E) : chaque témoin de trc-cuda et de trc-es doit tomber quand on force sa condition à faux, puis tenir une fois restauré (SOCLE, « Le niveau exigé »).	67
F: format measurements		
mesures/moteur/F/casser_copie.py	Cassure C1 sur une vraie copie du fichier (§ 6.5 : « une médiane faussée dans une copie ⟶ inexact détecté »). …	36
mesures/moteur/F/casser_lecture.py	Témoin des témoins de la lecture sans tampon de trc-codeur (gguf::lire_dans), F3.	83
mesures/moteur/F/casser_temoins_f1.py	Témoin des témoins de F1 (SOCLE : « force sa condition à faux, relance, vérifie que l'instrument tombe, puis restaure »).	72
mesures/moteur/F/comparer_f1.py	Juge des taux du lot F contre les seuils déclarés (mesures/moteur/F/00-seuils-declares.md).	70
mesures/moteur/F/lancer.ps1	Lance trc-codeur en priorité « sous la normale » (machine partagée), sorties redirigées, et note l'alimentation (Win32_Battery.BatteryStatus : 2 = sur secteur) avant et après. Usage …	17
mesures/moteur/F/lancer_py.ps1	Lance un outil Python du lot F en priorité « sous la normale » (machine partagée), sorties redirigées sous journaux/, alimentation (Win32_Battery.BatteryStatus …	18
mesures/moteur/F/second_chemin_c1.py	Second chemin de C1 (lot F), sur le fichier .trc et son GGUF source.	684
mesures/moteur/F/vers_csv.py	Données des figures du lot F, tirées des journaux de trc-codeur verifier (les octets viennent des fichiers .trc eux-mêmes) …	83
K: kernel measurements		
mesures/moteur/K/attendre_et_mesurer.ps1	Attend que la carte soit libre (aucun llama-*, aucun traceur, aucun verrou), puis lance le pilote des bancs de débit du lot K (pilote_bancs.ps1, qui revérifie tout et prend le verrou). …	30
mesures/moteur/K/k3_biais_attention.py	—	29
mesures/moteur/K/k3_vers_csv.py	K3 : les CSV des figures du mémoire, tirés des journaux commités sous k3/ (aucun chiffre recopié).	69
mesures/moteur/K/pilote_bancs.ps1	Pilote des bancs de débit du lot K (étapes K1 et K2), en séries A B B A de processus.	105
mesures/moteur/K/temoin-k3/src/chaine.rs	Mode chaine : la passe avant entière de gpt-oss enchaînée sur la carte par les seuls noyaux de K1 à K3, contre Reference::evaluer (f64), sur des jetons réels.	376
mesures/moteur/K/temoin-k3/src/main.rs	Témoin K3 : les douze noyaux de la passe avant, chacun contre trc-reference.	1 171
mesures/moteur/K/temoin-trc/src/main.rs	Témoin K1 sur un vrai fichier .trc du lot F.	432
mesures/moteur/K/vers_csv.py	Données des figures et tableaux du lot K pour le mémoire (CSV), tirées des journaux commités.	137
O: round of corrections		

Tool	Its own description (first line, quoted as written, in French)	Lines
mesures/moteur/O/corrections/analyse_base.py	Bloc 6 de seuils-declares.md : la base llama.cpp balayée (M2), et C6 recalculé.	71
mesures/moteur/O/corrections/analyse_kv.py	Bloc 5 de seuils-declares.md (M5) : le cache KV en f32 contre f16, jugé contre la vérité f64 du juge.	114
mesures/moteur/O/corrections/analyse_ronde.py	Analyse de la séance « vitesse » de la ronde de corrections (blocs 3 et 5 de seuils-declares.md).	102
mesures/moteur/O/corrections/construire-avec-vcvars.bat	Ronde de corrections J1, constat m2 : la recette du SOCLE (vcvars64.bat d abord) sur trc-noyaux, arbre de travail du depot, cible a part. Usage : construire-avec-vcvars.bat [sans]	16
mesures/moteur/O/corrections/pilote_ronde.py	Pilote des séances de la ronde de corrections J1 (blocs 3, 5 et 6 de seuils-declares.md).	100
mesures/moteur/O/corrections/rejouer_g1.py	Bloc 4 de seuils-declares.md : la passe figée de l'épreuve (G1) rejouée sous la même charge.	91
mesures/moteur/O/corrections/temoin_garde.py	Témoins de la garde de production (constat M1 de l'épreuve J1), sur le 20b, passes courtes.	104
mesures/moteur/O/corrections/temoin_sortie.py	Témoin des constats G1 et G2 de l'épreuve J1 : le programme rend-il la main, avec son erreur, sans laisser d'enfant vivant ?	136
O: engine measurements		
mesures/moteur/O/analyse_o4.py	Verdicts de la séance O4 = J1 (C5, C7, C8, C10) depuis les journaux de pilote_o4.py.	89
mesures/moteur/O/casser_mediane.py	Cassure de C3 (lot O) : une copie de gpt-oss-20b.trc où une seule médiane d'un seul expert change.	62
mesures/moteur/O/casser_tests.py	Cassures des témoins unitaires de trc-cache : chaque témoin qui peut tomber est forcé à faux une fois (SOCLE, « le niveau exigé »), dans une copie jetable de la crate, et l'on vérifie qu'il …	137
mesures/moteur/O/courbe_dense.py	Courbe dense du succès du cache global (LRU et Belady) contre le nombre d'emplacements, sur les traces réelles de gpt-oss-20b, par rejouer (trc-cache) …	66
mesures/moteur/O/pilote_j1plus.py	Pilote de la seconde séance du 120b (lot O) : diagnostic D-C7, variante L12, préchargement (J1+), C12 sur le fichier brut, et la conversation harmony publiée.	199
mesures/moteur/O/pilote_o3.py	Pilote de l'étape O3 du lot O : C3 (codé contre brut, au bit), le compte du cache contre rejouer.	201
mesures/moteur/O/pilote_o4.py	Pilote de l'étape O4 = J1 du lot O : vitesse (C5), fenêtre bornée (C7), octets lus (C8), VRAM (C10).	126
mesures/moteur/O/temoin-o2/src/main.rs	Témoin O2 : la passe avant de trc-modele contre la chaîne du lot K, ligne pour ligne.	273
mesures/moteur/O/temoin_o1.py	Témoin O1 du lot O : le simulateur du moteur (trc-cache, rejouer) contre les simulateurs Python des études 04 et gpt-oss, compté à l'unité (ARCHITECTURE.md § 6.5, ligne O1).	236
mesures/moteur/O/vers_csv_o.py	Données des figures du mémoire pour les étapes O2 à O4 du lot O, tirées des journaux commités.	81
R: reference measurements		
mesures/moteur/R/bascules.py	Les bascules de routage entre deux passes de la référence (lot R, R1, reprise du 2026-10-15).	90

Tool	Its own description (first line, quoted as written, in French)	Lines
`mesures/moteur/R/r0_ecart.py`	R0 : l'écart entre moteurs sur gpt-oss-20b (ARCHITECTURE.md § 6.5, lot R).	295
`mesures/moteur/R/r1_reference.py`	R1 : la référence f64 de gpt-oss, et son jugement contre llama.cpp (ARCHITECTURE.md § 6.5, lot R).	226
`mesures/moteur/R/rampe_yarn.py`	Second chemin de la rampe de YaRN de gpt-oss (figure R-M3 du mémoire ; lot R).	69
Trial J1 (the independent judge)		
`mesures/moteur/epreuve-J1/analyse_c8.py`	C8 refait par le juge, deux instruments.	36
`mesures/moteur/epreuve-J1/analyse_seance.py`	Analyse de la séance de l'épreuve (pilote_epreuve.py) : verdicts des seuils de seuils-declares.md.	134
`mesures/moteur/epreuve-J1/base_kld_invite.py`	Fabrique un .kld de base d'UNE fenêtre de n jetons donnés (format de llama-perplexity), pour que troncons evaluer passe ces jetons depuis la position 0 et écrive l'argmax de chaque …	22
`mesures/moteur/epreuve-J1/comparer_c3.py`	C3 par le chemin du juge : .kld et empreintes de logits, codé contre brut, et contre le lot O.	61
`mesures/moteur/epreuve-J1/comparer_invite.py`	Argmax position par position : vérité numpy (argmax_toutes) contre l'empreinte CSV de troncons.	39
`mesures/moteur/epreuve-J1/comparer_kld.py`	Juge de l'épreuve : KL(vérité ‖ fichier) et premier choix, position par position.	99
`mesures/moteur/epreuve-J1/comparer_routage.py`	Routage de troncons (trace du moteur, une ligne par pas, L groupes de k experts) contre celui de la vérité numpy (routage[L, P, k]) …	31
`mesures/moteur/epreuve-J1/construire-avec-vcvars.bat`	Epreuve J1, constat m2 : la recette du SOCLE (vcvars d'abord) sur la seule crate trc-noyaux.	9
`mesures/moteur/epreuve-J1/construire.bat`	Epreuve J1 : construit troncons depuis un clone propre, cible propre au juge, -j 4.	10
`mesures/moteur/epreuve-J1/decodeur_s4.py`	Épreuve indépendante du jalon J1, geste 1 (decisions/ARCHITECTURE.md § 8.1).	613
`mesures/moteur/epreuve-J1/ecart_logp.py`	Écart des log-probabilités entre la vérité numpy du juge et un .kld, hors plancher du format.	43
`mesures/moteur/epreuve-J1/echantillonner_fil.py`	Diagnostic d'un processus bloqué (lancé par le juge) : où tourne le fil donné ?	63
`mesures/moteur/epreuve-J1/gptoss_numpy.py`	Une troisième passe avant de gpt-oss, en numpy f64, écrite par le juge de l'épreuve J1.	283
`mesures/moteur/epreuve-J1/normalisation_kld.py`	Témoin de l'instrument du juge : log Σ exp(log q) de chaque entrée d'un .kld (0 si normalisé).	46
`mesures/moteur/epreuve-J1/pile_fil.py`	Diagnostic d'un fil bloqué : adresses de retour de troncons.exe trouvées sur sa pile, nommées.	89
`mesures/moteur/epreuve-J1/pilote_epreuve.py`	Épreuve indépendante de J1 (§ 8.3 à 8.5) : vitesse, RAM, VRAM et base llama.cpp, par mes commandes.	304
`mesures/moteur/epreuve-J1/temoin_compteurs_es.py`	Témoin de l'instrument C8 du juge : GetProcessIoCounters compte-t-il une lecture sans tampon ?	61
`mesures/moteur/epreuve-J1/verdict_c4.py`	C4 par la vérité du juge : comparaison appariée (troncons contre llama.cpp, mêmes positions).	35

Tool	Its own description (first line, quoted as written, in French)	Lines
`mesures/moteur/epreuve-J1/verifier_vocab.py`	§ 4.13 : le compagnon <nom>.vocab.gguf est un GGUF v3 valide, n_tenseurs = 0, toutes les paires du source octet pour octet, aucune donnée. Lecteur GGUF de decodeur_s4.py (écrit ici). …	19
The thesis, Volume II		
`memoire/outils/tome2_annexes.py`	Produit les tables des annexes du Tome II depuis le dépôt lui-même, et vérifie l'annexe des outils.	263
`memoire/outils/tome2_chemin_noyaux.py`	Données des figures des chapitres « Le chemin des octets » et « Les noyaux sur la carte » (Tome II).	249
`memoire/outils/tome2_format.py`	Données des chapitres « Architecture » et « Format .trc » du Tome II (memoire/tome2/).	158
`memoire/outils/tome2_juge.py`	Second chemin du juge technique du Tome II (memoire/tome2/JUGEMENT.md).	152
`memoire/outils/tome2_reference_cache.py`	Données des figures et chiffres refaits des chapitres « La référence exacte et le juge » et « Le cache, l'ordonnanceur et le moteur » du Tome II.	279
`memoire/outils/tome2_reprise.py`	Reprise du jugement du Tome II : refait, depuis les CSV commités, chaque chiffre que la reprise publie ou corrige (objections G1 et m1 à m21 de memoire/tome2/JUGEMENT.md), et vérifie dans …	175
`memoire/outils/tome2_resultats.py`	Données des chapitres « Résultats », « L'épreuve » et « Discussion » du Tome II.	332
Total	83 tools	14 504

B.4 The figures and their data

Every pgfplots graph of the volume reads a committed file; every TikZ diagram is a file under `memoire/figures/tome2/`. Table B.4 gives, for every figure command that reads data, the files it reads; the tool that generates it exits with an error if one of them is missing.

Table B.4: The figures of Volume II and the data files they read.

Figure (command)	Data files it reads
`\GrapheCchFalaise`	`mesures/moteur/O/csv/falaise-20b-dense.csv`
`\GrapheCchProp`	`memoire/figures/tome2/donnees/t2o-prop1.csv`
`\GrapheCchSucces`	`mesures/localite-gpt-oss/figures/succes-gpt-oss-20b-lecture.csv` `mesures/localite-gpt-oss/figures/succes-gpt-oss-120b-lecture.csv`
`\GrapheCchChrono`	`memoire/figures/tome2/donnees/t2o-chrono-couche.csv`
`\GrapheCchVitesse`	`memoire/figures/tome2/donnees/t2o-c6.csv`
`\GrapheChmSync`	`memoire/figures/tome2/donnees/t2c-sync.csv`
`\GrapheChmDma`	`memoire/figures/tome2/donnees/t2c-dma.csv`

Figure (command)	Data files it reads
\GrapheChmEun	memoire/figures/tome2/donnees/t2c-e1-s1.csv memoire/figures/tome2/donnees/t2c-e1-s3.csv
\GrapheChmLittle	memoire/figures/tome2/donnees/t2c-little.csv
\GrapheChmEdeux	memoire/figures/tome2/donnees/t2c-e2-n.csv memoire/figures/tome2/donnees/t2c-e2-v.csv memoire/figures/tome2/donnees/t2c-nvvn.csv
\GrapheChmEtages	memoire/figures/tome2/donnees/t2c-etages.csv
\GrapheChmChrono	mesures/moteur/E/journaux/e2-2026-10-15-seance1/chrono-1/chrono-A-t1-r0.csv mesures/moteur/E/journaux/e2-2026-10-15-v12/chrono-v12582912-1/chrono-A-t1-r0.csv
\GrapheChmLongueurs	mesures/moteur/E/csv/e2-balayage-2026-10-15-longueurs.csv
\GrapheChmCouche	memoire/figures/tome2/donnees/t2c-couche.csv
\GrapheChmLdouze	memoire/figures/tome2/donnees/t2c-l12.csv
\TdArchGrapheLignes	memoire/figures/donnees/tome2/caisses.csv
\TdFmtGrapheTaux	mesures/moteur/F/csv/taux-par-tenseur-120b.csv mesures/moteur/F/csv/taux-par-tenseur-20b.csv
\TdFmtGrapheTroncons	mesures/moteur/F/csv/troncons-par-couche-120b.csv mesures/moteur/F/csv/troncons-par-couche-20b.csv
\TdFmtGraphePointes	mesures/moteur/F/csv/pointes-memoire.csv
\GrapheNoyDecodeur	memoire/figures/tome2/donnees/t2n-k1.csv
\GrapheNoyDecomposition	mesures/moteur/K/csv/k1-decomposition.csv
\GrapheNoyGemv	memoire/figures/tome2/donnees/t2n-k2.csv
\GrapheNoyEcarts	mesures/moteur/K/csv/k2-ecarts-reels.csv
\GrapheNoyMarges	memoire/figures/tome2/donnees/t2n-marges.csv
\GrapheNoyChaine	mesures/moteur/K/csv/k3-chaine-kl.csv
\GrapheNoyFenetre	mesures/moteur/K/csv/k3-chaine-kl.csv
\GrapheNoyRoutage	memoire/figures/tome2/donnees/t2n-routage.csv
\GrapheNoyKV	memoire/figures/tome2/donnees/t2n-kv-120b.csv
\GrapheRefRampe	mesures/moteur/R/r1/csv/rampe-yarn.csv
\GrapheRefRepartition	memoire/figures/tome2/donnees/t2r-kl-repartition.csv
\GrapheRefBascules	memoire/figures/tome2/donnees/t2r-bascules-autres.csv memoire/figures/tome2/donnees/t2r-bascules-soi.csv
\GrapheRefKv	memoire/figures/tome2/donnees/t2r-kv-glissant.csv memoire/figures/tome2/donnees/t2r-kv-invite-glissant.csv

Figure (command)	Data files it reads
\GrapheRefRoutage	mesures/moteur/epreuve-J1/csv/routage-par-couche-invite.csv
\GrapheResVitesse	memoire/figures/tome2/donnees/res-vitesse-120b.csv
\GrapheResPlancher	memoire/figures/tome2/donnees/res-plancher.csv
\GrapheResMemoire	memoire/figures/tome2/donnees/res-memoire.csv
\GrapheResKL	memoire/figures/tome2/donnees/res-kl-120b-f0.csv memoire/figures/tome2/donnees/res-kl-invite.csv
\GrapheResRoutage	memoire/figures/tome2/donnees/res-routage.csv
\GrapheResSeance	memoire/figures/tome2/donnees/res-epreuve-seance.csv

B.5 The logs

The raw logs, CSV files and notes of every lot are committed under `mesures/moteur/`. Table B.6 gives their extent, folder by folder (the tools, counted in Table B.2, are excluded). Heavy files stay outside the repository: the `.trc` files and their broken copies under `~/dev/modeles/trc/`, the outputs of the passes (`.kld`, `.npz`) under `~/dev/essais/troncons/`, the built programs under `~/dev/cibles/`; the logs keep their SHA-256 fingerprints and the commands.

Table B.6: The committed logs of the engine, by folder.

Folder (logs, CSV, notes; tools excluded)	Files	Size (MB)
mesures/moteur/E/csv/	14	0.13
mesures/moteur/E/journaux/	223	1.82
mesures/moteur/E/miroir/	2	0.01
mesures/moteur/F/	6	0.04
mesures/moteur/F/csv/	9	0.03
mesures/moteur/F/journaux/	70	0.49
mesures/moteur/K/	23	0.12
mesures/moteur/K/csv/	9	0.03
mesures/moteur/K/journaux/	44	0.04
mesures/moteur/K/k3/	61	1.20
mesures/moteur/K/temoin-k3/	2	0.01
mesures/moteur/K/temoin-trc/	2	0.01
mesures/moteur/O/corrections/	94	0.82
mesures/moteur/O/csv/	8	0.30
mesures/moteur/O/journaux/	190	11.22
mesures/moteur/O/temoin-o2/	2	0.01
mesures/moteur/R/r0/	60	3.57

Folder (logs, CSV, notes; tools excluded)	Files	Size (MB)
`mesures/moteur/R/r1/`	41	3.82
`mesures/moteur/R/r2/`	11	0.19
`mesures/moteur/epreuve-J1/`	2	0.01
`mesures/moteur/epreuve-J1/csv/`	4	0.05
`mesures/moteur/epreuve-J1/journaux/`	134	1.90
Total	1 011	25.79

What could be wrong

- **The first line of a description is not the tool**: it says what its author meant it to do. The chapters cite tools by what they produced, with their logs.
- **The witness of the options compares names, not meanings**: a default changed in the code without Table B.1 changing would pass. The defaults of the table were reread in `moteur/crates/trc-ordo/src/lib.rs` (`Config::nouvelle`) and in `main.rs` on 16 October 2026.
- **The heavy files outside the repository** are reproducible only through their fingerprints and the commands in the logs; their folder may be freed by Stone.

Bibliography

[1] Open Compute Project. *OCP Microscaling Formats (MX) Specification, Version 1.0.* Définit MXFP4 : éléments E2M1, échelle partagée E8M0 par bloc de 32. 2023. URL: https://www.opencompute.org/documents/ocp-microscaling-formats-mx-v1-0-spec-final-pdf (visited on 2026-10-15).

[2] S. Kullback and R. A. Leibler. "On Information and Sufficiency". In: *The Annals of Mathematical Statistics* 22.1 (1951-03). La divergence d'information entre deux lois, positive et nulle seulement pour deux lois égales ; c'est la mesure du juge., pp. 79–86. DOI: 10.1214/aoms/1177729694. URL: https://doi.org/10.1214/aoms/1177729694 (visited on 2026-10-16).

[3] J. D. C. Little. "A Proof for the Queuing Formula: $L = \lambda W$". In: *Operations Research* 9.3 (1961). Métadonnées relues par l'API de Crossref (l'éditeur refuse la lecture automatique). Second chemin du débit du lecteur : X = q b / R (dérivation D3 du lot E)., pp. 383–387. DOI: 10.1287/opre.9.3.383. (Visited on 2026-10-15).

[4] NVIDIA Corporation. *CUDA Driver API Reference Manual, 6.27 Module Management.* Version 13.4. `cuModuleLoadData` accepte une image cubin ou fatbin produite par nvcc, ou du PTX terminé par un octet nul : c'est ce qui permet d'embarquer l'image des noyaux dans le binaire (`include_bytes!`) sans aucune DLL d'exécution de CUDA. 2026-10-12. URL: https://docs.nvidia.com/cuda/cuda-driver-api/group__CUDA__MODULE.html (visited on 2026-10-16).

[5] The Cargo Team. *Workspaces, The Cargo Book.* Membres par motifs (`crates/*`), `Cargo.lock` et dossier de sortie communs, héritage des versions par `[workspace.dependencies]` et `workspace = true` ; ce que fait `moteur/Cargo.toml`. URL: https://doc.rust-lang.org/cargo/reference/workspaces.html (visited on 2026-10-16).

[6] Microsoft. *File Buffering.* FILE_FLAG_NO_BUFFERING : taille et décalage multiples du secteur du volume, adresse du tampon alignée sur le secteur physique ; VirtualAlloc aligne sur la page (4 096 sur x64). trc-es exige 4 096 pour les trois. 2025-05-31. URL: https://learn.microsoft.com/en-us/windows/win32/fileio/file-buffering (visited on 2026-10-15).

[7] ggml-org. *GGUF: file format specification.* https://github.com/ggml-org/ggml/blob/master/docs/gguf.md. Licence MIT (dépôt ggml) ; paires clé-valeur, types 0 à 12, descriptions des tenseurs, alignement `general.alignment` ; lu et réécrit de zéro par trc-format et trc-codeur ; non reconsulté par le lot F le 2026-10-15.

[8] J. Duda. *Asymmetric numeral systems: entropy coding combining speed of Huffman coding with compression rate of arithmetic coding.* 2013. arXiv: 1311.2540. URL: https://arxiv.org/abs/1311.2540 (visited on 2026-10-15).

[9] F. Giesen. *Interleaved entropy coders.* 2014. arXiv: 1402.3392. URL: https://arxiv.org/abs/1402.3392 (visited on 2026-10-15).

[10] Open Compute Project. *OCP Microscaling Formats (MX) Specification, Version 1.0*. Tech. rep. MXFP4 : blocs de 32 éléments E2M1 et une échelle E8M0 ; non reconsulté par le lot F le 2026-10-15. 2023. URL: https://www.opencompute.org/documents/ocp-microscaling-formats-mx-v1-0-spec-final-pdf.

[11] National Institute of Standards and Technology. *Secure Hash Standard (SHS), FIPS PUB 180-4*. Tech. rep. SHA-256 ; empreinte64 = ses 8 premiers octets en petit-boutiste ; vecteur de test « abc » vérifié par le test empreinte64_connue ; non reconsulté le 2026-10-15. 2015. URL: https://doi.org/10.6028/NIST.FIPS.180-4.

[12] ggml-org. *llama.cpp b11515 : lecteur GGUF de ggml (gguf_init_from_file, ggml-base.dll)*. Version b11515. Licence MIT (vérifiée par l'étude 06). Binaire Windows CUDA 12.4 de la machine, empreinte vérifiée (attendu-llama.txt). Fonctions publiques appelées par ctypes depuis second_chemin_c1.py le 2026-10-15 (gguf_get_n_tensors, gguf_get_tensor_name, _offset, _type, _size, gguf_get_data_offset, gguf_get_alignment) : troisième lecture des décalages des tenseurs, aucune ligne copiée. URL: https://github.com/ggml-org/llama.cpp.

[13] NVIDIA. *CUDA Driver API Reference Manual*. Version 13.4. Gestion de la mémoire : cuMemHostAlloc (« Allocates page-locked host memory »), cuMemHostGetDevicePointer, copies asynchrones ; avertissement sur la mémoire épinglée en excès (« may degrade system performance »), sous cuMemAllocHost. Le moteur est construit contre les en-têtes de CUDA 13.1.2 et tourne sur le pilote 591.91 (13.1). URL: https://docs.nvidia.com/cuda/cuda-driver-api/group__CUDA__MEM.html (visited on 2026-10-15).

[14] NVIDIA. *CUDA Driver API Reference Manual, Primary Context Management*. Version 13.4. cuDevicePrimaryCtxSetFlags : CU_CTX_SCHED_SPIN, YIELD, BLOCKING_SYNC ; AUTO attend activement tant que le nombre de contextes actifs ne dépasse pas celui des processeurs logiques (le cas de la machine : 1 contexte, 16 fils). Mesuré par l'essai E0(c). URL: https://docs.nvidia.com/cuda/cuda-driver-api/group__CUDA__PRIMARY__CTX.html (visited on 2026-10-15).

[15] NVIDIA. *CUDA C++ Best Practices Guide*. Version 13.4. Recouvrement copie et calcul : asyncEngineCount ; un seul moteur de copie permet une copie asynchrone à la fois, en même temps que les noyaux ; deux moteurs permettent les deux sens à la fois. La carte de la machine rend 1 sous WDDM (relevé par trc-cuda). URL: https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/index.html (visited on 2026-10-15).

[16] Microsoft. *I/O Completion Ports*. Paquets de complétion en file FIFO ; valeur de concurrence ; le bloc d'état n'est mis à jour qu'au retrait du paquet, sauf échec synchrone (d'où, dans trc-es, l'OVERLAPPED gardé jusqu'à la récolte, et l'annulation suivie de l'attente de chaque paquet). 2025-07-18. URL: https://learn.microsoft.com/en-us/windows/win32/fileio/i-o-completion-ports (visited on 2026-10-15).

[17] NVIDIA. *CUDA Programming Guide, 5.4 C/C++ Language Extensions*. Version 13.4.2. Fonctions de vote du warp (__ballot_sync, § 5.4.6.2 : « Evaluates predicate for all non-exited threads in mask »), d'échange (__shfl_sync, __shfl_xor_sync, § 5.4.6.5), bornes de lancement (__launch_bounds__, § 5.4.3.2). Le pas rANS du décodeur repose sur le vote (quelles voies renormalisent) et le comptage de bits (où chacune lit). URL: https://docs.nvidia.com/cuda/cuda-programming-guide/05-appendices/cpp-language-extensions.html (visited on 2026-10-15).

[18] C. Wellons. *Prospecting for Hash Functions*. Constantes de la fonction de mélange 32 bits « lowbias32 » (domaine public), utilisées par les noyaux d'essai pour tirer des indices d'experts que l'hôte sait recalculer. 2018-07-31. URL: https://nullprogram.com/blog/2018/07/31/ (visited on 2026-10-15).

[19] Microsoft. *Calculating Counter Values*. Pour PERF_COUNTER_BULK_COUNT, valeur affichée (N1 - N0) / ((D1 - D0) / F) : la donnée brute N est un compte cumulé (ici d'octets lus), d'où les octets d'un intervalle par simple différence de deux relevés, sans intégrer un débit échantillonné. 2025-07-14. URL: https://learn.microsoft.com/en-us/windows/win32/perfctrs/calculating-counter-values (visited on 2026-10-15).

[20] Microsoft. *PdhGetRawCounterValue function (pdh.h)*. Rend la valeur brute d'un compteur (structure PDH_RAW_COUNTER). Utilisé par banc-chemin (disque.rs) pour les octets lus par toute la machine pendant une passe. 2025-07-01. URL: https://learn.microsoft.com/en-us/windows/win32/api/pdh/nf-pdh-pdhgetrawcountervalue (visited on 2026-10-15).

[21] S. Pronovost. *Hardware Accelerated GPU Scheduling*. L'ordonnancement de la carte passe d'un fil du processeur à un processeur dédié de la carte ; les applications peuvent soumettre des lots plus petits plus souvent. Cadre de la variante « boîte sans vidage » de l'essai E0(c) (le pilote retient-il un lancement sans appel qui le pousse ?). Microsoft, DirectX Developer Blog. 2020-06-30. URL: https://devblogs.microsoft.com/directx/hardware-accelerated-gpu-scheduling/ (visited on 2026-10-15).

[22] NVIDIA. *NVIDIA Ada GPU Architecture Tuning Guide*. Version 13.4. Capacité de calcul 8.9 : « The shared memory capacity per SM is 100 KB », 99 Kio au plus par bloc (1 Kio réservé par bloc), 48 warps résidents au plus par SM, 64 K registres de 32 bits par SM. D'où l'occupation de decoder_codes (48 Kio de LUT par bloc de 8 warps : 2 blocs, 16 warps par SM) et le calcul de la vague unique. 2026-10-12. URL: https://docs.nvidia.com/cuda/ada-tuning-guide/index.html (visited on 2026-10-15).

[23] ggml-org. *llama.cpp, ggml/src/ggml-common.h : block_mxfp4 et kvalues_fp4*. Licence MIT (vérifiée par l'étude 06). QK_MXFP4 = 32 ; bloc de 17 octets : un octet E8M0 puis 16 octets de demi-octets ; table des valeurs 0, 1, 2, 3, 4, 6, 8, 12 et leurs opposées (grandeurs E2M1 doublées). Lu pour cadrer la disposition, rien de copié ; la GEMV de troncons la redérive (03-derivations § 1.2) et la juge contre une référence f64 écrite depuis cette seule définition. URL: https://github.com/ggml-org/llama.cpp/blob/master/ggml/src/ggml-common.h (visited on 2026-10-15).

[24] IEEE. *IEEE Standard for Floating-Point Arithmetic (IEEE Std 754-2019)*. binary16 (11 bits de précision : u = 2^{-11}) et binary32 (u = 2^{-24}) ; l'arrondi du cache KV (M5). 2019. DOI: 10.1109/IEEESTD.2019.8766229. URL: https://doi.org/10.1109/IEEESTD.2019.8766229 (visited on 2026-10-16).

[25] N. J. Higham. *Accuracy and Stability of Numerical Algorithms*. 2nd ed. Bornes d'erreur d'un produit scalaire (gamma_n = n u / (1 − n u)) ; § 13.3 de O-pour-le-memoire. SIAM, 2002. DOI: 10.1137/1.9780898718027. URL: https://doi.org/10.1137/1.9780898718027 (visited on 2026-10-16).

[26] B. Zhang and R. Sennrich. "Root Mean Square Layer Normalization". In: *Advances in Neural Information Processing Systems (NeurIPS)*. Version unique. Définition du noyau rmsnorm. 2019-10-16. arXiv: 1910.07467. URL: https://arxiv.org/abs/1910.07467 (visited on 2026-10-15).

[27] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen, and Y. Liu. *RoFormer: Enhanced Transformer with Rotary Position Embedding*. v5 du 2023-11-08. Rotation par paires. gpt-oss tourne par moitiés (forme NEOX), avec les fréquences de YaRN (peng2023yarn) : c'est le noyau rope_yarn. 2021-04-20. arXiv: 2104.09864. URL: https://arxiv.org/abs/2104.09864 (visited on 2026-10-15).

[28] B. Peng, J. Quesnelle, H. Fan, and E. Shippole. "YaRN: Efficient Context Window Extension of Large Language Models". In: *arXiv* (2023-08-31). L'interpolation « NTK par parties » et le facteur de température $\sqrt{1/t} = 0{,}1 \ln s + 1$; cité par l'implémentation d'OpenAI. arXiv: 2309.00071. URL: https://arxiv.org/abs/2309.00071 (visited on 2026-10-15).

[29] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis. "Efficient Streaming Language Models with Attention Sinks". In: *International Conference on Learning Representations (ICLR)*. v4 du 2024-04-07. Le « puits » d'attention. Dans gpt-oss, il est appris par tête et entre dans le dénominateur du softmax (noyau attention). C'est pourquoi un biais de clé ne s'y annulerait pas, et le biais des clés vaut exactement 0 dans les deux GGUF (k3-biais-attention-*.log). 2023-09-29. arXiv: 2309.17453. URL: https://arxiv.org/abs/2309.17453 (visited on 2026-10-15).

[30] N. Shazeer. *GLU Variants Improve Transformer*. Version unique. SwiGLU. gpt-oss en emploie une forme bornée, avec α = 1,702 et u + 1 (openai2025gptossref) : c'est le noyau swiglu_oai. 2020-02-12. arXiv: 2002.05202. URL: https://arxiv.org/abs/2002.05202 (visited on 2026-10-15).

[31] NVIDIA. *CUDA Math API, Single Precision Intrinsics*. Version 13.4. Pour __fadd_rn et __fmul_rn : « This operation will never be merged into a single multiply-add instruction ». Le noyau combiner en dépend pour égaler au bit la somme de l'hôte. La cassure qui remet un fmaf fait tomber le test dès l'élément 649 (tests/passe.rs). 2026-10-12. URL: https://docs.nvidia.com/cuda/cuda-math-api/cuda_math_api/group__CUDA__MATH__INTRINSIC__SINGLE.html (visited on 2026-10-15).

[32] OpenAI. *gpt-oss : implémentation de référence en PyTorch (`gpt_oss/torch/model.py`)*. Version commit 243a1b02767da73bd2e3975be250afa801635866. Apache-2.0. Définit RMSNorm, la rotation YaRN « NTK par parties » (bornes de la rampe non arrondies, concentration $0{,}1 \ln s+1$ appliquée au cosinus et au sinus), l'attention à puits appris et fenêtre de 128 sur les couches paires, le routeur top-4 suivi d'un softmax sur les quatre valeurs, la SwiGLU bornée ($\alpha = 1{,}702$, limite 7). Lue en entier ; rien n'est copié. 2025-08-05. URL: https://github.com/openai/gpt-oss/blob/main/gpt_oss/torch/model.py (visited on 2026-10-15).

[33] OpenAI. *openai/gpt-oss-20b, `config.json`*. Version révision 6cee5e81ee83917806bbde320786a8fb61efebee. Apache-2.0. `rope_scaling` : facteur 32, contexte d'origine 4 096, β rapide 32, β lent 1, `"truncate": false` ; `layer_types` alterne fenêtre glissante (couches paires) et attention pleine. 2025-08-26. URL: https://huggingface.co/openai/gpt-oss-20b/blob/main/config.json (visited on 2026-10-15).

[34] The ggml authors. *llama.cpp, version b11515*. Version b11515, commit 3d65c90d0. MIT. Lus : `tools/perplexity/perplexity.cpp` (format du fichier `--kl-divergence-base`, codage des log-probabilités sur 16 bits, formule de la KL, quantiles), `src/models/openai-moe.cpp`, `src/llama-graph.cpp` (routage `SOFTMAX_WEIGHT`), `ggml/src/ggml.c` (`ggml_rope_yarn_corr_dims` : bornes de la rampe arrondies par `floorf` et `ceilf`), `ggml/src/ggml-cpu/ops.cpp` (`rope_yarn`, `swiglu_oai`), `src/llama-context.cpp` (facteur d'attention de YaRN), `include/llama.h` (API publique). Relus le

2026-10-15 (reprise, par raw.githubusercontent.com à l'étiquette b11515) : ggml/src/ggml-cpu/ggml-cpu.c (type_traits_cpu : MXFP4 et Q8_0 ont vec_dot_type = Q8_0, l'entrée d'un produit quantifié est donc ramenée en Q8_0), ggml/src/ggml-cpu/arch/x86/quants.c (quantize_row_q8_0 AVX2 : d = amax/127 en f16, id = 127/amax, arrondi au plus proche), ggml/src/ggml-cpu/ops.cpp (attention éclair du processeur : noyau pavé quand le lot de requêtes est grand, puits appliqués au premier morceau), ggml/src/ggml.c (ggml_rope_yarn_corr_dims, revérifié). Rien n'est copié. 2026. URL: https://github.com/ggml-org/llama.cpp (visited on 2026-10-15).

[35] K. Goto and R. A. van de Geijn. "Anatomy of High-Performance Matrix Multiplication". In: *ACM Transactions on Mathematical Software* 34.3 (2008-05). Notice vérifiée le 2026-10-15 par Crossref (api.crossref.org/works/10.1145/1356052.1356053 : titre, auteurs, revue, 34(3), pages 1–25, parution en ligne le 2008-05-16) ; le texte de l'article n'a pas été relu ce jour (dl.acm.org répond 403), pp. 1–25. DOI: 10.1145/1356052.1356053. URL: https://doi.org/10.1145/1356052.1356053 (visited on 2026-10-15).

[36] T. M. Cover and J. A. Thomas. *Elements of Information Theory*. 2nd ed. Chapitre 10 : théorème 10.3.2 (source gaussienne) et théorème du débit-distorsion. Hoboken, NJ: Wiley-Interscience, 2006. ISBN: 978-0-471-24195-9. DOI: 10.1002/047174882X. (Visited on 2026-10-15).

[37] E. B. Wilson. "Probable Inference, the Law of Succession, and Statistical Inference". In: *Journal of the American Statistical Association* 22.158 (1927). Intervalle de score utilisé pour le taux de premier choix identique (C4)., pp. 209–212. DOI: 10.1080/01621459.1927.10502953.

[38] S. Merity, C. Xiong, J. Bradbury, and R. Socher. *Pointer Sentinel Mixture Models*. Le corpus WikiText-2 (CC BY-SA 3.0) ; wiki.test.raw, empreinte dans les journaux de l'étude 06. 2016-09-26. arXiv: 1609.07843. URL: https://arxiv.org/abs/1609.07843 (visited on 2026-10-15).

[39] L. A. Belady. "A Study of Replacement Algorithms for a Virtual-Storage Computer". In: *IBM Systems Journal* 5.2 (1966), pp. 78–101. DOI: 10.1147/sj.52.0078. (Visited on 2026-10-15).

[40] R. L. Mattson, J. Gecsei, D. R. Slutz, and I. L. Traiger. "Evaluation techniques for storage hierarchies". In: *IBM Systems Journal* 9.2 (1970). Les algorithmes à pile (le contenu à la capacité C est inclus dans celui à C' > C) : LRU et MIN en sont ; c'est la propriété qui rend l'étage RAM inclusif de trc-cache égal à un cache unique de sa taille (lemme 2 du lot O)., pp. 78–117. DOI: 10.1147/sj.92.0078. URL: https://doi.org/10.1147/sj.92.0078 (visited on 2026-10-15).

[41] D. D. Sleator and R. E. Tarjan. "Amortized Efficiency of List Update and Paging Rules". In: *Communications of the ACM* 28.2 (1985). Rapport compétitif de LRU : au plus N fois les défauts de l'optimum hors ligne (plus une constante), et aucune politique en ligne déterministe ne fait mieux ; la suite cyclique sur N + 1 pages en est le pire cas., pp. 202–208. DOI: 10.1145/2786.2793. URL: https://doi.org/10.1145/2786.2793 (visited on 2026-10-16).

[42] NVIDIA. *CUDA Driver API Reference Manual, v13.4 : Stream Management (cuStreamWaitEvent)*. « Makes all future work submitted to hStream wait for all work captured in hEvent » : la sémantique sur laquelle repose l'ordre sûr des trois flux de trc-ordo (lemme 4). 2026-10-12. URL: https://docs.nvidia.com/cuda/cuda-driver-api/group__CUDA__STREAM.html (visited on 2026-10-15).

[43] National Institute of Standards and Technology. *Secure Hash Standard (SHS)*. Federal Information Processing Standards Publication 180-4. SHA-256, dont le préfixe de 64 bits est l'empreinte64 du format ; la garde de production (M1). 2015-08. DOI: 10.6028/NIST.FIPS.180-4. URL: https://doi.org/10.6028/NIST.FIPS.180-4 (visited on 2026-10-16).

[44] OpenAI. *harmony : renderer for the harmony response format to be used with gpt-oss*. Apache-2.0. Jetons <|start|>, <|message|>, <|end|>, canaux analysis, commentary, final ; la conversation du § 8 point 6 est construite d'après ce format, les identifiants des jetons spéciaux retrouvés par leur texte dans le vocabulaire du GGUF. URL: https://github.com/openai/harmony (visited on 2026-10-15).

[45] Microsoft. *TerminateProcess function (processthreadsapi.h)*. Arrêt sans notification des bibliothèques (contrairement à ExitProcess), annulation des E/S en cours ; la sortie sur erreur sans destruction du moteur (G1). 2026-01-07. URL: https://learn.microsoft.com/en-us/windows/win32/api/processthreadsapi/nf-processthreadsapi-terminateprocess (visited on 2026-10-16).

[46] Microsoft. *JOBOBJECT_BASIC_LIMIT_INFORMATION structure (winnt.h)*. JOB_OBJECT_LIMIT_KILL_ON_JOB_CLOSE : tous les processus de la tâche meurent à la fermeture de sa dernière poignée ; l'enfant nvidia-smi de l'échantillonneur (constat G2). 2024-02-22. URL: https://learn.microsoft.com/en-us/windows/win32/api/winnt/ns-winnt-jobobject_basic_limit_information (visited on 2026-10-16).

[47] Microsoft. *CreateProcessW function (processthreadsapi.h)*. bInheritHandles = TRUE : chaque poignée héritable du parent passe à l'enfant ; la cause du tuyau tenu par l'orphelin (G2) ; d'où les poignées standard rendues non héritables. 2025-07-01. URL: https://learn.microsoft.com/en-us/windows/win32/api/processthreadsapi/nf-processthreadsapi-createprocessw (visited on 2026-10-16).

[48] C. E. Shannon. "A Mathematical Theory of Communication". In: *Bell System Technical Journal* 27.3 (1948), pp. 379–423. DOI: 10.1002/j.1538-7305.1948.tb01338.x. (Visited on 2026-10-15).

[49] R. Shafipour, D. Harrison, M. Horton, et al. *SeedLM: Compressing LLM Weights into Seeds of Pseudo-Random Generators*. v1 du 2024-10-14, v2 du 2024-10-16 (la plus récente à la consultation). 2024-10-14. arXiv: 2410.10714. URL: https://arxiv.org/abs/2410.10714 (visited on 2026-10-16).

[50] D. Ha, A. Dai, and Q. V. Le. *HyperNetworks*. v4 du 2016-12-01 (la plus récente à la consultation). 2016-09-27. arXiv: 1609.09106. URL: https://arxiv.org/abs/1609.09106 (visited on 2026-10-16).

[51] S. Ohno. *Evolution by Gene Duplication*. Métadonnées relues par l'API Crossref. Berlin, Heidelberg: Springer, 1970. DOI: 10.1007/978-3-642-86659-3. (Visited on 2026-10-16).

[52] R. P. N. Rao and D. H. Ballard. "Predictive coding in the visual cortex: a functional interpretation of some extra-classical receptive-field effects". In: *Nature Neuroscience* 2.1 (1999). Métadonnées relues par l'API Crossref, pp. 79–87. DOI: 10.1038/4580. (Visited on 2026-10-16).

[53] P. R. Huttenlocher. "Synaptic density in human frontal cortex: Developmental changes and effects of aging". In: *Brain Research* 163.2 (1979-03). Métadonnées relues par l'API Crossref, pp. 195–205. DOI: 10.1016/0006-8993(79)90349-4. (Visited on 2026-10-16).

[54] Y. LeCun, J. S. Denker, and S. A. Solla. “Optimal Brain Damage”. In: *Advances in Neural Information Processing Systems 2 (NIPS 1989)*. 1989. URL: https://proceedings.neurips.cc/paper/1989/hash/6c9882bbac1c7093bd25041881277658-Abstract.html (visited on 2026-10-16).

[55] S. Han, H. Mao, and W. J. Dally. *Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding*. v5 du 2016-02-15 (la plus récente à la consultation). 2015-10-01. arXiv: 1510.00149. URL: https://arxiv.org/abs/1510.00149 (visited on 2026-10-16).